

Modellbildung und Simulation

—
Eine Einführung

Data Science & Scientific Computing / Informatik / Digital Engineering

Prof. Dr. W. Högele



Fakultät Informatik und Mathematik

Sommersemester 2026

Urheberrechtlicher Hinweis

Dies ist eine Skriptfassung aus dem Inhalt des 4. Kapitels folgenden Lehrbuchs:

*Aufbruch in die Angewandte Mathematik –
Vom Verständnis zum praktischen Werkzeug*

Wolfgang Högele, Springer Spektrum, 2026

ISBN: 978-3-662-72189-6, eBook ISBN: 978-3-662-72190-2.

Diese Skriptfassung darf nicht weiter verbreitet werden und darf nur für persönliche/interne Lernzwecke verwendet werden.

Inhalt

- ① Einführung und Begriffe
- ② Modell vs. Daten
- ③ Modelle und ihre Simulation

Ablauf der Vorlesung

- **Moodle ist Ihr Startpunkt für alles rund um die Vorlesung.** Dieser Bereich ist passwortgeschützt. Jede unauthorisierte Weitergabe der Dokumente auf Moodle wird hiermit untersagt.
- Der Ablauf der Vorlesung erfolgt in der Regel mit 2x 1.5h / Woche. Es wird die strikte Aufteilung (Vorlesung/Übung) im Wesentlichen befolgt (mit Ausnahmen bspw. zu Beginn von Übungen).
- Die Vorlesungen werden durch folgende Elemente lebendig gestaltet:
 - Kontinuierliche Beispiel- und Verständnisaufgaben, daher Schmierzettel bereitlegen
 - Zwischenzeitliche Abstimmungen bei Verständnisfragen (Peer Instruction)
 - Ihre fachlichen Fragen: Präsent: einfach Hand heben, Virtuell: Im Chat melden, und dann auf meine Bitte Mikro einschalten

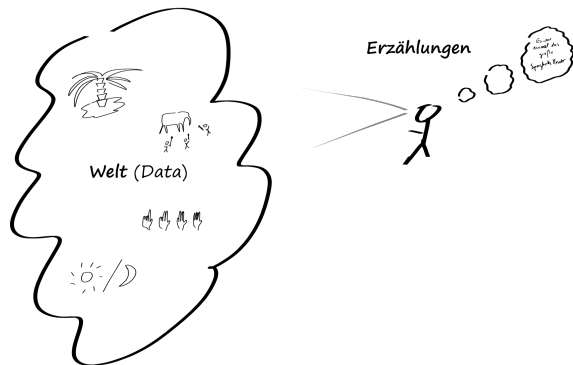
Zu den Unterlagen:

- Sie erhalten das Skript zur Vorlesung, das Sie für sich kommentieren bzw. ergänzen können bzw. sollten (entweder digital oder auf Papier)
 - Machen Sie in der Vorlesung aktiv mit! Damit steigert sich Ihr Verständnis enorm!
- Auch lohnt es sich einzelne Programm-Code-Beispiele im Skript selbst nachzubauen und dann damit auch eigene Experimente durchzuführen.

1. Einführung und Begriffe

Verortung in einer Pseudo-Historie

Verortung in einer Pseudo-Historie: Das „Prä-Scientificum“



Beobachtung: „Das ist interessant!“ Das möchte ich mir als wichtige Erkenntnis behalten und Anderen näher bringen.

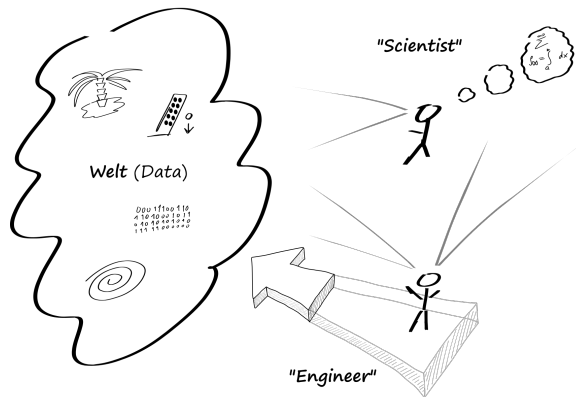
→ Erzählungen / Mythen / Skizzen. Der Nutzen ist dabei meist sehr praktisch.

Das Grundprinzip lautet:

Wir suchen Erzählungen, die für *viele verständlich* sind, uns einen Ausschnitt der Welt *erklären*, *Vorhersagen* / *Planungen* / *Optimierungen ermöglichen* und uns somit zu Handlungen führen.

→ Dies verstehen wir als ein Grundprinzip der Erkenntnisgewinnung.

Verortung in einer Pseudo-Historie: Das „Scientificum“



Grundlagenwissenschaft: Welche Gesetzmäßigkeit steckt hinter der Beobachtung?

⇒ „Scientist“

→ Entwicklung grundlegender mathematischer Modelle.

Angewandte Wissenschaft: Ich möchte diese Erkenntnisse anwenden, um die Welt zu verändern!

⇒ „Engineer“

→ Kenntnisse sowohl der relevanten Modelle, als auch der konkreten Rahmenbedingungen (Daten aus der Welt) notwendig.

→ Anwendung der Modelle, um konkrete Veränderung zu erreichen.

Frage: Warum benötigt man Modelle und probiert als „Engineer“ nicht einfach aus?

Antwort: Da man einen möglichst „guten“ Versuch zum Ausprobieren benötigt! D.h. ein Versuchsaufbau der zumindest nicht im Widerspruch zu bisheriger gesicherter Theorie steht, im besten Fall sogar „optimal“ bzgl. dieser ist.

Im Detail gibt es Probleme mit

- Kosten (wie viele Experimente?)
- Zeit (wie lang darf ich für die Aufgabe brauchen?)
- Möglichkeiten (kann man überhaupt Experimente durchführen?)
- Fähigkeiten (kann man Experimente auch praktisch umzusetzen?)

Denken Sie an praktische Beispiele wie

- *Der Bau einer Brücke:* Wie viele Pfeiler, welche Größe, für welche Belastung, wie lange soll diese stehen bleiben, Schwingungen/Resonanzeffekte,...?
- *Konstruktion einer Rakete:* Wie schwer darf die Rakete sein im Verhältnis zum benötigten Equipment? Wie viel Treibstoff benötigt man bei welcher Antriebskonstruktion? Wie wird die Rakete am besten gesteuert und geregelt?

Hinweis:

Wir Menschen verändern die Welt, welche dadurch auch neue Gesetzmäßigkeiten hervorbringt (*Rückkopplung*)!

D.h. aus Angewandter Wissenschaft kann einerseits weitere Angewandte Wissenschaft, andererseits aber auch wieder Grundlagenwissenschaft entstehen.

- Bspw. Wissen im Internet, Kommunikation,.. vs. Philosophie, Sprachwissenschaft, Soziologie,..

Beispiel 1.1 (Fragestellungen aus Rückkopplungen)

- *Wie erstellen wir ein stabiles Stromnetz für viele Haushalte, wenn die Stromquellen sehr ungleichmäßig Strom liefern?*
- *Welche Maßnahmen helfen, um die menschengemachte Klimakrise zu bewältigen?*
- *Wie kann man die Ausbreitung von Epidemien in den globalen Verkehrsströmen besser verstehen, verhindern, kontrollieren, ...?*
- *Wie lege ich ein konkretes Produkt optimal aus, sodass es in einem beweglichen Marktumfeld für Kunden attraktiv ist?*

1. Einführung und Begriffe

Modelle sind Abbildungen

Definition 1.1 (Modell)

Ein **Modell** ist eine Abbildung eines kleinen Ausschnitts der zu untersuchenden Welt unter Zuhilfenahme von bekannten Gesetzmäßigkeiten in einer *konkreten, nützlichen Darstellung*, um mit dieser zu arbeiten.

Das ist ein sehr dynamischer / flexibler Prozess, da er abhängt

- vom Geschick und dem Wissen der modellierenden Person,
- was man mit dem Modell überhaupt herausfinden will,
- wie groß bzw. genau der Ausschnitt der Welt sein soll/muss,
- welche Werkzeuge zur Darstellung eines Modells zur Verfügung stehen, ...

Da niemand die Welt oder auch nur einen Ausschnitt davon vollständig erfassen kann, muss man immer Abstrahieren, Idealisieren – d.h. Vereinfachen.

⇒ Modelle sind daher stets *vereinfachende Abbildungen von Ausschnitten der Welt!*

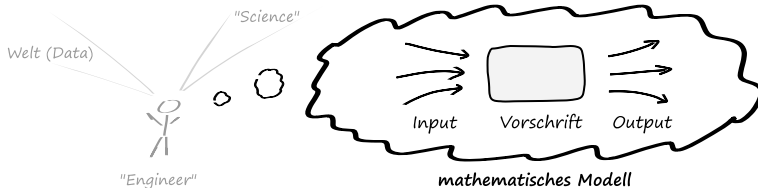
Definition 1.2 (Mathematisches Modell)

Ein **Mathematisches Modell** ist ein Modell, dessen konkrete, nützliche Darstellung durch ein *mathematisches Formelwerk*, bspw. Gleichungen, etc. beschrieben wird.

Es besteht dabei immer aus einem

- definierten zahlenmäßigen **Input** (*meist Daten, Annahmen, Rahmenbedingungen*),
 - einem definierten zahlenmäßigen **Output** (*häufig eine gesuchte Größe*)
 - und einer **Vorschrift**, welchen den Input in den Output überführt.
- ⇒ Alle drei Komponenten sind Teil des mathematischen Modells.

⇒ Mathematische Modelle sind daher *auch im engeren Sinn* Abbildungen!



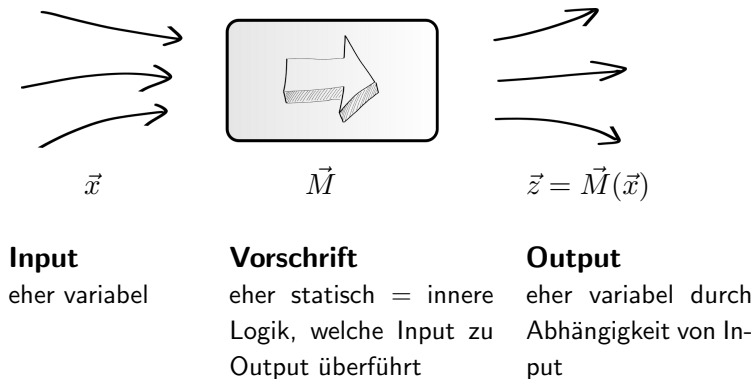
1. Einführung und Begriffe

Allgemeine Klassifikation von mathematischen Modellen

Allgemeine Klassifikation von mathematischen Modellen

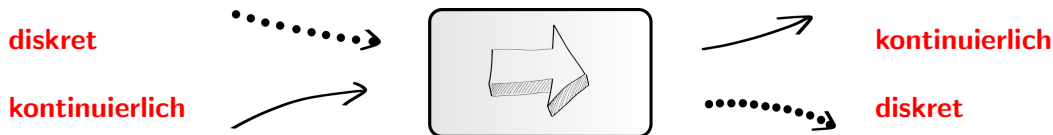
Mathematische Modelle (und das Arbeiten mit diesen) kann man anhand von Kriterien in verschiedene *Klassen* unterteilen:

Für alle Modelle gilt jedoch folgendes grundlegendes Schema:



Aufgabe des „*Engineer*“s: So wenig Blackbox wie möglich, Input–Vorschrift–Output so gut wie möglich verstehen, da Modelle unsere neue Form von „Erzählungen“ sind.

Allgemeine Klassifikation von mathematischen Modellen: Kriterium: Diskret / Kontinuierlich



⇒ 4 grundlegende Typen

Beispiel 1.2 (Diskret / Kontinuierlich)

Input:

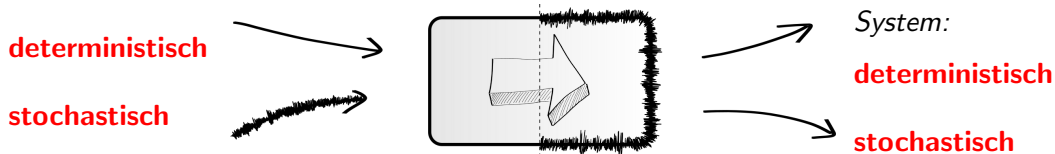
- *diskret*: zeitdiskret (schrittweiser Fortschritt), Optionen/Schalter für Parameter
- *kontinuierlich*: klassische physikalische Zeit, Koordinate, frei wählbare „Tuning“-Parameter

Output:

- *diskret*: Klassenzuordnung, ganzzahlige Ergebnisse (Rundung), boolean
- *kontinuierlich*: klassisch physikalische Messgröße, Geldmenge

⇒ Häufig ersetzt man Modelle mit kontinuierlichem Input/Output aufgrund der Berechenbarkeit durch Modelle mit diskretem Input/Output.

Allgemeine Klassifikation von mathematischen Modellen: Kriterium: Deterministisch / Stochastisch



⇒ 4 grundlegende Typen

Beispiel 1.3 (Deterministisch / Stochastisch)

Input:

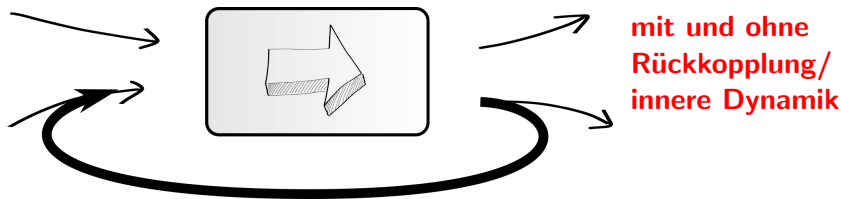
- *deterministisch*: klassisch physikalischer Zeitverlauf
- *stochastisch*: Messwerte als Input (immer mit Rauschen), unklares Vorwissen

Vorschrift:

- *deterministisch*: physikalisches Gesetz, einfache mathematische Funktion
- *stochastisch*: gleicher Input liefert nicht immer gleiches Ergebnis, Zeitreihen, „Rauschen“ im System, Zufallsentscheidungen

⇒ In stochastischen Fällen häufig statistische Auswertung des Outputs nötig!

Allgemeine Klassifikation von mathematischen Modellen: Kriterium: Mit und Ohne innerer Dynamik



⇒ 2 grundlegende Typen

Beispiel 1.4 (Mit und Ohne innerer Dynamik)

Systeme ohne innere Dynamik:

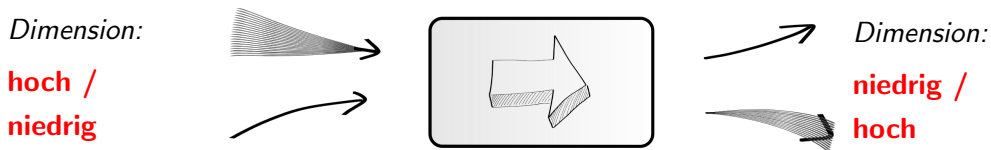
→ klassische Ursache/Wirkung-Beziehung, einfache physikalische Gesetze

Systeme mit innerer Dynamik:

→ Differentialgleichungen, Differenzengleichungen, iterative Modelle wie bspw. Optimierungen, Feedback-Loops, Zinseszins

⇒ Im dynamischen Fall sind qualitativ neue Eigenschaften, wie Stabilität, Periodizität, Chaotik, Resonanz, Gleichgewichtszustände,... zu beobachten!

Allgemeine Klassifikation von mathematischen Modellen: Kriterium: Dimensionalität



⇒ 4 grundlegende Typen

Beispiel 1.5 (Dimensionalität)

Input:

- *Dimension hoch*: Big Data Anwendungen, Machine Learning
- *Dimension niedrig*: Idealisierte physikalische Systeme

Output:

- *Dimension hoch*: Bildrekonstruktion (alle Pixelwerte), zeitliche+räumliche Verteilung von physikalischen Größen (bspw. Strömungssimulation)
- *Dimension niedrig*: typische Optimierung (Zielfunktionswert)

⇒ Die Dimensionalität eines Modells verändert häufig die möglichen Tools und verlangt Erfahrung in der Datenverwaltung

Allgemeine Klassifikation von mathematischen Modellen: Zusammenfassung

Allein mit dieser vereinfachten Klassifizierung gibt es bereits

$$4 \times 4 \times 2 \times 4 = 128$$

verschiedene mathematische Modelltypen!

Nicht zu vergessen: Mischungen innerhalb einzelner Klassen sind immer möglich!

- ⇒ Jede Anwendung erzeugt ein sehr individuell angepasstes Modell. Es gibt deshalb auch keine „klassische allgemeine Theorie“, die für alle Modelltypen anwendbar ist.
- ⇒ Auch die Art wie man mit einzelnen Modelltypen „arbeitet“ kann sich erheblich unterscheiden und benötigt sehr unterschiedliche mathematische Kenntnisse.
- ⇒ Daher gibt es auch typischerweise keine „Spezialisten“ für alle Arten von Modellen, sondern man vertieft sich häufig in einzelne Bereiche.

In der Praxis gibt es auch verschiedene Selbstverständnisse von *Modellierung*:

1. **konkrete Anwendung** → **mathematisches Modell**

Dabei ist die konkrete Anwendung sehr stark im Vordergrund, bspw. der *Balken dessen Durchbiegung* modelliert werden soll, oder der konkrete *Autotyp* für den die Luftströmung simuliert wird.

→ Diese Sichtweise ist besonders in einem konkret umrissenen Anwendungsfeld relevant bei dem man aus typischen, gegebenen mathematischen Modellen – häufig in einer Spezialsoftware – auswählt.

2. **mathematisches Modell** → **konkrete Anwendung**

Dabei geht es eher um den allgemeinen Blick was / wie grundsätzlich modelliert werden kann. Bspw. startet man mit den Gleichungen der *Fluidodynamik*, welche physikalische Grundprinzipien beschreiben und die man dann für verschiedene konkrete Anwendungen anpasst (bspw. Flugzeug, Autobau, Strömungssimulation unterschiedlicher Fluidarten,...).

→ Diese Sichtweise ist besonders dann relevant, wenn Sie an Grundprinzipien der Modellbildung interessiert sind und grundsätzlich neue mathematische Modelle entwerfen möchten bzw. müssen. Diese Vorlesung orientiert sich an Punkt 2.

1. Einführung und Begriffe

Simulation: Mit mathematischen Modellen arbeiten

Simulation: Mit mathematischen Modellen arbeiten

Eines der wichtigsten Eigenschaften beim praktischen Arbeiten mit mathematischen Modellen ist die *Berechenbarkeit*, d.h. wie man konkret die **Vorschrift** realisiert!

Da wir stets mit dem Computer arbeiten, gibt es dabei mehrere Wege: Man kann

- die Berechnung als *explizite Formel* auf dem Blatt lösen oder aus einer Quelle übernehmen und in den Computer eintippen,
- einen *direkten Algorithmus* an den Computer übergeben, welcher die Berechnung exakt ausführt (auch wenn man keine explizite Formel dafür hat), oder
- einen *approximativen Algorithmus* an den Computer übergeben, welcher die Berechnung näherungsweise („genau genug“) ausführt.

Insbesondere für die *algorithmische Herangehensweise* hat sich der Begriff des „Computer-Experiments“ etabliert, da man in der Praxis häufig den *Input* und/oder das *Berechnungsverfahren* oder *Parameter des Modells* variiert und dabei das Verhalten des *Outputs* beobachtet und auf *Plausibilität* prüft.

Definition 1.3 (Simulation)

Simulation ist die Durchführung von Berechnungen mit Hilfe mathematischer Modelle (typischerweise „Computer-Experimente“).

Beispiel 1.6 (Explizite Formel: explizite Lösungen von Differentialgleichungen)

- **Input:** Anfangswert $y(0) = y_0$, Parameter γ, N , Zeitpunkte t
- **Vorschrift:** $y'(t) = \gamma \cdot y(t) \cdot (N - y(t))$ (SI-Modell der Epidemiologie)
- **Output:** y -Werte zu t

$$y'(t) = \gamma \cdot y(t) \cdot (N - y(t))$$

$$\Rightarrow \int \frac{1}{y \cdot (N - y)} dy = \int \gamma dt$$

$$\Rightarrow \frac{1}{N} \int \frac{1}{y} + \frac{1}{N - y} dy = \gamma \cdot t + c$$

$$\Rightarrow \frac{1}{N} \ln \left(\frac{|y|}{|N - y|} \right) = \gamma \cdot t + c$$

$$y > 0 \wedge N - y > 0 \Rightarrow y(t) = \frac{N}{1 + e^{-\gamma \cdot N \cdot t} \cdot \tilde{c}}$$

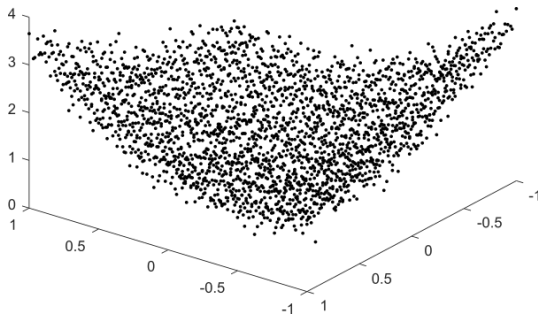
$$y(0) = y_0 \Rightarrow y(t) = \frac{N}{1 + e^{-\gamma \cdot N \cdot t} \left(\frac{N}{y_0} - 1 \right)}$$

Explizite Berechnung:

Anwendung der Separation der Variablen und Partialbruchzerlegung führt zu einer expliziten Darstellung von $y(t)$. Diese wird für alle t -Werte ausgewertet.

Beispiel 1.7 (Direkter Algorithmus: Diskrete, erschöpfende Suche eines Optimums)

- **Input:** Alle Werte (x, y, z)
- **Vorschrift:** Suche die Stelle (x, y) mit dem kleinsten z -Wert!
- **Output:** (x, y, z) dieses Minimums



Direkter Algorithmus:

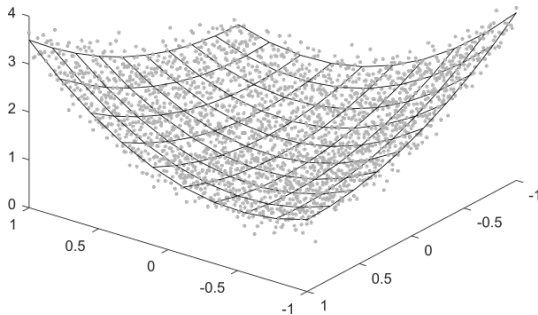
Gehe systematisch alle Punkte durch, und merke dir den (x, y, z) -Wert für das bis dahin kleinste z . Am Ende hat man garantiert das Minimum gefunden.

Beispiel 1.8 (Approximativer Algorithmus I: Heuristik in der diskreten Suche als Annäherung an das ursprüngliche Problem)

- **Input:** Alle Werte (x, y, z)
- **Vorschrift:** Suche die Stelle (x, y) mit dem kleinsten z -Wert, durch Annäherung mit einem Paraboloiden!
- **Output:** (x, y, z) dieses Minimums

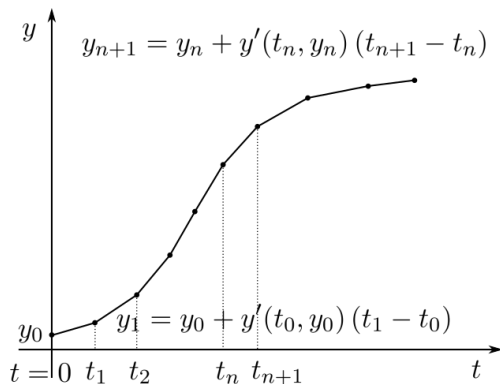
Approximativer Algorithmus:

Erkenne, dass die Punktemenge im Wesentlichen einen Paraboloiden beschreibt. Fitte einen Paraboloiden in die Punktemenge und finde über die explizite Berechnung der Nullstelle des Gradienten den Kandidaten des Extremums (x_p, y_p) . Suche nun in der Menge (x, y) welches am nächsten an (x_p, y_p) liegt.



Beispiel 1.9 (Approximativer Algorithmus II: numerische Verfahren, Diskretisierung bei kontinuierlichen Problemen)

- **Input:** Anfangswert $y(0) = y_0$, Parameter γ, N , Zeitpunkte t
- **Vorschrift:** $y'(t) = \gamma \cdot y(t) \cdot (N - y(t))$ (SI-Modell der Epidemiologie)
- **Output:** y -Werte zu t



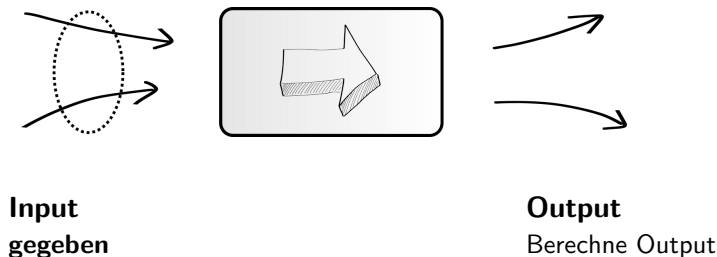
Approximativer Algorithmus:

Anwendung des expliziten Eulerschen Polygonzugverfahrens mit der Gleichung in der Vorschrift.

Hinweis: Dadurch erzeugen wir eine Dynamik!

Simulation: Mit mathematischen Modellen arbeiten: Vorwärtssimulation

Die **Vorwärtssimulation** ist die klassische Verwendungsart von mathematische Modellen:

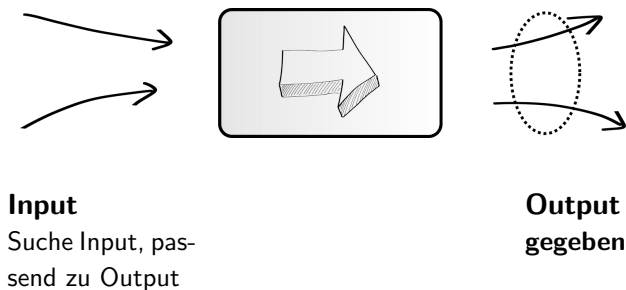


Beispiel 1.10 (Vorwärtssimulationen)

- physikalische Systeme mit bekannten Startbedingungen,
- Epidemiologische Entwicklung von Krankheiten,
- Simulation zur Auslegung von Produkten, ...

Simulation: Mit mathematischen Modellen arbeiten: Inverse Probleme

Inverse Probleme entstehen dann, wenn eine Vorwärtssimulation besonders einfach zu realisieren ist die *Rückrichtung jedoch nicht*. Dabei ist der *Output* schon gegeben und es wird ein passender *Input* gesucht:



Beispiel 1.11 (Inverse Probleme)

- Medizinische Bildgebung (Computer–Tomographie, Radon–Transformation),
- Zustandsschätzung von Systemen, welche beobachtet werden durch Messungen,
- Positionsbestimmungen, statistische Schätzungen,...

Simulation: Mit mathematischen Modellen arbeiten: Inverse Probleme

Praktisch werden inverse Systeme häufig als *iterative Optimierungsprobleme* gelöst, um den gegebenen Output $\vec{z}_{\text{target}}$ zu erreichen.

Dabei folgt man im Prinzip stets folgendem Algorithmus:

Algorithmus 1.1 (Allgemeines Inverses Problem)

1. Anfangsschätzung $n = 0$ für den Input $\vec{x}_0$.
(je nach Anwendung kann das sehr aufwändig sein!)
2. Berechnung des zugehörigen Outputs $\vec{z}_n$ mit Hilfe des Vorwärtsmodells.
3. Berechnung eines Maßes, welches den Abstand von $\vec{z}_n$ und $\vec{z}_{\text{target}}$ bemisst. Das ist die sog. Zielfunktion.
4. Mit einem Optimierer (bspw. Newton–Verfahren, Downhill–Simplex–Algorithmus, etc.) eine bessere Schätzung für $\vec{x}_{n+1}$ finden.
5. Iterative Wiederholung der Schritte 2–4 mit jeweils neuem $\vec{x}_{n+1}$ bis ein Optimierungsergebnis zufriedenstellend erreicht wurde.

Bspw. $\vec{z}_n$ ist ausreichend nahe an $\vec{z}_{\text{target}}$ dran, oder $\vec{x}_n$ bzw. $\vec{z}_n$ verändern sich von Iteration zu Iteration nur mehr sehr wenig.

Bemerkung 1.1 (zu Algorithmus 1.1)

Folgt man dem Algorithmus 1.1, hat man per Definition ein Problem mit innerer Dynamik konstruiert, da iterativ der momentan errechnete Output über die Zielfunktion und den Optimierer den darauffolgenden Input der Vorwärtssimulation definiert.

Bemerkung 1.2 (Inverse Probleme und Ursache/Wirkung)

Häufig entstehen inverse Probleme aus der Situation, dass man eine messbare Wirkung beobachtet und auf die dazugehörige Ursache schließen will. In den meisten Anwendungen ist es sehr leicht von der Ursache auf die Wirkung zu schließen, jedoch ist die Umkehrung dann unmöglich, wenn bspw. mehrere Ursachen eine ähnliche Wirkung erzeugen können.

Simulation: Mit mathematischen Modellen arbeiten: Schlecht gestellte Probleme und Regularisierung

Da das Lösen von inversen Problemen mit mathematischen Modellen und Simulationen in der Praxis (bspw. bei Mess-/Sensorik-systemen) häufig auftritt, diese jedoch meist leider nur schwer oder auch gar nicht lösbar sind, hat man sich in der Literatur auf den Begriff **gut gestellte** (well-posed) und **schlecht gestellte** (ill-posed) Probleme geeinigt.

Definition 1.4 (Gut gestelltes Problem / well-posed problem)

Man spricht von einem **gut gestellten** Problem, wenn folgende Garantien gelten: Wenn

1. für das Problem eine Lösung existiert,
2. diese Lösung eindeutig ist, und
3. diese Lösung stetig von den gegebenen Daten abhängt.

Der letzte Punkt bedeutet insbesondere, dass kleine Änderung der Rahmenbedingungen zu kleinen Änderungen in der Lösung führen, was eine wünschenswerte Eigenschaft ist.

Alle Probleme, bei denen mindestens eine Bedingung nicht erfüllt ist, nennt man **schlecht gestellt**. Viele Probleme in der Praxis sind schlecht gestellt oder man weiß nicht, ob sie gut oder schlecht gestellt sind.

Simulation: Mit mathematischen Modellen arbeiten: Schlecht gestellte Probleme und Regularisierung

Bei schlecht gestellten inversen Problemen verwendet man häufig einen Trick, um sie „besser zu stellen“: **Die Regularisierung**.

Wir verwenden hier den Begriff sehr allgemein und beschreiben damit, dass man bspw. durch

- Vereinfachung von Problemen,
- Einbringung von (un-/gerechtfertigten) zusätzlichen Bedingungen/Annahmen/Vorwissen,
- Lösen von alternativen/ähnlichen Ersatz-Problemen, ...

die Lösbarkeit verbessert oder eine Qualität der Lösung erzwingt, welche besonders vorteilhaft erscheint.

Beispiel 1.12 (Schlecht gestellte Probleme)

- Habe ein unscharfes/defokussiertes Bild mit einem Objektiv aufgenommen – wie sieht das scharfe „Originalbild“ aus?
- Habe ein starkes Rauschen in einem Messsignal – wie sieht das „Originalsignal“ aus?
- Habe eine Liste von sehr erfolgreichen und weniger erfolgreichen Menschen – welche Handlungsweisen entscheiden ob man erfolgreich wird?
- Ein Staubsaugerroboter hat innerhalb von 2 Stunden einen Raum gereinigt – wie sah der Weg genau aus, den der Roboter zurückgelegt hat?
- Mit einem EEG werden Spannungsschwankungen an der Kopfoberfläche gemessen – in welchem Areal des Gehirns ist viel Aktivität gewesen?
- Der Börsenkurs hat heute eine sehr ungewöhnliche Wendung genommen, obwohl keine wesentlichen Ankündigungen von Unternehmenszahlen oder Behörden waren – was ist die Ursache?
- Sie suchen 3 aufeinander senkrechte Geraden in der Ebene!

Simulation: Mit mathematischen Modellen arbeiten: Verifikation und Validierung

Unter **Verifikation** verstehen wir die Prüfung, ob die Berechnungen während der Simulation bei einem gegebenen mathematischen Modell (insbesondere bei direkten und approximativen Algorithmen) korrekt durchgeführt wurden.

Für diese Fragestellung ist stets eine detaillierte Überprüfung des Codes – meist auch im 4–Augen–Prinzip – notwendig.

Beispiel 1.13 (Verifikation)

- Werden die modellierten Gleichungen gut genug gelöst? Probe, wenn möglich!
- Berechnet die Zielfunktion bei inversen Problemen tatsächlich den Abstand zu $\vec{z}_{\text{target}}$? Bleibt ein ungeschickt eingestellter Optimierer frühzeitig in lokalen Minima stecken?
- Passen die Datentypen / -formate im gesamten Code oder verursacht fehlende Datenkonvertierung versteckte (Rundungs-)Fehler?

Simulation: Mit mathematischen Modellen arbeiten: Verifikation und Validierung

Im Gegensatz dazu verstehen wir unter **Validierung** die Prüfung, ob die Simulation mit dem mathematischen Modell den Ausschnitt der realen Welt tatsächlich gut genug beschreibt. Insbesondere bieten sich hier folgende Möglichkeiten zur Validierung an (wenn möglich):

- Vorhersagen vergleichen mit realen Experimenten (evtl. auf kleiner Skala)
- Retrospektive Betrachtung von schon real gemessenen Daten
- Plausibilitätstests (Widerspruchsfreiheit) von Vorhersagen der Simulation zu schon etablierten Theorien / anderen Modellen

Beispiel 1.14 (Validierung)

- Messungen im Straßenverkehr nach simulierter Veränderung des Verkehrsflusses.
- Umfangreiche Wiederholungsexperimente im Labor mit geeichten Referenzmaßen, bspw. bei der Entwicklung von Messmaschinen.
- Extreme Vorhersagen, welche leicht theoretisch überprüft werden können oder schon als Referenzrechnungen in wissenschaftlichen Journalen veröffentlicht wurden.

Simulation: Mit mathematischen Modellen arbeiten: Visualisierung und Kommunikation

Eine umfangreiche (und häufig oft auch arbeitsintensive) Aufgabe ist es durch Simulationen erzeugte umfangreiche Datensätze zu **visualisieren**. Diese können bspw. entstehen bei

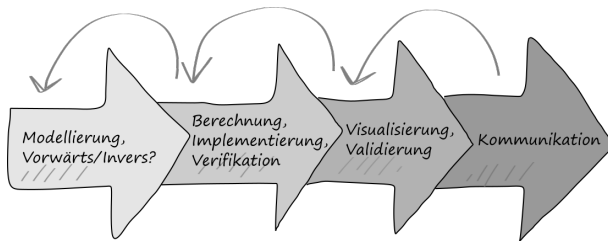
- Simulationen mit hohen Dimensionen im Output, oder
- bei sehr großen Wiederholungszyklen von Simulationen, bspw. um mit dem Output eine statistische Auswertung fahren zu können.

Die Visualisierung dient dabei zwei Zwecken:

- An die simulierende Person: Kann man recht schnell die erzeugten Simulationsergebnisse überblicken, um damit Verifikation und Validierung besser durchführen zu können?
- An die aufgabenstellende Person: Überzeugende und einfach einleuchtende Darstellungen erhöhen die Akzeptanz für die Modellierungs- und Simulationsarbeit. Dies ist unbedingter Teil einer ausgeglichenen **Kommunikationsstrategie** in Projekten.

Simulation: Mit mathematischen Modellen arbeiten: Die Simulationspipeline

Um die zuvor beschriebenen Aspekte der Simulationsaufgabe in ein Übersichtsbild zu stellen, ist es nützlich eine grobe Arbeitsreihenfolge in einer sog. **Simulationspipeline** darzustellen (ähnlich zu Bungartz et al. (2013)):



- Dies ist die Darstellung eines idealen Ablaufs. Im Lauf einer realen Simulationsaufgabe springt man stets zwischen den einzelnen Stationen.
- Insbesondere in der Station **Implementierung** der Berechnung als Programmcode/Software wird in größeren Projekten eine sehr große Aufmerksamkeit geschenkt.
- Eine effiziente **Kommunikation** nach „außen“ wird häufig unterschätzt und gehört als wesentlicher Punkt zur Aufgabe der Simulation.

Zusammenfassung der allgemeinen Begriffe:

- Die Modellbildung beschäftigt sich mit vereinfachenden Darstellungen von Ausschnitten der Welt.
- Der Vorgang der Modellbildung hin zu mathematischen Modellen ist sehr flexibel und hängt maßgeblich von der Anwendung und den Fähigkeiten der modellierenden Person ab.
- Anhand von Kriterien kann eine grobe Einordnung von mathematischen Modellen durchgeführt werden. Diese Kriterien helfen bspw. bei der Auswahl der richtigen Hilfsmittel/Herangehensweise an die Modellbildung.
- Simulationen sind „Computer-Experimente“ mit diesen mathematischen Modellen. Insbesondere spielt die exakte oder approximative algorithmische Berechenbarkeit der Modelle hier eine wichtige Rolle.
- Dabei kann man grundsätzlich zwischen klassischen Vorwärtssimulationen und inversen Problemen unterscheiden. Letztere folgen meist einem grundsätzlichen Schema, sind meist schwerer zu lösen und man versucht sie häufig durch Regularisierung „besser zu stellen“.

Zusammenfassung der allgemeinen Begriffe:

- Verifikation beantwortet die Frage, ob richtig gerechnet wird, und Validierung, ob man das richtige modelliert hat.
- Visualisierung von Simulationsergebnissen ist sowohl für die simulierende, als auch für die aufgabenstellende Person eine praktische Notwendigkeit.
- All diese Aufgaben können in einer Simulationspipeline zusammengefasst werden.
- Die vorgestellten Begriffe werden in der Praxis und Literatur meist nicht eindeutig verwendet, weshalb sich hier auch etwas Nachsicht in der Formulierung empfiehlt.

Beispiel 1.15 (Modellierung einer Simulationspipeline)

Modellieren Sie eine mögliche Simulationspipeline für folgende Aufgabe:

- Es sind 5 Kugeln mit Durchmesser 2 und
- 5 Würfel mit Kantenlänge 1
- in einer quaderförmigen Schachtel zu stapeln.
- Wie sieht die Schachtel mit diesem Inhalt mit dem kleinsten Volumen aus?

2. Modell vs. Daten

Einordnung des Kapitels

Einordnung des Kapitels

In dem Abschnitt *Modell vs. Daten* der Vorlesung beschäftigen wir uns insbesondere mit dem Einpassen von gegebenen Modellfamilien in gegebene Daten. Wir sind also stark durch die Daten getrieben und wollen aus einer gegebenen Modellfamilie das zu den Daten passendste Modell herausfinden. Das ist eine sehr universelle Aufgabe und es gibt unterschiedliche Bezeichnungen in unterschiedlichen Bereichen für diese Aufgabe.

- In der *Statistik* nennt man dieses Vorgehen klassischerweise *Regression*.
- Im Bereich des *Machine Learning* versteht man darunter das *Training* von Modellen.
- Wir bezeichnen es in diesem Kontext dem übergeordneten Begriff als *Model-Fitting*.

Es gehört damit grundsätzlich in den Bereich **Data Science**.

2. Modell vs. Daten

Spezielle Klassifikation von mathematischen Modellen

Spezielle Klassifikation von mathematischen Modellen: Lineares Modell

Wir erinnern uns: Wir sagen, eine Funktion $f : D \mapsto W$ ist linear, wenn für alle $x, y \in D$ und alle $c \in \mathbb{R}$ gilt:

1. $f(x + y) = f(x) + f(y)$ (*Additivität*)
2. $f(c \cdot x) = c \cdot f(x)$ (*Homogenität*)

Haben wir nun ein Modell $\vec{M}$, dann hat dies im Verständnis dieses Abschnitts zwei Eingaben, einmal die Daten $\vec{x} \in D$ und einmal (im allgemeinen) m Parameter $\vec{\alpha} \in \mathbb{R}^m$, welche die Flexibilität der Modellfamilie festlegen:

$$\vec{M}(\vec{x}; \vec{\alpha})$$

Definition 2.1 (Lineares Modell)

Ein Modell $\vec{M}(\vec{x}; \vec{\alpha})$, welches von den Parametern $\vec{\alpha} \in \mathbb{R}^m$ abhängt und grundsätzlich aus dem Input $\vec{x} \in D$ den Output $\vec{z} \in W$ berechnet, ist **linear**, wenn für alle $\vec{x} \in D$, für alle $\vec{\alpha}, \vec{\beta} \in \mathbb{R}^m$ und $c \in \mathbb{R}$ gilt

1. $\vec{M}(\vec{x}; \vec{\alpha} + \vec{\beta}) = \vec{M}(\vec{x}; \vec{\alpha}) + \vec{M}(\vec{x}; \vec{\beta})$
2. $\vec{M}(\vec{x}; c \cdot \vec{\alpha}) = c \cdot \vec{M}(\vec{x}; \vec{\alpha})$.

Spezielle Klassifikation von mathematischen Modellen: Lineares Modell

Eine wichtige Erkenntnis ist demnach: Die Linearität bezieht sich **nur** auf die Parameter!

Beispiel 2.1 (1D-Polynom als lineares Modell)

Ein in der Angewandten Mathematik weit verbreitetes Modell ist beispielsweise ein Polynom mit unbekannten Koeffizienten $\vec{\alpha} \in \mathbb{R}^{m+1}$ als Parameter:

$$p(x; \vec{\alpha}) := \alpha_0 + \alpha_1 x + \alpha_2 x^2 + \alpha_3 x^3 + \cdots + \alpha_m x^m ,$$

welches aus einem Wert $x \in \mathbb{R}$ einen Wert $z = p(x; \vec{\alpha}) \in \mathbb{R}$ berechnet. Offensichtlich ist dieses Modell *p nicht* linear in x , da allein schon für den Ausdruck im Allgemeinen $(x + y)^2 \neq x^2 + y^2$ gilt. Dennoch ist das Modell *linear*, denn:

$$\begin{aligned} p(x; \vec{\alpha} + \vec{\beta}) &= (\alpha_0 + \beta_0) + (\alpha_1 + \beta_1) x + (\alpha_2 + \beta_2) x^2 + \cdots + (\alpha_m + \beta_m) x^m \\ &= (\alpha_0 + \alpha_1 x + \alpha_2 x^2 + \cdots + \alpha_m x^m) + (\beta_0 + \beta_1 x + \beta_2 x^2 + \cdots + \beta_m x^m) \\ &= p(x; \vec{\alpha}) + p(x; \vec{\beta}) \end{aligned}$$

Spezielle Klassifikation von mathematischen Modellen: Lineares Modell

und

$$\begin{aligned} p(x; c \cdot \vec{\alpha}) &= c \alpha_0 + c \alpha_1 x + c \alpha_2 x^2 + \cdots + c \alpha_m x^m \\ &= c \cdot (\alpha_0 + \alpha_1 x + \alpha_2 x^2 + \cdots + \alpha_m x^m) \\ &= c \cdot p(x; \vec{\alpha}) . \end{aligned}$$

In der Praxis wird etwas „locker“ mit diesen Begriffen gearbeitet, weshalb man bei einem „linearen“ Polynom manchmal auch „nur“ $p(x, \vec{\alpha}) := \alpha_0 + \alpha_1 x$ versteht, und meint, dass es linear in x ist. Das ist jedoch genau genommen falsch und sogar irreführend. Wie in der Praxis üblich: *Fragen Sie im Zweifelsfall nach, was gemeint ist.*

Beispiel 2.2 (2D–Polynom als lineares Modell)

Natürlich gilt diese Beschreibung eines linearen Modells auch für höherdimensionale Polynome, d.h. für $\vec{\alpha} \in \mathbb{R}^m$ und $x, y \in \mathbb{R}$ (ein zweidimensionaler Input) ist auch

$$p(x, y; \vec{\alpha}) = \alpha_0 + \alpha_1 x + \alpha_2 y + \alpha_3 x^2 + \alpha_4 x y + \alpha_5 y^2 \dots$$

ein lineares Modell (vollziehen Sie die nötige Rechnung auch noch einmal nach!). D.h. eine Ebene oder ein Paraboloid kann ebenso als ein lineares Modell betrachtet werden.

Beispiel 2.3 (Polynom beliebiger Dimension als lineares Modell)

Dieses Spiel kann man natürlich beliebig erweitern auf höhere Dimensionen der Eingabedaten, wobei man dann auch die Eingabewerte besser in einem Vektor $\vec{x} \in \mathbb{R}^n$ zusammenfasst. D.h. mit $\vec{\alpha} \in \mathbb{R}^m$ könnte man ein solches Modell auch mit $p(\vec{x}; \vec{\alpha})$ abkürzen.

Bemerkenswert ist hier an diesem Beispiel, dass der Output $z = p(\vec{x}; \vec{\alpha}) \in \mathbb{R}$ immer eindimensional bleibt.

Beispiel 2.4 (Polynome mit höherdimensionalem Output als lineares Modell)

Natürlich kann man aus den bisher beschriebenen Polynomen auch einen Vektor von Polynomen generieren, was dann einem höherdimensionalen Output entspricht.

Beispielsweise ist

$$\vec{z} = \vec{p}(x, y; \vec{\alpha}) = \begin{bmatrix} \alpha_0 + \alpha_1 x + \alpha_2 x^2 + \alpha_3 x^3 \\ \alpha_4 + \alpha_5 y + \alpha_6 y^2 \\ \alpha_7 x^2 + \alpha_8 x y + \alpha_9 y^2 \end{bmatrix}$$

auch ein lineares Modell in dem Parametervektor $\vec{\alpha} \in \mathbb{R}^{10}$, dem zweidimensionalen Input $(x, y)^T \in \mathbb{R}^2$ und einem dreidimensionalen Output $\vec{z} \in \mathbb{R}^3$. Vollziehen Sie die nötige Rechnung für die Linearität auch noch einmal nach $\rightarrow$ Vektoreigenschaften.

Offensichtlich kann man mit Vektoren mit elementweisen Polynomen zwischen beliebigen Dimensionen in Input $\vec{x} \in D = \mathbb{R}^n$ und Output $\vec{z} \in W = \mathbb{R}^k$ vermitteln. Wie viele Parameter $\vec{\alpha} \in \mathbb{R}^m$ innerhalb dieses Modells verwendet werden, ist frei wählbar (je nach Flexibilität der elementweisen Polynome). Ganz allgemein kann man deshalb ein *lineares* polynomiales Modell schreiben als $\vec{p}(\vec{x}; \vec{\alpha})$.

Spezielle Klassifikation von mathematischen Modellen: Lineares Modell

Von dieser großen linearen Polynom-Familie ausgehend, wollen wir die Gestalt von allgemeinen linearen Modellen konstruktiv angeben mit:

Satz 2.1 (Lineares Modell)

Jedes lineare Modell $\vec{M}(\vec{x}; \vec{\alpha})$ mit $\vec{M} \in \mathbb{R}^k$, $\vec{x} \in \mathbb{R}^n$ und $\vec{\alpha} \in \mathbb{R}^m$ lässt sich schreiben als

$$\vec{M}(\vec{x}; \vec{\alpha}) := \sum_{i=1}^m \alpha_i \cdot \vec{f}_i(\vec{x})$$

mit m Modellfunktionen $\vec{f}_i(\vec{x})$.

Beweis: Die Richtung, dass diese beschriebene Darstellung ein lineares Modell ist, lässt sich einfach nachrechnen. Die umgekehrte Richtung kann man erkennen, indem man $\vec{f}_i(\vec{x}) := \vec{M}(\vec{x}; \vec{e}_i)$ mit dem i -ten Einheitsvektor (überall Eintrag 0, nur an Koordinate i eine 1) identifiziert und dann nur mit den Linearitätseigenschaften rechnet:

$$\vec{M}(\vec{x}; \vec{\alpha}) = \vec{M}\left(\vec{x}; \sum_{i=1}^m \alpha_i \cdot \vec{e}_i\right) = \sum_{i=1}^m \vec{M}(\vec{x}; \alpha_i \cdot \vec{e}_i) = \sum_{i=1}^m \alpha_i \cdot \vec{M}(\vec{x}; \vec{e}_i) .$$

Spezielle Klassifikation von mathematischen Modellen: Lineares Modell

Eine Darstellung, welche diese Linearität in $\vec{\alpha}$ schließlich komplett verdeutlicht ist gegeben durch die weitere Zusammenfassung

$$\vec{M}(\vec{x}; \vec{\alpha}) = \sum_{i=1}^m \alpha_i \cdot \vec{f}_i(\vec{x}) = \begin{bmatrix} \vec{f}_1(\vec{x}) & \dots & \vec{f}_m(\vec{x}) \end{bmatrix} \cdot \vec{\alpha} = A(\vec{x}) \cdot \vec{\alpha},$$

was direkt eine Matrix–Vektor–Multiplikation ist und damit die Brücke zur *linearen Algebra* schlägt.

Bemerkung 2.1 (Lineare Modelle)

Es gibt zwei wesentliche Gründe, warum es hilfreich und nützlich ist mit linearen Modellen zu arbeiten:

- Für gegebene $\vec{x}$ kann man die oben beschriebene Matrix $A(\vec{x})$ aufstellen und für diese die Reichhaltigkeit der Theorie der linearen Algebra verwenden, bspw. die Eigenwerte und –vektoren bestimmen, um die *innere Dynamik* bzgl. $\vec{\alpha}$ dieses linearen Modells besser zu verstehen. Dies ist somit für alle linearen Modelle möglich.
- Mit linearen Modellen kann man auch recht leicht „rechnen“ und damit bspw. Optimierungsaufgaben deutlich einfacher lösen (wie wir später sehen werden).

Beispiel 2.5 (Polynome mit höherdimensionalem Output als lineares Modell)

Betrachten wir erneut

$$\vec{z} = \vec{p}(x, y; \vec{\alpha}) = \begin{bmatrix} \alpha_0 + \alpha_1 x + \alpha_2 x^2 + \alpha_3 x^3 \\ \alpha_4 + \alpha_5 y + \alpha_6 y^2 \\ \alpha_7 x^2 + \alpha_8 x y + \alpha_9 y^2 \end{bmatrix}.$$

Wie sieht dies nun in dieser allgemeinen Darstellung aus?

$$\begin{aligned} \vec{p}(x, y; \vec{\alpha}) &= \alpha_0 \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} + \alpha_1 \begin{bmatrix} x \\ 0 \\ 0 \end{bmatrix} + \alpha_2 \begin{bmatrix} x^2 \\ 0 \\ 0 \end{bmatrix} + \alpha_3 \begin{bmatrix} x^3 \\ 0 \\ 0 \end{bmatrix} + \alpha_4 \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} + \alpha_5 \begin{bmatrix} 0 \\ y \\ 0 \end{bmatrix} + \cdots \\ &\quad \cdots + \alpha_6 \begin{bmatrix} 0 \\ y^2 \\ 0 \end{bmatrix} + \alpha_7 \begin{bmatrix} 0 \\ 0 \\ x^2 \end{bmatrix} + \alpha_8 \begin{bmatrix} 0 \\ 0 \\ x y \end{bmatrix} + \alpha_9 \begin{bmatrix} 0 \\ 0 \\ y^2 \end{bmatrix} \\ &= \begin{bmatrix} 1 & x & x^2 & x^3 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & y & y^2 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & x^2 & x y & y^2 \end{bmatrix} \cdot \vec{\alpha} \end{aligned}$$

Spezielle Klassifikation von mathematischen Modellen: Lineares Modell

Natürlich ist es nun leicht sich beliebige lineare Modelle (auch jenseits der Polynome) vorzustellen:

Beispiel 2.6 (Beispiel eines linearen Modells)

$$\vec{z} = \vec{M}(x, y; \vec{\alpha}) = \begin{bmatrix} \alpha_0 + \alpha_1 x + \alpha_2 e^x + \alpha_3 \sin(x) \\ \alpha_0 x^2 + \alpha_4 + \alpha_5 \log(y) + \alpha_6 y^2 \\ \alpha_7 x^2 + \alpha_8 \sinh(xy) + \alpha_9 y^2 \end{bmatrix}.$$

Wie sieht die zugehörige Matrix–Vektor–Darstellung aus?

Es zeichnet sich ab, dass (in den Parametern) lineare Modelle sehr vielgestaltig sein können und man damit schon viele Gegebenheiten als lineare Modelle darstellen kann.

Definition 2.2 (Nichtlineares Modell)

Ein Modell $\vec{M}(\vec{x}; \vec{\alpha})$, welches von den Parametern $\vec{\alpha} \in \mathbb{R}^n$ abhängt und grundsätzlich aus dem Input $\vec{x} \in D$ den Output $\vec{z} \in W$ berechnet, nennen wir i.R.d.V. **echt nichtlinear**, wenn die Eigenschaft der Linearität nicht erfüllt ist.

In anderen Worten: Wenn es nicht möglich ist, $\vec{M}(\vec{x}; \vec{\alpha}) = A(\vec{x}) \cdot \vec{\alpha}$ aufzustellen.

Bemerkung 2.2 (Nichtlineares Modell)

Hier ist wieder bei der Verwendung der Begriffe Vorsicht angebracht: Unter *nichtlinearen Modellen* versteht man in der Angewandten Mathematik meist die gemeinsame Menge der linearen und echt nichtlinearen Modelle (sozusagen markierend, dass Linearität nicht garantiert ist). Wir verwenden deshalb den künstlichen Begriff „echt nichtlinear“, wenn entweder Additivität oder Homogenität (oder beides) nicht erfüllt ist.

Beispiel 2.7 (Echt nichtlineares Modell)

Wir beginnen mit einem einfachen Beispiel:

$$M(\vec{x}; \vec{\alpha}) = x_1 \cdot \alpha_1 + x_2 \cdot \alpha_2 + x_1 \cdot$$

Ist dieses Modell echt nichtlinear?

Beispiel 2.8 (Echt nichtlineares Modell)

Weiter betrachten wir folgendes einfaches Beispiel:

$$M(\vec{x}; \vec{\alpha}) = x_1 \cdot x_2 \cdot \alpha_1 + \alpha_2 .$$

Ist dieses Modell echt nichtlinear?

Beispiel 2.9 (Allgemeines affines Modell)

Ein Beispiel für ein echt nichtlineares Modell in $\vec{\alpha}$ ist ein *affines* Modell, welches die Gestalt hat $\vec{b}(\vec{x}) \neq \vec{0}$:

$$\vec{M}(\vec{x}; \vec{\alpha}) = A(\vec{x}) \cdot \vec{\alpha} + \vec{b}(\vec{x}) ,$$

d.h. allein durch das Hinzuaddieren eines Terms unabhängig von $\vec{\alpha}$ wird das Modell sofort echt nichtlinear. Wie weist man das nach?

Beispiel 2.10 (Allgemeines quadratisches Modell)

Ein weiteres Beispiel für ein echt nichtlineares Modell in $\vec{\alpha}$ ist, beispielsweise das *quadratische* Modell

$$M(\vec{x}; \vec{\alpha}) = \vec{\alpha}^T \cdot A(\vec{x}) \cdot \vec{\alpha} + \vec{b}(\vec{x})^T \cdot \vec{\alpha} + d(\vec{x}) .$$

Wie weist man nach, dass dies ein echt nichtlineares Modell ist? Diskutieren Sie dabei die Bedeutung von $A(\vec{x})$, $\vec{b}(\vec{x})$ und $d(\vec{x})$.

Beispiel 2.11 (Allgemeines exponentielles Modell)

Ein weiteres Beispiel für ein echt nichtlineares Modell ist das *exponentielle* (komponentenweise Anwendung!) Modell

$$\vec{M}(\vec{x}; \vec{\alpha}) = e^{A(\vec{x}) \cdot \vec{\alpha} + \vec{b}(\vec{x})} = e^{A(\vec{x}) \cdot \vec{\alpha}} \cdot e^{\vec{b}(\vec{x})} .$$

Wie weist man das nach?

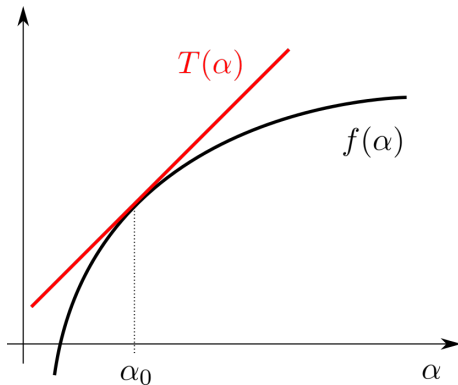
Bemerkung 2.3 (Nichtlineares Modell)

Im Gegensatz zur Arbeit mit linearen Modellen (mit den Rückgriffen auf die Lineare Algebra, wie wir das schon gesehen hatten), gibt es keine allgemeine Theorie für echt nichtlinearen Modelle. Das kommt daher, dass echt nichtlineare Modelle sehr überraschende und unintuitive Verhaltensweisen haben können. Praktisch bedeutet dies, entweder

- man versucht sehr häufig Modelle zu *linearisieren*, d.h. durch lineare Modelle bestmöglich anzunähern soweit es sinnvoll möglich ist (siehe nächsten Abschnitt), oder
- man fokussiert sich in der Praxis auf eine spezielle Familie von echt nichtlinearen Modellen (quadratische, exponentielle Modelle, etc.) und untersucht spezielle Eigenschaften dieser Modelle im Detail.

Spezielle Klassifikation von mathematischen Modellen: Lokale Linearisierung eines Modells

Eine Möglichkeit ein echt nichtlineares Modell zu *linearisieren* besteht in einer Erkenntnis aus der Analysis: Die Tangente $T(\alpha)$ an einer *differenzierbaren* Funktion $f(\alpha)$ ist in der Umgebung des Berührungspunkts α_0 eine gute Annäherung an f .



Spezielle Klassifikation von mathematischen Modellen: Lokale Linearisierung eines Modells

Aus der Analysis–Vorlesung wissen wir, dass sich diese Tangente am Berührungspunkt α_0 beschreiben lässt durch (Taylorpolynom 1. Grades)

$$T(\alpha) = f(\alpha_0) + f'(\alpha_0) \cdot (\alpha - \alpha_0) ,$$

d.h. die Steigung der Tangente ist die Ableitung der Funktion f an der Stelle α_0 . Schreiben wir das etwas um in die typische Geradenform, dann sieht das so aus:

$$T(\alpha) = f'(\alpha_0) \cdot \alpha + (f(\alpha_0) - f'(\alpha_0) \cdot \alpha_0) ,$$

d.h. wir haben ein *affines* Modell mit Hilfe des Taylorpolynoms 1. Grades erstellt. Man betrachtet den *konstanten* Teil dieses affinen Modells aber meist als nicht sehr störend, da es sich hierbei nur um eine *vertikale Verschiebung* der Tangente handelt. Ein lineares Modell wird daraus, wenn man diese konstante Verschiebung einfach auf die linke Seite setzt und sich diese Verschiebung merkt:

$$T(\alpha) - (f(\alpha_0) - f'(\alpha_0) \cdot \alpha_0) = f'(\alpha_0) \cdot \alpha .$$

Spezielle Klassifikation von mathematischen Modellen: Lokale Linearisierung eines Modells

Als erste Erweiterung halten wir fest, dass diese lineare Annäherung natürlich auch bei höherdimensionalen Funktionen $f(\vec{\alpha})$ koordinatenweise funktioniert:

$$T(\vec{\alpha}) = f(\vec{\alpha}_0) + \frac{\partial}{\partial \alpha_1} f(\vec{\alpha}_0) \cdot (\alpha_1 - \alpha_{0,1}) + \frac{\partial}{\partial \alpha_2} f(\vec{\alpha}_0) \cdot (\alpha_2 - \alpha_{0,2}) + \dots ,$$

bei der die einfache Ableitung f' mit der partiellen (koordinatenweisen) Ableitung $\frac{\partial}{\partial \alpha_i} f$ ersetzt wird.

Wichtig in dieser Erweiterung: die Funktion f bildet von $\vec{\alpha} \in \mathbb{R}^m$ immer noch einfach nach $\mathbb{R}$ ab.

Die obige Formel kann man auch zusammenfassen mit dem „Nabla“-Operator ∇ (eine Kurzschreibweise):

$$T(\vec{\alpha}) = f(\vec{\alpha}_0) + (\nabla f(\vec{\alpha}_0))^T \cdot (\vec{\alpha} - \vec{\alpha}_0) ,$$

mit dem *Gradienten* $\nabla f(\vec{\alpha})$ bei dem alle partiellen Ableitungen einfach in einen Vektor untereinander geschrieben werden.

Spezielle Klassifikation von mathematischen Modellen: Lokale Linearisierung eines Modells

Als zweite Erweiterung erkennen wir, wenn $\vec{f}(\vec{\alpha})$ eine Funktion ist die $\vec{\alpha} \in \mathbb{R}^m$ nach $\mathbb{R}^k$ abbildet, d.h. $\vec{f}$ ist selbst wieder ein Vektor, dann kann man diese Annäherung natürlich auch für jede einzelne Koordinate j von $\vec{f}$ durchführen:

$$T_j(\vec{\alpha}) = f_j(\vec{\alpha}_0) + (\nabla f_j(\vec{\alpha}_0))^T \cdot (\vec{\alpha} - \vec{\alpha}_0) .$$

Dies wiederum lässt sich nun selbst wieder vektoriell zusammenfassen zu

$$\begin{aligned} \vec{T}(\vec{\alpha}) &= \vec{f}(\vec{\alpha}_0) + \begin{bmatrix} (\nabla f_1(\vec{\alpha}_0))^T \\ (\nabla f_2(\vec{\alpha}_0))^T \\ \vdots \\ (\nabla f_k(\vec{\alpha}_0))^T \end{bmatrix} \cdot (\vec{\alpha} - \vec{\alpha}_0) \\ &= \vec{f}(\vec{\alpha}_0) + \begin{bmatrix} \frac{\partial}{\partial \alpha_1} f_1(\vec{\alpha}_0) & \frac{\partial}{\partial \alpha_2} f_1(\vec{\alpha}_0) & \dots & \frac{\partial}{\partial \alpha_m} f_1(\vec{\alpha}_0) \\ \frac{\partial}{\partial \alpha_1} f_2(\vec{\alpha}_0) & \frac{\partial}{\partial \alpha_2} f_2(\vec{\alpha}_0) & \dots & \frac{\partial}{\partial \alpha_m} f_2(\vec{\alpha}_0) \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial}{\partial \alpha_1} f_k(\vec{\alpha}_0) & \frac{\partial}{\partial \alpha_2} f_k(\vec{\alpha}_0) & \dots & \frac{\partial}{\partial \alpha_m} f_k(\vec{\alpha}_0) \end{bmatrix} \cdot (\vec{\alpha} - \vec{\alpha}_0) = \vec{f}(\vec{\alpha}_0) + J_{\vec{f}}(\vec{\alpha}_0) \cdot (\vec{\alpha} - \vec{\alpha}_0) \end{aligned}$$

mit der *Jacobi-Matrix* $J_{\vec{f}}(\vec{\alpha}_0)$.

Spezielle Klassifikation von mathematischen Modellen: Lokale Linearisierung eines Modells

Das sieht im Einzelfall kompliziert aus und bedarf viel Rechenaufwand, jedoch ermöglicht es uns jede beliebige differenzierbare höherdimensionale Funktion $\vec{f}(\vec{\alpha})$ lokal an einer Stelle $\vec{\alpha}_0$ als affine Funktion (= linear + konstante Verschiebung) mit der Matrix $J_{\vec{f}}(\vec{\alpha}_0)$ anzunähern:

$$\vec{T}(\vec{\alpha}) = J_{\vec{f}}(\vec{\alpha}_0) \cdot \vec{\alpha} + \underbrace{\left(\vec{f}(\vec{\alpha}_0) - J_{\vec{f}}(\vec{\alpha}_0) \cdot \vec{\alpha}_0 \right)}_{\text{Konstante}} .$$

Angewendet auf die Modell-Schreibweise bedeutet dies für eine affine Annäherung $\vec{T}(\vec{x}; \vec{\alpha})$ an das Modell $\vec{M}(\vec{x}; \vec{\alpha})$ an der Stelle $\vec{\alpha}_0$:

$$\vec{T}(\vec{x}; \vec{\alpha}) = J_{\vec{M}}(\vec{x}; \vec{\alpha}_0) \cdot \vec{\alpha} + \underbrace{\left(\vec{M}(\vec{x}; \vec{\alpha}_0) - J_{\vec{M}}(\vec{x}; \vec{\alpha}_0) \cdot \vec{\alpha}_0 \right)}_{\text{Konstante}} .$$

Auch wenn die Annäherung im engeren Sinne durch eine *affine* Funktion ist, spricht man von einer *Linearisierung*, da mit der bzgl. $\vec{\alpha}$ konstanten Verschiebung die qualitative Änderung für die Parameter $\vec{\alpha}$ gleich bleibt.

Bemerkung 2.4 (Linearisierung differenzierbarer Funktionen)

Warum ist es wichtig, diese lokale Linearisierung als *grundlegendes Prinzip* zu verstehen? In der Praxis gilt häufig Folgendes:

1. Auch wenn man viele Phänomene durch lineare Modelle beschreiben kann, treten häufig differenzierbare und echt nichtlineare Modelle als exaktere Beschreibung vieler Phänomene auf.
2. Echt nichtlineare Modelle haben die Eigenschaft, dass sie ein überraschendes Verhalten zeigen und man zuverlässige Vorhersagen nur schwer erreicht. Deshalb muss man solche Modelle häufig simulieren, um zu wissen, welche Ergebnisse sie liefern.
3. Viele Phänomene sind *hochdimensional*, d.h. es gibt bspw. einen sehr großen Parametersatz $\vec{\alpha}$ von denen Modelle abhängen (denken Sie an Neuronale Netze) – mit diesen muss man effizient rechnen!
4. **Dies führt zu:** Ein großer Teil aller Algorithmen, die differenzierbare, echt nichtlineare Modelle simulieren oder mit diesen optimieren, beruhen auf dieser beschriebenen Linearisierung. Dabei wird bspw. häufig der *Gradient* oder die *Jacobi-Matrix* praktisch numerisch angenähert.

Beispiel 2.12 (Linearisierung differenzierbarer Funktionen)

Finden Sie die affine Annäherung an das Modell $\vec{M}$ an der Stelle $\vec{\alpha}_0 = [1 \ 1]^T$

$$\vec{M}(\vec{x}; \vec{\alpha}) = \begin{bmatrix} \alpha_1 \cdot x_1 + \alpha_2 \cdot x_2 \\ \alpha_1^2 \cdot x_1^2 + 2 \alpha_1 \alpha_2 \cdot x_1 x_2 + \alpha_2^2 \cdot x_2^2 \\ \alpha_1 \cdot \alpha_2^3 + x_1^2 \cdot x_2 \end{bmatrix}.$$

Wir berechnen hierzu zuerst die *Jacobi-Matrix* und setzen dort $\vec{\alpha}_0$ ein:

Spezielle Klassifikation von mathematischen Modellen: Lokale Linearisierung eines Modells

Dann setzen wir in $\vec{M}(\vec{x}; \vec{\alpha})$ den Wert $\vec{\alpha}_0$ ein:

und berechnen nun die affine Funktion $\vec{T}(\vec{x}; \vec{\alpha})$:

Bemerkung 2.5 (Linearisierung eines Modells durch Reihenentwicklung)

Ein alternativer/ergänzender Weg zur Linearisierung eines Modells ist die Reihenentwicklung. Wir wissen ja bereits, dass jedes lineare Modell $\vec{M}(\vec{x}; \vec{\alpha})$ mit $\vec{M} \in \mathbb{R}^k$, $\vec{x} \in \mathbb{R}^n$ und $\vec{\alpha} \in \mathbb{R}^m$ sich schreiben lässt als

$$\vec{M}(\vec{x}; \vec{\alpha}) := \sum_{i=1}^m \alpha_i \cdot \vec{f}_i(\vec{x})$$

mit m Modellfunktionen $\vec{f}_i(\vec{x})$.

Wenn man nun $m \rightarrow \infty$ gehen lässt, kann man damit auch echt nichtlineare Funktionen approximieren, bspw. mit Hilfe der *Taylorreihen*– oder *Fourierreihen*–Entwicklung ($\rightarrow$ *Analysis, Approximationstheorie*), d.h.

$$\vec{M}(\vec{x}; \vec{\alpha}) := \sum_{i=1}^{\infty} \alpha_i \cdot \vec{f}_i(\vec{x}) .$$

2. Modell vs. Daten

Model–Fitting

Model-Fitting

Eine in der Praxis ständig wiederkehrende Aufgabe ist es ein gegebenes Modell $\vec{M}$ mit noch unbekannten Parametern $\vec{\alpha}$ als Freiheitsgrade in konkret gegebene Daten $(\vec{x}, \vec{z})$ bestmöglich „einzupassen“.

Es gibt mehrere Gründe für diese Aufgabe, welche auch häufig gleichzeitig gelten:

- Man kann mit der Menge an *Rohdaten nicht gut arbeiten*, d.h. sie ist aufgrund ihrer Größe oder Dimension unübersichtlich und evtl. auch im direkten Umgang schwer zu handhaben (bspw. wg. Speicher). → Informatik, Signal- und Bildverarbeitung
- Man möchte diese Daten *analysieren*, d.h. bspw. grundlegende Tendenzen und Verläufe in den Daten erkennen oder Störungen der Daten festhalten. → Statistik, Datenanalyse
- Man möchte ein Modell aus den Daten aktiv *synthetisieren*, um dann nur mit diesem Modell weiter zu arbeiten. Hierbei unterscheidet man typischerweise
 - Parameter mit *konkreter, direkter Bedeutung* (→ Natur- und Ingenieurwissenschaften), und
 - Modelle mit *freien Parametern ohne explizite Bedeutung* (→ Data Science mit Neuronalen Netzen, Polynomfitting).

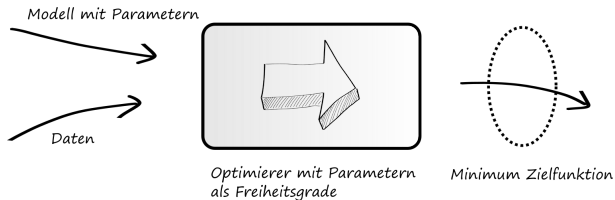
Model-Fitting

Die vier Bestandteile eines „Model-Fittings“ sind dabei grundsätzlich:

- Der gegebene **Datensatz** muss zugänglich und verarbeitbar (d.h. numerisch) sein.
Datenformat? Dimensionalität der Daten? Diskussion mit Datenerzeuger?
- Man benötigt ein **Modell** mit noch unbestimmten **Parametern**
Modellursprung vs. Vorwissen? Dimensionalität passt zu Daten? Anzahl Parameter vs. Anzahl Daten?
- Man benötigt eine **Zielfunktion**, welche den Abstand der gegebenen Daten zur Modellvorhersage bei gegebenem Parametersatz bemisst.
In wie weit berücksichtigt man Störung der Daten (Rauschen, Ausreißer, Messfehler)? Zielfunktion „Abstandsmaß zwischen Daten und Modell“.
- Final ist ein **Optimierablauf/programm** nötig, welche die Parameter als Freiheitsgrade verwendet, um damit die Zielfunktion (typischerweise) zu minimieren.
Explizite Berechnung auf Blatt? Iterative Optimierer? Strategie bzgl. Untermengen von Freiheitsgraden?

Model-Fitting

Wir erkennen, dass es sich hier offensichtlich um ein inverses Problem handelt:



Mit etwas Abstand wird offensichtlich: Der „Model-Fit“, d.h. ein gegebenes Modell in einen Datensatz einzupassen, bedarf selbst einer Modellierung mit Zielfunktion und Optimierer.

→ Model-Fitting benötigt eine Modellierung mit einer Abstraktionsstufe höher als das eigentlich einzupassende Modell.

Für diese Einpassung benötigen wir natürlich erst einmal *Daten*. Wir verstehen Daten dabei konkret als eine Menge von Input-Daten $\vec{x} \in \mathbb{R}^n$ mit zugehörigen Output-Daten $\vec{z} \in \mathbb{R}^k$. Gehen wir von $j = 1, \dots, N$ solcher Datensätze aus, dann schreiben wir das als

$$(\vec{x}_j, \vec{z}_j) \quad \forall j = 1, \dots, N .$$

Im Folgenden gibt es nun zwei klassische Unterscheidungen bzgl. der Modelleinpassung, welche von der Gestalt der Parameter im Modell abhängen, und die im Wesentlichen die Möglichkeiten für Zielfunktion und Optimierer stark beeinflussen.

Model-Fitting: Linearer Model-Fit via Least-Squares

1. Wie man schon erwarten darf, arbeitet man beim *linearen Model-Fit* mit einem *linearen Modell*, wie wir es schon zuvor kennengelernt haben:

$$\vec{z} = \vec{M}(\vec{x}; \vec{\alpha}) = \sum_{i=1}^m \alpha_i \cdot \vec{f}_i(\vec{x}) ,$$

welches Input-Daten $\vec{x} \in \mathbb{R}^n$ in Output-Daten $\vec{M} \in \mathbb{R}^k$ abbildet, und dabei von den Parametern $\vec{\alpha} \in \mathbb{R}^m$ abhängt. Beim *Daten-Fit* geht es darum, diese Parameter $\vec{\alpha}$ zu bestimmen, sodass die Verbindung $\vec{x}_j \rightarrow \vec{z}_j$ für alle $j = 1, \dots, N$ bestmöglich gilt.

2. Hierzu benötigt man eine *Zielfunktion* F , welche den Abstand zwischen Modell und Daten misst. Dieser Abstand ist eine reelle Zahl. Im linearen Model-Fit verwendet man dabei die *Quadratische Zielfunktion* (=Fehlerquadratsumme, „squares“):

$$F = \sum_{j=1}^N || \vec{z}_j - \vec{M}(\vec{x}_j; \vec{\alpha}) ||^2$$

Model-Fitting: Linearer Model-Fit via Least-Squares

Offensichtlich gilt, falls $\vec{M}(\vec{x}_j; \vec{\alpha})$ exakt mit $\vec{z}_j$ für alle $j = 1, \dots, N$ übereinstimmt, dass $F = 0$ ist. Alle Abweichungen führen zu $F > 0$. Daher ist es jetzt schon klar, dass eine *Minimierung* der Zielfunktion angestrebt wird (= „least squares“).

Die Kombination aus *linearem Modell* und *Quadratischer Zielfunktion* kann man nun ineinander einsetzen und man erhält:

$$\begin{aligned} F &= \sum_{j=1}^N \|\vec{z}_j - \vec{M}(\vec{x}_j; \vec{\alpha})\|^2 \\ &= \sum_{j=1}^N \left\| \vec{z}_j - \sum_{i=1}^m \alpha_i \cdot \vec{f}_i(\vec{x}_j) \right\|^2 \\ &= \sum_{j=1}^N \sum_{l=1}^k \left(z_{l,j} - \sum_{i=1}^m \alpha_i \cdot f_{l,i}(\vec{x}_j) \right)^2. \end{aligned}$$

Eine offene Frage: Warum nun genau diese Kombination?

Model-Fitting: Linearer Model-Fit via Least-Squares

Die Antwort hierzu liefert uns die letzte fehlende Komponente: Der *Optimierer*.

3. Wählt man diese Kombination, dann gibt es eine Möglichkeit *einen großen Teil* der Optimierung ganz allgemein *auf dem Blatt* durchzuführen! Aus der mehrdimensionalen Differentialrechnung wissen wir, ein Kandidat für ein Extremum (Erinnerung: wir suchen ja $\vec{\alpha}$ sodass Zielfunktion F minimal) lässt sich berechnen mit

$$\nabla F = \begin{bmatrix} \frac{\partial}{\partial \alpha_1} F \\ \frac{\partial}{\partial \alpha_2} F \\ \vdots \\ \frac{\partial}{\partial \alpha_m} F \end{bmatrix} \stackrel{!}{=} \vec{0},$$

d.h. der *Gradient der Zielfunktion* muss der Nullvektor werden. Das ist das aus der Analysis bekannte „Ableitung Null setzen“ auf jeden dieser Parameter $\vec{\alpha}$ angewendet.

Model-Fitting: Linearer Model-Fit via Least-Squares

Wendet man diese Rechnung auf einen Beispielparameter α_q an, so ergibt sich mit den einfachen Rechenregeln der eindimensionalen Ableitung:

$$\begin{aligned} \frac{\partial}{\partial \alpha_q} \sum_{j=1}^N \sum_{l=1}^k \left(z_{l,j} - \sum_{i=1}^m \alpha_i \cdot f_{l,i}(\vec{x}_j) \right)^2 &\stackrel{!}{=} 0 \\ \sum_{j=1}^N \sum_{l=1}^k 2 \cdot \left(z_{l,j} - \sum_{i=1}^m \alpha_i \cdot f_{l,i}(\vec{x}_j) \right) \cdot (-f_{l,q}(\vec{x}_j)) &\stackrel{!}{=} 0 \\ \sum_{j=1}^N \sum_{l=1}^k z_{l,j} \cdot f_{l,q}(\vec{x}_j) - \sum_{j=1}^N \sum_{l=1}^k \sum_{i=1}^m \alpha_i \cdot f_{l,i}(\vec{x}_j) \cdot f_{l,q}(\vec{x}_j) &\stackrel{!}{=} 0 \\ \sum_{i=1}^m \alpha_i \cdot \underbrace{\left(\sum_{j=1}^N \sum_{l=1}^k f_{l,i}(\vec{x}_j) \cdot f_{l,q}(\vec{x}_j) \right)}_{\text{Konstante}} &\stackrel{!}{=} \underbrace{\sum_{j=1}^N \sum_{l=1}^k z_{l,j} \cdot f_{l,q}(\vec{x}_j)}_{\text{Konstante}}, \end{aligned}$$

womit diese Bedingung bzgl. α_q zu *einer* linearen Gleichung für alle $\vec{\alpha}$ führt.

Model-Fitting: Linearer Model-Fit via Least-Squares

Wir schreiben das etwas ausführlicher auf:

$q = 1$:

$$\alpha_1 \cdot \left(\sum_{j=1}^N \sum_{l=1}^k f_{l,1}(\vec{x}_j) \cdot f_{l,1}(\vec{x}_j) \right) + \alpha_2 \cdot \left(\sum_{j=1}^N \sum_{l=1}^k f_{l,2}(\vec{x}_j) \cdot f_{l,1}(\vec{x}_j) \right) + \dots$$
$$= \sum_{j=1}^N \sum_{l=1}^k z_{l,j} \cdot f_{l,1}(\vec{x}_j)$$

$q = 2$:

$$\alpha_1 \cdot \left(\sum_{j=1}^N \sum_{l=1}^k f_{l,1}(\vec{x}_j) \cdot f_{l,2}(\vec{x}_j) \right) + \alpha_2 \cdot \left(\sum_{j=1}^N \sum_{l=1}^k f_{l,2}(\vec{x}_j) \cdot f_{l,2}(\vec{x}_j) \right) + \dots$$
$$= \sum_{j=1}^N \sum_{l=1}^k z_{l,j} \cdot f_{l,2}(\vec{x}_j)$$

...

Model-Fitting: Linearer Model-Fit via Least-Squares

Was man letztlich mit den Bedingungen für alle α_q ($q = 1, \dots, m$) zu folgendem linearen Gleichungssystem zusammenfassen kann:

$$\begin{bmatrix} \sum_{j=1}^N \sum_{l=1}^k f_{l,1}(\vec{x}_j) \cdot f_{l,1}(\vec{x}_j) & \sum_{j=1}^N \sum_{l=1}^k f_{l,2}(\vec{x}_j) \cdot f_{l,1}(\vec{x}_j) & \dots & \sum_{j=1}^N \sum_{l=1}^k f_{l,m}(\vec{x}_j) \cdot f_{l,1}(\vec{x}_j) \\ \sum_{j=1}^N \sum_{l=1}^k f_{l,1}(\vec{x}_j) \cdot f_{l,2}(\vec{x}_j) & \sum_{j=1}^N \sum_{l=1}^k f_{l,2}(\vec{x}_j) \cdot f_{l,2}(\vec{x}_j) & \dots & \sum_{j=1}^N \sum_{l=1}^k f_{l,m}(\vec{x}_j) \cdot f_{l,2}(\vec{x}_j) \\ \vdots & \vdots & \ddots & \vdots \\ \sum_{j=1}^N \sum_{l=1}^k f_{l,1}(\vec{x}_j) \cdot f_{l,m}(\vec{x}_j) & \sum_{j=1}^N \sum_{l=1}^k f_{l,2}(\vec{x}_j) \cdot f_{l,m}(\vec{x}_j) & \dots & \sum_{j=1}^N \sum_{l=1}^k f_{l,m}(\vec{x}_j) \cdot f_{l,m}(\vec{x}_j) \end{bmatrix} \cdot \vec{\alpha} = \begin{bmatrix} \sum_{j=1}^N \sum_{l=1}^k z_{l,j} \cdot f_{l,1}(\vec{x}_j) \\ \sum_{j=1}^N \sum_{l=1}^k z_{l,j} \cdot f_{l,2}(\vec{x}_j) \\ \vdots \\ \sum_{j=1}^N \sum_{l=1}^k z_{l,j} \cdot f_{l,m}(\vec{x}_j) \end{bmatrix}$$

Diese Erkenntnis ist **äußerst vorteilhaft**:

- D.h. man muss *nur* ein quadratisches $m \times m$ -lineares Gleichungssystem lösen, und hat damit die Lösung *jedes* linearen Model-Fits! Die Eindeutigkeit der Lösung ist gegeben, falls die Funktionen $\vec{f}_i(\vec{x})$ alle linear unabhängig sind. *Warum?*
- Diese Herleitung funktioniert für *alle* linearen Modelle mit beliebiger Anzahl von Parametern $\vec{\alpha}$ ($= m$), Dimensionen von Input- ($= n$) und Output-Daten ($= k$) und Größe der Datensätze ($= N$) (nirgendwo eingeschränkt)!

Model-Fitting: Linearer Model-Fit via Least-Squares

Man kann nun dieses Gleichungssystem noch etwas gründlicher betrachten und erkennen, dass dieses Gleichungssystem auch geschrieben werden kann als

$$A^T \cdot A \cdot \vec{\alpha} = A^T \cdot \vec{b},$$

d.h. dies sind nichts anderes als die klassischen *Normalgleichungen* mit der *Gram-Matrix* $A^T \cdot A$, welche Sie schon aus der *Linearen Algebra* kennen, mit der $(k \cdot N) \times m$ -Matrix A und dem $(k \cdot N) \times 1$ -Vektor $\vec{b}$

$$A = \begin{bmatrix} \vec{f}_1(\vec{x}_1) & \vec{f}_2(\vec{x}_1) & \dots & \vec{f}_m(\vec{x}_1) \\ \vec{f}_1(\vec{x}_2) & \vec{f}_2(\vec{x}_2) & \dots & \vec{f}_m(\vec{x}_2) \\ \vdots & & & \\ \vec{f}_1(\vec{x}_N) & \vec{f}_2(\vec{x}_N) & \dots & \vec{f}_m(\vec{x}_N) \end{bmatrix}, \quad \vec{b} = \begin{bmatrix} \vec{z}_1 \\ \vec{z}_2 \\ \vdots \\ \vec{z}_N \end{bmatrix}.$$

Daher wissen wir auch, dass die große Matrix des Gleichungssystems per Definition *positiv (semi-)definit* ist und sich daher zur Lösung das *Cholesky-Verfahren* anbietet.

Bemerkung 2.6 (Lösung der Normalengleichung)

Eine Alternative (und durchaus in der Praxis gebräuchliche Methode) die Normalengleichungen am Computer zu lösen beruht auf der Singulärwertzerlegung von A , mit der die *Pseudo-Inverse* A^+ berechnet wird. Dies kann Ihnen schon aus der *Linearen Algebra*– oder *Numerik*–Veranstaltung bekannt sein.

Beispiel 2.13 (Geraden-Fit)

Das klassischste Beispiel für *least squares* ist der Fit einer Gerade in (x, z) -Wertepaare. In diesem Fall gilt: $x \in \mathbb{R}$ ($n = 1$), $z \in \mathbb{R}$ ($k = 1$) und N Datenpunkte (x_j, z_j) ($j = 1, \dots, N$). Das zugehörige Modell der Gerade lautet:

$$M(x, \vec{\alpha}) = \alpha_1 x + \alpha_2 .$$

Damit ist $f_1(x) = x$ und $f_2(x) = 1$ und weiterhin:

$$A = \begin{bmatrix} x_1 & 1 \\ x_2 & 1 \\ \vdots & \\ x_N & 1 \end{bmatrix}, \quad \vec{b} = \begin{bmatrix} z_1 \\ z_2 \\ \vdots \\ z_N \end{bmatrix} \quad \text{bzw.} \quad \begin{bmatrix} \sum_{j=1}^N x_j^2 & \sum_{j=1}^N x_j \\ \sum_{j=1}^N x_j & N \end{bmatrix} \cdot \vec{\alpha} = \begin{bmatrix} \sum_{j=1}^N z_j x_j \\ \sum_{j=1}^N z_j \end{bmatrix}$$

für die *Normalgleichungen*.

Model-Fitting: Linearer Model-Fit via Least-Squares

Für dieses 2×2 -Gleichungssystem ergibt sich explizit die Lösung:

$$\begin{aligned}\vec{\alpha} &= \frac{1}{N \cdot \sum_{j=1}^N x_j^2 - \left(\sum_{j=1}^N x_j \right)^2} \begin{bmatrix} N - \sum_{j=1}^N x_j & \sum_{j=1}^N z_j x_j \\ -\sum_{j=1}^N x_j & \sum_{j=1}^N x_j^2 \end{bmatrix} \cdot \begin{bmatrix} \sum_{j=1}^N z_j x_j \\ \sum_{j=1}^N z_j \end{bmatrix} \\ &= \frac{1}{N \cdot \sum_{j=1}^N x_j^2 - \left(\sum_{j=1}^N x_j \right)^2} \begin{bmatrix} N \cdot \sum_{j=1}^N z_j x_j - \left(\sum_{j=1}^N x_j \right) \cdot \left(\sum_{j=1}^N z_j \right) \\ \left(\sum_{j=1}^N x_j^2 \right) \cdot \left(\sum_{j=1}^N z_j \right) - \left(\sum_{j=1}^N x_j \right) \cdot \left(\sum_{j=1}^N z_j x_j \right) \end{bmatrix}.\end{aligned}$$

Model-Fitting: Linearer Model-Fit via Least-Squares

Betrachten wir ein Rechenbeispiel:

Matlab/Octave

```
% Spaltenvektoren x,z gegeben
```

```
N = length(x);
```

```
sum_x = sum(x);
```

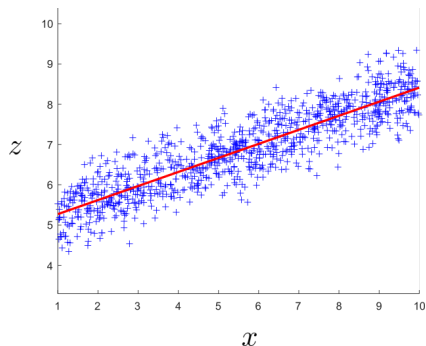
```
sum_z = sum(z);
```

```
sum_x2 = sum(x.^2);
```

```
sum_xz = sum(x.*z);
```

```
alpha_1 = (N*sum_xz - sum_x*sum_z) / (N*sum_x2 - sum_x^2);
```

```
alpha_2 = (sum_x2*sum_z - sum_x*sum_xz) / (N*sum_x2 - sum_x^2);
```



Model-Fitting: Linearer Model-Fit via Least-Squares

Es sei erwähnt, dass der Code auch noch deutlich kürzer geschrieben werden kann mit

Matlab/Octave

```
% Spaltenvektoren x,z gegeben
```

```
A = [x, ones(size(x))];
```

```
alpha = A\z;
```

Beispiel 2.14 (Polynom-Fit)

Natürlich kann man das auch für Polynome mit allgemeinem Grad p durchführen. Es ergibt sich:

$$A = \begin{bmatrix} x_1^p & x_1^{p-1} & \dots & x_1 & 1 \\ x_2^p & x_2^{p-1} & \dots & x_2 & 1 \\ \vdots & \vdots & & \vdots & \vdots \\ x_N^p & x_N^{p-1} & \dots & x_N & 1 \end{bmatrix}, \quad \vec{b} = \begin{bmatrix} z_1 \\ z_2 \\ \vdots \\ z_N \end{bmatrix},$$

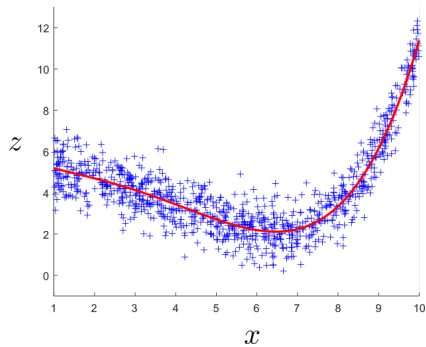
mit $m = p + 1$. Der zugehörige Code für bspw. $p = 5$ lautet:

Matlab/Octave

```
% Spaltenvektoren x,z gegeben  
A = [x.^5,x.^4,x.^3,x.^2,x, ones(size(x))];  
alpha = A\z;
```

Model-Fitting: Linearer Model-Fit via Least-Squares

Hier ein Beispielbild für einen solchen Polynom-Fit:



Beispiel 2.15 (2D-Polynom-Fit)

Dies funktioniert natürlich auch für ein 2D-Polynom, d.h. $\vec{x} \in \mathbb{R}^2$ ($n = 2$). Es ergibt sich bspw. für ein 2D-Polynom vom Grad 3:

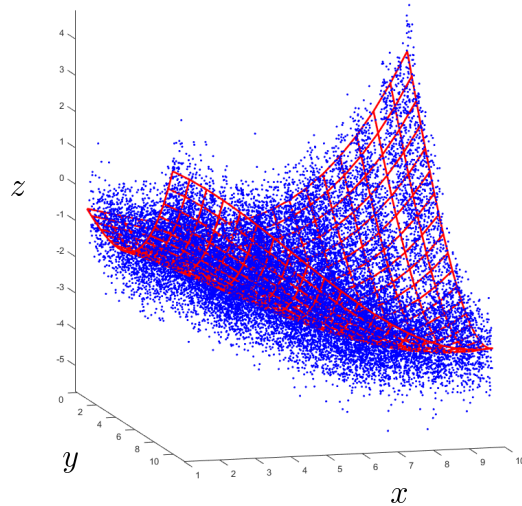
$$A = \begin{bmatrix} x_1^3 & x_1^2 y_1 & x_1 y_1^2 & y_1^3 & x_1^2 & x_1 y_1 & y_1^2 & x_1 & y_1 & 1 \\ x_2^3 & x_2^2 y_2 & x_2 y_2^2 & y_2^3 & x_2^2 & x_2 y_2 & y_2^2 & x_2 & y_2 & 1 \\ \vdots & & & & & & & & & \\ x_N^3 & x_N^2 y_N & x_N y_N^2 & y_N^3 & x_N^2 & x_N y_N & y_N^2 & x_N & y_N & 1 \end{bmatrix}, \quad \vec{b} = \begin{bmatrix} z_1 \\ z_2 \\ \vdots \\ z_N \end{bmatrix},$$

mit $m = 10$. Der zugehörige Code lautet:

```
% Spaltenvektoren x,y,z gegeben
A = [x.^3,x.^2.*y,x.*y.^2,y.^3, x.^2,x.*y,y.^2, x,y, ones(size(x))];
alpha = A\z;
```

Model-Fitting: Linearer Model-Fit via Least-Squares

Hier ein Beispielbild für einen solchen 2D-Polynom-Fit:



Beispiel 2.16 (Parametrisierte Raumkurve)

Um die Vielgestaltigkeit des linearen Model-Fits noch einmal zu verdeutlichen, wird eine polynomiale *parametrisierte Raumkurve* vom Grad 3 allgemein beschrieben durch

$$\vec{M}(t, \vec{\alpha}) = \begin{bmatrix} x(t) \\ y(t) \\ z(t) \end{bmatrix} = \begin{bmatrix} \alpha_1 t^3 + \alpha_2 t^2 + \alpha_3 t + \alpha_4 \\ \alpha_5 t^3 + \alpha_6 t^2 + \alpha_7 t + \alpha_8 \\ \alpha_9 t^3 + \alpha_{10} t^2 + \alpha_{11} t + \alpha_{12} \end{bmatrix}$$

Wie lauten die Funktionen $\vec{f}_i$ ($i = 1, \dots, 12$)?

Model-Fitting: Linearer Model-Fit via Least-Squares

Und wie lauten die Matrizen A und b bei gegebenen Datensätzen (t_j, x_j, y_j, z_j) ($j = 1, \dots, N$)?

Model-Fitting: Linearer Model-Fit via Least-Squares

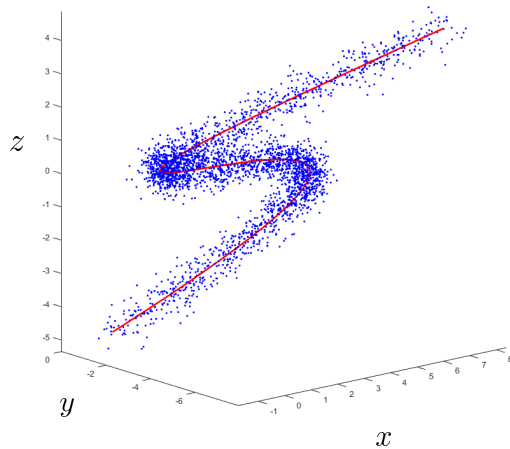
Hier der zugehörige Beispielcode:

Matlab/Octave

```
% Spaltenvektoren t,x,y,z gegeben
col = zeros(length(t)*3,4);
col(1:3:end,1) = t.^3;
col(1:3:end,2) = t.^2;
col(1:3:end,3) = t;
col(1:3:end,4) = 1;
A = [col,col([end,1:end-1],:),col([end-1:end,1:end-2],:)]';
b = zeros(length(t)*3,1);
b(1:3:end) = x;
b(2:3:end) = y;
b(3:3:end) = z;
alpha = A\b;
```

Model-Fitting: Linearer Model-Fit via Least-Squares

Hier ein Beispielbild für den Raumkurven-Fit:



Bemerkung 2.7 (Linearer Model-Fit)

Es drängen sich ein paar Erkenntnisse zum *linearen Model-Fit* auf:

- *lineare Modelle* können sehr vielgestaltig und flexibel sein, und sollten nicht unterschätzt werden.
- Die *Quadratische Zielfunktion* ist leider starr mit diesem Konzept verwoben. D.h. durch eine Änderung der Zielfunktion würde man nicht mehr auf die *Normalengleichungen* kommen und der Vorteil des Verfahrens (Lösung nur eines linearen Gleichungssystems) würde verloren gehen.
- Bei Modellen mit vielen Parametern $\vec{\alpha}$ kann die Matrix $A^T \cdot A$ der Normalengleichung numerisch schlecht konditioniert werden, was der Tatsache entspricht, dass die einzelnen Parameter „nahezu linear abhängig“ sind, d.h. dass sich die Funktionen $\vec{f}_i$ ($i = 1, \dots, m$) auf diesem Datensatz teilweise (nahezu) gegenseitig ersetzen können.

Dies gilt es bei der Wahl eines geschickten Modells zu vermeiden, da dann die Parameter über das lineare Gleichungssystem nicht mehr stabil bestimmt werden können. Eine übliche Lösungsstrategie wäre die Reduktion der Funktionsbasen ohne (nahezu) linearer Abhängigkeit.

Model-Fitting: Nichtlinearer Model-Fit

Es gibt mehrere Gründe, warum man in verschiedensten Anwendungsfeldern von dem *linearen Model-Fit* Abstand nimmt. Bspw.

- Viele Modelle sind (*echt*) *nichtlinear* in den Parametern $\vec{\alpha}$ bzw. lassen sich auch nur sehr schwer durch lineare Modelle annähern.
- Die *Quadratische Zielfunktion* kann unangebracht sein – warum *muss* denn die Abweichung eines Datenpunkts vom Modell *quadratisch* bestraft werden?
Vorsicht: Manchmal ist hier auch die Begründung: damit ich einen linearen Model-Fit durchführen kann, der einfach zu berechnen ist. Das kann aber gerade bzgl. des Wissens um *Ausreißer* in den Daten (bzw. deren Verteilung) auch schlecht gewählt sein!
- Manchmal ist es auch effizienter, die allgemeinere Methode des nichtlinearen Model-Fits zu verwenden, da man nicht viel Zeit für das Aufstellen der Normalengleichungen aufwenden möchte.

Model-Fitting: Nichtlinearer Model-Fit

Da dieser Abschnitt somit der *allgemeinere* Fall ist, müssen wir den *nichtlinearen Model-Fit* auch als einen solchen begreifen, d.h.

1. Das nichtlineare Modell wird allgemein (ohne eine alternative Darstellbarkeit) beschrieben als $\vec{M}(\vec{x}; \vec{\alpha})$.
2. Die *Zielfunktion* F kann auch nicht weiter detailliert werden, jedoch machen wir die Abhängigkeit einerseits von den Daten $(\vec{x}_j, \vec{z}_j)$ ($j = 1, \dots, N$) und andererseits von dem konkreten Modell deutlich:

$$F \left((\vec{x}_j, \vec{z}_j), \vec{M}(\vec{x}; \vec{\alpha}) \right)$$

Ohne Beschränkung der Allgemeinheit gehen wir davon aus, dass F den *Abstand* zwischen Datensatz und Modell beschreibt und damit, dass beim *nichtlinearen Model-Fit* F bzgl. $\vec{\alpha}$ minimiert werden soll. Ansonsten einfach $-F$ verwenden.

$$\operatorname{argmin}_{\vec{\alpha}} F \left((\vec{x}_j, \vec{z}_j), \vec{M}(\vec{x}; \vec{\alpha}) \right)$$

Beispiel 2.17 (Zielfunktionen)

Es gibt eine Vielzahl von Zielfunktionen, welche den Abstand als Zahl in $\mathbb{R}$ bemisst:

→ Natürlich gibt es die *Quadratische Zielfunktion* auch beim nichtlinearen Model-Fit:

$$F = \sum_{j=1}^N ||\vec{z}_j - \vec{M}(\vec{x}_j; \vec{\alpha})||^2$$

→ Ebenso ist die *Absolutdistanz* als Zielfunktion üblich, welche Abweichungen in den Datensätzen nicht mehr *quadratisch* sondern nur mehr *linear* bestraft:

$$F = \sum_{j=1}^N \sum_{l=1}^k |z_{l,j} - M_l(\vec{x}_j; \vec{\alpha})|$$

Diese Zielfunktion ist jedoch i.A. nicht mehr überall differenzierbar in $\vec{\alpha}$, da der Absolutbetrag nicht überall differenzierbar ist, die sog. *Manhattan-Norm*.

Model-Fitting: Nichtlinearer Model-Fit

- Etwas exotischer können auch statistische Größen verwendet werden, bspw. der *Median*

$$F = \text{median}_{j=1,\dots,N; l=1,\dots,k} (|z_{l,j} - M_l(\vec{x}_j; \vec{\alpha})|)$$

welcher auch *Median Absolute Deviation (MAD)* genannt wird. Alternativ könnten auch andere *Quantile* verwendet werden. Diese sind *extrem robust*, da sie die Größe der größten Ausreißer nicht berücksichtigt.

- Zielfunktionen können auch aus der *erwarteten Fehlerverteilung* der Residuen $z_{l,j} - M_l(\vec{x}_j; \vec{\alpha})$ konstruiert werden. Erwartet man, dass diese Restfehler unabhängig nach der Dichte f verteilt sind (bspw. normalverteilt), dann ist bei gegebenem $\vec{\alpha}$ der Dichte-Wert $f(z_{l,j} - M_l(\vec{x}_j; \vec{\alpha}))$ für jeden Datenpunkt j und Koordinate l . Man will nun die Parameter $\vec{\alpha}$ finden, sodass die (unabhängigen) Restfehler bzgl. aller Datenpunkte und Koordinaten am Wahrscheinlichsten werden und maximiert daher das Produkt dieser Dichte-Werte für alle Datenpunkte, die sog. *Likelihood-Funktionen (Maximum Likelihood)*, bzw. minimiert die Zielfunktion

$$F = -\ln \left(\prod_{j=1}^N \prod_{l=1}^k f(z_{l,j} - M_l(\vec{x}_j; \vec{\alpha})) \right) = -\sum_{j=1}^N \sum_{l=1}^k \ln (f(z_{l,j} - M_l(\vec{x}_j; \vec{\alpha}))) .$$

Bemerkung 2.8 (Zusammenhang normalverteilter Fehler und Quadratische Zielfunktion)

Eine bemerkenswerte Erkenntnis liefert die Zielfunktion nach *Maximum Likelihood* unter Verwendung (unabhängiger und identischer, mit Erwartungswert 0) *normalverteilter* Residuen, dies liefert nämlich

$$\begin{aligned} F &= - \sum_{j=1}^N \sum_{l=1}^k \ln \left(\frac{1}{\sigma \sqrt{2\pi}} e^{-\frac{1}{2\sigma^2} (z_{l,j} - M_l(\vec{x}_j; \vec{\alpha}))^2} \right) \\ &\propto \sum_{j=1}^N \sum_{l=1}^k (z_{l,j} - M_l(\vec{x}_j; \vec{\alpha}))^2 = \sum_{j=1}^N \|\vec{z}_j - \vec{M}(\vec{x}_j; \vec{\alpha})\|^2, \end{aligned}$$

d.h. führt automatisch zur Quadratischen Zielfunktion (unabhängig von σ !). Dies erlaubt einen (vorsichtigen) Umkehrschluss: *Verwendet man eine Quadratische Zielfunktion nimmt man implizit damit normalverteilte (oder damit kompatible) Fit-Fehler an.* Dies hat insbesondere Auswirkungen für das praktische Verständnis des linearen Model-Fits.

Model-Fitting: Nichtlinearer Model-Fit

Entlang dieser Beispiele lassen sich natürlich vielfältige Zielfunktionen formulieren. Die drei vorgestellten Zielfunktionen können im Code beispielsweise mit Hilfe von *anonymen* Funktionen definiert werden:

```
Matlab/Octave
% Datenmatrizen x (n x N), z (k x N), sowie Modellfunktion M(x,alpha) ( $\mathbb{R}^n \rightarrow \mathbb{R}^k$ ) gegeben
F_SE = @(alpha) sum( sum( (z - M(x,alpha)).^2 ) );
F_Manhattan = @(alpha) sum( sum( abs(z - M(x,alpha)) ) );
F_MAD = @(alpha) median( abs(z - M(x,alpha)), 'all' );
```

Model-Fitting: Nichtlinearer Model-Fit

Ein letzter fehlender Baustein ist dann noch der *Optimierer*:

3. Der *Optimierer* hat die Aufgabe die Parameter $\vec{\alpha}$ so zu verändern („optimieren“), dass F *kleiner* wird, und sich damit einem *Minimum* nähert.

Hierzu werden typischerweise *iterative* Algorithmen verwendet, welche

- Von einem *Startwert* $\vec{\alpha}_0$ (oder auch mehreren) beginnend
- in jedem *Iterationsschritt* ein besseres $\vec{\alpha}$ berechnet, sodass F kleiner wird.
- Dieser Vorgang wiederholt sich so oft, bis zwischen Iterationsschritten entweder $\vec{\alpha}$ oder F sich nicht mehr stark verändern.

Hier wird natürlich deutlich, dass ein *nichtlinearer Model-Fit* grundsätzlich nichts anderes als ein *Inverses Problem* ist.

Beispiel 2.18 (Optimierer, differenzierbare Zielfunktion und Modell)

Zuerst beschränken wir uns darauf, dass $\vec{\alpha}$ so in der Zielfunktion und dem Modell enthalten ist, dass diese Verbindung *differenzierbar* ist, d.h. es existiert der Gradient

$$\nabla_{\vec{\alpha}} F \left((\vec{x}_j, \vec{z}_j), \vec{M}(\vec{x}; \vec{\alpha}) \right) = \begin{bmatrix} \frac{\partial}{\partial \alpha_1} F \\ \frac{\partial}{\partial \alpha_2} F \\ \vdots \\ \frac{\partial}{\partial \alpha_m} F \end{bmatrix}.$$

→ Der Gradient ist ja die Richtung des steilsten Anstiegs, weshalb der Vektor $-\nabla_{\vec{\alpha}} F$ in die Richtung des steilsten Abstiegs schaut. Dann ist eine der bekanntesten Methoden der *steilste Gradientenabstieg* (*steepest descent*), d.h. im Iterationsschritt p wird das neue $\vec{\alpha}^{p+1}$ berechnet mit der Methode

$$\vec{\alpha}^{p+1} = \vec{\alpha}^p - \beta \cdot \nabla_{\vec{\alpha}} F \left((\vec{x}_j, \vec{z}_j), \vec{M}(\vec{x}; \vec{\alpha}^p) \right),$$

wobei die positive *Schrittweite* β meist in einer *Unteroptimierung* (*line search*) bestimmt wird.

Model-Fitting: Nichtlinearer Model-Fit

- Eine direkte Erweiterung des steilsten Gradientenabstiegs ist der *stochastische Gradientenabstieg* (*stochastic gradient descent, SGD*), welcher gerne in Fit-Problemen angewendet wird, welche auf großen Datenmengen beruhen (wie bspw. bei ANNs).

Idee ist dabei, dass man den Gradienten $\nabla_{\vec{\alpha}} F \left((\vec{x}_j, \vec{z}_j), \vec{M}(\vec{x}; \vec{\alpha}^p) \right)$ nicht mit allen Datenpunkten $(\vec{x}_j, \vec{z}_j)$ mit $j = 1, \dots, N$ auswertet, sondern wechselnde Untermengen der Datenpunkte verwendet.

Dies hat zwei praktische Vorzüge:

- Man spart sich Zeit und Aufwand bei der Berechnung des Gradienten (Hauptgrund), und
- diese „Schätzungen“ des Gradienten führen zu „leicht zufällig gestörten“ Gradienten, was sich praktisch als hilfreich erwiesen hat bei großen Optimierungsproblemen mit vielen Sattelpunkten / lokalen Minima.

Model-Fitting: Nichtlinearer Model-Fit

→ Eine weitere Methode ist das *Gauß-Newton-Verfahren*, welches nur für die Quadratische Zielfunktion funktioniert. Als erstes wird das nichtlineare Modell an der Stelle $\vec{\alpha}^p$ *linearisiert* und in die Quadratische Zielfunktion eingesetzt:

$$F = \sum_{j=1}^N \| \vec{z}_j - \vec{M}(\vec{x}_j; \vec{\alpha}) \|^2 \approx \sum_{j=1}^N \| \vec{z}_j - \vec{M}(\vec{x}_j; \vec{\alpha}^p) - J_{\vec{M}}(\vec{x}_j; \vec{\alpha}^p) \cdot (\vec{\alpha} - \vec{\alpha}^p) \|^2 ,$$

was als *linearer Model-Fit* identifiziert werden kann und zur Lösung der *Normalgleichungen* für die den Vektor $(\vec{\alpha} - \vec{\alpha}^p)$ führt. Setzt man danach $\vec{\alpha}^{p+1} := \vec{\alpha}$, so hat man für diesen Iterationsschritt die beste Schätzung nach *least squares* erhalten.

Dies kann zu folgendem Algorithmus zusammengefasst werden:

$$\vec{\alpha}^{p+1} = \vec{\alpha}^p + (A^T \cdot A)^{-1} A^T \cdot \vec{b},$$

mit

$$A = \begin{bmatrix} \frac{\partial}{\partial \alpha_1} \vec{M}(\vec{x}_1; \vec{\alpha}^p) & \frac{\partial}{\partial \alpha_2} \vec{M}(\vec{x}_1; \vec{\alpha}^p) & \dots & \frac{\partial}{\partial \alpha_m} \vec{M}(\vec{x}_1; \vec{\alpha}^p) \\ \frac{\partial}{\partial \alpha_1} \vec{M}(\vec{x}_2; \vec{\alpha}^p) & \frac{\partial}{\partial \alpha_2} \vec{M}(\vec{x}_2; \vec{\alpha}^p) & \dots & \frac{\partial}{\partial \alpha_m} \vec{M}(\vec{x}_2; \vec{\alpha}^p) \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial}{\partial \alpha_1} \vec{M}(\vec{x}_N; \vec{\alpha}^p) & \frac{\partial}{\partial \alpha_2} \vec{M}(\vec{x}_N; \vec{\alpha}^p) & \dots & \frac{\partial}{\partial \alpha_m} \vec{M}(\vec{x}_N; \vec{\alpha}^p) \end{bmatrix}, \quad \vec{b} = \begin{bmatrix} \vec{z}_1 - \vec{M}(\vec{x}_1; \vec{\alpha}^p) \\ \vec{z}_2 - \vec{M}(\vec{x}_2; \vec{\alpha}^p) \\ \vdots \\ \vec{z}_N - \vec{M}(\vec{x}_N; \vec{\alpha}^p) \end{bmatrix}.$$

Natürlich wird algorithmisch nicht (wie hier auf dem Blatt dargestellt) die inverse Matrix direkt berechnet, sondern das zugehörige lineare Gleichungssystem gelöst.

Für das *Gauß-Newton-Verfahren* gibt es vielfältige Erweiterungen:

- Bspw. kann man kontinuierlich die beiden Update-Gleichungen des *steilsten Gradientenabstiegs* und des *Gauß-Newton-Verfahrens* mit einem Gewichtungsfaktor kombinieren und erhält im Wesentlichen den *Levenberg-Marquardt-Algorithmus* (auch unter *damped least squares* bekannt).
- Oder man führt einen Schrittweitenfaktor β vor der inversen Matrix ein, welcher wie beim *steilsten Gradientenabstieg* in einer *Unteroptimierung* bestimmt wird.

Man erkennt, die beiden Algorithmen können zwar auf dem Blatt kompliziert aussehen, aber im Grunde effizient implementiert werden, wenn *Gradient* oder *Jacobi-Matrix* bekannt sind.

Beispiel 2.19 (Optimierer, nicht-differenzierbare Zielfunktion und Modell)

Für nicht in $\vec{\alpha}$ differenzierbare Zielfunktionen bzw. Modelle, kann man nicht auf die zuvor beschriebenen Optimierer zurückgreifen. Möglichkeiten hierzu sind

- Annäherungen des *Gradienten* oder der *Jacobi-Matrix* aus den vorherigen Beispielen durch endliche Differenzenquotienten (für kleine $h \in \mathbb{R}^+$)

$$\frac{\partial}{\partial \alpha_i} f(\vec{\alpha}) \approx \frac{1}{h} (f(\vec{\alpha} + h \cdot \vec{e}_i) - f(\vec{\alpha})) .$$

Bekannt hierfür ist auch das *Quasi-Newton-Verfahren*, welches im *mehrdimensionalen Newton-Verfahren* die *Hesse-Matrix* durch endliche Differenzenquotienten annähert.

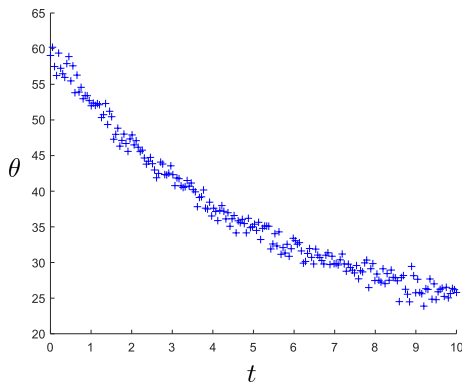
- Komplette unabhängig davon gibt es auch das *Downhill-Simplex-Verfahren* (auch *Nelder-Mead Algorithmus*), welches nach einer vorgegebenen Logik in einer einfachen Schematik eine endliche Anzahl von Richtungen für die $\vec{\alpha}$ testet und wenn keine Verbesserung gefunden werden kann, die Schrittweite verändert.

Model-Fitting: Nichtlinearer Model-Fit

Diese Auflistung von *Optimierern* soll Ihnen helfen verschiedene Algorithmen und Begriffe, denen Sie beim *nichtlinearen Model-Fit* begegnen können grob einzuordnen. Details zu solchen Algorithmen werden typischerweise in einer *Numerik-* oder *Optimierungsveranstaltung* näher gebracht.

Beispiel 2.20 (Newtonsches Abkühlungsgesetz)

Im Folgenden betrachten wir einen Datensatz, welcher aus der Messung der Temperaturabkühlung eines heißen Körpers besteht und zu unterschiedlichen Zeitpunkten $t \in \mathbb{R}$ ($n = 1$) die Temperatur $\theta \in \mathbb{R}$ ($k = 1$) enthält.



Wir wollen zwei verschiedene Wege begehen, das *Newtonsche Abkühlungsgesetz* in diese Daten zu fitten.

Model-Fitting: Nichtlinearer Model-Fit

Das zugehörige physikalische Modell ist *echt nichtlinear* mit

$$M(t, \vec{\alpha}) = \alpha_1 + (\alpha_2 - \alpha_1) e^{-\alpha_3 t},$$

und α_1 der *Umgebungstemperatur*, α_2 der *Anfangstemperatur des Körpers* und α_3 einer *Abkühlungskonstante*.

In der ersten Lösung verwenden wir die *Manhattan-Norm* als *Zielfunktion* und den ableitungsfreien *Downhill-Simplex-Algorithmus* als *Optimierer*:

```
Matlab/Octave
% Spaltenvektoren t,theta gegeben
M = @(t,alpha) alpha(1) + (alpha(2)-alpha(1))*exp(-alpha(3)*t);
F_Manhattan = @(alpha) sum( abs( theta - M(t,alpha) ) );
alpha_start = [theta(end);theta(1);0.5];
alpha_opt = fminsearch(F_Manhattan,alpha_start); % Downhill-Simplex
```

und erhalten dabei die Werte $\vec{\alpha}^T = [19.8390 \ 59.4256 \ 0.1948]$.

Model-Fitting: Nichtlinearer Model-Fit

Für die zweite Variante wollen wir den *Gauß-Newton-Algorithmus* verwenden. Hierzu benötigen wir erstmal die Ableitungen:

$$\frac{\partial}{\partial \alpha_1} M(t; \vec{\alpha}) =$$

$$\frac{\partial}{\partial \alpha_2} M(t; \vec{\alpha}) =$$

$$\frac{\partial}{\partial \alpha_3} M(t; \vec{\alpha}) =$$

Damit lassen sich nun die Matrix A und der Vektor $\vec{b}$ bestimmen:

$$A = \begin{bmatrix} 1 - e^{-\alpha_3^p t_1} & e^{-\alpha_3^p t_1} & -t_1 \cdot (\alpha_2^p - \alpha_1^p) e^{-\alpha_3^p t_1} \\ 1 - e^{-\alpha_3^p t_2} & e^{-\alpha_3^p t_2} & -t_2 \cdot (\alpha_2^p - \alpha_1^p) e^{-\alpha_3^p t_2} \\ \vdots & \vdots & \vdots \\ 1 - e^{-\alpha_3^p t_N} & e^{-\alpha_3^p t_N} & -t_N \cdot (\alpha_2^p - \alpha_1^p) e^{-\alpha_3^p t_N} \end{bmatrix}, \quad \vec{b} = \begin{bmatrix} \theta_1 - (\alpha_1^p + (\alpha_2^p - \alpha_1^p) e^{-\alpha_3^p t_1}) \\ \theta_2 - (\alpha_1^p + (\alpha_2^p - \alpha_1^p) e^{-\alpha_3^p t_2}) \\ \vdots \\ \theta_N - (\alpha_1^p + (\alpha_2^p - \alpha_1^p) e^{-\alpha_3^p t_N}) \end{bmatrix}.$$

Model-Fitting: Nichtlinearer Model-Fit

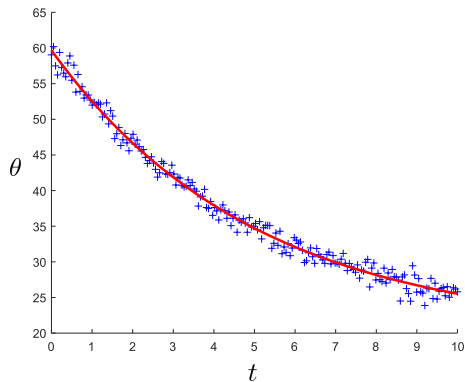
Lassen wir nun den *Gauß-Newton-Algorithmus* einfach 10 Iterationen laufen, dann kann das wie folgt umgesetzt werden:

```
Matlab/Octave
% Spaltenvektoren t,theta gegeben
M = @(t,alpha) alpha(1) + (alpha(2)-alpha(1))*exp(-alpha(3)*t);
alpha = [theta(end);theta(1);0.5];
for i = 1:10
    A = [1-exp(-alpha(3)*t),exp(-alpha(3)*t),-t.*(alpha(2)-alpha(1)).*exp(-alpha(3)*t)];
    b = [theta - M(t,alpha)];
    alpha = alpha + A\b;
end
```

und erhält dabei die Werte $\vec{\alpha}^T = [20.1701 \ 59.6690 \ 0.1997]$.

Model-Fitting: Nichtlinearer Model-Fit

Die Lösung beider *nichtlinearer Model-Fits* sehen graphisch identisch aus:



Beispiel 2.21 (Fit einer Wahrscheinlichkeitsdichte)

Eine häufige Anwendung besteht bei einer gegebenen Verteilung von Werten eine zugehörige Wahrscheinlichkeitsdichte in Histogramm-Daten zu fiten. Die Wahrscheinlichkeitsdichten sind bzgl. ihrer Parameter praktisch immer *nichtlinear*, bspw. wie in der *Normalverteilung*:

$$M(x, \vec{\alpha}) = \frac{1}{\sqrt{2\pi} \alpha_2} e^{-\frac{(x-\alpha_1)^2}{2\alpha_2^2}},$$

wobei man die Parameter typischerweise $\mu = \alpha_1$ (Erwartungswert), $\sigma = \alpha_2$ (Standardabweichung) bezeichnet. Für die Anwendung des *Gauß-Newton-Verfahrens* benötigen wir die partiellen Ableitungen:

$$\begin{aligned}\frac{\partial}{\partial \alpha_1} M(x, \vec{\alpha}) &= \frac{x - \alpha_1}{\sqrt{2\pi} |\alpha_2|^3} e^{-\frac{(x-\alpha_1)^2}{2\alpha_2^2}} \\ \frac{\partial}{\partial \alpha_2} M(x, \vec{\alpha}) &= \frac{1}{\sqrt{2\pi} \alpha_2^2} \left(\frac{(x - \alpha_1)^2}{\alpha_2^2} - 1 \right) e^{-\frac{(x-\alpha_1)^2}{2\alpha_2^2}}.\end{aligned}$$

Model-Fitting: Nichtlinearer Model-Fit

Der zugehörige Code für zwei *nichtlineare Model-Fits* sieht bspw. wie folgt aus:

```
% Matlab/Octave

% Vektor data gegeben

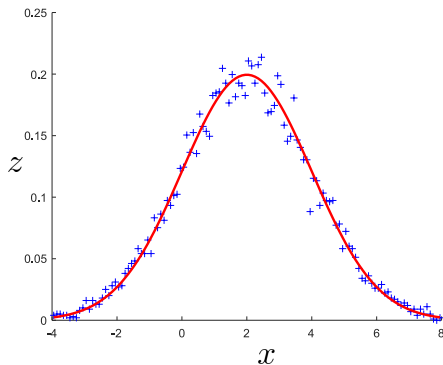
% 1) Aufbereitung der Histogramm-Daten:
edges = [-4:0.1:8]; % Festlegung eines geeigneten Rasters für das Histogramm
[counts] = histcounts(data,edges); % Berechnung der Histogramm-Werte
x = 0.5*(edges(1:end-1)+edges(2:end)); % Berechnen der Mittelpunkte
z = counts/( sum(counts*(edges(2)-edges(1))) ); % Normalisierung auf Flächeninhalt 1

% 2a) Nichtlinearer Model-Fit (allgemein):
M = @(x,alpha) 1/sqrt(2*pi*alpha(2)^2) * exp( -(x-alpha(1)).^2/(2*alpha(2)^2) );
F_SE = @(alpha) sum( ( z - M(x,alpha) ).^2 );
alpha_start = [0;1];
alpha_opt = fminunc(F_SE,alpha_start); % Quasi-Newton, d.h. ableitungsfrei

% 2b) Nichtlinearer Model-Fit (Gauß-Newton):
x = x'; z = z'; % Spaltenvektoren
alpha = [0;1];
for i = 1:10
    A = [(x-alpha(1))/(sqrt(2*pi)*abs(alpha(2))^3).*exp( -(x-alpha(1)).^2/(2*alpha(2)^2) ), ...
          1/(sqrt(2*pi)*alpha(2)^2)*((x-alpha(1)).^2/alpha(2)^2-1).*exp(-(x-alpha(1)).^2/(2*alpha(2)^2))];
    b = [z - M(x,alpha)];
    alpha = alpha + A\b;
end
```

Model-Fitting: Nichtlinearer Model-Fit

Auch hier sieht die Lösung beider *nichtlinearer Model-Fits* graphisch identisch aus:



Es sei hier nur erwähnt, dass im Speziellen für die *Normalverteilung* durch die direkte *Schätzung* des Erwartungswerts und der Standardabweichung schneller eine Lösung generiert werden kann. Wir haben bewusst hier darauf verzichtet, damit Sie das allgemeine Verfahren sehen.

Model-Fitting: Modell-Auswahl durch Model-Fit

Praktisch ist häufig die Herausforderung, dass man noch kein fixes Modell hat (meist jedoch eine grobe Idee einer Auswahl von möglichen Modellen), welches zu den vorliegenden Daten passt. Eine naheliegende Idee ist es nun, dass man das Model-Fitting in die *Bestimmung eines geeigneten Modells* als Kriterium hinzunimmt. Man nimmt dann das Modell, welches nach dem Model-Fit bspw. die kleinste Abweichung zu den Daten erzeugt.

Dabei gibt es jedoch mehrere Aspekte zu beachten:

1. Es muss ein geeignetes Vergleichskriterium der unterschiedlichen Modelle verwendet werden. Deshalb sollte man entweder für alle Modelle die *gleiche* Zielfunktion verwenden und dann das Modell mit dem kleinsten Zielfunktionswert nehmen, oder eine eigene (unabhängige) *Vergleichsfunktion*, welche den Abstand zwischen Daten und optimiertem Modell direkt berechnet.

Model-Fitting: Modell-Auswahl durch Model-Fit

In einfachen Model-Fitting Situationen für Modelle, die nach $\mathbb{R}$ abbilden (d.h. für Daten $(\vec{x}_j, z_j)$) hat sich der statistische *Bestimmtheitskoeffizient* R^2 als Vergleichsstandard etabliert. Er ist definiert als

$$R^2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}},$$

mit der *Varianz des Restfehlers zwischen Modell und z -Daten* (Vorfaktor kürzt sich)

$$SS_{\text{res}} = \sum_{j=1}^N (z_j - M(\vec{x}_j; \vec{\alpha}))^2$$

und der *grundsätzlich vorhandenen Varianz der z -Daten*

$$SS_{\text{tot}} = \sum_{j=1}^N (z_j - \bar{z})^2$$

mit dem arithmetischen Mittel $\bar{z}$.

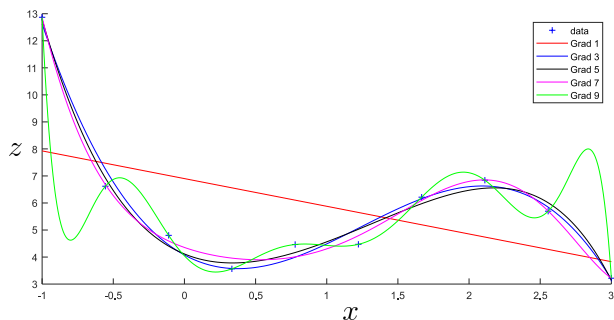
R^2 drückt damit aus, zu welchem Prozentsatz sich die Streuung der Daten durch das Modell komplett vorausgesagt werden ($R^2 = 1$) oder das Modell nichts zur Erklärung der Streuung in den Daten beiträgt ($R^2 = 0$).

Model-Fitting: Modell-Auswahl durch Model-Fit

2. Bei der Auswahl der Modelle muss darauf geachtet werden, dass die Anzahl der Parameter m deutlich *kleiner* als die Anzahl der Datensätze N sein sollte ($m < N$). Ansonsten besteht die Gefahr in das *Overfitting* (*Überanpassung*) zu laufen. Ein klassisches Beispiel hierfür ist der Polynomfit in folgendem Beispiel. D.h. *Flexibilität* vs. *Steifheit* des Modells muss einerseits zum zu Grunde liegenden Problem und andererseits auch zu der Datenmenge passen.

Beispiel 2.22 (Overfitting)

Eine zu hohe *Flexibilität* des Modells kann auch eine *Gefahr* sein. Wir haben einen eindimensionalen Datensatz $n = 1$, $k = 1$ gegeben mit wenigen Datenpunkten $N = 10$. In diesen Datensatz *fitten* wir nun Polynome vom Grad 1, 3, 5, 7, 9.



Es zeigt sich, dass Grad 1 dem Datensatz kaum folgt (*zu steif*), Grad 3 bis 7 alle gut sind, aber auch augenscheinlich keines zu bevorzugen ist, und Grad 9 zwar den Fehler der Zielfunktion auf 0 drückt, jedoch zwischen den Datenwerten unzufriedenstellende Ergebnisse liefert.

Model-Fitting: Modell-Auswahl durch Model-Fit

Generell gilt bzgl. der *Flexibilität* und der damit einhergehenden Schwierigkeit des Umgangs mit vielen Parametern $\vec{\alpha}$ typischerweise die Richtlinie (welche *Albert Einstein* zugeschrieben wird):

„Everything Should Be Made as Simple as Possible, But Not Simpler.“

Oder in einer älteren Version auch aus dem dem 14. Jahrhundert mit „*Occam's razor*“ (informieren Sie sich!).

Solche *Modellierungsparadigma* können insbesondere sonst nur aufwändig zu beherrschende *Overfitting*-Probleme vermeiden helfen und würde in dem Beispiel dem Grad 3-Polynom im Vergleich zum Grad 5 bzw. Grad 7 Polynom (welche ähnliche Ergebnisse liefern) den Vorrang geben.

3. Auf den Optimierer beim *nichtlinearen Model-Fit* muss man besonders aufpassen:
- Mit seinen Einstellungen kann der Optimierer bei einem Modell besonders gut und bei einem anderen eher schlecht funktionieren, das hängt ganz vom Zusammenspiel des Modells mit der Zielfunktion zusammen (bspw. steile, tiefe vs. ausgedehnte, flache Minima, etc.). Deshalb muss man auch für jeden Fit den Optimierer „*tunen*“.
 - Es gibt den sog. „*Fluch der Dimensionen*“, der besagt, dass je größer m (Anzahl der Parameter) ist, es auch viel mehr Möglichkeiten für Sattelpunkte gibt, d.h. Minima bzgl. nur einer Untermenge von Parametern und bzgl. der anderen nicht. Der Optimierer wird von diesen *Sattelpunkten* angezogen, und kann damit vom gesuchten *lokalen oder globalen Minimum* recht leicht abgelenkt werden.

Stellen wir uns ein Problem mit $m = 3$ Parametern vor, so gibt es 1 Konstellation für ein echtes lokales Minimum (bzgl. aller drei Parameter ein Minimum) und 6 Konstellationen für Sattelpunkte bzw. 1 Maximum (mindestens bzgl. einem Parameter kein Minimum).

Bei $m = 30$ verschiebt sich dies schon auf 1 zu 1073741822 Sattelpunkte.

Allgemein ergibt sich die Formel mit 1 zu $2^m - 2$.

Model-Fitting: Modell-Auswahl durch Model-Fit

- Die *Generierung der Startwerte* kann sehr anspruchsvoll sein, insbesondere, wenn man kein unabhängiges qualitatives Wissen über die Entstehung des Datensatzes hat oder die Dimension von m von $\vec{\alpha}$ einfach sehr groß ist (siehe bspw. ANNs). Häufig verwendet man dann bei anspruchsvollen Optimierungsaufgaben mehrere Startwerte, und vergleicht dann die Fit-Ergebnisse oder verwendet *vortrainierte* Startwerte aus verwandten Problemen.

4. Natürlich kommt es auf die Qualität der Daten an!

- Natürlich muss der Datensatz repräsentativ sein und einer passenden Verteilung für das zu modellierende Phänomen folgen.
- Die Qualität (bspw. bzgl. *Ausreißern*) muss bekannt sein, und/oder die Datensätze werden in einem *Pre-Processing*-Schritt aufbereitet.
- Ratsam ist es den Model-Fit mit Teilmengen der Daten durchzuführen (*Trainings-Daten*) und dann auf einen anderen Teil der Daten (*Test-Daten*) anzuwenden, um die Qualität des Modells zu beurteilen. Weiterhin empfiehlt es sich beim Model-Fitting auf Teilmengen der Daten (zumindest zeitweise während des Fittings) zu setzen. Dies kann typischerweise die *Robustheit* eines Modells erhöhen.

Model-Fitting: Anwendungsfall Machine Learning mit künstlichen neuronalen Netzen (ANNs)

Eine sehr populäre Anwendung von Model-Fitting ist das *Machine Learning* und insbesondere die Verwendung von *künstlichen Neuronalen Netzen (ANNs)* als Teil des *supervised learning*. Wir halten fest:

Das *Trainieren eines ANNs* ist nichts anderes als ein *nichtlinearer Model-Fit*.

Wir diskutieren kurz die einzelnen Komponenten und deren Besonderheiten:

- Der *Datensatz* $(\vec{x}_j, \vec{z}_j)$ ($j = 1, \dots, N$) ist groß (*Big Data*). Dies hat zwei Aspekte:
 - In den einzelnen Dimensionen von $\vec{x}$ (n sehr groß) und $\vec{z}$ (k sehr groß) – bspw. wenn die Input- und Output-Daten *Bilder* sind, dann kann ein einzelnes 512×512 -Bild einem Vektor im $512 \cdot 512 = 262144$ -dim. Raum entsprechen.
 - N ist ebenso sehr groß, man hat sehr viele Datenpaare in diesem sehr hochdimensionalen Raum. Häufig bläht man den Datensatz sogar noch einmal künstlich auf (→ *Augmentation*).
 - Durch die Größe des Datensatzes wird meist eine sehr ausgefeilte *Sampling*-Strategie verwendet (mindestens die Aufteilung in Trainings- und Test-Datensatz), sowie Teilmengen innerhalb der Trainingsdaten sog. „*Batches*“ während des Optimiervorgangs.

Model-Fitting: Anwendungsfall Machine Learning mit künstlichen neuronalen Netzen (ANNs)

- Das Modell $\vec{M}(\vec{x}; \vec{\alpha})$ ist *nichtlinear* und besteht aus sehr vielen Parametern $\vec{\alpha}$, d.h. m sehr groß. Das Modell ist sehr flexibel, da es sich um eine große *Funktionskomposition* handelt, d.h. eine lange Hintereinanderausführung von einfachen Funktionen, welche jeweils Teile der Parameter $\vec{\alpha}$ enthalten. Typischerweise hat ein *Feed-Forward-ANN* mit r Schichten („Layers“) folgende Struktur:

$$\vec{M}(\vec{x}; \vec{\alpha}) = W_{r-1}(\vec{\alpha}) \cdot \vec{f}_{r-2} \left(\dots \vec{f}_2 \left(W_2(\vec{\alpha}) \cdot \vec{f}_1 (W_1(\vec{\alpha}) \cdot \vec{x}) \right) \dots \right) ,$$

wobei

- $W_1, \dots, W_{r-1}$ die *Gewichtsmatrizen* sind, deren Einträge optimiert werden (d.h. auf die sich typischerweise viele tausend/millionen $\vec{\alpha}$ verteilen).
- $\vec{f}_1, \dots, \vec{f}_{r-2}$ auf die jeweilige Dimension des Layers angepasste Funktionen sind, welche komponentenweise typischerweise die immer gleiche *Aktivierungsfunktion* anwenden. Hier wird gerne der *Tangens Hyperbolicus* $\tanh(x)$, die *Sigmoid-Funktion* $\text{sig}(x)$ oder *Rectified Linear Unit* (ReLU) verwendet.

Die genaue Struktur und der Aufbau des Modells, d.h. des ANNs, ist dabei die Kunst für die jeweilige Anwendung und wird meist durch *Ausprobieren* bestimmt.

Model-Fitting: Anwendungsfall Machine Learning mit künstlichen neuronalen Netzen (ANNs)

- Als *Zielfunktion* wird bspw. die *Quadratische Zielfunktion* (SE) verwendet, sodass man insbesondere die Differenzierbarkeit der Zielfunktion erhält. Für Klassifikationsaufgaben (d.h. $\vec{z}$ diskret) gibt es auch spezielle Zielfunktionen, wie den *log-loss*.

Nicht selten wird auch eine Regularisierung zur Zielfunktion hinzugefügt, bspw.

$$F^*(\vec{\alpha}) = F(\vec{\alpha}) + \lambda \sum_{i=1}^m |\alpha_i| ,$$

die *Lasso-Regularisierung*, welche die einzelnen Gewichte $\vec{\alpha}$ tendenziell klein hält.

- Als *Optimierer* verwendet man gerne den *steilsten Gradientenabstieg* bzw. Varianten davon. Meist wird die Optimierung auch frühzeitig gestoppt („*early stopping*“) und der Optimieralgorithmus wird aktiv verändert bzw. gestört bzw. es werden abwechselnd unterschiedliche Teildaten (*Batches*) in der Zielfunktion verwendet (→ *Stochastic Gradient Descent*).

Eine weitere Strategie ist das temporäre entfernen von einzelnen Neuronen und deren Gewichten während der iterativen Optimierung, das sog. *Dropout*. Dies erlaubt es eine höhere Robustheit im trainierten Netz zu erzielen.

Model-Fitting: Anwendungsfall Machine Learning mit künstlichen neuronalen Netzen (ANNs)

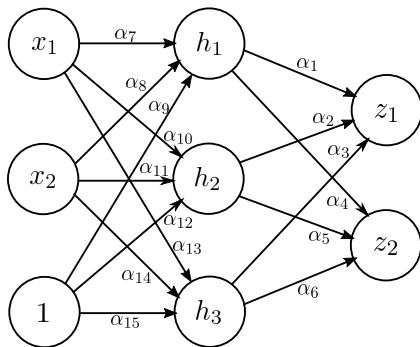
Es wird deutlich, dass das „Lernen aus Daten“ im *Machine Learning* mit ANNs nichts anders als ein *nichtlinearer Model-Fit* mit großen und hochdimensionalen Datensätzen und einer sehr flexiblen Modell-Familie mit vielen Parametern $\vec{\alpha}$ ist. Oder anders: Diese große Flexibilität des Modells muss durch eine sehr große Anzahl von Trainingsdaten, und ausgefeilten weiteren Strategien (bspw. Augmentation, Regularisierung, Dropout, etc.) „gezähmt“ werden.

Die große Kunst ist es ein Modell zu finden, welches den komplizierten Verläufen der Trainingsdaten folgt jedoch Overfitting vermeidet und damit letztlich gute Vorhersagen für Daten generiert, welche nicht im Trainingsdatensatz waren.

Man nennt diese Aufgabe: Die *Generalisierung* des Modells.

Beispiel 2.23 (ANN: 3-Layer Perzeptron / Data Science, Scheid/Vogl, 2021)

Betrachten wir konkret das Modell das häufig zur Verständnissgewinnung von ANNs verwendeten wird: Das *3-Layer Perzeptron*.



Es gibt das *Input layer* mit $\vec{x} \in \mathbb{R}^2$ (links), *hidden layer* (mitte), und *Output layer* $\vec{z} \in \mathbb{R}^2$ (rechts). Die Gewichte sind $\vec{\alpha} \in \mathbb{R}^{15}$ und die „1“ im *Input layer* ist nur die Konstante des Aktivierungspotenzials im *hidden layer*.

Model-Fitting: Anwendungsfall Machine Learning mit künstlichen neuronalen Netzen (ANNs)

Wir wollen mit diesem ANN *Regression* betreiben, d.h. wir betrachten das Modell:

$$\begin{aligned}\vec{M}(\vec{x}, \vec{\alpha}) &= W_2(\vec{\alpha}) \cdot \tanh \left(W_1(\vec{\alpha}) \cdot \begin{bmatrix} \vec{x} \\ 1 \end{bmatrix} \right) \\ &= \begin{bmatrix} \alpha_1 & \alpha_2 & \alpha_3 \\ \alpha_4 & \alpha_5 & \alpha_6 \end{bmatrix} \cdot \tanh \left(\begin{bmatrix} \alpha_7 & \alpha_8 & \alpha_9 \\ \alpha_{10} & \alpha_{11} & \alpha_{12} \\ \alpha_{13} & \alpha_{14} & \alpha_{15} \end{bmatrix} \cdot \begin{bmatrix} x_1 \\ x_2 \\ 1 \end{bmatrix} \right),\end{aligned}$$

mit der komponentenweisen Anwendung des $\tanh$. Ausgerechnet ergibt sich:

$$\vec{M}(\vec{x}, \vec{\alpha}) = \begin{bmatrix} \alpha_1 \cdot \tanh(\alpha_7 x_1 + \alpha_8 x_2 + \alpha_9) + \alpha_2 \cdot \tanh(\alpha_{10} x_1 + \alpha_{11} x_2 + \alpha_{12}) + \alpha_3 \cdot \tanh(\alpha_{13} x_1 + \alpha_{14} x_2 + \alpha_{15}) \\ \alpha_4 \cdot \tanh(\alpha_7 x_1 + \alpha_8 x_2 + \alpha_9) + \alpha_5 \cdot \tanh(\alpha_{10} x_1 + \alpha_{11} x_2 + \alpha_{12}) + \alpha_6 \cdot \tanh(\alpha_{13} x_1 + \alpha_{14} x_2 + \alpha_{15}) \end{bmatrix},$$

d.h. durch die Hintereinanderausführung entsteht schnell ein recht unübersichtliches Modell. Erhöht man nun die Anzahl und jeweilige Dimension der *hidden layers* ist dieses Modell jedoch sehr flexibel (m wird sehr groß).

Typischerweise wird dann die *Quadratische Zielfunktion* für die *Trainingsdaten* ($\vec{x}_j, \vec{z}_j$) ($j = 1, \dots, N$) mit dem *Gradientenabstieg* und einer geschickten Verwendung der mehrdimensionalen Kettenregel („*Backpropagation*“) minimiert:

$$\operatorname{argmin}_{\vec{\alpha}} F = \sum_{j=1}^N \|\vec{z}_j - \vec{M}(\vec{x}_j; \vec{\alpha})\|^2$$

Bemerkung 2.9 (Training künstlicher Neuronaler Netze)

Es gibt offensichtlich viele Modellierungsfreiheiten und *Meta-Parameter* der Optimierer, welche in dezidiert Software berücksichtigt und meist durch Graphen schön aufbereitet werden. Insbesondere der Graph, welcher über den Optimiervorgang den kleiner werdenden Zielfunktionswert anzeigt, hilft die Optimierung zu verfolgen.

Bemerkung 2.10 (Flexibilität von Modellen)

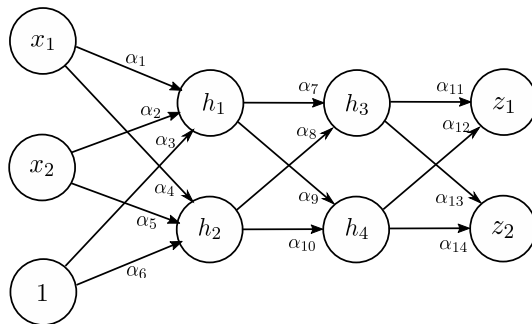
Eine Besonderheit von sehr flexiblen Modellen (wie bei ANNs) ist dabei, dass

- *vorteilhaft* wirkt, dass damit auch von der modellierenden Person nicht einmal in ihrer groben Struktur bekannte Vorhersagen möglich sind (d.h. „man lässt die Daten sprechen“),
- *nachteilig* wirkt, dass die einzelnen Parameter typischerweise nicht mehr interpretiert werden können. D.h. das Modell an sich hat nur mehr ein geringes „Erklärpotential“. Ob man ein solches Modell vor sich hat kann man typischerweise mit der Frage herausfinden:

Wenn ein falscher Output $\tilde{z}$ in einem Test-Datensatz herauskommt, kann ich das auf einzelne Parameter α zurückführen? (Verbesserungsversuch: „Explainable AI“).

Beispiel 2.24 (ANN mit zwei hidden layers)

Schreiben Sie das explizite Modell zu folgendem ANN hin mit dem $\tanh$ als Aktivierungsfunktion:



3. Modelle und ihre Simulation

Einordnung des Kapitels

Einordnung des Kapitels

In dem Abschnitt *Modelle und ihre Simulation* der Vorlesung beschäftigen wir uns insbesondere mit dem Aufstellen und Simulieren von Modellen welche meist einer inneren Dynamik folgen, oder welche wir in der Simulation einer künstlichen Dynamik unterwerfen. Wir wollen über diese Simulationsergebnisse mehr über die allgemeinen Eigenschaften oder die konkrete Anwendung der Modelle erfahren. Er gehört damit grundsätzlich in den Bereich **Scientific Computing** (*Wissenschaftliches Rechnen*).

3. Modelle und ihre Simulation

Deterministische Modelle mit Differentialgleichungen

Deterministische Modelle mit Differentialgleichungen: Simulation gewöhnlicher Dgln (ODE)

Viele schwer zu verstehende (und deshalb besonders untersuchenswerte) Systeme haben eine *innere Dynamik*. D.h. bspw. von Startzuständen ausgehend ist es nicht offensichtlich klar, wie sich das System über die Zeit entwickeln wird. Offensichtlich lohnt es sich deshalb solche Systeme zu simulieren.

Mathematisch können viele solcher Systeme mit Hilfe von sog. *Differentialgleichungen* dargestellt werden, welche genau das beinhalten:

→ Man beschreibt mit dieser Gleichung die *Dynamik an sich*, nicht jedoch die exakte Entwicklung des Systems.

Bspw. sei der zeitliche Verlauf einer Größe $z(t)$ gesucht, wobei man nur weiß, die Veränderung (d.h. Zu- oder Abnahme) von z ist abhängig davon, wie groß z zum Zeitpunkt t schon gerade ist. Mathematisch kann man das via der Ableitung schreiben:

$$z'(t) = a \cdot z(t) ,$$

mit der Konstanten $a \in \mathbb{R}$. Diese Gleichung sagt nicht, wie $z(t)$ aussieht, sondern wie sich $z(t)$ dynamisch verhält.

→ Je nach Startzustand zum Zeitpunkt t_0 kann die Dynamik ganz anders aussehen.

Deterministische Modelle mit Differentialgleichungen: Simulation gewöhnlicher Dgln (ODE)

Ausgehend von dieser *Dynamik-Vorschrift* ist dann immer eine *explizite Darstellung* von $z(t)$ für gegebene Start- oder Randbedingungen gesucht.

Wir betrachten in einem Überblick folgende dynamische Systeme, die Ihnen einerseits die Breite dieser Modellierungsfamilie verdeutlicht, andererseits Ihnen auch evtl. konkrete Anknüpfungspunkte zum Spielen und Ausprobieren liefern:

- *Elektrische Schaltungen* mit Anregungsspannungen und Resonanzeffekten.
- *Populationsmodelle* und deren dynamische Entwicklungen, wenn es mind. zwei Gegenspieler in den Populationen gibt. Dies beinhaltet auch *Infektionsmodelle* bei ansteckenden Krankheiten.
- Der sehr komplizierte *Bewegungsablauf von mehreren Planeten*, unter dem Einfluss ihrer Gravitation.
- Ein erster Einblick in kompliziertere Differentialgleichungen (sog. *Partielle Differentialgleichungen*).

Definition 3.1 (Explizite gewöhnliche Differentialgleichung)

Eine *gewöhnliche Differentialgleichung* (*Ordinary Differential Equation, ODE*) der n -ten Ordnung für die unbekannte/gesuchte Funktion $z(t) \in \mathbb{R}$ und $t \in \mathbb{R}$ ist definiert durch

$$f\left(t, z, z', z'', \dots, z^{(n)}\right) = 0 ,$$

mit einer mindestens stetigen Funktion $f : \mathbb{R}^{n+2} \mapsto \mathbb{R}$ und der n -ten Ableitung $z^{(n)}$. Kann man die Differentialgleichung auf $z^{(n)}$ auflösen, d.h.

$$z^{(n)}(t) = g\left(t, z, z', z'', \dots, z^{(n-1)}\right) ,$$

mit $g : \mathbb{R}^{n+1} \mapsto \mathbb{R}$ stetig, dann spricht man von einer *expliziten gewöhnlichen Differentialgleichung*.

Definition 3.2 (Gewöhnliches Differentialgleichungssystem mit expliziten Gleichungen)

Hat man mehrere explizite gewöhnliche Differentialgleichungen welche voneinander abhängen, bspw.

$$\begin{aligned} z_1^{(n_1)}(t) &= g_1 \left(t, z_1, z_1', \dots, z_1^{(n_1-1)}, z_2, z_2', \dots, z_2^{(n_2-1)}, \dots, z_m, z_m', \dots, z_m^{(n_m-1)} \right) \\ z_2^{(n_2)}(t) &= g_2 \left(t, z_1, z_1', \dots, z_1^{(n_1-1)}, z_2, z_2', \dots, z_2^{(n_2-1)}, \dots, z_m, z_m', \dots, z_m^{(n_m-1)} \right) \\ &\vdots \\ z_m^{(n_m)}(t) &= g_m \left(t, z_1, z_1', \dots, z_1^{(n_1-1)}, z_2, z_2', \dots, z_2^{(n_2-1)}, \dots, z_m, z_m', \dots, z_m^{(n_m-1)} \right), \end{aligned}$$

dann spricht man von einem *gewöhnlichen Differentialgleichungssystem* mit m expliziten Gleichungen jeweils der n_i -ten Ordnung mit den beschreibenden Funktionen $g_i : \mathbb{R}^{(\sum_{j=1}^m n_j + 1)} \mapsto \mathbb{R}$ für $i = 1, \dots, m$.

Deterministische Modelle mit Differentialgleichungen: Simulation gewöhnlicher Dgln (ODE)

Für die Modellierung von solchen durchaus komplizierten, expliziten Differentialgleichungen bzw. –systemen ist **folgender Trick** von besonderer Bedeutung: Diese ODEs können immer in *größere Systeme* (d.h. mit mehr Gleichungen) jeweils 1. Ordnung überführt werden, durch die Einführung von künstlichen Hilfsfunktionen $w_{i,j}(t)$ (für $i = 1, \dots, m$ und $j = 1, \dots, n_i - 1$). Betrachten wir hierzu die Modifikation der $i = 1$. Gleichung:

$$w_{1,1}(t) := z_1'(t)$$

$$w_{1,2}(t) := z_1''(t) = w_{1,1}'(t)$$

$$w_{1,3}(t) := z_1^{(3)}(t) = w_{1,2}'(t)$$

$$\vdots$$

$$w_{1,n_1-1}(t) := z_1^{(n_1-1)}(t) = w_{1,n_1-2}'(t)$$

$$w_{1,n_1-1}'(t) = g_1 \left(t, z_1, w_{1,1}, \dots, w_{1,n_1-1}, z_2, z_2', \dots, z_2^{(n_2-1)}, \dots, z_m, z_m', \dots, z_m^{(n_m-1)} \right)$$

Deterministische Modelle mit Differentialgleichungen: Simulation gewöhnlicher Dgln (ODE)

Schreibt man diese erste Gleichung um (Reihenfolge der ersten Gleichungen umkehren), so ergibt sich:

$$z_1'(t) = w_{1,1}(t)$$

$$w_{1,1}'(t) = w_{1,2}(t)$$

$$w_{1,2}'(t) = w_{1,3}(t)$$

$$\vdots$$

$$w_{1,n_1-2}'(t) = w_{1,n_1-1}(t)$$

$$w_{1,n_1-1}'(t) = g_1 \left(t, z_1, w_{1,1}, \dots, w_{1,n_1-1}, z_2, z_2', \dots, z_2^{(n_2-1)}, \dots, z_m, z_m', \dots, z_m^{(n_m-1)} \right),$$

d.h. aus dieser 1. Differentialgleichung der n_1 . Ordnung werden n_1 Differentialgleichungen 1. Ordnung.

Deterministische Modelle mit Differentialgleichungen: Simulation gewöhnlicher Dgln (ODE)

Dies kann man natürlich für jede beliebige Gleichung in dem Differentialgleichungssystem durchführen. D.h. für die i -te Differentialgleichung ($i = 1, \dots, m$) ergibt sich nach Einsetzen aller Hilfsfunktionen in g_i :

$$z'_i(t) = w_{i,1}(t)$$

$$w'_{i,1}(t) = w_{i,2}(t)$$

$$w'_{i,2}(t) = w_{i,3}(t)$$

$$\vdots$$

$$w'_{i,n_i-2}(t) = w_{i,n_i-1}(t)$$

$$w'_{i,n_i-1}(t) = g_i(t, z_1, w_{1,1}, \dots, w_{1,n_1-1}, z_2, w_{2,1}, \dots, w_{2,n_2-1}, \dots, z_m, w_{m,1}, \dots, w_{m,n_m-1}) \quad .$$

Dadurch ist es möglich ein Differentialgleichungssystem mit höheren Ordnungen immer in ein großes Differentialgleichungssystem 1. Ordnung umzuformen. Konkret werden dann aus den m Gleichungen der vorherigen Definition $\sum_{j=1}^m n_j$ Gleichungen 1. Ordnung.

Deterministische Modelle mit Differentialgleichungen: Simulation gewöhnlicher Dgln (ODE)

Da diese Umformung immer funktioniert, müssen numerische Lösungsalgorithmen für nahezu alle Differentialgleichungen nur für Differentialgleichungssysteme 1. Ordnung erstellt werden.

Beispiel 3.1 (ODE–System 1. Ordnung (van der Pol))

Gegeben sei folgende Differentialgleichung:

$$z''(t) = (1 - z^2(t)) \cdot z'(t) - z(t) .$$

Wie sieht das zugehörige Differentialgleichungssystem 1. Ordnung aus?

$$\begin{aligned} z'(t) &= w_1(t) \\ w_1'(t) &= (1 - z^2(t)) \cdot w_1(t) - z(t) . \end{aligned}$$

Nach dem Lösen hat man Interesse an $z(t)$ und ignoriert die (Hilfs–)Lösung für $w_1(t)$.

Beispiel 3.2 (ODE–System 1. Ordnung)

Gegeben sei das Differentialgleichungssystem

$$z_1''(t) = -2 \cdot z_1'(t) + 3 \cdot z_2'(t) - 4 \cdot z_2(t) + 2 \cdot z_3(t)$$

$$z_2''(t) = 3 \cdot z_2'(t) - 4 \cdot z_3'(t) + 2 \cdot z_3(t) - 3 \cdot z_1(t)$$

$$z_3''(t) = -4 \cdot z_3'(t) + 2 \cdot z_1'(t) - 3 \cdot z_1(t) + 4 \cdot z_2(t) .$$

Wie sieht das zugehörige Differentialgleichungssystem 1. Ordnung aus?

$$z_1'(t) = w_{1,1}(t)$$

$$w_{1,1}'(t) = -2 \cdot w_{1,1}(t) + 3 \cdot w_{2,1}(t) - 4 \cdot z_2(t) + 2 \cdot z_3(t)$$

$$z_2'(t) = w_{2,1}(t)$$

$$w_{2,1}'(t) = 3 \cdot w_{2,1}(t) - 4 \cdot w_{3,1}(t) + 2 \cdot z_3(t) - 3 \cdot z_1(t)$$

$$z_3'(t) = w_{3,1}(t)$$

$$w_{3,1}'(t) = -4 \cdot w_{3,1}(t) + 2 \cdot w_{1,1}(t) - 3 \cdot z_1(t) + 4 \cdot z_2(t) .$$

Nach dem Lösen hat man Interesse an $z_1(t)$, $z_2(t)$ und $z_3(t)$ und ignoriert die (Hilfs–)Lösung für $w_{1,1}(t)$, $w_{2,1}(t)$ und $w_{3,1}(t)$.

Beispiel 3.3 (ODE–System 1. Ordnung)

Gegeben sei das Differentialgleichungssystem ($a, b \in \mathbb{R}$)

$$z_1^{(3)}(t) = z_1'(t) \cdot z_2''(t) + a \cdot z_1(t)$$

$$z_2^{(4)}(t) = z_1''(t) \cdot z_2'(t) + b .$$

Wie sieht das zugehörige Differentialgleichungssystem 1. Ordnung aus?

Deterministische Modelle mit Differentialgleichungen: Simulation gewöhnlicher Dgln (ODE)

Deshalb übergibt man typischerweise an ODE–Solver (Algorithmen zur numerischen/näherungsweise Lösung von ODEs) immer eine explizite Funktion, welche die Informationen über das Differentialgleichungssystem 1. Ordnung enthält.

Beispielsweise sieht diese in Beispiel 3.1 wie folgt aus:

```
function dydt = f(t,y)           Matlab/Octave
    dydt = zeros(2,1);
    dydt(1) = y(2);
    dydt(2) = (1-y(1)^2)*y(2) - y(1);
end
```

Wobei die gesuchte Funktion $z(t)$, dann im ersten Eintrag der Lösung „ $y(:,1)$ “ zu finden sein wird.

Im Beispiel 3.2 sieht die dazugehörige Funktion dann wie folgt aus:

Matlab/Octave

```
function dydt = f(t,y)
    dydt = zeros(6,1);
    dydt(1) = y(2);
    dydt(2) = -2*y(2) + 3*y(4) - 4*y(3) + 2*y(5);
    dydt(3) = y(4);
    dydt(4) = 3*y(4) - 4*y(6) + 2*y(5) - 3*y(1);
    dydt(5) = y(6);
    dydt(6) = -4*y(6) + 2*y(2) - 3*y(1) + 4*y(3);
end
```

Wobei die gesuchten Funktionen $z_1(t)$, $z_2(t)$ und $z_3(t)$, dann im ersten, dritten und fünften Eintrag der Lösung „y(:,1)“, „y(:,3)“, „y(:,5)“ zu finden sein wird.

Deterministische Modelle mit Differentialgleichungen: Simulation gewöhnlicher Dgln (ODE)

Wie sieht dann die zugehörige Funktion für das Beispiel 3.3 aus?

Deterministische Modelle mit Differentialgleichungen: Simulation gewöhnlicher Dgln (ODE)

Hat man nun diese Darstellung der Differentialgleichung erreicht, dann muss man nur mehr

- diese Systemfunktion,
- den Startzustand des Systems „ y “ (genau so viel Einträge wie „ y “) zum Startzeitpunkt t_0 , als einfachen Vektor angeben, und
- die Zeitspanne $[t_0, t_E]$ für welche die Lösung berechnet werden soll an den Lösungsalgorithmus übergeben.

Bemerkung 3.1 (Startzustand des ODE-Systems)

Meist will man die dynamische Entwicklung der Funktionen von einem Startzustand zum Startzeitpunkt t_0 des Systems beginnend bis zu einem Endzeitpunkt t_E berechnen. Die Suche nach solchen Lösungen nennt man *Anfangswertprobleme* und wir werden uns in diesem Abschnitt auch nur auf solche beziehen. Eine tiefgreifende Erkenntnis der ODE-Theorie ist, dass es für Anfangswertprobleme immer eine Lösung in einem Intervall $[t_0, t_E]$ existiert (→ *Satz von Picard-Lindelöf*) – es kann jedoch sein, dass man den Endzeitpunkt t_E je nach Aufgabe geschickt wählen muss.

Deterministische Modelle mit Differentialgleichungen: Simulation gewöhnlicher Dgln (ODE)

Der finale Aufruf zur Lösung eines Systems lautet dann beispielsweise

Matlab/Octave

```
t_0 = 0;  
t_E = 1;  
t_span = [t_0, t_E];  
y_0 = [0,0,0,...];  
[t,y] = ode45(f, t_span, y_0);
```

Die zugehörige Lösung ist dann eine Matrix „y“, welche zu jedem Zeitpunkt „t“ die Lösungseinträge enthält.

Deterministische Modelle mit Differentialgleichungen: Simulation gewöhnlicher Dgln (ODE)

Ein Aufruf könnte hierbei beispielsweise zu Beispiel 3.1 sein:

Matlab/Octave

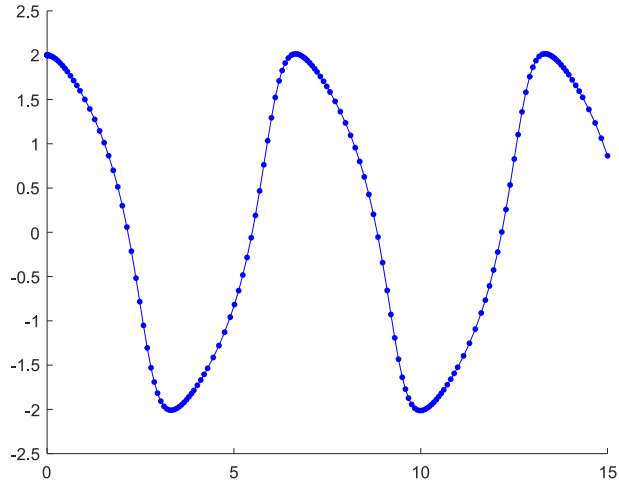
```
% calculate
t_0 = 0;
t_E = 15;
t_span = [t_0,t_E];
y_0 = [2,0];
[t,y] = ode45(@f,t_span,y_0);

% plotting
figure('color','white');
hold on
plot(t,y(:,1),'-b');
plot(t,y(:,1),'b','MarkerSize',10);

% define system function
function dydt = f(t,y)
    dydt = zeros(2,1);
    dydt(1) = y(2);
    dydt(2) = (1-y(1)^2)*y(2) - y(1);
end
```

Deterministische Modelle mit Differentialgleichungen: Simulation gewöhnlicher Dgln (ODE)

Was folgenden Plot liefert:



Deterministische Modelle mit Differentialgleichungen: Simulation gewöhnlicher Dgln (ODE)

Ein Aufruf könnte hierbei beispielsweise zu Beispiel 3.2 sein:

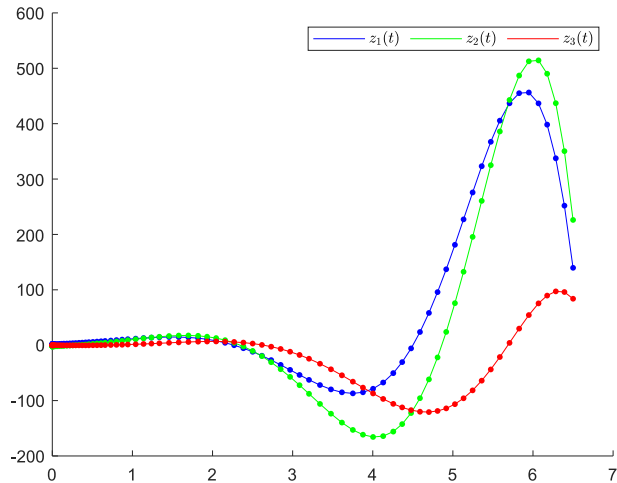
```
% calculate
t_0 = 0;
t_E = 6.5;
t_span = [t_0,t_E];
y_0 = [2,5,-2,5,0,-1];
[t,y] = ode45(@f,t_span,y_0);

% plotting
figure('color','white');
hold on
plot(t,y(:,1),'-b');
plot(t,y(:,3),'-g');
plot(t,y(:,5),'-r');
plot(t,y(:,1),'.b','MarkerSize',10);
plot(t,y(:,3),'.g','MarkerSize',10);
plot(t,y(:,5),'.r','MarkerSize',10);
legend('$z_1(t)$','$z_2(t)$','$z_3(t)$','Interpreter','LaTeX','Orientation','horizontal')
```

mit der zuvor definierten Systemfunktion.

Deterministische Modelle mit Differentialgleichungen: Simulation gewöhnlicher Dgln (ODE)

Was folgenden Plot liefert:



Bemerkung 3.2 (ODE–Solver in Python)

Ein ODE–Solver in Python findet man im Paket `scipy` unter `scipy.integrate.solve_ivp` (kurz für „solve initial value problem“) mit einer intuitiv ähnlichen Verwendungsart.

Wir betrachten im Folgenden

- einfache *Netzwerke/Graphen* in denen *Ströme* $I_k(t)$ entlang von Gefällen $u_k(t)$ fließen.
- insbesondere das weit verbreitete Beispiel der elektronischen Schaltungen, welche aus *in Reihe* oder *parallel geschalteten* passiven Bauelementen, wie *Widerständen*, *Spulen* und *Kondensatoren* aufgebaut sind.

Es gelten folgende Erhaltungsregeln in dem Netzwerk:

$$\sum_{k=1}^n I_k(t) = 0 \quad \text{Knotenregel für alle } n \text{ Ströme die durch einen Knoten fließen}$$

$$\sum_{k=1}^n u_k(t) = 0 \quad \text{Maschenregel für alle } n \text{ Gefälle in einer Masche.}$$

Dies sind die *Kirchhoffschen Regeln*, und besagen, dass

- in einem Knoten keine Ströme verschwinden oder neu entstehen können, und
- geht man im Netzwerk im Kreis (Masche), man wieder auf das Ausgangsniveau der Gefälle kommt.

Deterministische Modelle mit Differentialgleichungen: *Elektrische Schwingkreise und Antennentechnik

In elektrischen Schaltungen gelten insbesondere für diese elementaren Bauteile folgende bekannten Grundregeln zwischen Stromfluss $i(t)$ und dem Spannungsabfall $u(t)$

$$u(t) = R \cdot I(t) \qquad \text{Ohm'sches Gesetz – Widerstand } R$$

$$u(t) = \frac{1}{C} \int_{-\infty}^t I(\tau) \, d\tau \qquad \text{Kondensator mit Kapazität } C$$

$$u(t) = L \frac{d}{dt} I(t) \qquad \text{Spule mit Induktivität } L$$

Im Folgenden betrachten wir die für das Verständnis der **Sende– und Empfangsantennen** (Dipolantennen) wichtigsten elektrischen Schaltung: den *Reihenschwingkreis* und den *Parallelschwingkreis*. Die realen Dipolantennen sind dabei praktisch sehr einfach realisierbar und stellen in unserer vereinfachten Darstellung einfach nur einen speziell geformten Kondensator dar.

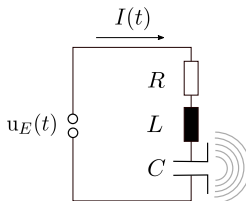
Warum sind Sende– und Empfangsantennen so wichtig in unserem Alltag?

Beispiel 3.4 (Reihenschwingkreis)

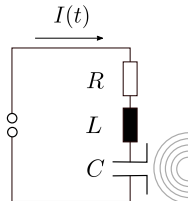
Der Reihenschwingkreis hat folgendes Schaltungsbild rechts.

Wir interessieren uns hier für den fließenden Strom $I(t)$, da er für alle Bauteile gleich ist, jedoch sich zeitlich ändert.

Sender:



Empfänger:



Aus der *Kirchhoffschen Maschenregel* ergibt sich:

$$u_R(t) + u_L(t) + u_C(t) = u_E(t)$$

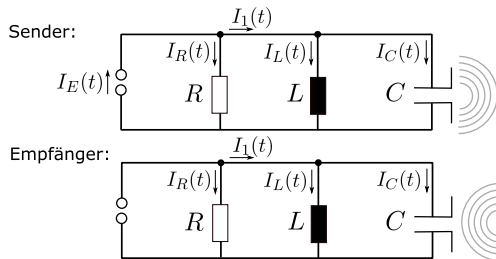
$$\Leftrightarrow R \cdot I(t) + L \cdot \dot{I}(t) + u_C(0) + \frac{1}{C} \cdot \int_0^t I(\tau) \, d\tau = u_E(t)$$

$$\Leftrightarrow \boxed{\ddot{I}(t) + \frac{R}{L} \cdot \dot{I}(t) + \frac{1}{LC} \cdot I(t) = \frac{1}{L} \dot{u}_E(t)} .$$

Beispiel 3.5 (Parallelschwingkreis)

Der Parallelschwingkreis hat folgendes Schaltungsbild rechts.

Wir interessieren uns hier für die Spannung $u(t)$, da diese für alle Bauteile gleich ist, jedoch sich zeitlich ändert.



Aus der *Kirchhoffschen Knotenregel* ergibt sich:

$$\left. \begin{aligned} I_E(t) &= I_R(t) + I_1(t) \\ I_1(t) &= I_L(t) + I_C(t) \end{aligned} \right\} \Rightarrow I_E(t) = I_R(t) + I_L(t) + I_C(t)$$
$$\Rightarrow \dot{I}_E(t) = \dot{I}_R(t) + \dot{I}_L(t) + \dot{I}_C(t)$$

Aus der *Kirchhoffschen Maschenregel* ergibt sich:

$$u_R(t) = u_L(t) = u_C(t) =: u(t)$$

Setzen wir nun die bekannten Verhältnisse zwischen Spannungen und den passenden Strömen für jedes Bauteil ein, erhalten wir:

$$u(t) = u_R(t) = R I_R(t) \Rightarrow \dot{I}_R(t) = \frac{1}{R} \dot{u}(t)$$

$$u(t) = u_L(t) = L \dot{I}_L(t) \Rightarrow \dot{I}_L(t) = \frac{1}{L} u(t)$$

$$u(t) = u_C(t) = \frac{1}{C} \int_{-\infty}^t I_C(\tau) d\tau \Rightarrow \dot{I}_C(t) = C \ddot{u}(t)$$

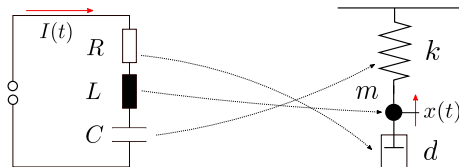
Setzt man dies nun in die Knotengleichung ein, so ergibt sich:

$$\begin{aligned} \dot{I}_R(t) + \dot{I}_L(t) + \dot{I}_C(t) &= \dot{I}_E(t) \\ \Leftrightarrow \frac{1}{R} \dot{u}(t) + \frac{1}{L} u(t) + C \ddot{u}(t) &= \dot{I}_E(t) \\ \Leftrightarrow \ddot{u}(t) + \frac{1}{RC} \cdot \dot{u}(t) + \frac{1}{LC} \cdot u(t) &= \frac{1}{C} \dot{I}_E(t) \end{aligned}$$

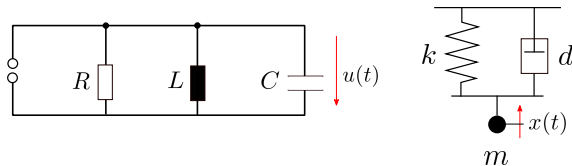
Bemerkung 3.3 (Elektrischer Schwingkreis und Mechanische Schwingung)

Es gibt eine sehr starke Ähnlichkeit zwischen elektrischen Schwingkreisen und mechanischen Schwingungen. So verwendet man zwar andere Konstanten, jedoch ist die Dynamik (inkl. Differentialgleichungstyp) die gleiche, siehe Vergleichsschaltbilder:

Reihenordnung:



Parallelanordnung:



Im Allgemeinen ergeben sich für solche elektronischen Schaltungen (egal wie kompliziert sie zusammengeschaltet werden) immer folgende Form von linearen Differentialgleichungen

$$a_0 x(t) + a_1 \dot{x}(t) + a_2 \ddot{x}(t) + \dots = b_0 y(t) + b_1 \dot{y}(t) + b_2 \ddot{y}(t) + \dots ,$$

mit konstanten (aber für viele Bauelemente durchaus sehr komplizierten) Koeffizienten $a_k, b_k \in \mathbb{R}$ mit der unbekannten gesuchten Funktion $x(t)$ und einer Anregung $y(t)$.

Bemerkung 3.4 (Lineare ODE mit konstanten Koeffizienten)

- Die *Modellbildung* ist mit der Bestimmung der ODE im Wesentlichen abgeschlossen. Lösungsberechnungen für verschiedene Parameter dieser ODE, sind dann *Simulationen*.
- Für solche Gleichungen benötigen wir keinen allgemeinen numerischen ODE-Solver, sondern können die Lösung direkt berechnen (bekannt aus der Theorie der ODE).
- Im Folgenden wollen wir uns auf eine einfache Untermenge dieser Gleichungen konzentrieren und am Beispiel des bekannten *Reihenschwingungskreises* näher anschauen.

Definition 3.3 (Lineare ODE 2. Ordnung mit konstanten Koeffizienten)

Wir nennen eine Gleichung für alle $t \in I \subset \mathbb{R}$

$$\ddot{x}(t) + a_1 \dot{x}(t) + a_0 x(t) = y(t) \quad (1)$$

eine gewöhnliche *lineare Differentialgleichung 2. Ordnung mit konstanten Koeffizienten*, mit

$x(t) : I \subset \mathbb{R} \rightarrow \mathbb{R}$, der gesuchten Funktion,

$a_1, a_0 \in \mathbb{R}$, reelle Koeffizienten,

$y(t) : I \rightarrow \mathbb{R}$ eine bekannte Anregungs-/Störfunktion.

Weiter nennen wir die Differentialgleichung (1) **homogen**, wenn $y(t) = 0$ für alle $t \in I$ gilt, d.h.

$$\ddot{x}(t) + a_1 \dot{x}(t) + a_0 x(t) = 0. \quad (2)$$

Ansonsten nennen wir die Differentialgleichung **inhomogen**.

Definition 3.4 (Anfangswertproblem)

Wir nennen eine Differentialgleichung (1) oder (2) ein **Anfangswertproblem (AWP)**, falls zusätzlich folgende Anfangswerte für $x(t)$ als bekannt vorausgesetzt werden:

$$x(0) = \alpha, \quad \dot{x}(0) = \beta, \quad \text{mit vorgegebenen } \alpha, \beta \in \mathbb{R}.$$

Wie sieht die allgemeine Lösung einer inhomogenen Differentialgleichung aus? Der wichtigste Baustein hierfür ist folgender Satz:

Satz 3.1 (Die Additivität der homogenen zur inhomogenen Lösung)

Sei $x_p(t)$ (*partikuläre/inhomogene Lösung*) eine Lösung der inhomogenen Differentialgleichung (1), so kann man zu dieser jede beliebige homogene Lösung $x_h(t)$ hinzuaddieren

$$z(t) := x_p(t) + x_h(t) ,$$

und erhält dann immer noch eine Lösung der inhomogenen Differentialgleichung.

Beweis:

$$\begin{aligned} \ddot{z}(t) + a_1 \dot{z}(t) + a_0 z(t) &= (\ddot{x}_p(t) + \ddot{x}_h(t)) + a_1 (\dot{x}_p(t) + \dot{x}_h(t)) + a_0 (x_p(t) + x_h(t)) \\ &= \underbrace{(\ddot{x}_p(t) + a_1 \dot{x}_p(t) + a_0 x_p(t))}_{=y(t)} + \underbrace{(\ddot{x}_h(t) + a_1 \dot{x}_h(t) + a_0 x_h(t))}_{=0} = y(t) \end{aligned}$$

Bemerkung 3.5 (Die Additivität der homogenen zur inhomogenen Lösung)

Allgemein gilt: Addieren wir *alle Lösungen der homogenen ODE* auf (irgend)eine *Lösung der inhomogenen ODE*, dann haben wir *alle Lösungen der inhomogenen ODE*.

Betrachten wir nun zuerst die **allgemeine Lösung der homogenen ODE**, d.h. wir legen im Schwingkreis *keine* externe Strom/Spannungsquelle an (bspw. Empfangsantenne):

Satz 3.2 (Allgemeine Gestalt der Lösungsmenge eines AWP's mit homogener ODE)

Jede Lösung eines AWP's mit linearer homogener ODE 2. Ordnung mit konstanten Koeffizienten lässt sich eindeutig als Linearkombination

$$x(t) := c_1 \cdot x_{h_1}(t) + c_2 \cdot x_{h_2}(t)$$

darstellen, wobei $c_1, c_2 \in \mathbb{R}$ passend gewählt werden müssen.

Man nennt $x_{h_1}(t), x_{h_2}(t)$ *Basislösungen* der homogenen linearen Differentialgleichung.

Ansatz:

$$x(t) = e^{\lambda t} \quad \lambda \text{ unbekannter Parameter}$$

$$\Rightarrow \dot{x}(t) = \lambda \cdot e^{\lambda t}, \quad \ddot{x}(t) = \lambda^2 \cdot e^{\lambda t}.$$

Setzt man nun diesen Ansatz in die ODE $\ddot{x}(t) + a_1 \dot{x}(t) + a_0 x(t) = 0$ ein:

$$\begin{aligned} \lambda^2 \cdot e^{\lambda t} + a_1 \lambda \cdot e^{\lambda t} + a_0 \cdot e^{\lambda t} &= 0 \\ e^{\lambda t} (\lambda^2 + a_1 \lambda + a_0) &= 0. \end{aligned}$$

Diese Gleichung ist insbesondere dann erfüllt, wenn λ eine Nullstelle des quadratischen Polynoms

$$\lambda^2 + a_1 \lambda + a_0 = 0$$

ist. Man nennt dieses Polynom auch das **charakteristische Polynom**. Aus der Algebra ist bekannt, dass das quadratische Polynom immer folgende Lösungen hat

$$\lambda_{1,2} = -\frac{a_1}{2} \pm \sqrt{\frac{a_1^2}{4} - a_0}.$$

Satz 3.3 (Konkrete Gestalt der Lösungsmenge von AWP's mit homogener ODE)

1. Fall: $\frac{a_1^2}{4} - a_0 > 0 \Rightarrow \lambda_1 \neq \lambda_2 \text{ reell. Dann gilt:}$

$$x_{h_1} = e^{\lambda_1 t}$$

$$x_{h_2} = e^{\lambda_2 t}$$

2. Fall: $\frac{a_1^2}{4} - a_0 = 0 \Rightarrow \lambda_1 = \lambda_2 =: \lambda \text{ reell. Dann gilt:}$

$$x_{h_1} = e^{\lambda t}$$

$$x_{h_2} = t \cdot e^{\lambda t}$$

3. Fall: $\frac{a_1^2}{4} - a_0 < 0 \Rightarrow \lambda_1 \neq \lambda_2, \text{ komplex konjugiert. Dann gilt:}$

$$x_{h_1} = e^{-\frac{a_1}{2} t} \cdot \cos \left(\sqrt{a_0 - \frac{a_1^2}{4}} \cdot t \right)$$

$$x_{h_2} = e^{-\frac{a_1}{2} t} \cdot \sin \left(\sqrt{a_0 - \frac{a_1^2}{4}} \cdot t \right)$$

Beispiel 3.6 (Fortführung Reihenschwingkreis)

Konkret haben wir im Reihenschwingkreis:

$$\Rightarrow a_1 = \frac{R}{L}, a_0 = \frac{1}{LC}.$$

Gilt $R = L = C = 1$, so ergibt sich:

$$a_1 = 1, a_0 = 1 \Rightarrow \frac{a_1^2}{4} - a_0 = -\frac{3}{4} < 0 \quad (3. \text{ Fall})$$

und somit nach Satz 2.3 die allgemeine Lösung

$$I(t) = e^{-\frac{1}{2}t} \cdot \left(c_1 \cdot \cos\left(\frac{\sqrt{3}}{2} \cdot t\right) + c_2 \cdot \sin\left(\frac{\sqrt{3}}{2} \cdot t\right) \right).$$

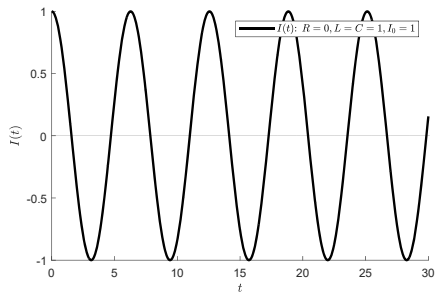
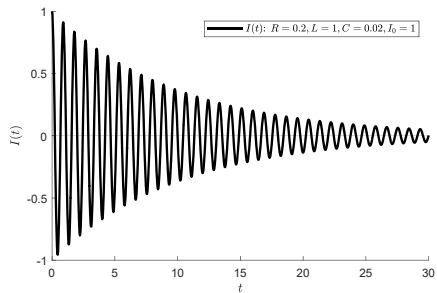
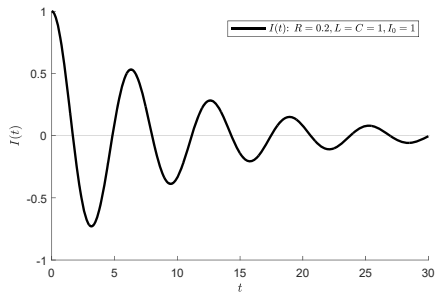
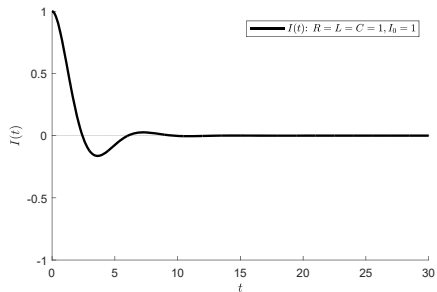
Zur Lösung des Anfangswertproblems $I(0) = I_0 \in \mathbb{R}$ und $\dot{I}(0) = 0$:

1. Einsetzen von $t = 0$ und Gleichsetzen mit I_0
2. Ableiten von $I(t)$ und ebenso Einsetzen von $t = 0$ und Gleichsetzen mit 0

$$\Rightarrow c_1 = I_0, c_2 = I_0 \cdot \frac{1}{\sqrt{3}}$$

$$\Rightarrow I(t) = I_0 \cdot e^{-\frac{1}{2}t} \cdot \left(\cos\left(\frac{\sqrt{3}}{2} \cdot t\right) + \frac{1}{\sqrt{3}} \cdot \sin\left(\frac{\sqrt{3}}{2} \cdot t\right) \right)$$

Deterministische Modelle mit Differentialgleichungen: *Elektrische Schwingkreise und Antennentechnik



Wir suchen nun **eine Lösung der inhomogenen ODE**, d.h. wir legen jetzt an den Schwingkreis eine Strom/Spannungsquelle an (bspw. Sendeantenne).

Wie findet man (irgend)eine inhomogene Lösung der ODE?

Die Antwort: Man „probiert“ mit Ansatzfunktionen, die ähnlich zu der inhomogenen Anregung $y(t)$ ist!

Ein Auszug von typischen Ansatzfunktionen bzgl. Anregungen $y(t)$ wären bspw.:

$y(t)$	Lösungsansatz $x_p(t)$
$y(t) = e^{\omega t}$	$x_p(t) = \begin{cases} A \cdot e^{\omega t}, & \omega \text{ keine Nst. des charakt. Polyn.} \\ A \cdot t \cdot e^{\omega t}, & \omega \text{ einfache Nst. des charakt. Polyn.} \\ A \cdot t^2 \cdot e^{\omega t}, & \omega \text{ zweifache Nst. des charakt. Polyn.} \end{cases}$
$y(t) = \cos(\omega t)$ $y(t) = \sin(\omega t)$	$x_p(t) = \begin{cases} A \cdot \cos(\omega t) + B \cdot \sin(\omega t), & j\omega \text{ keine Nst. des charakt. Polyn.} \\ t \cdot [A \cdot \cos(\omega t) + B \cdot \sin(\omega t)], & j\omega \text{ Nst. des charakt. Polyn.} \end{cases}$

Die dabei neu eingeführten Parameter $A, B, \dots$ etc. müssen dann einfach durch Koeffizientenvergleich mit der ganz konkreten Anregungsfunktion bestimmt werden.

Führen wir im Reihenschwingkreis folgende Inhomogenität ein:

$$y(t) = u_E(t) := U_0 \cdot \sin(\omega_1 t)$$
$$\Rightarrow \ddot{I}(t) + \frac{R}{L} \cdot \dot{I}(t) + \frac{1}{LC} \cdot I(t) = \frac{U_0 \omega_1}{L} \cos(\omega_1 t)$$

Gilt weiterhin: $R = 0, L = C = 1$ so ergibt sich

$$\ddot{I}(t) + I(t) = U_0 \omega_1 \cos(\omega_1 t)$$

Wir nehmen im Folgenden an für die Anregungsfrequenz (Tabelle): $|\omega_1| \neq 1$.

Damit vermeiden wir einen Konflikt mit den Nullstellen des charakt. Polynoms.

Aus der Tabelle der Ansatzfunktionen ergibt sich:

$$I_p(t) = A \cdot \cos(\omega_1 t) + B \cdot \sin(\omega_1 t) .$$

Setzt man diese Ansatzfunktion in die inhomogene Gleichung ein, ergibt sich

$$A (1 - \omega_1^2) \cdot \cos(\omega_1 t) + B (1 - \omega_1^2) \cdot \sin(\omega_1 t) = U_0 \omega_1 \cdot \cos(\omega_1 t) .$$

und mit dem Koeffizientenvergleich schließlich:

$$A (1 - \omega_1^2) = U_0 \omega_1 \Rightarrow A = \frac{U_0 \omega_1}{1 - \omega_1^2}$$

$$B (1 - \omega_1^2) = 0 \Rightarrow B = 0$$

Setzt man diese A, B in die Ansatzfunktion ein, so erhält man die partikuläre Lösung:

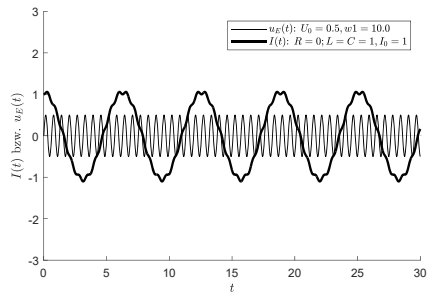
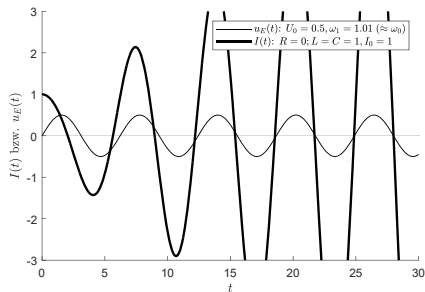
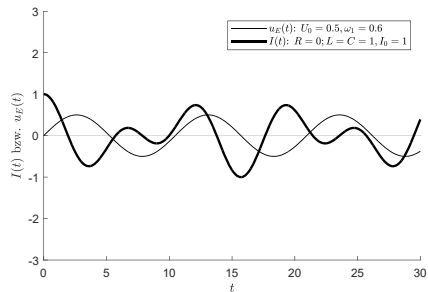
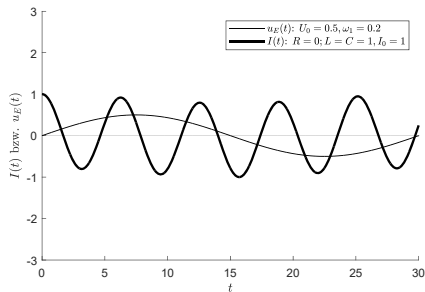
$$I_p(t) = \frac{U_0 \omega_1}{1 - \omega_1^2} \cdot \cos(\omega_1 t) .$$

Somit lautet die allgemeine Lösung der inhomogenen Differentialgleichung:

$$I(t) = I_p(t) + c_1 \cdot I_{h_1}(t) + c_2 \cdot I_{h_2}(t)$$
$$\Rightarrow \boxed{I(t) = \frac{U_0 \omega_1}{1 - \omega_1^2} \cdot \cos(\omega_1 t) + c_1 \cdot \cos(t) + c_2 \cdot \sin(t)},$$

mit den Unbekannten c_1, c_2 , die man weiter aus den Anfangswerten $I(0) = I_0$ und $\dot{I}(0) = 0$ bestimmt.

Deterministische Modelle mit Differentialgleichungen: *Elektrische Schwingkreise und Antennentechnik



Bemerkung 3.6 (Flüsse und Ströme / elektrische Schaltungen)

Das an dieser **Modellierung** ist bemerkenswert, dass man

- mit sehr elementaren Erhaltungsregeln (Kirchoff'sche Regeln) und den physikalischen Spannungsabfällen anfängt, und
- dabei (egal wie kompliziert die Bauteile zusammengeschaltet werden) immer bei dem im Prinzip gleichen mathematischen Objekt rauskommt: Der linearen ODE mit konstanten Koeffizienten, welche die Dynamik der Flüsse und Ströme im Netzwerk beschreibt.

Aus **Simulationssicht** ist spannend, dass

- man keinen numerischen Algorithmus benötigt, sondern wie am Beispiel des Reihenschwingkreises *auf dem Blatt* zu Lösungen kommen kann, und
- wir bei der Simulation dieses Modells mit unterschiedlichen Parametern praktisch das Phänomen der *Resonanz* als ein Aufschaukeln des dynamischen Systems bei spezieller Anregung kennen gelernt haben. Weiterhin ist die *Periodizität/Schwingung* der Lösung von besonderem Interesse, da diese nicht offensichtlich im Modell ersichtlich ist.

Deterministische Modelle mit Differentialgleichungen: Populationsmodelle: Räuber–Beute–Modelle

Eine weitere Anwendung in der Modellierung sind *Populationsmodelle*, welche sich aus der *Biologie* motivieren. Ziel ist es dabei die Veränderung von einer oder mehrerer Populationen über die Zeit zu beschreiben.

Das einfachste Beispiel einer ODE, um das Wachstum einer Population zu beschreiben ist das *exponentielle Wachstum*.

Beispiel 3.7 (Exponentielles Wachstum)

Ist die ODE der Gestalt

$$\dot{x} = \beta x - \alpha x = (\beta - \alpha) x \quad \text{mit } \alpha, \beta > 0 ,$$

so beschreibt die Lösung ein exponentielles Wachstum, mit

→ $x(t)$ ist die gesuchte Populationsgröße

→ β ist die Zuwachsrage (z.B. Geburt)

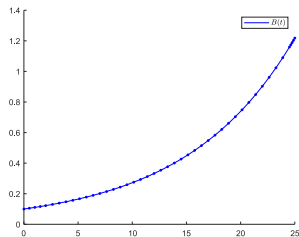
→ α die Schwundrate (z.B. Tod)

Die Lösung ist dabei offensichtlich

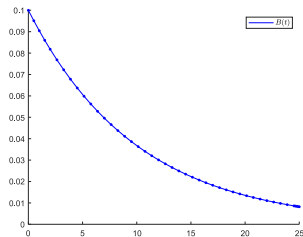
$$x(t) = c \cdot e^{(\beta - \alpha) t} \quad \text{mit } c > 0 .$$

Deterministische Modelle mit Differentialgleichungen: Populationsmodelle: Räuber–Beute–Modelle

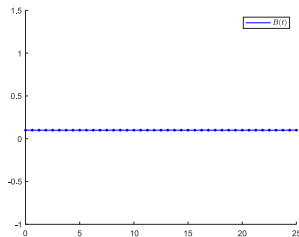
Parameterstudie:



$$\beta > \alpha$$



$$\beta < \alpha$$



$$\beta = \alpha$$

Deterministische Modelle mit Differentialgleichungen: Populationsmodelle: Räuber–Beute–Modelle

Hat man zwei Populationen (Räuber $R(t)$ und Beute $B(t)$) so kann man dies mit zwei *gekoppelten ODEs* beschreiben:

Beispiel 3.8 (Lotka–Volterra)

Das Lotka–Volterra (Räuber-Beute) ODE–System ist mit

$$\dot{B} = \beta B - \gamma_1 B R$$

$$\dot{R} = \gamma_2 B R - \alpha R$$

beschrieben, wobei α , β , γ_1 und $\gamma_2 > 0$, mit

- β drückt die effektive Wachstumsrate der Beute aus
- α drückt die Todesrate der Räuber aus, wenn Sie keine Beute bekommen
- „ BR “ drückt die Begegnungshäufigkeit von Beute und Räuber aus (*Warum?*)
 - γ_1 „gefressen werden“ der Beute wenn man sich trifft
 - γ_2 „Vermehrungsrate der Räuber“ nach Beute-Essen

Der dazugehörige Code könnte bspw. wie folgt aussehen:

```
Matlab/Octave
t_0 = 0; t_E = 25; t_span = [t_0,t_E];
y_0 = [2,1];

[t,y] = ode45(@f,t_span,y_0);

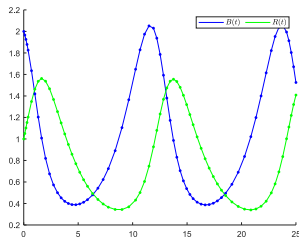
% plotting
figure('color','white');
hold on
plot(t,y(:,1),'-b');
plot(t,y(:,2),'-g');
legend('$B(t)$','$R(t)$','Interpreter','LaTeX','Orientation','horizontal')

% define system function
function dydt = f(t,y)
    beta = 0.6;
    gamma_1 = 0.75;
    gamma_2 = 0.5;
    alpha_1 = 0.5;

    dydt = zeros(2,1);
    dydt(1) = beta*(y(1))-gamma_1*y(1)*y(2);
    dydt(2) = gamma_2*y(1)*y(2) - alpha_1*y(2);
end
```

Deterministische Modelle mit Differentialgleichungen: Populationsmodelle: Räuber–Beute–Modelle

Parameterstudie:



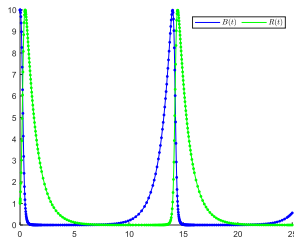
$$B(0) = 2, R(0) = 1$$

$$\beta = 0.6$$

$$\gamma_1 = 0.75$$

$$\gamma_2 = 0.5$$

$$\alpha = 0.5$$



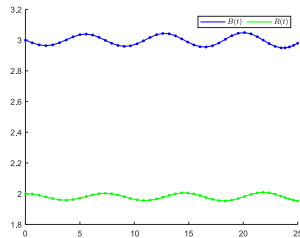
$$B(0) = 10, R(0) = 1$$

$$\beta = 1$$

$$\gamma_1 = 1$$

$$\gamma_2 = 1$$

$$\alpha = 1$$



$$B(0) = 3, R(0) = 2$$

$$\beta = 0.99$$

$$\gamma_1 = 0.5$$

$$\gamma_2 = 0.25$$

$$\alpha = 0.75$$

Eine Erweiterung des Modells erfährt man, wenn man eine zusätzliche „Fangquote“ der Räuber als einen äußeren Einfluss hinzufügt:

Beispiel 3.9 (Lotka–Volterra mit Fangquote)

Das Lotka–Volterra (Räuber–Beute) ODE–System mit *Fangquote der Räuber* ist mit

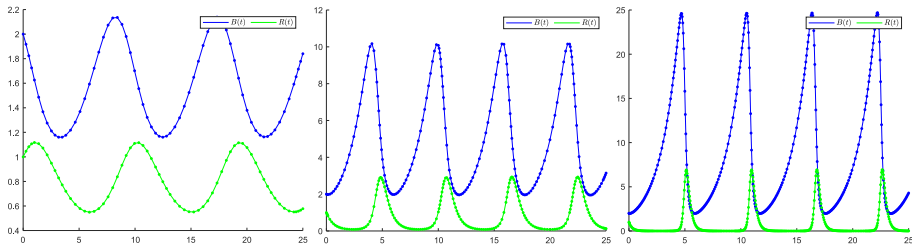
$$\begin{aligned}\dot{B} &= \beta B - \gamma_1 B R \\ \dot{R} &= \gamma_2 B R - \alpha R \boxed{-\eta R}\end{aligned}$$

beschrieben, wobei $\alpha, \beta, \gamma_1, \gamma_2$ und $\eta > 0$, mit

- η stellt die Fangquote der Räuber–Population durch einen äußeren Einfluss dar
- Bsp.: „Lachs–Bären–Verhältnis“ mit der Berücksichtigung von Bärenjagd durch den Menschen
- Da „ $-\alpha R - \eta R = -(\alpha + \eta) R$ “, kann man auch $\alpha + \eta$ als eine *neue, kombinierte Todesrate* der Räuber–Population zusammenfassen, welche tendenziell zu einer geringeren Räuber–Population führt, und daher im Gegenzug zu höheren Beute–Populationen.

Deterministische Modelle mit Differentialgleichungen: Populationsmodelle: Räuber–Beute–Modelle

Parameterstudie:



$$B(0) = 2, R(0) = 1$$

$$\beta = 0.6$$

$$\gamma_1 = 0.75$$

$$\gamma_2 = 0.5$$

$$\alpha = 0.5$$

$$\eta = 0.3$$

$$B(0) = 2, R(0) = 1$$

$$\beta = 0.6$$

$$\gamma_1 = 0.75$$

$$\gamma_2 = 0.5$$

$$\alpha = 0.5$$

$$\eta = 2$$

$$B(0) = 2, R(0) = 1$$

$$\beta = 0.6$$

$$\gamma_1 = 0.75$$

$$\gamma_2 = 0.5$$

$$\alpha = 0.5$$

$$\eta = 4$$

Deterministische Modelle mit Differentialgleichungen: Populationsmodelle: Räuber–Beute–Modelle

Hier ergibt sich jedoch ein „Modellierungsproblem“:

Gäbe es eine **sehr hohe Fangquote** („Ausrottung der Räuber-Population durch den Menschen“), so könnte sich die Beute **beliebig vermehren!**

→ *unrealistisch*, da es dennoch nur begrenzten Lebensraum für die Beute gibt.

Deshalb benötigt man zusätzlich eine maximale Schranke (bzw. Kapazität) für die Anzahl der Beutetiere

→ **Erinnerung:** *logistisches Wachstum* für die Vermehrung der Beutetiere:

Aus dem Zuwachs βB wird $\beta B (K - B)$, wobei K der Kapazität der Beutetiere B entspricht.

Dies führt zu folgenden Dgln:

Beispiel 3.10 (Lotka–Volterra mit Fangquote und max. Beute)

Das Lotka–Volterra (Räuber–Beute) ODE–System mit *Fangquote der Räuber* und einer *maximalen Kapazität der Beute–Population* sind mit

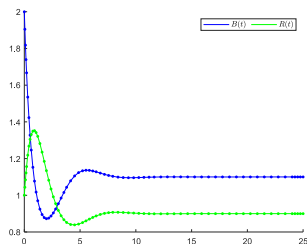
$$\begin{aligned}\dot{B} &= \boxed{\beta B (K - B)} - \gamma_1 B R \\ \dot{R} &= \gamma_2 B R - \alpha R - \eta R\end{aligned}$$

beschrieben, wobei $\alpha, \beta, \gamma_1, \gamma_2$ und $\eta > 0$

→ K entspricht der *maximalen Kapazität* der Anzahl der Beutetiere

Deterministische Modelle mit Differentialgleichungen: Populationsmodelle: Räuber–Beute–Modelle

Parameterstudie:



$$B(0) = 2, R(0) = 1$$

$$\beta = 1$$

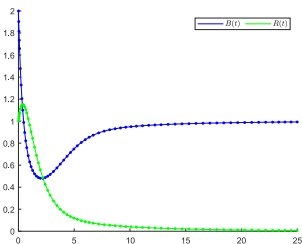
$$K = 2$$

$$\gamma_1 = 1$$

$$\gamma_2 = 1$$

$$\alpha = 1$$

$$\eta = 0.1$$



$$B(0) = 2, R(0) = 1$$

$$\beta = 1$$

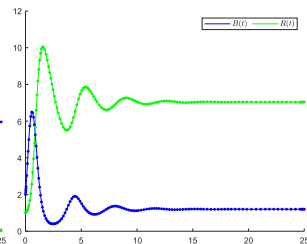
$$K = 1$$

$$\gamma_1 = 1$$

$$\gamma_2 = 1$$

$$\alpha = 1$$

$$\eta = 0.1$$



$$B(0) = 2, R(0) = 1$$

$$\beta = 0.6$$

$$K = 10$$

$$\gamma_1 = 0.75$$

$$\gamma_2 = 0.5$$

$$\alpha = 0.5$$

$$\eta = 0.1$$

Deterministische Modelle mit Differentialgleichungen: Populationsmodelle: Räuber–Beute–Modelle

Offensichtlich kommt es zu Gleichgewichtslösungen zwischen den Populationen. Kann man diese auch direkt berechnen?

Ja! Was zeichnet das Gleichgewicht aus?

Es ändert sich nichts mehr an den Populationen, d.h. $\dot{B} = 0$ und $\dot{R} = 0$. In der Gleichung führt dies zum nichtlinearen Gleichungssystem

$$0 = \beta B (K - B) - \gamma_1 B R$$

$$0 = \gamma_2 B R - \alpha R - \eta R$$

was nach ein paar Umformungen bspw. die Lösungen (*Vollziehen Sie das selbst nach!*)

$$\text{Für } \frac{\alpha + \eta}{\gamma_2} < K : \begin{pmatrix} B \\ R \end{pmatrix} = \begin{pmatrix} \frac{\alpha + \eta}{\gamma_2} \\ \frac{\beta}{\gamma_1} \left(K - \frac{\alpha + \eta}{\gamma_2} \right) \end{pmatrix}, \text{ sonst } \begin{pmatrix} B \\ R \end{pmatrix} = \begin{pmatrix} K \\ 0 \end{pmatrix}$$

hat. Man muss jedoch beachten, dass das Lösen von nichtlinearen Gleichungssystemen im Allgemeinen sehr anspruchsvoll sein kann.

Deterministische Modelle mit Differentialgleichungen: Populationsmodelle: Räuber–Beute–Modelle

Mehrere Populationen, bspw. drei, wobei für die Population M die Räuber R die Beute sind, werden beschrieben mit

Beispiel 3.11 (Lotka–Volterra für drei Populationen mit Fangquoten und max. Beute)

Das Lotka–Volterra (Räuber–Beute) ODE–System mit *Fangquoten der Räuber* und einer *maximalen Kapazität der Beute-Population* für drei Populationen sind mit

$$\dot{B} = \beta B (K - B) - \gamma_1 B R$$

$$\dot{R} = \gamma_2 B R - \gamma_3 M R - \alpha_1 R - \eta_1 R$$

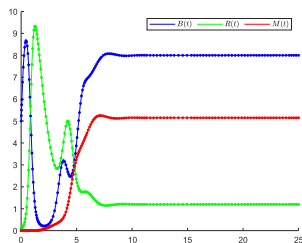
$$\dot{M} = \gamma_4 M R - \alpha_2 M - \eta_2 M$$

beschrieben, wobei $\alpha_1, \alpha_2, \beta, \gamma_1, \gamma_2, \gamma_3, \gamma_4, \eta_1$ und $\eta_2 > 0$.

→ K entspricht der *maximalen Kapazität* der Anzahl der Beutetiere

Deterministische Modelle mit Differentialgleichungen: Populationsmodelle: Räuber–Beute–Modelle

Parameterstudie:



$$B(0) = 5, R(0) = 0.1, \\ M(0) = 0.001$$

$$\beta = 0.6$$

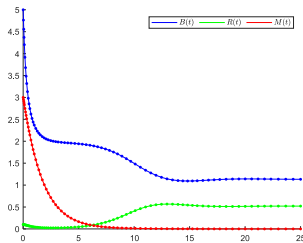
$$K = 10$$

$$\gamma_1 = 1, \gamma_2 = 0.75$$

$$\gamma_3 = 1, \gamma_4 = 0.5$$

$$\alpha_1 = 0.75, \alpha_2 = 0.5$$

$$\eta_1 = 0.1, \eta_2 = 0.1$$



$$B(0) = 5, R(0) = 0.1, \\ M(0) = 3$$

$$\beta = 0.6$$

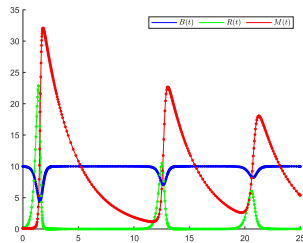
$$K = 2$$

$$\gamma_1 = 1, \gamma_2 = 0.75$$

$$\gamma_3 = 1, \gamma_4 = 0.5$$

$$\alpha_1 = 0.75, \alpha_2 = 0.5$$

$$\eta_1 = 0.1, \eta_2 = 0.1$$



$$B(0) = 10, R(0) = 0.1, \\ M(0) = 0.1$$

$$\beta = 0.6$$

$$K = 10$$

$$\gamma_1 = 0.2, \gamma_2 = 0.5$$

$$\gamma_3 = 0.5, \gamma_4 = 0.5$$

$$\alpha_1 = 0.25, \alpha_2 = 0.25$$

$$\eta_1 = 0.1, \eta_2 = 0.1$$

Beispiel 3.12 (Lotka–Volterra für drei Populationen mit Fangquoten und max. Beute mit variablen Parametern)

Bisher haben wir die Parameter α_1 , α_2 , β , γ_1 , γ_2 , γ_3 , γ_4 , η_1 , η_2 und K als Konstante beschrieben. Natürlich können sich diese Parameter auch über die Zeit (bspw. über den Jahresverlauf) ändern.

Beispielsweise bietet der Winter weniger Lebensraum als der Sommer für die Beutetiere und so könnte sich in dem vorherigen Beispiel $K(t)$ über das Jahr verändern.

Ein dazugehöriger Code könnte bspw. wie folgt aussehen:

```
Matlab/Octave
t_0 = 0; t_E = 60; t_span = [t_0,t_E];
y_0 = [1,0.1,0.01];

[t,y] = ode45(@f,t_span,y_0);

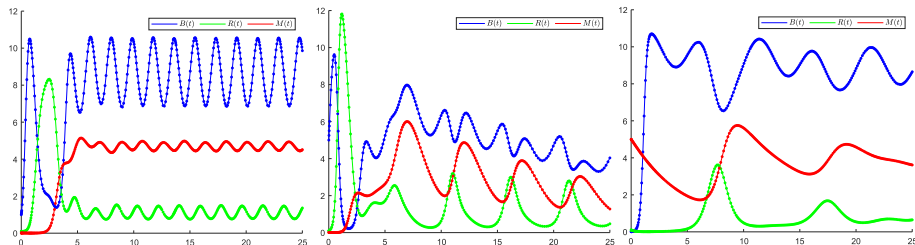
% plotting
figure('color','white'); hold on
plot(t,y(:,1),'-b'); plot(t,y(:,2),'-g'); plot(t,y(:,3),'-r');
legend('$B(t)$','$R(t)$','$M(t)$','Interpreter','LaTeX','Orientation','horizontal')

% define system function
function dydt = f(t,y)
    beta = 0.6;
    K = (10 + 2*sin(t*2*pi/1.86));
    gamma_1 = 0.75; gamma_2 = 0.5;
    gamma_3 = 0.75; gamma_4 = 0.5;
    alpha_1 = 0.5; alpha_2 = 0.25;
    eta_1 = 0.3; eta_2 = 0.3;

    dydt = zeros(3,1);
    dydt(1) = beta*(y(1))*(K-y(1))-gamma_1*y(1)*y(2);
    dydt(2) = gamma_2*y(1)*y(2) - gamma_3*y(3)*y(2) - alpha_1*y(2) - eta_1*y(2);
    dydt(3) = gamma_4*y(3)*y(2) - alpha_2*y(3) - eta_2*y(3);
end
```

Deterministische Modelle mit Differentialgleichungen: Populationsmodelle: Räuber–Beute–Modelle

Parameterstudie:



$$B(0) = 1, R(0) = 0.1, \\ M(0) = 0.01 \\ \beta = 0.6$$

$$B(0) = 5, R(0) = 0.1, \\ M(0) = 0.01 \\ \beta = 0.6$$

$$B(0) = 0.01, R(0) = 0.1, \\ M(0) = 5 \\ \beta = 0.6$$

$$K = 10 + 2 \cdot \sin(2\pi/1.86 t)$$

$$K = (10 + 3 \cdot \sin(2\pi/5 t)) e^{-t/40}$$

$$K = (10 + 1 \cdot \sin(2\pi/5 t)) e^{-t/400}$$

$$\gamma_1 = 0.75, \gamma_2 = 0.5$$

$$\gamma_1 = 1, \gamma_2 = 0.75$$

$$\gamma_1 = 0.5, \gamma_2 = 0.25$$

$$\gamma_3 = 0.75, \gamma_4 = 0.5$$

$$\gamma_3 = 1, \gamma_4 = 0.5$$

$$\gamma_3 = 0.5, \gamma_4 = 0.25$$

$$\alpha_1 = 0.5, \alpha_2 = 0.25$$

$$\alpha_1 = 0.75, \alpha_2 = 0.5$$

$$\alpha_1 = 0.1, \alpha_2 = 0.1$$

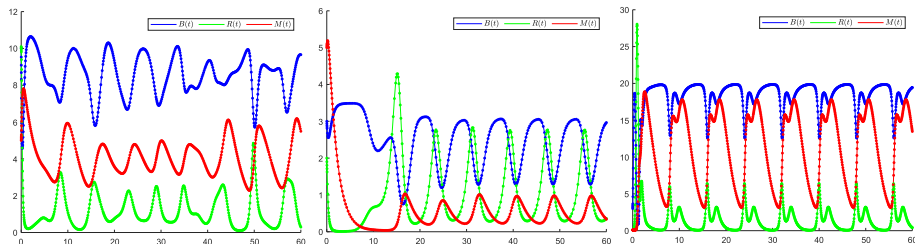
$$\eta_1 = 0.3, \eta_2 = 0.3$$

$$\eta_1 = 0.1, \eta_2 = 0.1$$

$$\eta_1 = 0.1, \eta_2 = 0.1$$

Deterministische Modelle mit Differentialgleichungen: Populationsmodelle: Räuber–Beute–Modelle

Parameterstudie:



$$B(0) = 10, R(0) = 10, \\ M(0) = 5$$

$$\beta = 0.6$$

$$K = 10 + 1 \cdot \sin(2\pi/8 t)$$

$$\gamma_1 = 0.75 + 0.2 \cdot \sin(2\pi/5 t)$$

$$\gamma_3 = 1 - 0.2 \cdot \sin(2\pi/7 t)$$

$$\gamma_2 = 0.5, \gamma_4 = 0.25$$

$$\alpha_1 = 0.2 + 0.2 \cdot \sin(2\pi/3 t)$$

$$\alpha_2 = 0.2, \eta_1 = 0.1, \eta_2 = 0.1$$

$$B(0) = 3, R(0) = 2, \\ M(0) = 5$$

$$\beta = 0.6$$

$$K = 3.5$$

$$\gamma_1 = 0.75 + 0.5 \cdot \sin(2\pi/8 t)$$

$$\gamma_2 = 0.5, \gamma_3 = 0.75, \gamma_4 = 0.5$$

$$\alpha_1 = 0.5 + 0.25 \cdot \sin(2\pi/8 t)$$

$$\alpha_2 = 0.25, \eta_1 = 0.3, \eta_2 = 0.3$$

$$B(0) = 3, R(0) = 0.1, \\ M(0) = 0.1$$

$$\beta = 0.6 + 0.25 \cdot \sin(2\pi/8 t)$$

$$K = 20$$

$$\gamma_2 = 0.5 + 0.25 \cdot \sin(2\pi/8 t)$$

$$\gamma_4 = 0.5 - 0.25 \cdot \sin(2\pi/8 t)$$

$$\gamma_1 = 0.75, \gamma_3 = 0.75$$

$$\alpha_1 = 0.5, \alpha_2 = 0.25$$

$$\eta_1 = 0.3, \eta_2 = 0.3$$

Deterministische Modelle mit Differentialgleichungen: Populationsmodelle: Räuber–Beute–Modelle

Aufgabe: Wie ist das modellierte System zu erweitern, wenn man auch den tödlichen Konkurrenzkampf beim Aufeinandertreffen von Artgenossen innerhalb jeder Population berücksichtigen will?

Bemerkung 3.7 (SI–Modell der Epidemiologie)

Eine interessante Querverbindung ergibt sich durch die Allgemeinheit dieser Modellierung: In der *Epidemiologie* wird die Ausbreitung von Infektionskrankheiten untersucht. Dieses basiert auf sehr ähnlichen Annahmen wie das Lotka–Volterra–Modell. Dabei sind die Populationen „Infizierbare Personen“, „Infizierte Personen“, „vorübergehend immune Personen durch Impfung/Erholung“, „gestorbene Personen“, etc. zu betrachten.

Besondere Bekanntheit hat hierbei das *SI–Modell* mit all seinen Erweiterungen erlangt.

Bemerkung 3.8 (Populationsmodelle: Räuber–Beute–Modelle)

Das Besondere an dieser **Modellierung** ist bspw., dass

- anstatt von physikalischen Gesetzen mit biologischen Abhängigkeiten argumentiert wird, und
- man damit recht einfach zu verstehende und erweiterbare ODEs erhält.

Bzgl. der **Simulation** kann man feststellen, dass

- durch die *Nichtlinearität* der ODE–Systeme sehr unintuitive Populationsveränderungen entstehen können – so ist nicht im Vorhinein klar, wann *Quasi*–Periodizität auftritt, wann ein Gleichgewicht entsteht, oder wann eine Auslöschung von Populationen geschieht.
- Die sehr einfache und schnelle numerische Lösung der ODE–Systeme lädt zum „Spielen“ und „Erkunden“ ein.

Deterministische Modelle mit Differentialgleichungen: Das Dreikörper–Problem

Das *Dreikörper–Problem* ist ein klassisches Problem der Astrophysik, welches viele Mathematiker und Physiker über Jahrhunderte beschäftigte. Erst mit Computersimulationen kommt man zu zufriedenstellenden Ergebnissen bei der Bearbeitung dieser Probleme.

Aufgabenbeschreibung: Es geht darum, dass man den exakten Bewegungsablauf von (mind.) drei massereichen Körpern beschreiben will, zwischen denen die *Gravitationskraft* wirkt.

Herausforderung: Ein Grundproblem dabei ist, dass es keine *explizite Lösung* für die Bewegung der Körper gibt (d.h. welche man als einfache Funktion beschreiben kann).

Deterministische Modelle mit Differentialgleichungen: Das Dreikörper-Problem

Wir beginnen diese physikalische Modellierung mit einfachen Erkenntnissen, die wir stückweise erweitern, um uns dem Problem zu nähern. Wir gehen davon aus, zwischen jeweils zwei *Körpern* gilt das sogenannte *Newton'sche Gravitationsgesetz*:

$$F = G \cdot \frac{m_1 \cdot m_2}{r^2} ,$$

welches durch Beobachtungen von *Isaac Newton* (1643–1727) festgestellt wurde.

Dabei ist

- F die momentane Kraft, welche zwischen den Körper 1 und 2 mit Masse m_1 und m_2 wirkt,
- G die Gravitationskonstante $G \approx 6.67430 \cdot 10^{-11} \frac{\text{m}^3}{\text{kg s}^2}$ und
- r der direkte Abstand zwischen den beiden Körpern.

Deterministische Modelle mit Differentialgleichungen: Das Dreikörper-Problem

Will man nun konkret wissen, wie stark beispielsweise Körper 1 unter dieser Kraft beschleunigt wird, benötigt man weiterhin das *zweite Newton'sche Axiom*:

$$F = m_1 \cdot a_1$$

mit a_1 der Beschleunigung des Körpers 1.

Setzt man dies nun in das Newton'sche Gravitationsgesetz ein, so erhält man

$$a_1 = \frac{F}{m_1} = G \cdot \frac{m_2}{r^2} .$$

Dies ist eine Beschreibung ohne *explizite* Lage der Körper im Raum und zueinander (es geht ja nur r als Abstand ein) und gilt auch nur für 2 Körper.

Im Folgenden werden wir diese Einschränkungen stückweise auflösen.

Deterministische Modelle mit Differentialgleichungen: Das Dreikörper-Problem

Erweiterung 1: Als erstes wollen wir Lage der Körper im Raum in diese Beschreibung hinzufügen, welche wir mit $\vec{r}_1$ und $\vec{r}_2$ den beiden Ortsvektoren von Körper 1 und 2 beschreiben. Mit dieser Beschreibung wird der Abstand zu

$$r = ||\vec{r}_2 - \vec{r}_1|| ,$$

und die Richtung der Kraft ist einfach der *normierte* Richtungsvektor zwischen den beiden Körpern:

$$\frac{\vec{r}_2 - \vec{r}_1}{||\vec{r}_2 - \vec{r}_1||} .$$

Setzt man dies nun in das bekannte Gesetz ein, ergibt sich die *vektorielle* Form des Newton'schen Gravitationsgesetzes:

$$\vec{F} = G \cdot \frac{m_1 \cdot m_2}{||\vec{r}_2 - \vec{r}_1||^2} \cdot \frac{\vec{r}_2 - \vec{r}_1}{||\vec{r}_2 - \vec{r}_1||} = G \cdot \frac{m_1 \cdot m_2}{||\vec{r}_2 - \vec{r}_1||^3} \cdot (\vec{r}_2 - \vec{r}_1)$$

Deterministische Modelle mit Differentialgleichungen: Das Dreikörper-Problem

Und mit der *vektoriellen* Form des *zweiten Newton'schen Axioms*

$$\vec{F} = m \cdot \vec{a} \ ,$$

ergibt sich für die Beschleunigung des Körpers 1 bspw.

$$\vec{a}_1 = G \cdot m_2 \cdot \frac{\vec{r}_2 - \vec{r}_1}{||\vec{r}_2 - \vec{r}_1||^3} \ .$$

Deterministische Modelle mit Differentialgleichungen: Das Dreikörper-Problem

Erweiterung 2: Als nächstes wollen wir dieses Problem auf 3 Körper erweitern. Hierzu benötigt man die Erkenntnis, dass Kräfte, welche auf einen Körper wirken sich vektoriell addieren:

$$\vec{F}_1 = \vec{F}_{12} + \vec{F}_{13} ,$$

mit der Kraft $\vec{F}_1$, welche auf den Körper 1 insgesamt wirkt und den beiden (Teil-)Kräften $\vec{F}_{12}$ bzw. $\vec{F}_{13}$ welche zwischen Körper 1 und 2 bzw. Körper 1 und 3 wirken.

Dies führt uns zu den drei Gleichungen:

$$\begin{aligned}\vec{F}_1 &= G \cdot \frac{m_1 \cdot m_2}{\|\vec{r}_2 - \vec{r}_1\|^3} \cdot (\vec{r}_2 - \vec{r}_1) + G \cdot \frac{m_1 \cdot m_3}{\|\vec{r}_3 - \vec{r}_1\|^3} \cdot (\vec{r}_3 - \vec{r}_1) \\ \vec{F}_2 &= G \cdot \frac{m_2 \cdot m_1}{\|\vec{r}_1 - \vec{r}_2\|^3} \cdot (\vec{r}_1 - \vec{r}_2) + G \cdot \frac{m_2 \cdot m_3}{\|\vec{r}_3 - \vec{r}_2\|^3} \cdot (\vec{r}_3 - \vec{r}_2) \\ \vec{F}_3 &= G \cdot \frac{m_3 \cdot m_1}{\|\vec{r}_1 - \vec{r}_3\|^3} \cdot (\vec{r}_1 - \vec{r}_3) + G \cdot \frac{m_3 \cdot m_2}{\|\vec{r}_2 - \vec{r}_3\|^3} \cdot (\vec{r}_2 - \vec{r}_3)\end{aligned}$$

Deterministische Modelle mit Differentialgleichungen: Das Dreikörper-Problem

Bzw. in der Darstellung der jeweiligen Beschleunigungen der Körper 1, 2 und 3:

$$\vec{a}_1 = G \cdot m_2 \cdot \frac{\vec{r}_2 - \vec{r}_1}{\|\vec{r}_2 - \vec{r}_1\|^3} + G \cdot m_3 \cdot \frac{\vec{r}_3 - \vec{r}_1}{\|\vec{r}_3 - \vec{r}_1\|^3}$$

$$\vec{a}_2 = G \cdot m_1 \cdot \frac{\vec{r}_1 - \vec{r}_2}{\|\vec{r}_1 - \vec{r}_2\|^3} + G \cdot m_3 \cdot \frac{\vec{r}_3 - \vec{r}_2}{\|\vec{r}_3 - \vec{r}_2\|^3}$$

$$\vec{a}_3 = G \cdot m_1 \cdot \frac{\vec{r}_1 - \vec{r}_3}{\|\vec{r}_1 - \vec{r}_3\|^3} + G \cdot m_2 \cdot \frac{\vec{r}_2 - \vec{r}_3}{\|\vec{r}_2 - \vec{r}_3\|^3}$$

Deterministische Modelle mit Differentialgleichungen: Das Dreikörper-Problem

Erweiterung 3: Als letztes benötigen wir noch eine elementare Erkenntnis aus den Bewegungsgleichungen. Es gilt die Erkenntnis, dass die vektorielle Beschleunigung die zweite zeitliche Ableitung der Einträge des Ortsvektors ist:

$$\vec{a} = \ddot{\vec{r}},$$

und damit

$$\begin{aligned}\ddot{\vec{r}}_1 &= G \cdot m_2 \cdot \frac{\vec{r}_2 - \vec{r}_1}{\|\vec{r}_2 - \vec{r}_1\|^3} + G \cdot m_3 \cdot \frac{\vec{r}_3 - \vec{r}_1}{\|\vec{r}_3 - \vec{r}_1\|^3} \\ \ddot{\vec{r}}_2 &= G \cdot m_1 \cdot \frac{\vec{r}_1 - \vec{r}_2}{\|\vec{r}_1 - \vec{r}_2\|^3} + G \cdot m_3 \cdot \frac{\vec{r}_3 - \vec{r}_2}{\|\vec{r}_3 - \vec{r}_2\|^3} \\ \ddot{\vec{r}}_3 &= G \cdot m_1 \cdot \frac{\vec{r}_1 - \vec{r}_3}{\|\vec{r}_1 - \vec{r}_3\|^3} + G \cdot m_2 \cdot \frac{\vec{r}_2 - \vec{r}_3}{\|\vec{r}_2 - \vec{r}_3\|^3}.\end{aligned}$$

Dies ist nun ein *gewöhnliches Differentialgleichungssystem*. Beachten Sie, dass in dieser Form schon im Dreidimensionalen Raum $3 \cdot 3 = 9$ Gleichungen zweiter Ordnung stehen (für jeden Eintrag der Vektoren gilt ja diese Gleichung), welche in ein ODE-System erster Ordnung mit 18 Gleichungen umgeformt werden kann. Führen wir die Notation $\vec{r}_1 = (x_1, y_1, z_1)^T$ ein, so ergibt sich das ODE-System:

Deterministische Modelle mit Differentialgleichungen: Das Dreikörper-Problem

$$\begin{aligned}\ddot{x}_1 &= G \cdot m_2 \cdot \frac{x_2 - x_1}{\|\vec{r}_2 - \vec{r}_1\|^3} + G \cdot m_3 \cdot \frac{x_3 - x_1}{\|\vec{r}_3 - \vec{r}_1\|^3} \\ \ddot{y}_1 &= G \cdot m_2 \cdot \frac{y_2 - y_1}{\|\vec{r}_2 - \vec{r}_1\|^3} + G \cdot m_3 \cdot \frac{y_3 - y_1}{\|\vec{r}_3 - \vec{r}_1\|^3} \\ \ddot{z}_1 &= G \cdot m_2 \cdot \frac{z_2 - z_1}{\|\vec{r}_2 - \vec{r}_1\|^3} + G \cdot m_3 \cdot \frac{z_3 - z_1}{\|\vec{r}_3 - \vec{r}_1\|^3} \\ \ddot{x}_2 &= G \cdot m_1 \cdot \frac{x_1 - x_2}{\|\vec{r}_1 - \vec{r}_2\|^3} + G \cdot m_3 \cdot \frac{x_3 - x_2}{\|\vec{r}_3 - \vec{r}_2\|^3} \\ \ddot{y}_2 &= G \cdot m_1 \cdot \frac{y_1 - y_2}{\|\vec{r}_1 - \vec{r}_2\|^3} + G \cdot m_3 \cdot \frac{y_3 - y_2}{\|\vec{r}_3 - \vec{r}_2\|^3} \\ \ddot{z}_2 &= G \cdot m_1 \cdot \frac{z_1 - z_2}{\|\vec{r}_1 - \vec{r}_2\|^3} + G \cdot m_3 \cdot \frac{z_3 - z_2}{\|\vec{r}_3 - \vec{r}_2\|^3} \\ \ddot{x}_3 &= G \cdot m_1 \cdot \frac{x_1 - x_3}{\|\vec{r}_1 - \vec{r}_3\|^3} + G \cdot m_2 \cdot \frac{x_2 - x_3}{\|\vec{r}_2 - \vec{r}_3\|^3} \\ \ddot{y}_3 &= G \cdot m_1 \cdot \frac{y_1 - y_3}{\|\vec{r}_1 - \vec{r}_3\|^3} + G \cdot m_2 \cdot \frac{y_2 - y_3}{\|\vec{r}_2 - \vec{r}_3\|^3} \\ \ddot{z}_3 &= G \cdot m_1 \cdot \frac{z_1 - z_3}{\|\vec{r}_1 - \vec{r}_3\|^3} + G \cdot m_2 \cdot \frac{z_2 - z_3}{\|\vec{r}_2 - \vec{r}_3\|^3} ,\end{aligned}$$

welches offensichtlich stark gekoppelt ist.

Deterministische Modelle mit Differentialgleichungen: Das Dreikörper-Problem

Ein dazugehöriger Code für die Systemfunktion könnte bspw. wie folgt aussehen:

```
function [dydt] = f(t,y)
    G = 1; % vereinfachte Gravitationskonstante
    m = [1,1,1]; % vereinfachte Massen

    r12_3 = sqrt( (y( 1)-y( 7))^2 + (y( 3)-y( 9))^2 + (y( 5)-y(11))^2 )^3;
    r23_3 = sqrt( (y(13)-y( 7))^2 + (y(15)-y( 9))^2 + (y(17)-y(11))^2 )^3;
    r13_3 = sqrt( (y( 1)-y(13))^2 + (y( 3)-y(15))^2 + (y( 5)-y(17))^2 )^3;

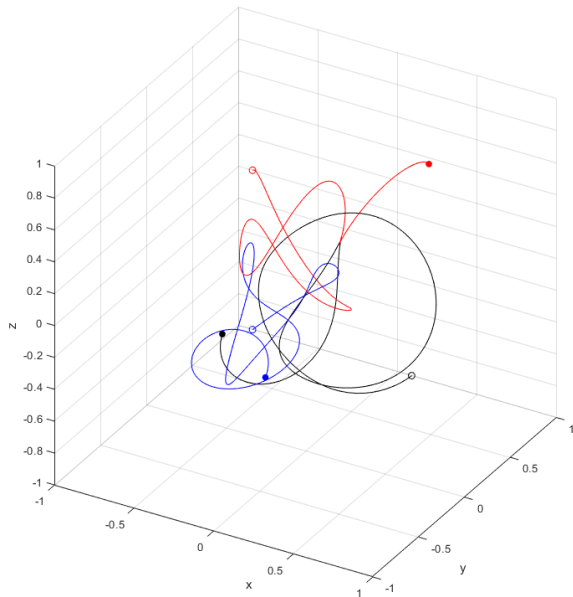
    dydt = zeros(18,1);
    dydt( 1) = y( 2); dydt( 2) = G * ( m(2)*(y( 7)-y( 1))/r12_3 + m(3)*(y(13)-y( 1))/r13_3 );
    dydt( 3) = y( 4); dydt( 4) = G * ( m(2)*(y( 9)-y( 3))/r12_3 + m(3)*(y(15)-y( 3))/r13_3 );
    dydt( 5) = y( 6); dydt( 6) = G * ( m(2)*(y(11)-y( 5))/r12_3 + m(3)*(y(17)-y( 5))/r13_3 );

    dydt( 7) = y( 8); dydt( 8) = G * ( m(1)*(y( 1)-y( 7))/r12_3 + m(3)*(y(13)-y( 7))/r23_3 );
    dydt( 9) = y(10); dydt(10) = G * ( m(1)*(y( 3)-y( 9))/r12_3 + m(3)*(y(15)-y( 9))/r23_3 );
    dydt(11) = y(12); dydt(12) = G * ( m(1)*(y( 5)-y(11))/r12_3 + m(3)*(y(17)-y(11))/r23_3 );

    dydt(13) = y(14); dydt(14) = G * ( m(1)*(y( 1)-y(13))/r13_3 + m(2)*(y( 7)-y(13))/r23_3 );
    dydt(15) = y(16); dydt(16) = G * ( m(1)*(y( 3)-y(15))/r13_3 + m(2)*(y( 9)-y(15))/r23_3 );
    dydt(17) = y(18); dydt(18) = G * ( m(1)*(y( 5)-y(17))/r13_3 + m(2)*(y(11)-y(17))/r23_3 );

end
```

Deterministische Modelle mit Differentialgleichungen: Das Dreikörper-Problem



Bemerkung 3.9 (Darstellung des Dreikörper–Problems)

Um sinnvolle Plots bzw. Animationen erzeugen zu können muss man insbesondere beachten:

- Beim Plotten sollte man zu jedem Zeitpunkt den momentanen Schwerpunkt der drei Körper abziehen (d.h. immer um den momentanen Schwerpunkt zentrieren). Damit werden insbesondere die interessanten Relativbewegungen der Körper zueinander dargestellt.
- Die Zeitschrittweite die aus dem ODE–Solver kommt variiert und läuft daher nicht gleichmäßig in der physikalischen Zeit ab. Deshalb ist eine Interpolation (meist auf ein feineres, aber gleichmäßiges Zeitschrittegitter) notwendig, wenn Sie eine Animation in physikalischer Zeit darstellen wollen.

Deterministische Modelle mit Differentialgleichungen: Das Dreikörper–Problem

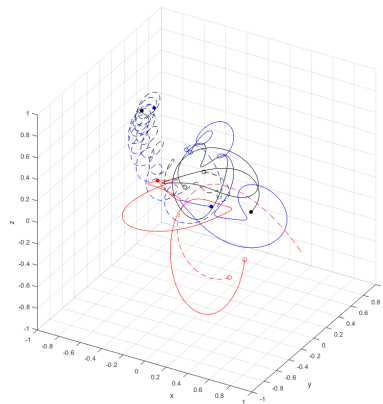
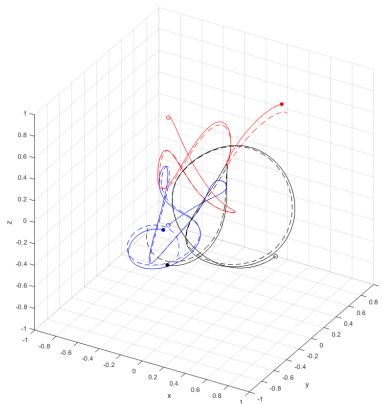
⇒ *Animation in physikalischer Zeit!*

Deterministische Modelle mit Differentialgleichungen: Das Dreikörper-Problem

Aufgabe: Wie ist es im Code zu modellieren, wenn man einen 1%-tigen Abfall von Masse der Körper pro Zeiteinheit T_M modellieren will?

Deterministische Modelle mit Differentialgleichungen: Das Dreikörper-Problem

Als Rechenbeispiel im Folgenden eine Simulation mit Zeitausschnitt $t \in [0, 5 \cdot T_M]$ (links) und einmal für einen späteren Ausschnitt $t \in [15 \cdot T_M, 20 \cdot T_M]$ (rechts), wobei die durchgezogenen Linien die Trajektorien ohne Masseverlust, und die gestrichelten Linien mit Masseverlust darstellen. Offensichtlich sind bei $0.99^5 (\approx 95\%$ der Ausgangsmasse) schon sichtbare Unterschiede und ab $(0.99^{15} \approx 86\%$ der Ausgangsmasse) keine Ähnlichkeiten mehr erkennbar.



Deterministische Modelle mit Differentialgleichungen: Das Dreikörper-Problem

Eine für die Astronomie wichtige Frage lautet: Wo kann man ein Raumschiff oder einen Satelliten stabil zwischen Himmelskörpern positionieren ohne viel Energie hierfür zu verbrauchen? Für die Antwort darauf wird meist ein vereinfachtes Dreikörper-Problem betrachtet.

Betrachten wir 2 Körper (wie bspw. Erde-Sonne), so umkreisen sich diese Körper. In den Gleichungen muss man nur $m_3 = 0$ setzen und die dritte Gleichung für $\vec{r}_3$ weglassen. Nach leichten Umformungen ergeben sich folgende Gleichungen:

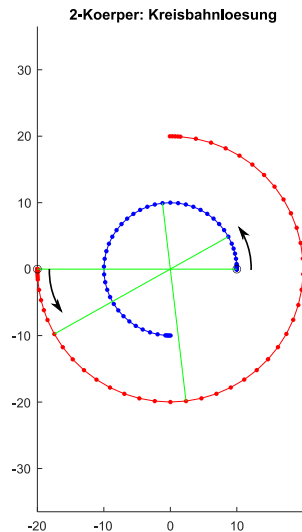
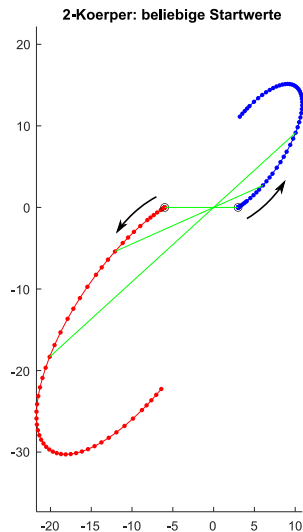
$$\begin{aligned}\ddot{\vec{r}}_1 &= G m_2 \frac{\vec{r}_2 - \vec{r}_1}{\|\vec{r}_2 - \vec{r}_1\|^3} \\ \ddot{\vec{r}}_2 &= -\frac{m_1}{m_2} \ddot{\vec{r}}_1 \quad \Rightarrow \quad \vec{r}_2 = -\frac{m_1}{m_2} \vec{r}_1 + t \cdot \vec{c}_1 + \vec{c}_2 .\end{aligned}$$

Diese kann man direkt lösen, wenn man die zweite Gleichung in die erste einsetzt und erkennt, dass die erste Gleichung dann eine ODE in $\vec{r}_1$ ist. Wenn man im Nachgang dieses Ergebnis direkt wieder in die zweite Gleichung einsetzt, erhält man auch die Lösung für $\vec{r}_2$.

Eine Sonderlösung ist die *stabile Kreisbewegung* der beiden Körper.

Deterministische Modelle mit Differentialgleichungen: Das Dreikörper-Problem

Zwei Beispiele für solche Lösungen sehen wie folgt aus ($m_1 = 2m_2$):



Welche ist die schwerere Masse?

Deterministische Modelle mit Differentialgleichungen: Das Dreikörper-Problem

Für diese Kreisbewegung gibt es sog. 5 *Lagrange*-Punkte, an denen am passenden Ort mit passender Startgeschwindigkeit ein „masseloses“ Objekt durch die Beschleunigungs- und Gravitationskräfte in einem Gleichgewicht statisch mit den beiden Planeten „mitwandert“.

Um die Bahn dieses Objekts $\vec{r}$ zwischen den Kreisbewegungen zu simulieren müssen wir nun einfach die dritte Gleichung isoliert betrachten, die gerade berechneten Bahnkurven $\vec{r}_1(t)$ und $\vec{r}_2(t)$ einsetzen und dann diese ODE lösen:

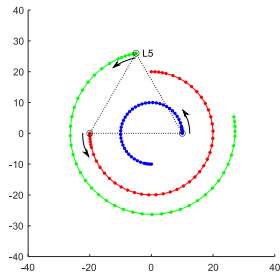
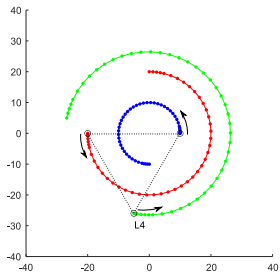
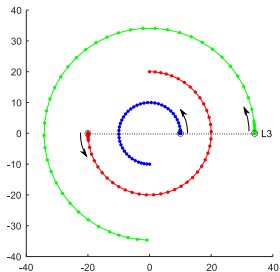
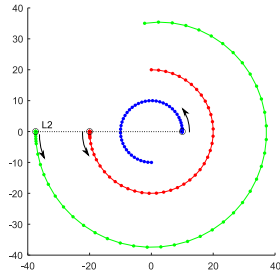
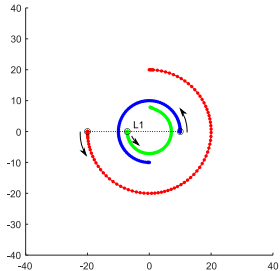
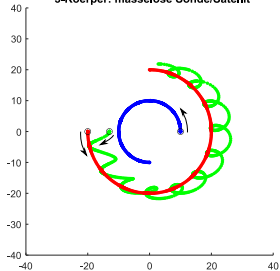
$$\ddot{\vec{r}} = G \cdot m_1 \cdot \frac{\vec{r}_1 - \vec{r}}{||\vec{r}_1 - \vec{r}||^3} + G \cdot m_2 \cdot \frac{\vec{r}_2 - \vec{r}}{||\vec{r}_2 - \vec{r}||^3}.$$

In den graphischen Ergebnissen sind die Objektbahnen für die 5 *Lagrange*-Punkte (L1–L5) berechnet (links oben ist ein beliebiger Nicht-*Lagrange*-Punkt als Beispiel für eine „typische“ Bahnkurve).

Das *James Webb-Weltraumteleskop* befindet sich am *Lagrange*-Punkt L2 des Erde–Sonnen 2–Körpersystems (wobei gilt $m_1 \approx 330000 m_2$).

Deterministische Modelle mit Differentialgleichungen: Das Dreikörper-Problem

3-Körper: masselose Sonde/Satellit



Bemerkung 3.10 (Das Dreikörper–Problem)

Das Besondere an dieser **Modellierung** ist bspw., dass

- nur unter Verwendung eines einzigen Kräftegesetzes (*Newton'sches Gravitationsgesetz*) sehr komplizierte Bahntrajektorien beschrieben werden können und
- es im Modell sehr naheliegende, systematische Erweiterungen auf N -Körper-Simulationen erkennbar sind, aber auch die Komplexität eines solches Modells merklich steigt. Ab einer größeren Zahl von N Körpern (bspw. $N = 10$ führt zu 60 gekoppelte ODEs 1. Ordnung) kann man über alternative Modellierungen nachdenken.

Bzgl. der **Simulation** kann man feststellen, dass

- durch die *Nichtlinearität* des ODE-Systems mit Hilfe des ODE-solvers Vorhersagen getroffen werden können, welche anders nicht mehr erreichbar sind.
- ein „post-processing“ von simulierten Daten benötigt wird, um daraus anschauliche Ergebnisse zu erhalten.
- in diesem Beispiel schon kleine Änderungen am System über die Laufzeit zu dramatischen Effekten führen.

Deterministische Modelle mit Differentialgleichungen: Simulation partieller Dgln (PDE)

Bisher haben wir mit ODEs die Lösung einer Funktion $\vec{z}(t)$ gesucht, welche von einer Variablen t abhängt, und in deren Gleichungen insbesondere nur die Ableitungen nach t auftauchen.

Es gibt viele Situationen, in denen eine gesuchte Funktion von mehreren Variablen abhängt, bspw. $\vec{u}(t, x, y, z)$ gibt den Funktionswert an einer Stelle (t, x, y, z) wie beispielsweise die Zeit- und Raumkoordinaten.

Suchen wir solche Funktionen $\vec{u}$ in Differentialgleichungen, so können diese deshalb auch die Ableitungen nach x, y, z und t enthalten. Man nennt diese die *partiellen Ableitungen*. Besteht eine Differentialgleichung aus partiellen Ableitungen, dann nennen wir diese eine *partielle Differentialgleichung (PDE)*.

Definition 3.5 (Partielle Differentialgleichung)

Eine *partielle Differentialgleichung* (*Partial Differential Equation, PDE*) für die unbekannte/gesuchte Funktion $u(t, x, y, z, \dots) \in \mathbb{R}$ und $t, x, y, z, \dots \in \mathbb{R}$ ist definiert durch

$$f(t, x, y, z, \dots, u, u_t, u_x, u_y, u_z, \dots, u_{tt}, u_{tx}, u_{xx}, u_{xy}, \dots) = 0,$$

mit einer mindestens stetigen Funktion $f : \mathbb{R}^m \mapsto \mathbb{R}$.

Wir sprechen von der n -ten Ordnung der PDE, wenn die höchste auftauchende Ableitung vom Grad n ist. Weiterhin haben wir in dieser Definition die Kurzschreibweise

$$u_t := \frac{\partial}{\partial t} u(t, x, y, z, \dots), \quad u_{xy} := \frac{\partial}{\partial y} \left(\frac{\partial}{\partial x} u(t, x, y, z, \dots) \right) = \frac{\partial^2}{\partial y \partial x} u(\dots), \dots$$

verwendet.

Bemerkung 3.11 (Partielle Differentialgleichung)

- Neben der Ordnung einer PDE ist natürlich auch die Anzahl der Variablen entscheidend.
- Natürlich kann man sich auch Systeme von PDEs (analog wie bei den ODE-Systemen) definieren. Bei mehreren gekoppelten Gleichungen sprechen wir von einem *System von partiellen Differentialgleichungen*.
- Leider gibt es für PDEs *keine allgemeine Umformulierung* in Systeme 1. Ordnung wie bei ODEs. Man betrachtet jede PDE für sich.

Deterministische Modelle mit Differentialgleichungen: Entstehung und Beispiele von PDEs

Die meisten PDEs entstehen in Zusammenhängen der Technik und Naturwissenschaft und die gesuchte Funktion $\vec{u}$ hängt von Zeit- und Raumkoordinaten ab. Hinter der Modellierung von PDEs stehen meist sehr einfache physikalische Grundprinzipien, welche mathematisch beschrieben werden.

- *Masseerhaltung* (häufig in der Form: Innerhalb eines gegebenen Berechnungsgebiets kann keine Masse verschwinden oder neu entstehen – die Massebilanz lässt sich komplett über den Zu- und Abfluss von Masse an den Rändern des Berechnungsgebiets bestimmen)
- *Energieerhaltung* (ähnlich wie die Masse kann sich die Gesamtenergie in einem Berechnungsgebiet als geschlossenes System nur durch aktive Energiequellen verändern)
- *Impulserhaltung* (unabhängig von der Energieerhaltung ist dies eine weitere unabhängige Erhaltungsgleichung, die insbesondere an jedem Ort bspw. eines Fluids gefordert wird)

Neben solchen Erhaltungssätzen der Mechanik gibt es auch viele andere physikalische Gesetze aus anderen Gebieten (Elektromagnetismus, Elementarteilchentheorie, etc.), sowie allgemein die *Variationsrechnung*, welche Grundlage von PDEs sind.

Beispiel 3.13 (Poisson–Gleichung)

In der *Elektrostatik* und *Gravitationstheorie* stößt man auf die *Poisson–Gleichung*:

$$u_{xx} + u_{yy} + u_{zz} = f(x, y, z) .$$

Ist $f(x, y, z) \equiv 0$, dann spricht man von der *Laplace–Gleichung*.

Beispiel 3.14 (Wärmeleitungs–Gleichung)

Bei der zeitlichen Veränderung der *Temperaturausbreitung* in einem Objekt stößt man auf die *Wärmeleitungs–Gleichung*:

$$u_t - a \cdot (u_{xx} + u_{yy} + u_{zz}) = 0 ,$$

mit $a > 0$ der Temperaturleitfähigkeit des Objekts. Häufig geht man von einer Anfangstemperaturverteilung aus und hat Randbedingungen für den Objektrand. Gibt es Wärmequellen, so steht auf der rechten Seite ein $f \neq 0$.

Beispiel 3.15 (Wellen–Gleichung)

Gibt es wellenartige Ausbreitungen (bspw. Tropfen in ein flüssiges Medium) stößt man auf die *Wellen–Gleichung*

$$\frac{1}{c^2} u_{tt} - (u_{xx} + u_{yy} + u_{zz}) = 0 ,$$

mit $c > 0$, der Ausbreitungsgeschwindigkeit der Welle im Medium. Mit einer rechten Seite $f \neq 0$ kann man auch Quellen von Wellen berücksichtigen.

Beispiel 3.16 (Wärmeleitungs–Gleichung in einem Draht)

Als konkretes Beispiel wollen wir die Wärmeleitungsgleichung in einem Draht ohne eine aktive Wärmequelle betrachten, d.h. es gibt eine Zeitvariable t und eine Raumvariable x . Gesucht ist die Wärmeverteilung $u(t, x)$. Verwenden wir die Temperaturleitfähigkeit $a = 1$ ergibt sich die PDE zu

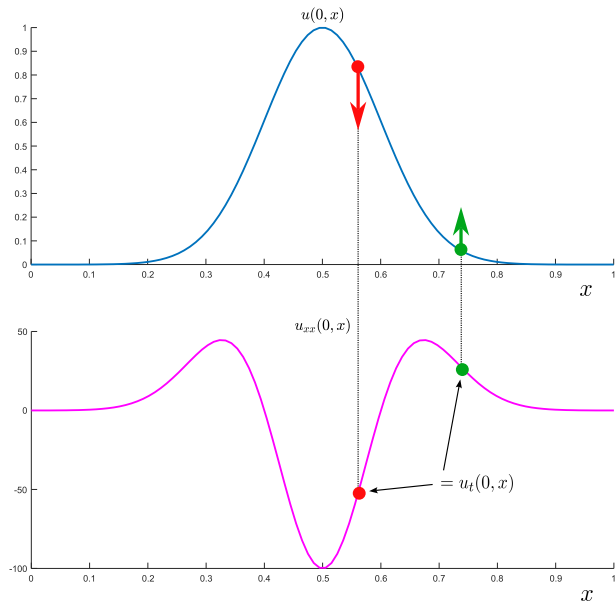
$$u_t - u_{xx} = 0 \quad \text{bzw.} \quad u_t = u_{xx} .$$

Gäbe es aktive Wärmequellen, so würden diese mit einer *rechten Seite* modelliert:

$$u_t - u_{xx} = f(t, x).$$

In der folgenden Abbildung erkennt man wie sich durch $u_t = u_{xx}$ von einem Startzustand $u(0, x)$ ausgehend die Wärme im Draht ausbreitet. Dort wo die zweite Ableitung negativ ist, wird die Temperatur abnehmen (linker Pfeil) und dort wo die zweite Ableitung positiv ist, wird die Wärme zunehmen (rechter Pfeil). Dort wo die zweite Ableitung nahe 0 ist, verändert sich erst einmal zeitlich nichts.

Deterministische Modelle mit Differentialgleichungen: Entstehung und Beispiele von PDEs



Bemerkung 3.12 (Bekannte Systeme von PDEs)

- Die *Navier–Stokes–Gleichungen* der *Strömungsmechanik*, welche für kompressible und inkompressible Fluide formuliert werden und deren gesuchte Funktionen die *Dichte* $\rho(t, x, y, z)$, der *Druck* $p(t, x, y, z)$, sowie die vektorielle *Geschwindigkeit* $\vec{v}(t, x, y, z)$ an jedem Ort des Fluids sind. Sie bilden eine allgemeine Form, wovon es für spezielle Fluide einfachere Formen wie die *Euler–Gleichungen* gibt.
- Die *Maxwell–Gleichungen* des *Elektromagnetismus*, welche die Grundgesetze der Elektrizität, des Magnetismus und deren zeitlicher Veränderungen (Induktion und Ampèresches Gesetz) zusammenfasst und für die gesuchten Funktionen der *elektrischen Feldstärke* $E(t, x, y, z)$ und der *magnetischen Feldstärke* $H(t, x, y, z)$ beschrieben wird. Speziallösungen hiervon sind beispielsweise die wellenförmige Ausbreitung des Lichts.
- Die *Schrödinger–Gleichung* der *Quantenmechanik*, welche Grundgrößen der Elementarteilchentheorie zusammenfasst und für eine *Wellenfunktion* $\psi(t, x, y, z)$ aufgestellt wird. Insbesondere die Größe $|\psi(t, x, y, z)|^2$ ist dabei von besonderer Bedeutung, da dies als die Wahrscheinlichkeitsverteilung für den Aufenthaltsort eines Teilchens interpretiert wird.

Deterministische Modelle mit Differentialgleichungen: Konkrete Anfangswerte und Randbedingungen

All diese Gleichungen werden praktisch meist so modelliert, dass man die gesuchte Funktion $u(t, x, y, z)$ auf einem *räumlichen Gebiet* $D \subset \mathbb{R}^3$ lösen will und die zeitliche Veränderung von u über diesem Gebiet D in einem *Zeitintervall* $[t_A, t_E]$ sucht.

Damit das Problem neben der PDE für die konkrete Anwendung vollständig formuliert wird, benötigt man noch *Anfangsbedingungen* für u zum Zeitpunkt $t = 0$ oder *Randbedingungen* für u am Rand des räumlichen Gebiets.

Im Gegensatz zu Anfangswertproblemen bei ODEs ist die *Existenz einer Lösung* bei PDEs im Allgemeinen *nicht klar*. Ob es für gegebene Anfangs- und Randbedingungen Lösungen gibt ist im Einzelfall zu untersuchen.

Bspw. stellen wir uns mit u eine Temperatur in einem Draht vor, der an beiden Seiten auf 0° Celsius gekühlt ist. D.h. $u(t, x_A) = u(t, x_E) = 0$ ist eine Randbedingung und zusätzlich ist $u(0, x) = g(x)$ die Anfangstemperaturverteilung im Draht. Natürlich dürfen sich diese Bedingungen nicht widersprechen, woraus $g(x_A) = g(x_E) = 0$ folgern muss.

Deterministische Modelle mit Differentialgleichungen: Konkrete Anfangswerte und Randbedingungen

Klassische Beschreibungen für diese Bedingungen sind:

- *Anfangsbedingungen*, welche den Startzustand $u(0, x, y, z)$ zum Zeitpunkt $t = 0$ über ganz D vorgeben.
- *Dirichlet-Randbedingungen*, welche auf dem Rand von D (geschrieben „ ∂D “) für alle t fixe Funktionswerte vorgeben $u(t, \partial D) = g(\partial D)$.
- *Neumann-Randbedingungen*, welche auf eine Veränderung von u über den Rand ∂D (d.h. in Normalenrichtung des Rands) für alle t vorgeben, d.h. für $\frac{\partial u}{\partial \mathbf{n}}(t, \partial D) = g(\partial D)$.
- Häufig tritt eine Kombination aus diesen Bedingungen für eine konkrete Anwendung auf.

Deterministische Modelle mit Differentialgleichungen: Numerische Lösungsstrategien

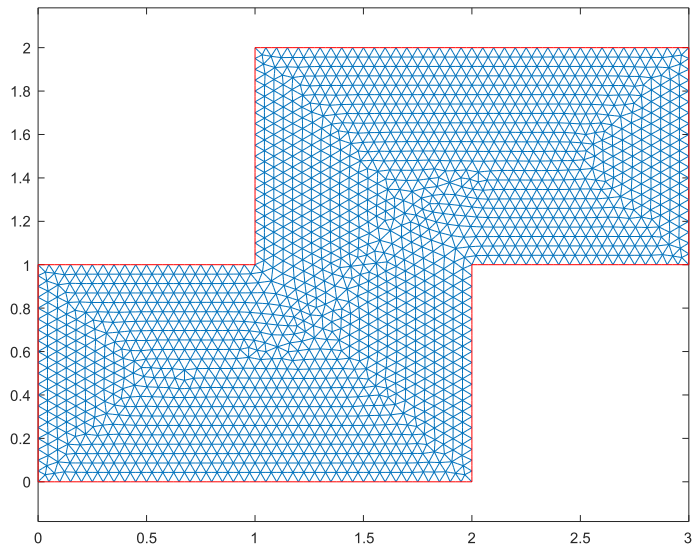
Da man nur in sehr ausgewählten Einzelfällen zu Lösungen von PDEs auf dem Blatt kommt, werden diese praktisch nahezu immer über Simulationen am Computer gelöst.

Meist wird folgende Strategie erfolgreich verwendet: Man diskretisiert sowohl das räumliche Gebiet D wie auch das Berechnungszeitintervall $[t_A, t_E]$. Dabei behandelt man typischerweise die *Zeit- und Raumvariablen unterschiedlich*:

- Typischerweise wird die räumliche Geometrie D sehr detailliert angepasst in einem Gitter diskretisiert. Hierzu bieten sich je nach Aufgabenstellung gleichmäßige quadratische oder rechteckige Raumelemente an, oder – mittlerweile der Standard – die Aufteilung in je nach benötigter Rechengenauigkeit angepasster Dreiecke oder Tetraeder (sog. *Triangulierung*). Letztere erlauben eine enorme Anpassungsfähigkeit an beliebige räumliche Gebiete.
- Für die Auflösung nach der Zeit werden ebenso typischerweise diskrete Zeitschritte $t_j \in [t_A, t_E]$ eingeführt. Dies wird jedoch meist unabhängig von der räumlichen Diskretisierung modelliert.

Die Idee: Bei *zeitabhängigen* PDEs erhält man letztlich immer große ODE-Systeme, welche man mit Hilfe der Zeitdiskretisierung numerisch löst. Bei *zeitunabhängigen* PDEs entstehen große algebraische Gleichungssysteme.

Deterministische Modelle mit Differentialgleichungen: Numerische Lösungsstrategien



Deterministische Modelle mit Differentialgleichungen: Numerische Lösungsstrategien

Konkret wird diese Diskretisierung in drei numerischen Lösungsstrategien verwendet:

- Den *Finite-Differenzen-Methoden* (FDM), welche alle Ableitungen direkt durch endliche Differenzenquotienten annähert. Bspw. für aufeinanderfolgende, diskrete Orte x_{j-1}, x_j, x_{j+1} :

$$u_{xx}(t, x_j) \approx \frac{u(t, x_{j+1}) - 2u(t, x_j) + u(t, x_{j-1}))}{(x_{j+1} - x_j)(x_j - x_{j-1})}$$
$$\vdots$$

Setzt man nun diese Näherungen der räumlichen Ableitungen in eine PDE ein, so erhält man große Systeme von ODEs (wenn die Zeitableitung nicht diskretisiert wird). Diese kann man dann mit Standardmethoden der ODE-solver lösen.

Für diese Klasse bietet sich ein regelmäßiges quadratischer Gitter an.

Beispiel 3.17 (Weiterführung: Wärmeleitungs–Gleichung in einem Draht, FDM)

Gesucht sei die Lösung $u(t, x)$ folgendes Anfangs– und Randwertproblems der Wärmeleitungs–Gleichung:

$$u_t - u_{xx} = f(t, x)$$

$$u(0, x) = g(x) \quad (\text{zeitliche Anfangswerte})$$

$$u(t, x_A) = u(t, x_E) = 0 \quad (\text{räumliche Randwerte}).$$

Führt man mit FDM nun eine gleichmäßige Diskretisierung der Ortsvariable x ein, d.h. $x_j = x_A + (j - 1) \cdot \Delta x$ und $\Delta x = (x_E - x_A)/N$ und $j = 1, \dots, N$, dann ergibt sich

$$u_t(t, x_j) \approx \frac{u(t, x_{j+1}) - 2u(t, x_j) + u(t, x_{j-1}))}{\Delta x^2} + f(t, x_j)$$

für alle $j = 1, \dots, N$ und damit ein großes System von ODEs in den N Funktionen $u_j(t) = u(t, x_j)$, wobei man die räumlichen Randwerte in den Gleichungen direkt berücksichtigen muss.

Deterministische Modelle mit Differentialgleichungen: Numerische Lösungsstrategien

- Eine weitere Lösungsstrategie sind die *Finite-Volumen-Methoden* (FVM), für welche man die PDE in eine Integralgleichung umformt. Anders als bei der FDM betrachtet man nicht die Lösung von u an einem Punkt, sondern stellt für jedes Raumelement Bilanzgleichungen auf (bspw. wie viel Temperatur entweicht über die Raumelementgrenzen bzw. wird hinzugeführt). Dies führt wiederum auf große (dünn besetzte) ODE-Systeme.
- Die letzte (und wichtigste) Lösungsstrategie sind die sehr verbreiteten *Finite-Elemente-Methoden* (FEM), bei denen man die PDE in eine schwache Integralform überführt. Wesentliche Grundidee ist dabei, dass man für die gesuchte Lösung u eine Ansatzfunktion (nach dem *Ritz-Ansatz*) annimmt:

$$u(t, \vec{x}) := \sum_{j=1}^N c_j(t) \cdot \phi_j(\vec{x}) ,$$

wobei man die $\phi_j(\vec{x})$ so wählt, dass die $\phi_j(\vec{x})$ ganz D abdecken und nur über wenige Raumelemente des Raumgitters $\neq 0$ sind und dort, wo sie überlappen, die auftretenden Integrale sehr einfach zu lösen sind (wie bspw. bei *B-Splines*). Damit wird die PDE wieder in große (dünn besetzte) ODE-Systeme überführt.

Beispiel 3.18 (Weiterführung: Wärmeleitungs–Gleichung in einem Draht, FEM)

Gesucht sei weiterhin die Lösung $u(t, x)$ für die Wärmeleitungsgleichung

$$u_t - u_{xx} = f(t, x)$$

Die Idee ist nun bei der FEM, dass man die Lösung als Ansatz schreibt

$$u(t, x) := \sum_{j=1}^N c_j(t) \cdot \phi_j(x) ,$$

mit sog. Testfunktionen $\phi_j(x)$, welche das gesamte Drahtintervall D abdecken, jedoch jeweils nur in wenigen Raumelementen $\neq 0$ sind. Bevor wir diesen Ansatz einsetzen, schreiben wir jedoch die PDE selbst in eine *schwache Form* um mit Hilfe dieser Testfunktionen (sog. *Galerkin-Verfahren*). Wäre die PDE erfüllt, dann auch die räumliche integrierte Version für alle Testfunktionen $\phi_i(x)$ ($i = 1, \dots, N$):

$$\int_D u_t \cdot \phi_i(x) \, dx - \int_D u_{xx} \cdot \phi_i(x) \, dx = \int_D f(t, x) \cdot \phi_i(x) \, dx$$

Deterministische Modelle mit Differentialgleichungen: Numerische Lösungsstrategien

In diesem Fall ergibt sich für den Ansatz weiterhin

$$u_t = \sum_{j=1}^N \frac{\partial}{\partial t} c_j(t) \cdot \phi_j(x), u_{xx} = \sum_{j=1}^N c_j(t) \cdot \frac{\partial^2}{\partial x^2} \phi_j(x)$$

mit bekanntem $\frac{\partial^2}{\partial x^2} \phi_j(x)$ und damit eingesetzt in die schwache Form der PDE für alle $i = 1, \dots, N$:

$$\begin{aligned} \int_D \left(\sum_{j=1}^N \frac{\partial}{\partial t} c_j(t) \cdot \phi_j(x) \right) \cdot \phi_i(x) \, dx - \int_D \left(\sum_{j=1}^N c_j(t) \cdot \frac{\partial^2}{\partial x^2} \phi_j(x) \right) \cdot \phi_i(x) \, dx \\ = \int_D f(t, x) \cdot \phi_i(x) \, dx \end{aligned}$$

bzw. ein großes (durch die Wahl der Testfunktionen) dünn besetztes ODE-System in den Koeffizienten $c_j(t)$ (für alle $i, j = 1, \dots, N$):

$$\sum_{j=1}^N \frac{\partial}{\partial t} c_j(t) \cdot \int_D \phi_j(x) \cdot \phi_i(x) \, dx - \sum_{j=1}^N c_j(t) \cdot \int_D \frac{\partial^2}{\partial x^2} \phi_j(x) \cdot \phi_i(x) \, dx = \int_D f(t, x) \cdot \phi_i(x) \, dx.$$

Deterministische Modelle mit Differentialgleichungen: Numerische Lösungsstrategien

Dies erkennt man beispielsweise daran, wenn man das System in Matrix-Form umschreibt mit

$$\underbrace{\begin{bmatrix} \vdots \\ \dots \int_D \phi_j(x) \cdot \phi_i(x) \, dx \dots \\ \vdots \end{bmatrix}}_{=:A} \cdot \dot{\vec{c}} - \underbrace{\begin{bmatrix} \vdots \\ \dots \int_D \frac{\partial^2}{\partial x^2} \phi_j(x) \cdot \phi_i(x) \, dx \dots \\ \vdots \end{bmatrix}}_{=:B} \cdot \vec{c} = \underbrace{\begin{bmatrix} \vdots \\ \int_D f(t,x) \cdot \phi_i(x) \, dx \\ \vdots \end{bmatrix}}_{=: \vec{d}}$$

bzw. mit Auflösung in die explizite Form des ODE-Systems

$$\dot{\vec{c}} = A^{-1} B \vec{c} + A^{-1} \vec{d}.$$

Randbedingungen kann man hier direkt über die Koeffizienten $\vec{c}$ einbringen (bspw. entsprechen *Dirichlet-Randbedingungen* fest vorgegebenen Werten für $\vec{c}$ am Rand) und die *Anfangswerte* der PDE werden über die Koeffizienten $\vec{c}$ zu den Anfangswerten des ODE-Systems.

Deterministische Modelle mit Differentialgleichungen: Numerische Lösungsstrategien

Will man nun praktisch eine gegebene PDE mit Anfangs- und Randwerten mit Hilfe von Simulationen lösen, dann gibt es hierzu offensichtlich *keine allgemeine Methode*. Je nach Gleichungstyp gibt es unterschiedliche Strategien (FDM/FVM/FEM) welche zufriedenstellend funktionieren. Daraus ergibt sich:

- Es gibt nicht die *eine* Software, welche alle möglichen auf PDEs basierenden Modelle löst (dies ist durchaus im Gegensatz zu ODEs zu verstehen, wo es durchaus Pakete gibt, die die meisten praktischen ODEs zufriedenstellend lösen).
- Für einzelne Anwendungsaufgaben (bspw. Strömungsmechanik, Elektromagnetismus, Quantenmechanik) gibt es eigens angepasste (und innerhalb dieser Anwendungen auch wieder zahlreiche) Softwarepakete. Dies macht natürlich die Auswahl für ein konkretes Tool nicht unbedingt leicht.
- Im Rahmen dieser Modellbildungs- und Simulations-Vorlesung kann natürlich nur ein grober erster Eindruck für dieses weite Feld gegeben werden.

Deterministische Modelle mit Differentialgleichungen: Konkretes Beispiel

Wir zeigen an einem kleinen Beispiel, wie PDEs speziellen Bautyps beispielsweise in Matlab/Octave gelöst werden können. Matlab bietet einen PDE-solver mit der FEM an für 1D-PDEs in $u(t, x)$ mit der allgemeinen Form:

$$c(x, t, u, u_x) \cdot u_t = x^{-m} \frac{\partial}{\partial x} (x^m f(x, t, u, u_x)) + s(x, t, u, u_x)$$

Für die einfache Wärmeleitungs-Gleichung in einem Draht ohne aktive Wärmequellen (Fortführung des Beispiels) gilt nach dieser Definition:

$$u_t = u_{xx} \quad \Rightarrow \quad c = 1, \quad m = 0, \quad f = u_x, \quad s = 0.$$

Dies beschreibt man im Code mit (m wird später festgelegt):

```
function [c,f,s] = heatpde(x,t,u,dudx)  Matlab/Octave
    c = 1;
    f = dudx;
    s = 0;
end
```

Deterministische Modelle mit Differentialgleichungen: Konkretes Beispiel

In ähnlicher Form werden Anfangswerte $u(0, x) = g(x)$ vorgegeben mit einer eigenen Funktion $g(x) = (\sin(4\pi x) + 1) \cdot x(1 - x)$:

```
function u0 = heatic(x)           Matlab/Octave
    u0 = (sin(x*4*pi)+1)*(x*(1-x));
end
```

Sowie für die Randbedingungen $u(t, \partial D)$ gilt folgende allgemeine Form

$$p(x, t, u) + q(x, t, u) f(x, t, u, u_x) = 0 ,$$

was für folgende einfache Dirichlet-Randbedingungen ($u(t, \partial D) = 0$) liefert:

```
function [pL,qL,pR,qR] = heatbc(xL,uL,xR,uR,t)           Matlab/Octave
    pL = uL;
    qL = 0;
    pR = uR;
    qR = 0;
end
```

Deterministische Modelle mit Differentialgleichungen: Konkretes Beispiel

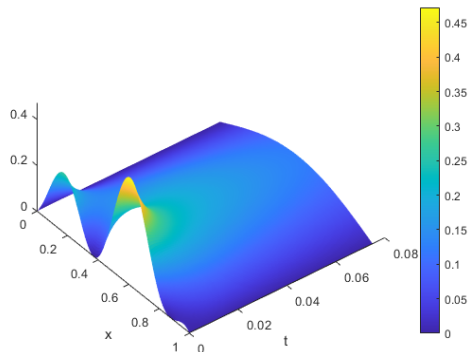
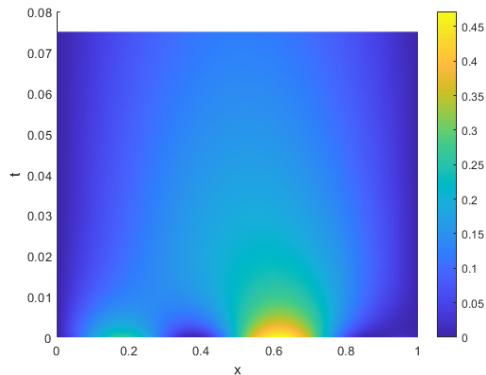
Der Aufruf und der Plot des PDE-solvers pdepe mit der Diskretisierung in x und t sieht dann beispielsweise wie folgt aus:

Matlab/Octave

```
x = linspace(0,1,200);  
t = [linspace(0,0.075,200)];  
  
m=0;  
sol = pdepe(m,@heatpde,@heatic,@heatbc,x,t);  
  
figure('Color','white')  
colormap default  
[X,T] = meshgrid(x,t);  
surface(X,T,sol,'EdgeColor','none')  
colorbar  
xlabel('x'); ylabel('t')
```

Deterministische Modelle mit Differentialgleichungen: Konkretes Beispiel

Und führt zu folgendem Ergebnis für $u(t, x)$:



Man erkennt den klassischen Abkühlungsvorgang über die Zeit bei dieser sehr inhomogenen initialen Erwärmung entlang von x und die stete Abkühlung am Rand.

Deterministische Modelle mit Differentialgleichungen: Konkretes Beispiel

Fordert man nun eine Punkterhitzung in der Anfangsverteilung, so kann man diese bspw. wie folgt annähern:

Matlab/Octave

```
function u0 = heatic(x)
    u0 = zeros(size(x));
    if (x>0.495) & (x<0.505)
        u0 = 20;
    end
end
```

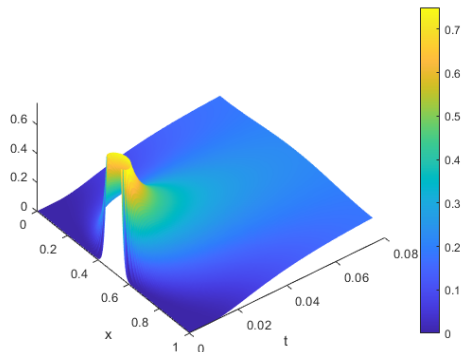
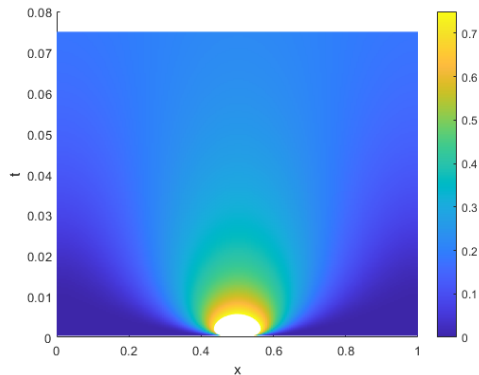
Und die isolierte Neumann-Randbedingung (über den Rand entweicht keine Temperatur, d.h. $u_x = 0$ für $x \in \partial D$) kann man realisieren durch:

Matlab/Octave

```
function [pL,qL,pR,qR] = heatbc(xL,uL,xR,uR,t)
    pL = 0;
    qL = 1;
    pR = 0;
    qR = 1;
end
```

Deterministische Modelle mit Differentialgleichungen: Konkretes Beispiel

Dies führt zu folgendem Ergebnis für $u(t, x)$:



Hier erkennt man, wie die initiale Punkterhitzung über die Zeit sich über die gesamte Länge x verteilt.

Bemerkung 3.13 (Simulation von PDEs)

Das Besondere an dieser **Modellierung** ist bspw., dass

- hier allein die saubere Formulierung des Modells als PDE schon einen recht anspruchsvollen mathematischen Werkzeugkasten benötigt.
- die Grundprinzipien, wie zuvor auch schon, jedoch einfach bleiben (altbekannte Erhaltungssätze in neuer mathematischer Form, etc.).
- neben der eigentlichen PDE, die Anfangs- und Randbedingungen einen viel größeren und wesentlicheren Anteil an der Modellierung haben im Vergleich zu ODEs.

Bzgl. der **Simulation** kann man feststellen, dass

- diese deutlich aufwändiger ist und selbst wieder sehr ausgefeilte Strategien, wie bspw. FDM/FVM/FEM (und evtl. noch schwerere mathematischer Methoden) benötigt. Das Ziel dieser Simulationsmethoden ist jedoch immer, diese neuen Probleme auf bekannte Probleme (ODE-Systeme oder algebraische Gleichungssysteme, etc.) zurückzuführen.
- aufgrund der vielfältigen Abhängigkeiten der Lösung von Raum und Zeit, geschickte graphische Darstellungen der Ergebnisse nötig werden.

3. Modelle und ihre Simulation

Stochastische Modelle mit Monte-Carlo-Simulationen

Die Grundideen hinter **Monte-Carlo-Simulationen** sind sehr einfach:

1. Es ist ein Modell gegeben $\vec{z} = \vec{M}(\vec{x}, \vec{\alpha})$, für das man Vorwärtssimulationen durchführen möchte.
2. Teile dieses Modells (bspw. Teile der Parameter $\vec{\alpha}$) sind entweder *unbekannt* oder werden *zufällig gestört*. Diese Teile des Modells sind zu bestimmen und Wahrscheinlichkeitsdichten sind für die *Unbekanntheit* oder für die *zufälligen Störungen* festzulegen. Dies ist ein zusätzlicher Teil der Modellbildung.
3. Man führt nun L unabhängige Ziehungen aus den Wahrscheinlichkeitsdichten für $\vec{\alpha}$ aus und führt damit L Vorwärtssimulationen des Modells $\vec{M}(\vec{x}, \vec{\alpha})$ aus. Man erhält damit L Ergebnisse $\vec{z}_l$ ($l = 1, \dots, L$) der Vorwärtssimulationen.
4. Final wird die Ergebnisverteilung der Punkte $\vec{z}_l$ ($l = 1, \dots, L$) als Menge statistisch und graphisch ausgewertet.

Damit erhält man final die Information, wie stark sich die *Unbekanntheiten* bzw. *zufälligen Störungen* im Modell auf das Ergebnis auswirken.

Bemerkung 3.14 (Monte-Carlo-Simulationen)

Der Begriff „Monte-Carlo“ ist in Anlehnung an das Glücksspiel im Stadtbezirk *Monte-Carlo* in *Monaco* gewählt, da man ja für jeden Simulationslauf die Parameter oder Störungen neu würfelt. Diese Simulationsmethode ist seit den 1940er Jahren durch die Einführung von leistungsstarken Rechnern sehr populär geworden.

Bemerkung 3.15 (Unbekanntheit vs. Störung)

Es ist bemerkenswert aus Sicht der Stochastik, dass man „Unbekanntheit“ und „Störung“ bei Monte-Carlo-Simulationen praktisch absolut gleich behandelt. Man muss unter Umständen für die „Unbekanntheit“ bspw. eines Parameters eine Annahme zur Wahrscheinlichkeitsdichte treffen. Dies wird meist mit der „Bayes'schen“ Sichtweise auf die Wahrscheinlichkeitsrechnung in Verbindung gebracht.

Stochastische Modelle mit Monte–Carlo–Simulationen: Pseudo–Zufallszahlen

Offensichtlich ist das *unabhängige zufällige Ziehen* aus einer Wahrscheinlichkeitsdichte von besonderer Bedeutung (Punkt 3 in vorhergehender Auflistung). Es sollte sehr schnell klar sein, dass wenn

- dieser „Ziehungsalgorithmus“ bzgl. der Unabhängigkeit qualitativ nicht gut ist, bzw.
- die dahinter liegende Wahrscheinlichkeitsdichte nicht gut genug approximiert wird, wir beliebig stark verzerrte bzw. manipulierte Ergebnisse erhalten können. Deshalb muss man sich mit der Generierung von Zufallszahlen sehr viel Mühe geben.

Weiter sei bemerkt, dass man auf einem Computer (eine deterministische Maschine) keinen „echten“ Zufall erzeugen kann, da der Computer immer eine Folge von deterministischen Befehlen ausführt. Deshalb spricht man bei von Computern erzeugten Zufallszahlen immer von **Pseudo–Zufallszahlen**. Diese sollten sich so verhalten wie es ein echtes Zufallsexperiment würde.

Welche Ideen haben Sie für das Erzeugen von Zufallszahlen am Computer?

Startpunkt ist dabei immer das Erzeugen von *gleichverteilten* Zufallszahlen, d.h. Zahlen, welche im Intervall $[0, 1)$ mit gleicher Wahrscheinlichkeit gezogen werden. Hierzu gibt es verschiedene (mittlerweile auch historische) Algorithmen:

Algorithmus 3.1 (Gleichverteilte Zufallszahlen durch lineare Kongruenz)

1. Gegeben sei ein günstig gewähltes ganzzahliges Parameterset a , c und m , sowie ein Startwert $x_0 \in \{0, \dots, m\}$ (sog. „Seed“).
2. In einer Schleife wird eine Zahlenfolge erzeugt:

$$x_n = (a \cdot x_{n-1} + c) \bmod m ,$$

wobei „mod“ die Modulo–Operation darstellt (der ganzzahlige Rest beim Teilen).

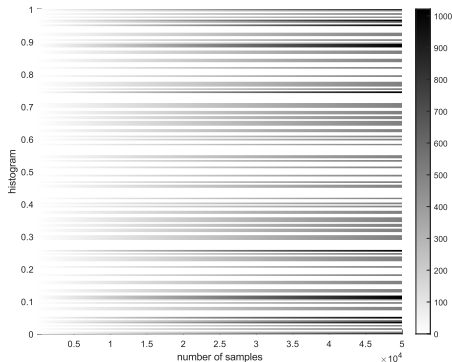
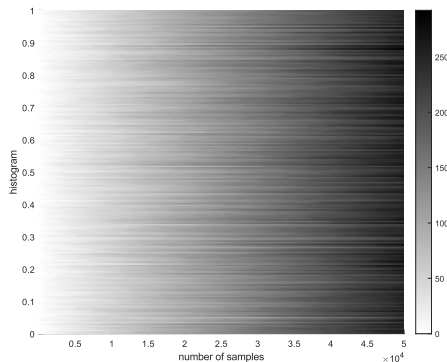
3. Teilt man nun im Reellen

$$u_n = \frac{x_n}{m} \quad (n = 0, \dots),$$

so erhält man bei günstiger Parameterwahl eine gleichverteilte Zufallsfolge u_n .

Stochastische Modelle mit Monte-Carlo-Simulationen: Pseudo-Zufallszahlen

Diese Algorithmen führen zwangsweise zu Wiederholungen, die bei ungeschickter Wahl schon sehr kurz sein kann. Deshalb sucht man Parameter a, c, m , sodass die Periodenlänge der Wiederholungen sehr lang sind und sich die Annäherung an die Gleichverteilung gutmütig gestaltet. Ein historisches Beispiel (von *Lewis, Goodman* und *Miller* – aus dem Buch von Rubinstein) sind dabei $m = 2^{31} - 1$, $a = 7^4$, $c = 0$. Vergleichen Sie folgenden Histogrammverlauf (links: gute Pseudozufallszahlen mit den erwähnten Parametern, rechts; schlechte Pseudozufallszahlen mit ungünstigen Parametern).



Stochastische Modelle mit Monte–Carlo–Simulationen: Pseudo–Zufallszahlen

Heutzutage benötigt man jedoch ausgefeiltere Methoden, damit die umfangreichen Simulationen mit Monte–Carlo–Methoden qualitativ hochwertig sind. Ein Beispiel hierfür ist der folgende (durchaus aktuelle) Algorithmus.

Algorithmus 3.2 (MRG32k3a nach L'Ecuyer – aus Rubinstein (2017))

1. Nehme folgende Parameter $m_1 = 2^{32} - 209$, $m_2 = 2^{32} - 22853$. Wähle Startwerte $x_0, x_1, x_2 \in \{0, \dots, m_1\}$ bzw. $y_0, y_1, y_2 \in \{0, \dots, m_2\}$.
2. Berechne folgende Zahlenfolge (ab $n = 3, \dots$)

$$x_n = (1403580 x_{n-2} - 810728 x_{n-3}) \bmod m_1$$

$$y_n = (527612 y_{n-1} - 1370589 y_{n-3}) \bmod m_2$$

3. Die Pseudo-Zufallszahlen werden erzeugt durch

$$u_n = \begin{cases} \frac{x_n - y_n + m_1}{m_1 + 1} & \text{falls } x_n \leq y_n \\ \frac{x_n - y_n}{m_1 + 1} & \text{falls } x_n > y_n \end{cases}$$

Die Periodenlänge liegt hierbei bei ca. $3 \cdot 10^{57}$ Zahlen.

Die Frage die sich nun ergibt ist: Wie kommt man von *gleichverteilten* zu *beliebig anders* verteilten Pseudo-Zufallszahlen in 1D? Die Antwort darauf ist tatsächlich vielfältig.

- Ein klassischer Ansatz ist die *Inversionsmethode*, welche wir hier gleich vorstellen.
- Eine weiterer klassischer Ansatz sind die *acceptance-rejection*-Methoden, welche wir ebenso vorstellen werden.
- Eine Weiterentwicklung der *acceptance-rejection*-Methoden sind die populären *Markov-Chain-Monte-Carlo*-Methoden (MCMC), insbesondere der *Metropolis-Hastings-Algorithmus* und Verwandte hierzu. Bei diesen Methoden verwendet man die aktuelle Ziehung, um auf eine neue Ziehung zu kommen. Auf diese Methoden gehen wir im Folgenden jedoch nicht ein, sondern verweisen auf die einschlägige Literatur.
- Zuletzt wollen wir noch auf Methoden hinweisen, welche ein „gleichmäßigeres“ *Ziehen* von Zufallszahlen ermöglichen, bspw. sog. *Latin Hypercube sampling* Methoden. Diese haben die Idee, garantiert aus allen Bereichen einer Wahrscheinlichkeitsdichte zu ziehen und dadurch die Gesamtzahl der benötigten Samples bei Monte-Carlo-Simulationen zu reduzieren.

Die *Inversionsmethode* hat folgende Idee: Sei X eine Zufallsvariable, welche einer gewünschten Wahrscheinlichkeitsdichte $f(x)$ mit der dazugehörigen

Wahrscheinlichkeitsverteilung $F(x) = \int_{-\infty}^x f(t) dt$ folgt. Nach Definition (im Bereich mit $f(x) > 0$) ist $F(x)$ eine streng monoton steigende Funktion und daher existiert ihre Umkehrfunktion F^{-1} .

Wir suchen nun eine Funktion $X = T(U)$ mit U gleichverteilt in $[0, 1)$, sodass X mit $F(x)$ verteilt ist. Wir beginnen mit

$$F(x) = P(X \leq x) = P(T(U) \leq x) = P(U \leq T^{-1}(x)) = T^{-1}(x) .$$

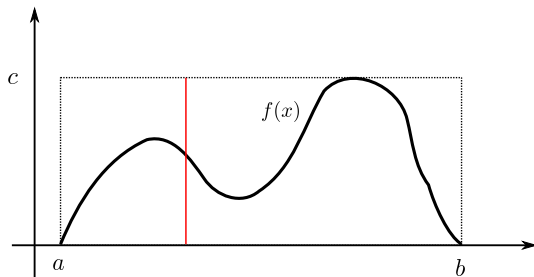
Der letzte Schritt ist klar, da die Verteilungsfunktion von U mit $F_U(x) = x$ ist. Insgesamt bedeutet dies, dass $T^{-1}(x) = F(x)$ bzw. $T(x) = F^{-1}(x)$ gilt. D.h.

Algorithmus 3.3 (Inversionsmethode)

1. Ziehe eine gleichverteilte Zufallszahl $u \in [0, 1)$.
2. Errechne $x = F^{-1}(u)$.
3. Durch Wiederholung erhält man zufällige x -Werte die mit f bzw. F verteilt sind.

Stochastische Modelle mit Monte-Carlo-Simulationen: Pseudo-Zufallszahlen

Die Idee der *acceptance-rejection*-Methoden (*Verwerfungsmethoden*) ist ebenso recht einfach. Hat man eine Wahrscheinlichkeitsdichte $f(x)$ gegeben, so kann man um diese Wahrscheinlichkeitsdichte eine Box legen:



Die Idee für einen Algorithmus ist nun einfach:

Algorithmus 3.4 (Acceptance–Rejection)

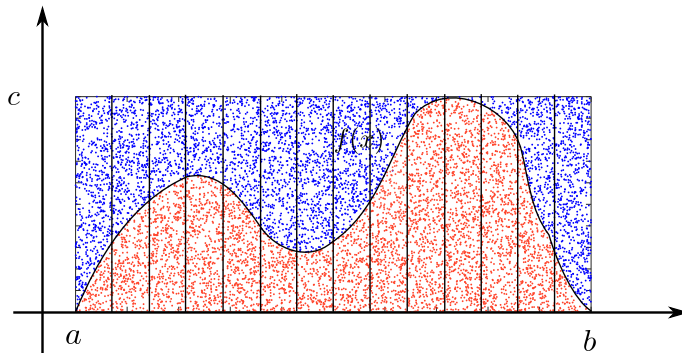
1. Ziehe eine gleichverteilte Zufallszahl $x \in (a, b)$.
2. Ziehe eine gleichverteilte Zufallszahl $y \in (0, c)$.
3. Falls $y \leq f(x)$, dann akzeptiere x –Wert, ansonsten verwirfe.
4. Die Menge aller akzeptierten x –Werte entsprechen der Verteilung nach $f(x)$.

Bemerkung 3.16 (Acceptance–Rejection)

Natürlich ist diese Version recht ineffizient, da jede „Verwerfung“ eine unnötige Rechnung/Ziehung ist. Angepasstere „einhüllende“, jedoch noch einfach zu „sampling“, Funktionen schaffen hier Abhilfe.

Stochastische Modelle mit Monte-Carlo-Simulationen: Pseudo-Zufallszahlen

Eine intuitive Begründung liefert folgende Darstellung: Mit dem Acceptance-Rejection-Algorithmus zieht man durch die Wahl zweier unabhängiger gleichverteilter Zufallsvariablen in x und y eine Stichprobe der 2D-Gleichverteilung in der Box. Durch die Bedingung bleiben nur noch die roten Punkte übrig. Betrachtet man nun ein Histogramm über die roten Punkte mit kleinen bin-Intervallen (senkrechte Linien), so wird das Histogramm die Dichtefunktion in der Form annähern. Bei beliebig großer Stichprobe und beliebig feinen bin-Intervallen ist die Annäherung beliebig genau. Daher sind die roten Punkte nach $f(x)$ verteilt.



Bemerkung 3.17 (Aus mehrdimensionale Wahrscheinlichkeitsdichten ziehen)

Will man nun mehrdimensionale Wahrscheinlichkeitsdichten sampeln benötigt man eine einfache Erkenntnis aus den bedingten Wahrscheinlichkeiten. Haben wir bspw. folgende 2D-Dichte $f(x_1, x_2)$, so können wir diese mit Hilfe bedingter Wahrscheinlichkeiten umschreiben zu

$$f(x_1, x_2) = g(x_1) \cdot h(x_2|x_1) ,$$

wobei $g(x_1)$ nun die (aufintegrierte) Randdichte von x_1 ist und $h(x_2|x_1)$ die bedingte Wahrscheinlichkeitsdichte für x_2 , wenn x_1 bekannt ist.

Damit kann man mehrdimensional sampeln indem man

1. eindimensional aus $g(x_1)$ zieht und x_1 erhält, und danach
2. eindimensional aus $h(x_2|x_1)$ zieht (unter Verwendung von x_1).

Dieses iterative Prinzip funktioniert natürlich auch für das Sampeln aus höherdimensionalen Wahrscheinlichkeitsdichten und ist daher grundlegend, bspw.

$$f(x_1, x_2, x_3) = f(x_1) \cdot f(x_2|x_1) \cdot f(x_3|x_1, x_2).$$

In Punkt 4 der Monte-Carlo-Simulationen steht die statistische Auswertung der Ergebnisse z_l bzw. $\vec{z}_l$ ($l = 1, \dots, L$) im Vordergrund. Das hierzu benötigte Werkzeug ist überschaubar, dennoch wollen wir hier etwas ausführlicher darauf eingehen:

Definition 3.6 (Erwartungswert und Varianz einer 1D-Stichprobe)

Gegeben sei eine Stichprobe $z_l \in \mathbb{R}$ mit $l = 1, \dots, L$.

→ Eine Schätzung des *Erwartungswerts* $\mu = \mathbb{E}[z]$ ist gegeben durch

$$\hat{\mu} = \frac{1}{L} \sum_{l=1}^L z_l .$$

→ Eine Schätzung der *Varianz* $\sigma^2 = \mathbb{V}\text{ar}[z]$ ist gegeben durch

$$\hat{\sigma}^2 = \frac{1}{L-1} \sum_{l=1}^L (z_l - \hat{\mu})^2 ,$$

mit $\hat{\sigma} = \sqrt{\hat{\sigma}^2}$ der Schätzung der *Standardabweichung*.

Bemerkung 3.18 (Erwartungswert und Varianz einer 1D–Stichprobe)

- Wir sagen zur Schätzung des Erwartungswerts auch einfach den *Mittelwert* (engl. *mean*).
- Es gibt noch eine weitere (nicht exakt identische) Formulierung für die Schätzung der Varianz mit

$$\hat{\sigma}^2 = \frac{1}{L-1} \left(\sum_{l=1}^L z_l^2 \right) - \hat{\mu}^2 ,$$

die sog. *Steinersche Verschiebungsformel*.

Gerade bei 1D-Stichproben sind folgende Größen von besonderem Interesse:

Definition 3.7 (Quantil und Median einer 1D-Stichprobe)

Gegeben sei eine Stichprobe $z_l \in \mathbb{R}$ mit $l = 1, \dots, L$. Das p -Prozent-Quantil z_p (bspw. $p = 95\%$) ist dadurch charakterisiert, dass mindestens $(1 - p) \cdot 100\%$ (bspw. 5%) der Werte von z_l einen Wert größer oder gleich dem Wert z_p annehmen. Hierzu wird wie folgt vorgegangen:

Zuerst muss die Stichprobe sortiert werden, sodass gilt

$$z_1 \leq z_2 \leq \dots \leq z_L .$$

Dann ergibt sich der p -Quantilwert zu

$$z_p = \begin{cases} z_{\lceil L \cdot p \rceil} & \text{wenn } L \cdot p \text{ nicht ganzzahlig} \\ \frac{1}{2} (z_{L \cdot p} + z_{L \cdot p + 1}) & \text{wenn } L \cdot p \text{ ganzzahlig} \end{cases}$$

Der Ausdruck $\lceil L \cdot p \rceil$ bedeutet dabei, die nächstgrößere ganze Zahl von $L \cdot p$ (*obere Gaußklammer*). Den 50%-Quantilwert ($p = 0.5$) nennt man *Median*.

Bemerkung 3.19 (Quantil und Median einer 1D-Stichprobe)

Die Quantile bzw. der Median gelten als besonders robust, d.h. werden durch einzelne „Ausreißer“ in der Stichprobe nicht stark beeinflusst – im Gegensatz zur Schätzung der Varianz bzw. Standardabweichung und auch des Erwartungswerts.

Arbeitet man mit Datensätzen wo durchaus „Ausreißer“ vorkommen (können), die jedoch nicht vom beobachteten Phänomen kommen, sondern ein Artefakt der Beobachtung sind, dann kann man entweder den Datensatz vor der Auswertung um die Ausreißer bereinigen (und dann alle klassischen Maße verwenden) oder von Beginn an auf solche robusten Maße ausweichen.

Will man nun nicht nur Kennwerte eines beobachteten oder simulierten Datensatzes z_l ($l = 1, \dots, L$) berechnen, sondern diese mit einer bekannten *Grundwahrheit* $z^\star$ vergleichen, so bietet sich die *Mittlere Quadratische Abweichung* (engl. *Mean Squared Error*) an.

Definition 3.8 (MSE und RMS)

Gegeben sei eine Stichprobe $z_l \in \mathbb{R}$ mit $l = 1, \dots, L$ und der *bekannte, wahre Wert* $z^\star$.

→ Der *Mean Squared Error* (MSE) ist dann definiert durch

$$\text{Mse}[z_l, z^\star] := \frac{1}{L} \sum_{l=1}^L (z_l - z^\star)^2 .$$

→ Wir definieren den *Root Mean Squared Error* (kurz RMS) mit

$$\text{Rms} := \sqrt{\text{Mse}} .$$

Bemerkung 3.20 (MSE und RMS)

- Der $\mathbb{Rms}$ wird sehr häufig in Simulationsanwendungen angegeben, da in Simulationen die wahre Größe z^* oft bekannt ist. Dies ist also ein äußerst wichtiges Maß in der statistischen Auswertung von Monte-Carlo-Simulationen.
- Eine etwas unerwartete, aber sehr bedeutsame Erkenntnis ist, dass der $\mathbb{Mse}$ sich schreiben lässt als die Summe aus Varianz und der Verzerrung (engl. *bias*):

$$\mathbb{Mse}[z_l, z^*] = \hat{\sigma}^2 + \underbrace{\left(\frac{1}{L} \sum_{l=1}^L (z_l - z^*) \right)}_{bias}^2 .$$

Dies führt auf den *bias-variance-tradeoff*, welcher besagt, dass man in vielen Anwendungen für einen kleineren MSE (bzw. RMS) eine (unbekannte) Verzerrung in Kauf nimmt (da die Varianz dabei überproportional kleiner wird).

Häufig werden jedoch höherdimensionale Datensätze simuliert. Dafür definieren wir:

Definition 3.9 (Höherdimensionaler Erwartungswert)

Gegeben sei eine Stichprobe $\vec{z}_l \in \mathbb{R}^k$ mit $l = 1, \dots, L$.

Eine Schätzung des *Erwartungswerts* $\vec{\mu} = \mathbb{E}[\vec{z}]$ ist gegeben durch

$$\hat{\vec{\mu}} = \frac{1}{L} \sum_{l=1}^L \vec{z}_l ,$$

was nach dieser Definition ein Vektor ist $\hat{\vec{\mu}} \in \mathbb{R}^k$.

Die Erweiterung der Varianz ins höherdimensionale ist jedoch etwas komplizierter

Definition 3.10 (Kovarianz, Kovarianzmatrix)

Gegeben sei eine Stichprobe $\vec{z}_l \in \mathbb{R}^k$ mit $l = 1, \dots, L$.

→ Eine Schätzung der *Kovarianz* $\sigma_{ij} = \text{Cov}[z_i, z_j]$ zwischen zwei Koordinaten i, j der Vektoren $\vec{z}_l$ ist gegeben durch

$$\hat{\sigma}_{i,j} = \frac{1}{L-1} \sum_{l=1}^L (z_{i,l} - \hat{\mu}_i) \cdot (z_{j,l} - \hat{\mu}_j) ,$$

und bemisst, in wie weit die Koordinaten i und j des Vektors $\vec{z}$ *korreliert* sind.

→ Eine Schätzung der *Kovarianzmatrix* $\Sigma \in \mathbb{R}^{k \times k}$ ist durch die Berechnung aller Kombinationen der Kovarianzen aufgestellt (mit den Varianzen auf der Diagonalen):

$$\hat{\Sigma} = \begin{bmatrix} \hat{\sigma}_1^2 & \hat{\sigma}_{1,2} & \hat{\sigma}_{1,3} & \dots & \hat{\sigma}_{1,k} \\ \hat{\sigma}_{2,1} & \hat{\sigma}_2^2 & \hat{\sigma}_{2,3} & \dots & \hat{\sigma}_{2,k} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \hat{\sigma}_{k,1} & \hat{\sigma}_{k,2} & \hat{\sigma}_{k,3} & \dots & \hat{\sigma}_k^2 \end{bmatrix} .$$

Ein Maß für die *Gesamtstreuung* einer höherdimensionalen Stichprobe ist gegeben mit

Definition 3.11 (Totale Varianz)

Gegeben sei eine Stichprobe $\vec{z}_l \in \mathbb{R}^k$ mit $l = 1, \dots, L$.

Dann ist die Summe der Diagonaleinträge der Schätzung der Kovarianzmatrix (also die Summe der geschätzten Varianzen) die geschätzte *totale Varianz* des Datensatzes. Die totale Varianz ist damit die *Spur* von $\hat{\Sigma}$.

Bemerkung 3.21 (Totale Varianz)

Will man also herausfinden, welcher bei zwei gegebenen höherdimensionalen Datensätzen (unabhängig von der Koordinatenrichtung der Streuung) in Summe mehr oder weniger streut, so muss man die totale Varianz beider berechnen und vergleichen.

Aus der Linearen Algebra ist bekannt: Die *Spur* einer Matrix ändert sich unter Rotationen (oder allgemein: durch Multiplikation mit orthogonalen Matrizen) nicht – etwas was wir von einer *Gesamtstreuung* erwarten würden: Drehe ich den Datensatz, dann bleibt die Streuung gleich. Die Spur ist auch die Summe der Eigenwerte der Kovarianzmatrix, was in der *Hauptkomponentenanalyse (PCA)* verwendet wird.

Der $\mathbb{M}\text{se}$ bzw. $\mathbb{R}\text{ms}$ verallgemeinert sich auch ins Höherdimensionale:

Definition 3.12 (Höherdimensionaler MSE und RMS)

Gegeben sei eine Stichprobe $\vec{z}_l \in \mathbb{R}^k$ mit $l = 1, \dots, L$ und der *bekannte, wahre* Wert $\vec{z}^* \in \mathbb{R}^k$.

→ Der *Mean Squared Error* (MSE) ist dann definiert durch

$$\mathbb{M}\text{se}[\vec{z}_l, \vec{z}^*] := \frac{1}{L} \sum_{l=1}^L \|\vec{z}_l - \vec{z}^*\|^2 = \frac{1}{L} \sum_{l=1}^L \sum_{i=1}^k (z_{i,l} - z_i^*)^2 .$$

→ Wir definieren den *Root Mean Squared Error* (kurz RMS) auch in diesem Fall weiterhin mit

$$\mathbb{R}\text{ms} := \sqrt{\mathbb{M}\text{se}} .$$

Was ist die Aufgabe der Objektlokalisierung?

Definition 3.13 (Objektlokalisierung)

Voraussetzung: Einem Objekt kann eine Positionscoordinate $\vec{p}$ zugeordnet werden.

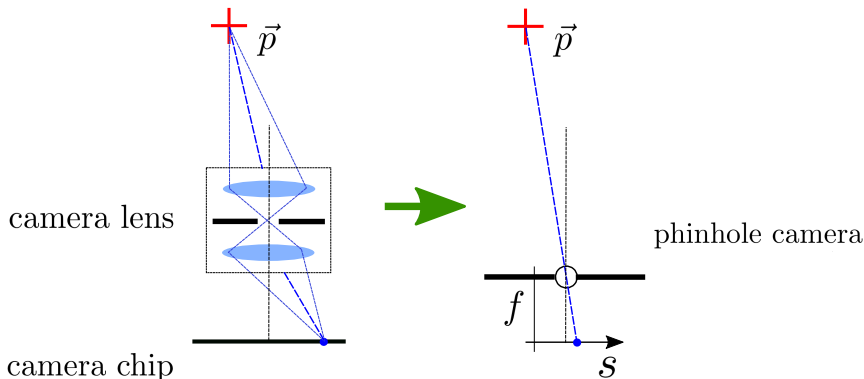
Aufgabe der Objektlokalisierung: Aus Messungen rund um das zu beobachtende Objekt eine möglichst gute Schätzung dieser Positionscoordinate $\vec{p}$ erhalten.

Zwei typische Methoden für diese Aufgabe sind:

- **Angulation** unter Verwendung von **Kameras**.
computer vision, Messtechnik, biologisches Sehen, ...
- **Lateration** unter Verwendung von **Abstandsmesssystemen**.
Radar, LiDAR, GPS (ToA), Lokalisierung von Mobiltelefonen, ...

Stochastische Modelle mit Monte-Carlo-Simulationen: Auslegung eines Positionsmesssystems

Was sind in diesem Kontext eigentlich „Kameras“?

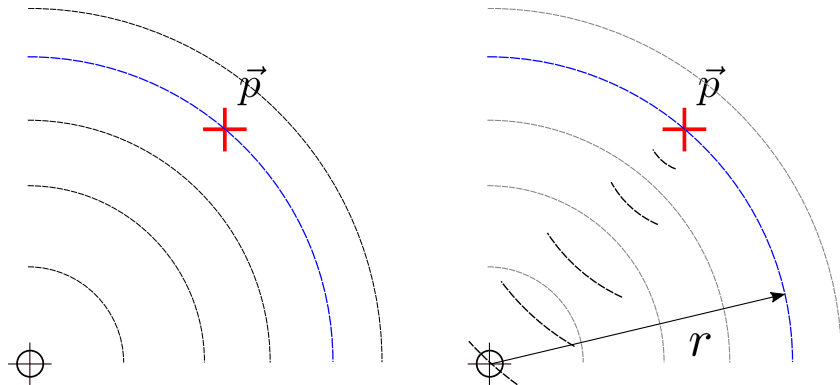


→ Bei genauer Betrachtung sind Kameras einfach *Winkelmesssysteme*. Ein relativ zur „Kamera“ positioniertes Objekt wird je nach relativem Winkel auf den Kamera-Chip projiziert. Eine typische Vereinfachung ist die *pinhole camera* (Lochkameramodell).

Pro: (typischerweise) kostengünstig, weit verbreitet, nicht-leuchtende Objekte

Contra: Kontrolle über die Lichtumgebung, Objektivverzeichnungen kalibrieren

Was sind in diesem Kontext „Abstandsmesssysteme“ (ranging systems)?



→ Ein Objekt wird von einer ausgesandten Welle getroffen, und gibt ein Rücksignal, welches am Startort detektiert wird. Über die Dauer zwischen Senden und Empfangen ergibt sich über die Ausbreitungsgeschwindigkeit der Welle eine Entfernung.

Pro: Typischerweise störungsresistent, schnell

Contra: (typischerweise) teuer, aktive oder retroreflective Objekte nötig

Im Rahmen der *Validierungsschritts* ergibt sich häufig die Frage: Wann hat man allgemein ein „gutes“ Modell erstellt? Hilfestellungen sind dabei allgemein:

- *Vollständigkeit*: alle wesentlichen Eigenschaften des zu modellierenden Phänomens sind enthalten.
- *Einfachheit*: alle nicht–wesentlichen Teile sind weggelassen.
- *Berechenbarkeit*: klare mathematische Beschreibung, welche mindestens durch den Computer zu Rechenlösungen führen.

Will man das auf die Objektlokalisierung anwenden, so bieten sich im ersten Schritt folgende Vereinfachungen an, um das Prinzip der Angulation und Lateration (und ihrer Fehlerverteilungen) besser zu verstehen:

- Modellbildung und Simulation in 2D
- Objektlokalisierung mit zwei Beobachtern

Stochastische Modelle mit Monte-Carlo-Simulationen: Auslegung eines Positionsmesssystems

Eine interessante Frage für solche Systeme lautet: *Welchen Einfluss haben zufällige Messfehler bzgl. der Bildkoordinaten auf die Objektlokalisierung?* Dies wollen wir durch *Simulationen* beantworten.

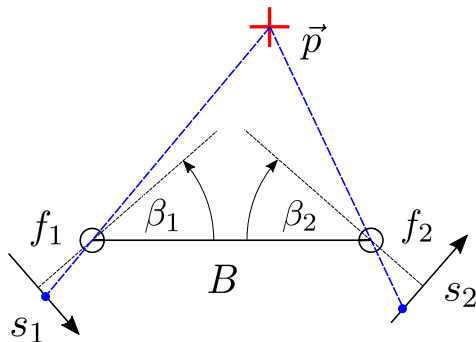
Ansatz: Simuliere Messungen mit zufälligen Messfehlern (bspw. normalverteilt) und berechne durch die Streuung der errechneten Positionskoordinaten des Objekts eine Fehlerverteilung der Objektlokalisierung.

Im Speziellen:

1. Wähle eine wahre Objektposition $\vec{p}$ und errechne die „wahren Messungen“ (s_1, s_2) (Bildkoordinaten) bzw. (r_1, r_2) (Entfernungen). „ $\vec{p} \rightarrow (s_1, s_2)$ “
2. Führe zufällige Messfehler in einer Schleife ein:
 - *Angulation:* Bildkoordinaten $(s_1^*, s_2^*) = (s_1, s_2) + (\varepsilon_{s_1}, \varepsilon_{s_2})$
 - *Lateralation:* Entfernungsmessungen $(r_1^*, r_2^*) = (r_1, r_2) + (\varepsilon_{r_1}, \varepsilon_{r_2})$
3. Für jede simulierte Messung muss die Position des Objekts berechnet werden $\vec{p}^*$. „ $(s_1^*, s_2^*) \rightarrow \vec{p}^*$ “
4. Untersuche die statistische Verteilung aller Objektpositionen $\vec{p}^*$.

Stochastische Modelle mit Monte-Carlo-Simulationen: Auslegung eines Positionsmesssystems

Betrachten wir ein solch vereinfachtes Modell zur *Angulation*



mit den Parametern:

B Basislänge

f_1, f_2 Bildabstände des Lochkammermodells

β_1, β_2 Ausrichtungswinkel der Kameras

s_1, s_2 Bildkoordinaten

Stochastische Modelle mit Monte–Carlo–Simulationen: Auslegung eines Positionsmesssystems

In diesem Fall entspricht die Lokalisierung dem Schnitt von zwei Geraden. Dies führt also auf folgendes lineare Gleichungssystem:

$$y = a_1 \cdot x$$

$$y = a_2 \cdot x + b_2$$

welche als Lösung folgenden Schnittpunkt hat „ $(s_1, s_2) \rightarrow \vec{p}$ “:

$$\vec{p} = \begin{pmatrix} p_1 \\ p_2 \end{pmatrix} = \frac{b_2}{a_1 - a_2} \cdot \begin{pmatrix} 1 \\ a_1 \end{pmatrix}$$

mit

$$a_1 = \tan \left(\beta_1 + \operatorname{atan} \left(\frac{s_1}{f_1} \right) \right)$$

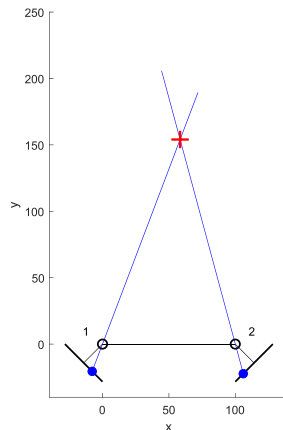
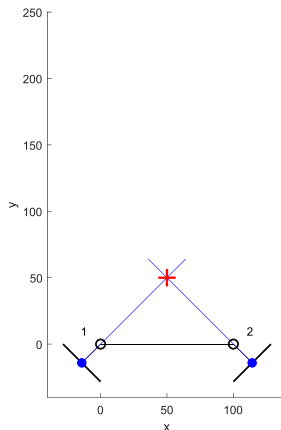
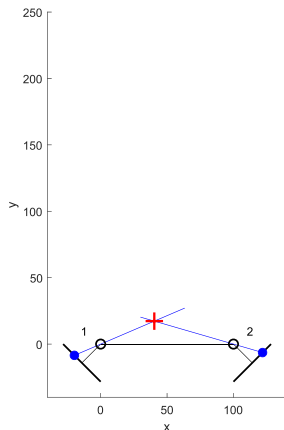
$$a_2 = -\tan \left(\beta_2 - \operatorname{atan} \left(\frac{s_2}{f_2} \right) \right)$$

$$b_2 = -a_2 \cdot B$$

Für die Simulation benötigt man aber auch noch die andere Richtung der Rechnung (d.h. wenn $\vec{p}$ gegeben, wie errechnet sich s_1, s_2 ? „ $\vec{p} \rightarrow (s_1, s_2)$ “). Dabei ergibt sich:

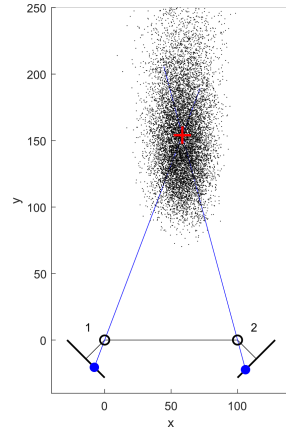
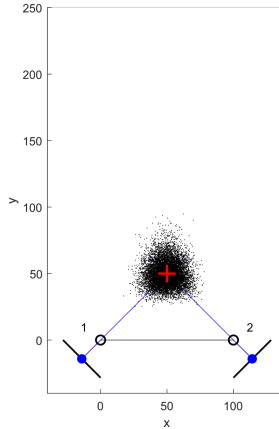
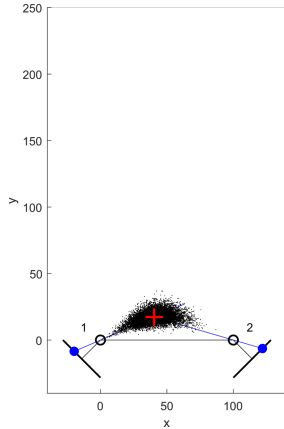
$$s_1 = f_1 \cdot \frac{\cos(\beta_1) p_2 - \sin(\beta_1) p_1}{\sin(\beta_1) p_2 + \cos(\beta_1) p_1}$$
$$s_2 = f_2 \cdot \frac{\sin(\beta_2) (p_1 - B) + \cos(\beta_2) p_2}{\cos(\beta_2) (p_1 - B) - \sin(\beta_2) p_2}$$

Stochastische Modelle mit Monte-Carlo-Simulationen: Auslegung eines Positionsmesssystems



→ Offensichtlich schneiden sich die „Sichtstrahlen“ des Objekts je nach Abstand relativ unter unterschiedlichen Winkeln.

Stochastische Modelle mit Monte-Carlo-Simulationen: Auslegung eines Positionsmesssystems



- Im Nahbereich führt das zu hauptsächlich horizontalen Fehlerverteilungen.
- Im Fernbereich ist die vertikale Fehlerverteilung dominant.

Im Code kann diese Simulation bspw. wie folgt realisiert werden:

```
% schon bekannte Parameter:
% p_true : wahre Objektkoordinate
% s1, s2 : wahre Bildkoordinaten (errechnet durch f1, f2 bzw. beta1, beta2 und Basislänge b)

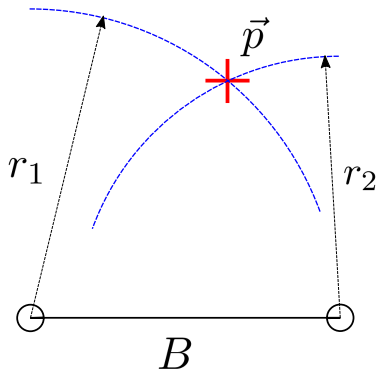
sigma_pixel_noise_1 = 2.5; % pixel noise camera 1
sigma_pixel_noise_2 = 2.5; % pixel noise camera 2

% calculating the intersection point with random noise
N = 10000; p = zeros(N,2);
for i = 1:N
    s1_rand = s1+sigma_pixel_noise_1*randn(1);
    s2_rand = s2+sigma_pixel_noise_2*randn(1);

    a1_rand = tan(beta1 + atan2(s1_rand,f1));
    a2_rand = -tan(beta2 - atan2(s2_rand,f2));
    b2_rand = b*a2_rand;
    p(i,:) = b2_rand/(a2_rand-a1_rand)*[1,a1_rand];
end
```

Stochastische Modelle mit Monte-Carlo-Simulationen: Auslegung eines Positionsmesssystems

Betrachten wir nun das dazu passende Modell der *Lateralation*



mit den Parametern:

B Basislänge

r_1, r_2 Entfernungsmessungen

Stochastische Modelle mit Monte-Carlo-Simulationen: Auslegung eines Positionsmesssystems

In diesem Fall entspricht die Lokalisierung dem Schnitt von zwei Kreisen. Dies führt auf folgendes *nichtlineare* Gleichungssystem

$$\begin{aligned}x^2 + y^2 &= r_1^2 \\(x - B)^2 + y^2 &= r_2^2\end{aligned}$$

welches zu den zwei folgenden Schnittpunkten führt „ $(r_1, r_2) \rightarrow \vec{p}$ “:

$$\vec{p} = \begin{pmatrix} p_1 \\ p_2 \end{pmatrix} = \begin{pmatrix} \frac{B^2 + r_1^2 - r_2^2}{2B} \\ \pm \sqrt{r_1^2 - \left(\frac{B^2 + r_1^2 - r_2^2}{2B} \right)^2} \end{pmatrix}$$

Bemerkung: In dieser Anwendung macht natürlich nur die Koordinate mit positivem y -Wert Sinn.

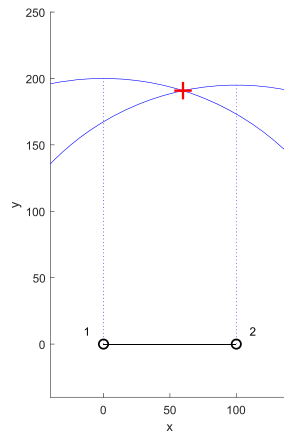
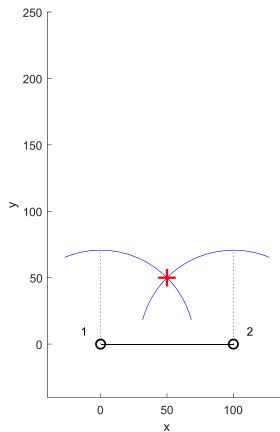
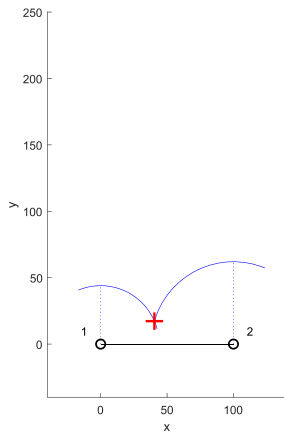
Stochastische Modelle mit Monte–Carlo–Simulationen: Auslegung eines Positionsmesssystems

Für die Simulation benötigt man aber auch noch die andere Richtung der Rechnung (d.h. wenn $\vec{p}$ gegeben, wie errechnet sich r_1, r_2 ? „ $\vec{p} \rightarrow (r_1, r_2)$ “). Dies ergibt sich aber leicht zu:

$$r_1 = \sqrt{p_1^2 + p_2^2}$$

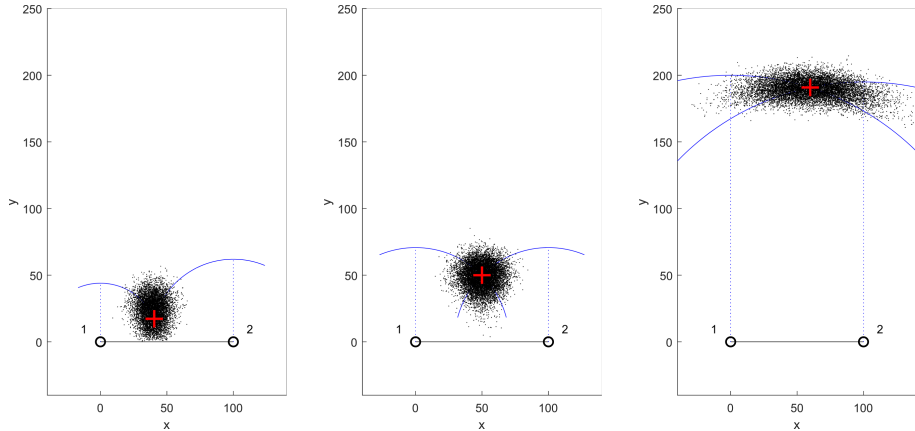
$$r_2 = \sqrt{(p_1 - B)^2 + p_2^2}$$

Stochastische Modelle mit Monte-Carlo-Simulationen: Auslegung eines Positionsmesssystems



→ Offensichtlich unterliegt hier der Schnitt auch unterschiedlichen Winkeln der Kurven, abhängig von der Objektposition.

Stochastische Modelle mit Monte-Carlo-Simulationen: Auslegung eines Positionsmesssystems



→ Im Nahbereich gibt es eine leicht vertikale Fehlerverteilung, jedoch auch einen deutlichen Bias.

→ Im Fernbereich wächst der Fehler nicht so stark wie bei der Angulation und wird hauptsächlich horizontal.

Im Code kann diese Simulation bspw. wie folgt realisiert werden:

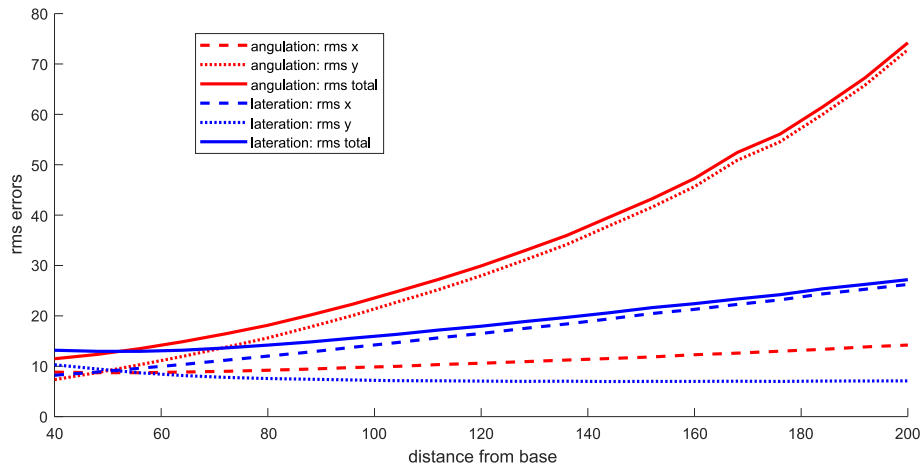
```
% schon bekannte Parameter:
% p_true : wahre Objektkoordinate
% r1, r2 : wahre Entfernungsmessungen (errechnet durch Basislänge b)

sigma_distance_noise_1 = 9; % distance noise camera 1
sigma_distance_noise_2 = 9; % distance noise camera 2

% calculating the intersection point with random noise
N = 10000;
p = zeros(N,2);
for i = 1:N
    r1_rand = r1+sigma_distance_noise_1*randn(1);
    r2_rand = r2+sigma_distance_noise_2*randn(1);

    % Test auf Schnittpunkt und Berechnung:
    if ((b^2+r1_rand^2-r2_rand^2)/(2*b))^2 < r1_rand^2
        p(i,:) = [(b^2+r1_rand^2-r2_rand^2)/(2*b), ...
                  sqrt( r1_rand^2 - ((b^2+r1_rand^2-r2_rand^2)/(2*b))^2 )];
    else
        p(i,:) = nan(1,2);
    end
end
```

Stochastische Modelle mit Monte–Carlo–Simulationen: Auslegung eines Positionsmesssystems



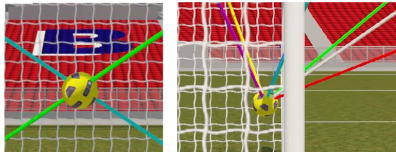
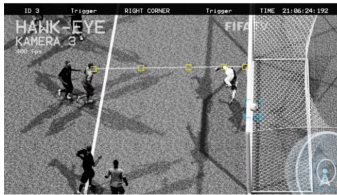
→ Bei einer systematischen Untersuchung RMS–Fehler (in x, y und total), zeigt sich, dass die Lateration auch systematisch vorteilhaft ist bei großen Distanzen des Objekts zur Basis.

Es ergeben sich folgende Erkenntnisse:

- Die meisten Lokalisierungsmethoden lassen sich grundsätzlich in die zwei Klassen *Angulation* mit Kameras und *Lateralation* mit Abstandsmessgeräten einteilen.
- Beide haben unterschiedliche charakteristische Fehlerverteilungen in Abhängigkeit des Messrauschens. Basierend auf der Simulation scheint die Lateralation vorteilhafter für Entfernungen welche im Verhältnis zur Basislänge groß sind (Fernbereich), und die Angulation im Nahbereich. Im Nahbereich führt Lateralation zusätzlich zu starken Verzerrungen (bias).
- Daraus ergeben sich mögliche typische Anwendungen wie folgt dargestellt.

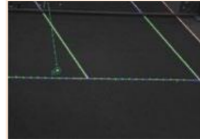
Stochastische Modelle mit Monte-Carlo-Simulationen: Auslegung eines Positionsmesssystems

Praktische Anwendung bspw. für *Angulationsmethoden im Sport*:



Camera Set Up

Up to ten cameras set up around the court to capture live images.



2D (x,y)

Vision processing is used to identify the centre of the ball.

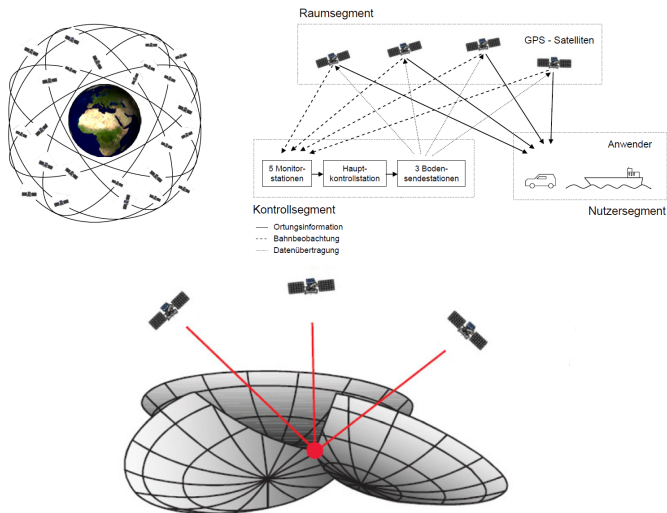


3D (x,y,z)

System triangulates information from each calibrated camera to provide a 3D position of ball.

Stochastische Modelle mit Monte-Carlo-Simulationen: Auslegung eines Positionsmesssystems

Praktische Anwendung bspw. für *Laterationsmethoden in der Navigation*:



Quelle: https://elib.dlr.de/54309/1/Lokalisierungsverfahren_v22.pdf

Aufgabe 1: Wie untersucht man die Empfindlichkeit der Systeme zusätzlich auf die auf die *Unbekanntheit* der Basislänge B ?

Aufgabe 2: Welches Setup benötigt man mindestens für 3D–Positionsmessungen mit Angulation und Lateration?

Bemerkung 3.22 (Auslegung eines Positionsmessgeräts)

Das Besondere an dieser **Modellierung** ist bspw., dass

- hier die Modelle mathematisch sehr einfach und anschaulich sind,
- und sich sehr einfach erweitern lassen.

Bzgl. der **Simulation** kann man feststellen, dass

- die einfache Störung dieser Modelle („Messrauschen“) zu unintuitiven und nicht erwartbaren Ergebnissen für die Fehlerverteilung der Systeme führen (was ja die *entscheidende* Größe bei einem Messsystem ist),
- und sich daraus ganz konkrete Schlussfolgerungen für Anwendungen ergeben.

Stochastische Modelle mit Monte-Carlo-Simulationen: Wahlsimulation

In diesem Beispiel wollen wir zeigen, wie Monte-Carlo-Simulationen gerade auch in nicht trivialen Abhängigkeiten und Entscheidungspfaden zu tieferen Einsichten führen können.

Wir wollen hierzu die US-Wahlen 2020 und 2024 analysieren, in der sich die Kandidaten Joe Biden (D) und Donald Trump (R) 2020 und Kamala Harris (D) und Donald Trump (R) 2024 gegenüber standen. Wir führen hierzu ein paar Vereinfachungen durch:

- Wir gehen davon aus, dass es jeweils nur diese beiden Kandidaten gab. Real gab es noch mehr Kandidat:innen.
- Jedem Wahlgebiet (meist sind das die Bundesstaaten) werden Wahlmänner zugordnet, welche für die tatsächliche Bestimmung des Wahlsiegers entscheidend sind.
- Wenn die Mehrheit in einem Wahlgebiet von einem Kandidaten gewonnen wurde, dann erhält der Kandidat alle Wahlmänner („*The winner takes it all*“).
- Die Wahl hat gewonnen, wer die meisten Wahlmänner (≥ 270) auf sich vereint.

Solche Modelle lohnen sich in zwei Studienarten zu verwenden

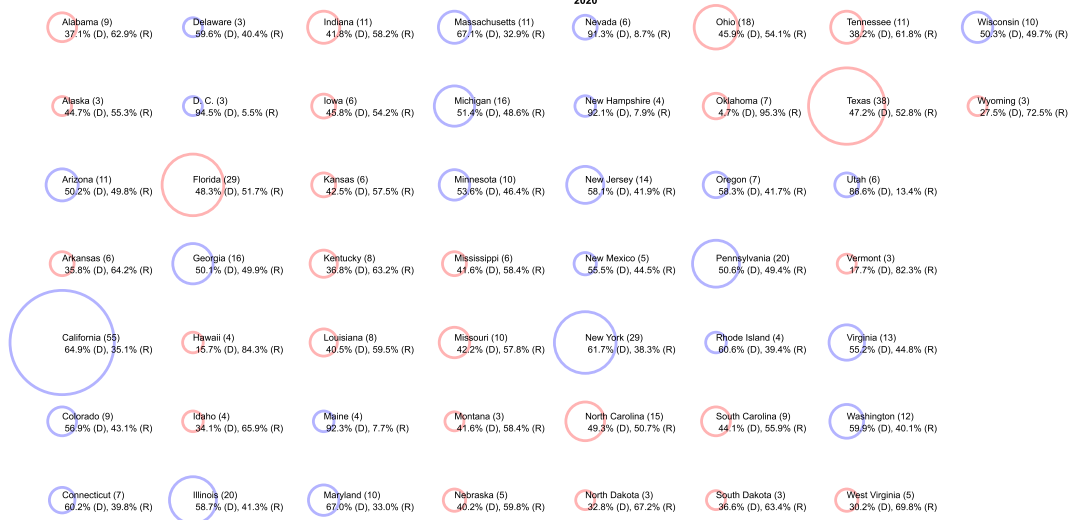
- *Prospektive Studie*: D.h. man hat beispielsweise vor der Wahl Wahlumfragen in allen Wahlgebieten zur Verfügung und interessiert sich für den wahrscheinlichen Gewinner. Diese Studienrichtung fand im Vorfeld der Wahlen natürlich sehr häufig und intensiv in den USA statt und wurde publiziert.
- *Retrospektive Studie*: D.h. wir gehen von dem amtlichen Wahlergebnis als festgestellte Grundwahrheit aus und versuchen dann im Nachhinein etwas über die Wahl herauszufinden. Diese Studienrichtung wird meist von der Öffentlichkeit ignoriert und nur von einer kleineren Statistik–Gemeinde durchgeführt.

In diesem Kapitel wollen wir eine *retrospektive Studie* durchführen.

Stochastische Modelle mit Monte-Carlo-Simulationen: Wahlsimulation

Starten wir mit dem Wahlergebnis 2020:

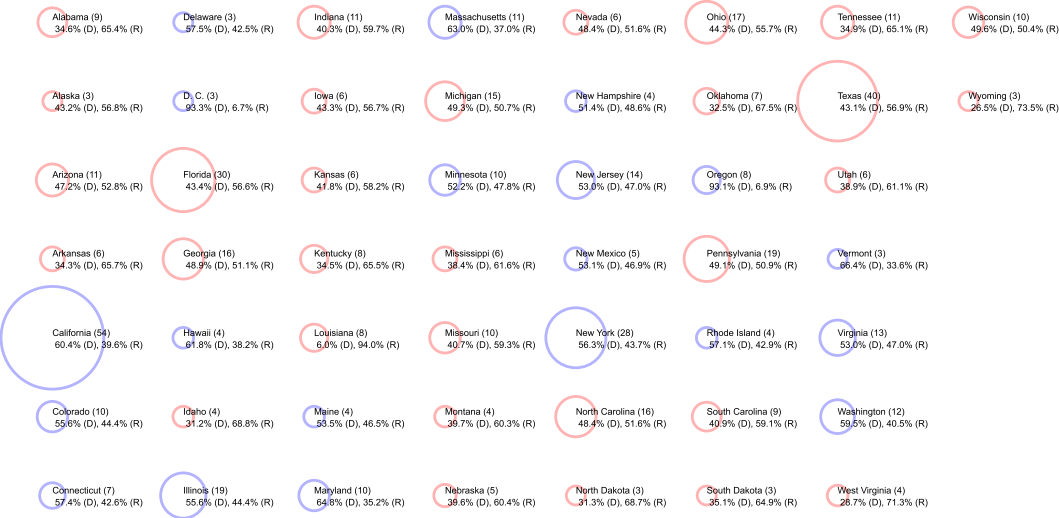
2020



Stochastische Modelle mit Monte-Carlo-Simulationen: Wahlsimulation

Und dann das Wahlergebnis 2024:

2024



Stochastische Modelle mit Monte-Carlo-Simulationen: Wahlsimulation

Da es immer eine rege Diskussion um „Auszählfehler“ gibt, können wir einmal untersuchen, wie die Ergebnisse aussehen würden, falls es einen zufälligen Auszählfehler mit Standardabweichung von 1% gegeben hätte. Der Code hierzu:

```
Matlab/Octave
% gegeben:
% - D_percentage (alle realen Ergebnisse in D-Prozent für jedes Wahlgebiet)
% - state_ec (die Wahlmänner pro Wahlgebiet)

sigma = 0.01; % Auszählfehler

N = 200000; % Anzahl der gewürfelten Auszählungen
D_ec = zeros(N,1);
for i = 1:N

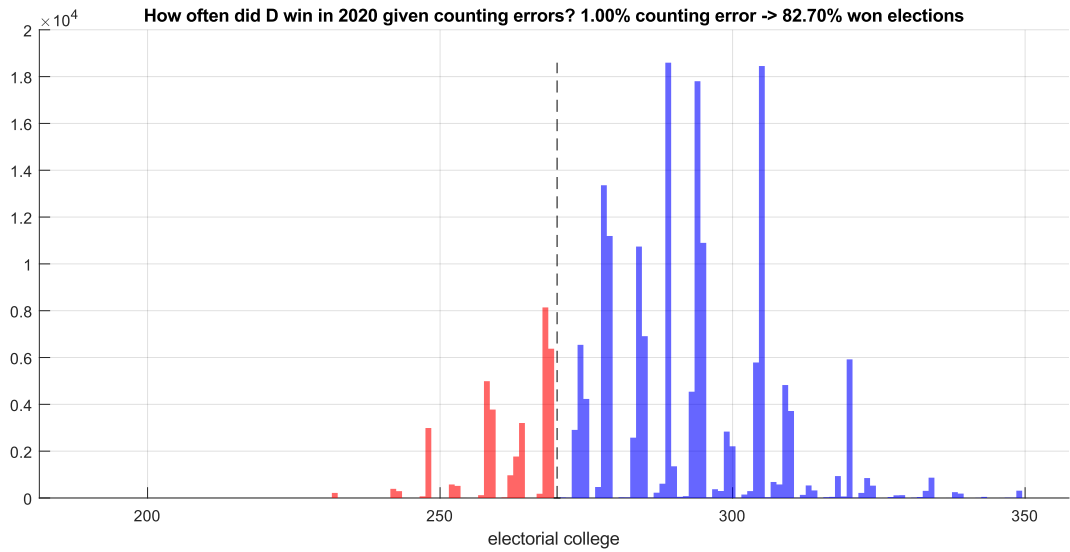
    random_shift = sigma*randn(size(D_votes)); % zufällige Prozentverschiebungen

    D_percentage_current = D_percentage + random_shift; % zufälliges Ergebnis
    D_percentage_current(D_percentage_current<0)=0;
    D_percentage_current(D_percentage_current>1)=1;

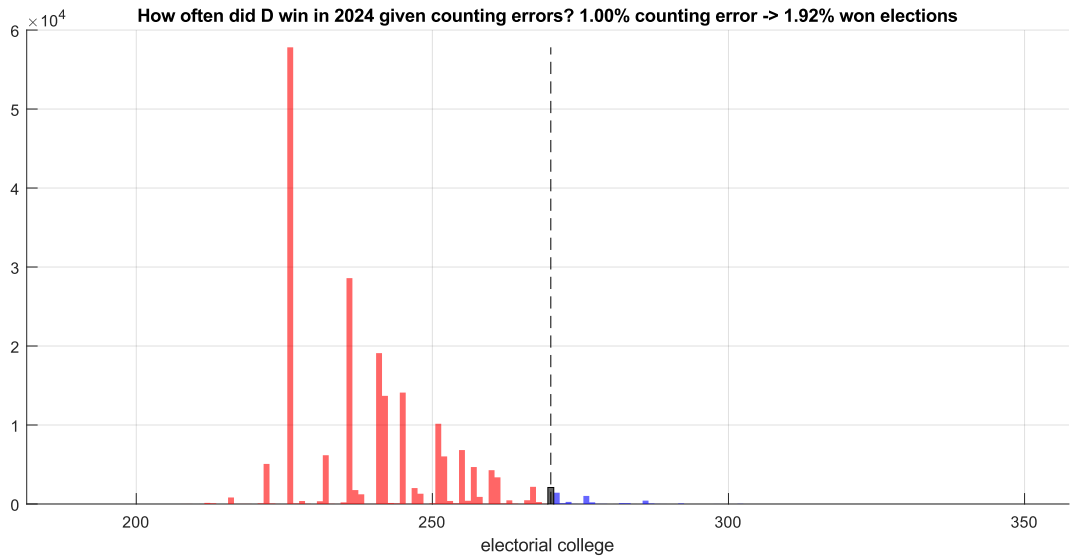
    D_ec(i) = sum(state_ec(D_percentage_current > 0.5)); % gewonnene Wahlmänner
end

D_win = sum(D_ec>=270)/N*100; % Prozentsatz der von D gewonnenen Wahlen
```

Stochastische Modelle mit Monte-Carlo-Simulationen: Wahlsimulation



Stochastische Modelle mit Monte-Carlo-Simulationen: Wahlsimulation



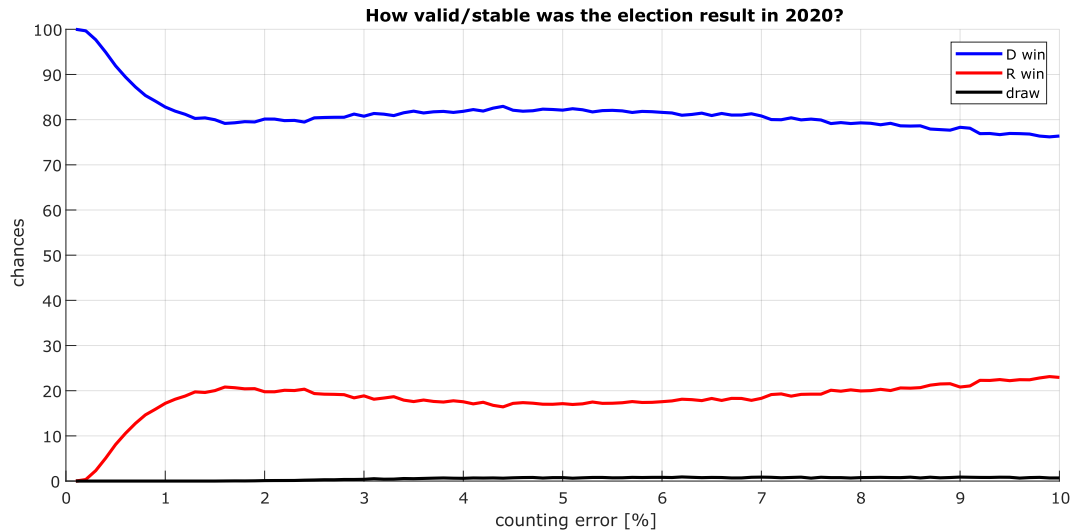
Stochastische Modelle mit Monte-Carlo-Simulationen: Wahlsimulation

Man erkennt deutlich, dass es nur eine diskrete Möglichkeit für Wahlmänner-Konstellationen gibt. Daher dieses „zerklüftete“ Histogramm.

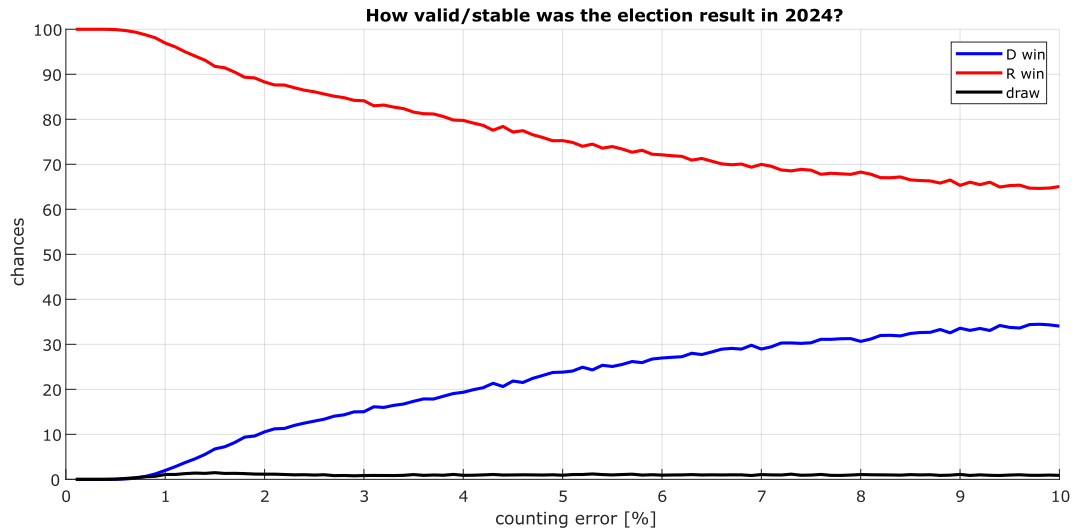
Es wird deutlich, dass es bei einem 1%-Auszählfehler teilweise durchaus Möglichkeiten gegeben hätte, dass der/die Unterlegene gewonnen hätte. D.h. wenn es zu Bedenken in dieser Größenordnung gekommen wäre, dann hätten sich „Neuauszählungen“ durchaus real lohnen können.

Weiterhin könnte man diese Wahrscheinlichkeit nun in Abhängigkeit der Größe des Auszählfehlers $\in [0.1, 10]\%$ betrachten. Hierzu führen wir die obige Simulation in einer Schleife mit wachsendem Auszählfehler durch und plotten den Anteil der gewonnenen Wahlen für jeden Kandidaten.

Stochastische Modelle mit Monte-Carlo-Simulationen: Wahlsimulation



Stochastische Modelle mit Monte-Carlo-Simulationen: Wahlsimulation



Man erkennt:

- 2020 kam es bis zu ca. 1.5% Auszählfehler zu einer drastischen Reduktion der Gewinnchancen um ca. 20% für Biden. Ein Zeichen dafür, dass es doch insgesamt eine knappe Wahl war (wenn Auszählfehler schon ein solches Gewicht hätten).
- 2024 gibt es für vergleichbare Auszählfehler nicht so einen drastischen Rückgang der Gewinnchancen für Trump. D.h. diese Wahlentscheidung wirkt eindeutiger/stabiler als 2020.
- Andererseits wird deutlich, dass es danach bis ca. 8% Auszählfehler 2020 keinen großen Einfluss mehr auf das Wahlergebnis gibt, der sich 2024 bis dorthin stetig verstärkt. Dies lässt sich durch die starke Nichtlinearität in diesem Modell erklären („Granularität“ der Wahlmänner in den Wahlgebieten + „Winner takes it all“).

Stochastische Modelle mit Monte-Carlo-Simulationen: Wahlsimulation

Neben den Auszählfehlern nach der Wahl ist eine stete Diskussion bzgl. der *Güte* von Wahlumfragen vor der Wahl ein wichtiges Thema. Insbesondere ist die Frage, wie sehr man sich auf Wahlumfragen *verlassen* kann entscheidend.

Wichtig ist die Erkenntnis, dass zu jeder Wahlumfrage immer ein typischer Standard-Fehler mitgeliefert wird, d.h. das ist der erwartbare Fehlerbereich der jeweiligen Umfrage.

Wir nähern uns der Frage wie folgt: Ausgehend von dem echten Wahlergebnis würfeln wir uns Wahlumfragen mit einem gegebenem Standard-Fehler und untersuchen dann, für jede so fix gewürfelte Wahlumfrage, durch Monte-Carlo-Simulationen, wie hoch die Gewinnchancen (genau unter der Verwendung dieser Wahlumfrage + seines Standard-Fehlers) für Biden bzw. Trump wäre.

Betrachten wir ein paar solcher Simulationen für jeweils eine Wahlumfrage.

Im Code könnte dies bspw. wie folgt realisiert werden:

```
sigma = 0.02; % Standard-Fehler der Wahlumfrage

% würfeln des Umfrageergebnisses
random_shift = sigma*randn(size(D_votes));
D_percentage_poll = D_percentage + random_shift;
D_percentage_poll(D_percentage_poll<0)=0;

% Monte-Carlo-Simulation unter Annahme der Umfrage + Standard-Fehler
N = 20000;
D_ec = zeros(N,1);
for i = 1:N

    random_shift = sigma*randn(size(D_votes));

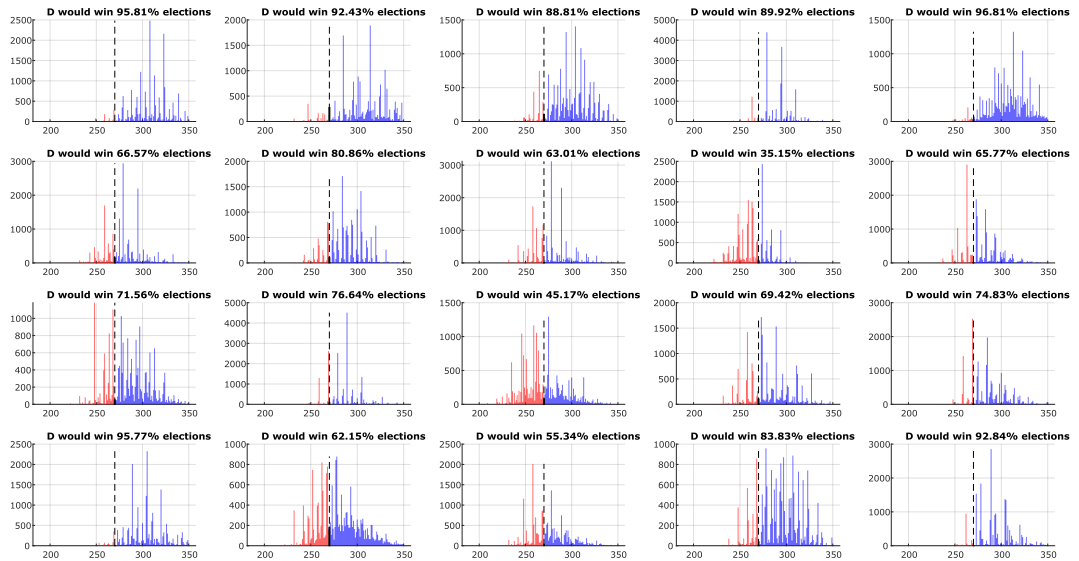
    D_percentage_current = D_percentage_poll + random_shift;
    D_percentage_current(D_percentage_current<0)=0;
    D_percentage_current(D_percentage_current>1)=1;

    D_ec(i) = sum(state_ec(D_percentage_current > 0.5));
end

D_win = sum(D_ec>=270)/N*100;
```

Stochastische Modelle mit Monte-Carlo-Simulationen: Wahlsimulation

How meaningful are polls with a 2.00% standard error in 2020?

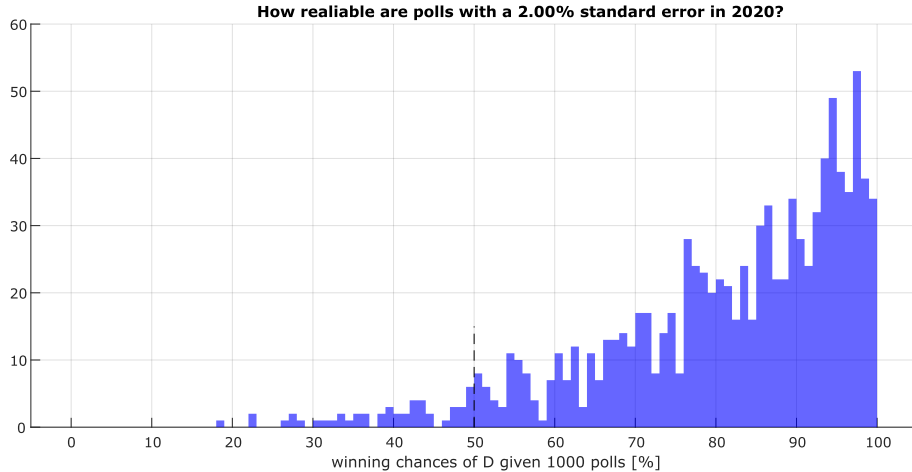


Man erkennt dabei:

- Für 2020 gibt es einzelne Umfragen, die bei einem nur 2% Standard-Fehler Biden (teils deutlich) als Verlierer sehen würden. Das ist *nicht utopisch*, da Wahlumfragen häufig einen Standard-Fehler im Bereich von 2 – 5% haben.
- D.h. selbst bei qualitativ guten Umfragen, findet man sehr leicht noch Umfrageergebnisse, die Trump als deutlichen Sieger gesehen hätten.

Auch das wollen wir noch etwas statistisch untermauern. Wir können anstatt der 20 Wahlumfragen in dem Bild 1000 Wahlumfragen würfeln und für jede dieser Wahlumfragen eine Gewinnchance für Biden 2020 errechnen. Interessant ist nun, wie viele Wahlumfragen gibt es, die Biden als Verlierer gesehen hätten.

Stochastische Modelle mit Monte-Carlo-Simulationen: Wahlsimulation



⇒ 4.90% der Umfragen würden Biden 2020 nicht siegen sehen. Teilweise sehr deutlich.

Man erkennt dabei:

- Es gibt noch einen nicht zu vernachlässigenden Prozentsatz an Wahlumfragen, die 2020 Trump als Sieger gesehen hätten.
- Teilweise gibt es einzelne Wahlumfragen, die Trump als „haushohen“ Favoriten gesehen hätten und Biden nur knapp 20% Gewinnchance eingeräumt hätten.
- Hier könnte natürlich eine psychologische Falle für eine:n Trump–Unterstützer:in 2020 vor der Wahl lauern: Wenn es diese eine qualitativ hochwertige Umfrage gibt, welche eine 80%ige Gewinnchance für Trump liefert, dann sieht es durch selektiven Fokus auf diese Umfrage eigentlich immer gut aus.

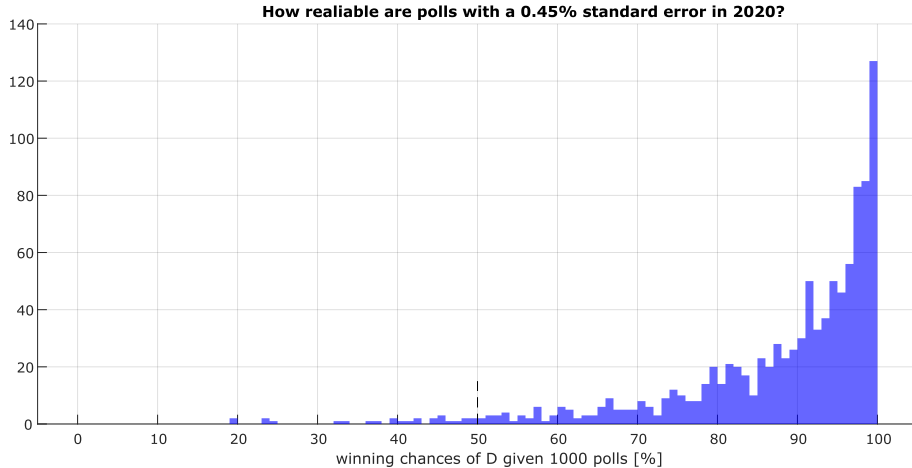
Die Auflösung des Fehlers: Es gibt nun mal sehr viel mehr Wahlumfragen, die 2020 Biden als Sieger gesehen hätten.

Stochastische Modelle mit Monte–Carlo–Simulationen: Wahlsimulation

Real passiert es ja nicht, dass man ganze Wahlsimulationen mit nur einer einzigen Wahlumfrage durchführt, sondern diese ebenso auch mittelt. D.h. wenn man annimmt, dass die Wahlumfragen alle qualitativ gleich hochwertig und unabhängig waren, müsste sich beim Mitteln von M Wahlumfragen der Standard–Fehler um den Faktor $\frac{1}{\sqrt{M}}$ reduzieren. *Warum?*

Gehen wir von einer Mittelung von $M = 20$ Wahlumfragen aus (die es zum Ende der Wahl sicherlich nahezu gleichzeitig gegeben haben könnte), dann reduziert sich der Standard–Fehler von 2% auf ca. 0.45%. Dies hätte folgendes Simulationsergebnis zur Folge:

Stochastische Modelle mit Monte-Carlo-Simulationen: Wahlsimulation



⇒ Diese Reduktion entspricht der Mittelung von ca. 20 Umfragen mit 2% Standard-Fehler. In diesem Fall würden 2.60% der Umfragen Biden 2020 nicht siegen sehen.

Stochastische Modelle mit Monte–Carlo–Simulationen: Wahlsimulation

D.h. zwar reduziert sich der Anteil der Umfragen, welche Trump 2020 als Sieger gesehen hätten, aber dann doch sehr „unterproportional“ von 4.90% auf 2.60%. Weiterhin ist zu erkennen, dass es dann (statistisch ganz erwartbar) immer noch einzelne Wahlumfragen mit gut 80% Siegchancen für Trump gibt – dies aber nur sehr selten.

Kurzum, die Zuverlässigkeit von Wahlergebnisvorhersagen durch Wahlumfragen ist eine sehr unsichere Geschichte. Problematisch ist zusätzlich, dass die realen Wahlumfragen in einem zeitlichen Kontext stattfinden (und die Akteure der Wahl ja versuchen aktiv die öffentliche Meinung unaufhörlich zu beeinflussen und es Skandale etc. gibt) und man auch deutlich weniger als die 1000 Wahlumfragen dieser Simulation für einen kleinen Zeitbereich zur Verfügung hat.

Aufgabe 1: Fallen Ihnen neben Auszählfehler und Wahlumfragen–Einschätzungen noch weitere interessante Fragestellungen für eine retrospektive Studie ein? Und wie würden Sie die Antwort darauf simulieren?

Aufgabe 2: Wie würden Sie ein Maß für die „Knappheit“ einer Wahl retrospektive formulieren? Wie würden Sie das prospektiv machen?

Bemerkung 3.23 (Wahlsimulation)

Das Besondere an dieser **Modellierung** ist bspw., dass

- das Modell der US–Wahl in einfachen Regeln ausgedrückt werden kann, welche sich jedoch insbesondere aufgrund der Diskretheit der Daten (bspw. Wahlmänner) mathematisch nicht mehr schön beschreiben lassen (Nichtlinearität + Diskretheit).
- Abhilfe schafft hier, dass wir den gesamten Wahlvorgang an sich simulieren können, und damit das Modell den realen Vorgang der Wahl einfach direkt nachbildet.

Bzgl. der **Simulation** kann man feststellen, dass

- man mit Monte–Carlo–Simulationen prospektive aber auch retrospektive Studien durchführen, und damit statistisch relevante Aussagen findet – auch nach der Wahl. Die Simulation ist sehr einfach und ohne viel Aufwand durchzuführen.
- Die Schwierigkeit steckt also weder in der Modellierung, noch in der Simulation, sondern in der Formulierung von interessanten und relevanten Fragen!

Stochastische Modelle mit Monte–Carlo–Simulationen: Sensitivitätsanalyse an zufälligen Dgln (RDE)

Eine weitere Anwendung von Monte–Carlo–Simulationen ist den Einfluss von einzelnen Parameteränderungen in komplizierten bspw. dynamischen Modellen abzuschätzen. Insbesondere die Auswirkungen und ihre Vielfalt bzw. Spannweite abzuschätzen ist von besonderem Interesse. Diese Untersuchungen werden unter dem Begriff *random differential equations* (RDE) durchgeführt und wird als Teilgebiet der Uncertainty Quantification (UQ) verstanden.

Hierfür wollen wir die etwas aufwändigeren ODEs betrachten, welche selbst bei kleineren Parameteränderungen sehr unterschiedliche Dynamiken entfalten können.

Diese Parametervariationen können (wie allgemein schon erwähnt) entweder von einer

- *Unbekanntheit* der Parameter herrühren,
- oder reale, zufällige *Störungen* des Systems beschreiben.

Im Folgenden werden wir uns eher der Sichtweise der *Unbekanntheit* der Parameter bedienen.

Wir betrachten erneut das 3–Populationsmodell nach *Lotka–Volterra*:

$$\dot{B} = \beta B (K - B) - \gamma_1 B R$$

$$\dot{R} = \gamma_2 B R - \gamma_3 M R - \alpha_1 R - \eta_1 R$$

$$\dot{M} = \gamma_4 M R - \alpha_2 M - \eta_2 M$$

mit den Parametern $\alpha_1, \alpha_2, \beta, \gamma_1, \gamma_2, \gamma_3, \gamma_4, \eta_1$ und $\eta_2 > 0$. Die Parameter sind dabei zu Beginn gewählt mit

$$B(0) = 5, R(0) = 0.1,$$

$$M(0) = 0.001$$

$$\beta = 0.6$$

$$K = 10$$

$$\gamma_1 = 1, \gamma_2 = 0.75$$

$$\gamma_3 = 1, \gamma_4 = 0.5$$

$$\alpha_1 = 0.75, \alpha_2 = 0.5$$

$$\eta_1 = 0.1, \eta_2 = 0.1$$

Stochastische Modelle mit Monte–Carlo–Simulationen: Sensitivitätsanalyse an zufälligen Dgln (RDE)

In diesem Modell wollen wir einzelne oder Gruppen von Parametern in einer Monte–Carlo–Simulation variieren. Dies bedeutet:

- Es wird ein Parametersatz nach vorgegebenen Wahrscheinlichkeitsdichten gewürfelt.
- Mit jedem Parametersatz wird mit Hilfe eines ODE–solvers eine numerische Lösung errechnet und abgespeichert.
- In einer Schleife finden die beiden vorherigen Schritte für bspw. $N = 1000$ Simulationen statt.
- Da jede Lösung auf einem eigenen Zeitraster (vom ODE–solver vorgegeben) gelöst wird, werden alle Lösungen im ersten Schritt noch auf ein gleiches Zeitraster interpoliert.
- Durch diese Vielzahl von Lösungen kann man dann Statistiken für jeden Zeitpunkt aufstellen.

Stochastische Modelle mit Monte-Carlo-Simulationen: Sensitivitätsanalyse an zufälligen Dgln (RDE)

Im Code könnte das bspw. für die Variation der Startwerte beim numerischen Lösen einer ODE wie folgt realisiert werden:

```
% Allgemeine Parameter setzen
t_0 = 0; t_E = 15; t_span = [t_0,t_E];
y_0 = [5,0.1,0.001]; % Startwerte

t_span_int = linspace(t_0,t_E,1000); % Zeitraster auf das alle Lösungen interpoliert werden

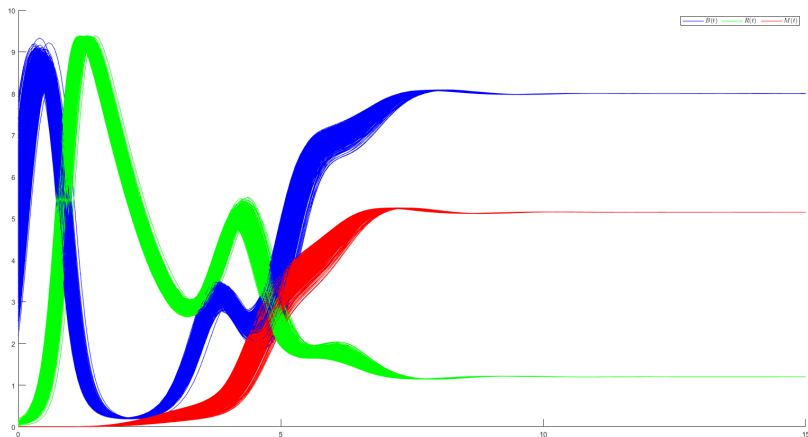
N = 1000; % Anzahl der Monte-Carlo-Wiederholungen
y_int = zeros(length(t_span_int),3,N); % Container in dem alle Lösungen liegen sollen

for i = 1:N
    % Würfeln der Startwerte (20% der Ausgangswerte):
    random_error = (y_0/5.*randn(1,3));
    y_start = abs(y_0 + random_error);

    % Lösen und ablegen der interpolierten Lösung
    [t,y] = ode45(@f,t_span,y_start);
    y_int(:, :, i) = interp1(t,y,t_span_int);
end
```

Stochastische Modelle mit Monte–Carlo–Simulationen: Sensitivitätsanalyse an zufälligen Dgln (RDE)

⇒ Plot aller Lösungen wenn nur die Startwerte $B(0), R(0), M(0)$ mit $\sigma = 20\%$ variiert werden.



Stochastische Modelle mit Monte–Carlo–Simulationen: Sensitivitätsanalyse an zufälligen Dgln (RDE)

Offensichtlich ist diese Darstellung recht unübersichtlich, da man kein Gefühl dafür bekommt, wie häufig sich die Kurven tatsächlich überlagern.

Um mehr Einsicht zu gewinnen, kann man auch eine klassische statistische Auswertung aller Kurvenverläufe für jeden einzelnen Zeitpunkt erstellen, d.h. in diesem Fall für jede Population den Mittelwert der Lösungen, die Standardabweichungen und die Minimum–Maximum–Spannweite ausrechnen.

Diese kann man dann graphisch auch sehr ansprechend aufbereiten.

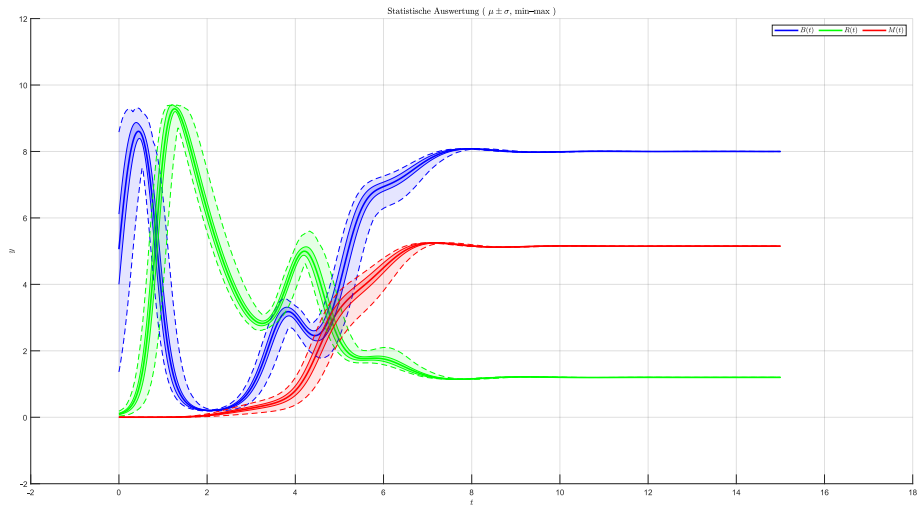
Beispielsweise kann man das in einem Code wie folgt auswerten:

Matlab/Octave

```
y_std = zeros(length(t_span_int),3);
y_max = zeros(length(t_span_int),3);
y_min = zeros(length(t_span_int),3);
y_mean = zeros(length(t_span_int),3);
for k = 1:3 % loop über die drei Populationen
    for i = 1:length(t_span_int) % loop über jeden Zeitpunkt
        y_mean(i,k) = mean(y_int(i,k,:));
        y_std(i,k) = std(y_int(i,k,:));
        y_max(i,k) = max(y_int(i,k,:));
        y_min(i,k) = min(y_int(i,k,:));
    end
end
```

Stochastische Modelle mit Monte-Carlo-Simulationen: Sensitivitätsanalyse an zufälligen Dgln (RDE)

⇒ Darstellung der statistischen Auswertung. Zu jedem Zeitpunkt wurde Minimum und Maximum (gestrichelte Linien), sowie Mittelwert μ (dicke Linie) und Standardabweichung σ (schmale Linien) der Simulationen berechnet.



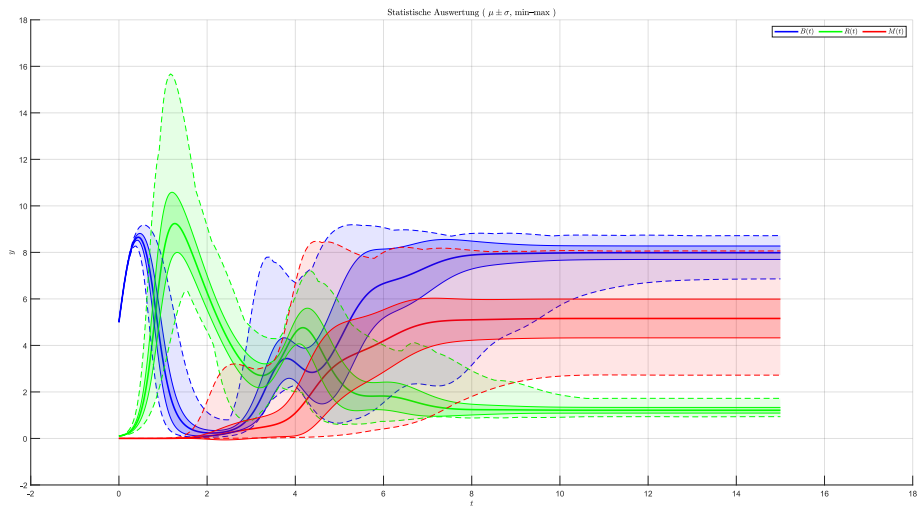
Stochastische Modelle mit Monte–Carlo–Simulationen: Sensitivitätsanalyse an zufälligen Dgln (RDE)

Eine spannende Aufgabe ist es nun die Empfindlichkeit von unterschiedlichen Parametern auf das gesamte System zu untersuchen. Im Folgenden betrachten wir mehrere Simulationsszenarien:

Stochastische Modelle mit Monte-Carlo-Simulationen: Sensitivitätsanalyse an zufälligen Dgln (RDE)

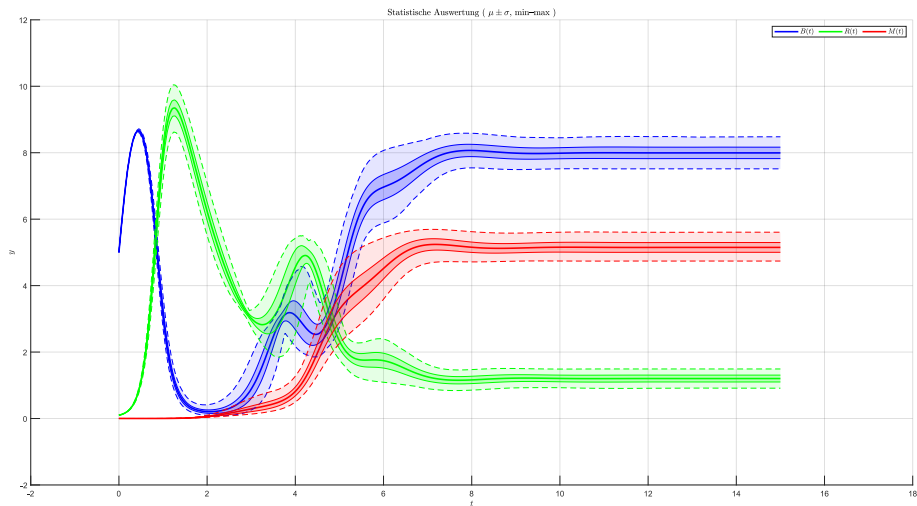
⇒ Darstellung der statistischen Auswertung wenn nur die „Kollisionsterme“

$\gamma_1, \gamma_2, \gamma_3, \gamma_4$ mit $\sigma = 10\%$ variiert werden.



Stochastische Modelle mit Monte–Carlo–Simulationen: Sensitivitätsanalyse an zufälligen Dgln (RDE)

⇒ Darstellung der statistischen Auswertung wenn nur die „Reduktionssterme“ $\alpha_1, \alpha_2, \eta_1, \eta_2$ mit $\sigma = 10\%$ variiert werden.



Man erhält mehrere Erkenntnisse:

- Man erkennt sofort, dass für die finale Vorhersage manche Parameter empfindlicher reagieren als andere. Beispielsweise sind hier die „Kollisionsparameter“ am Empfindlichsten bzgl. der finalen Gleichgewichte.
- Dies erlaubt auch einen Umkehrschluss zu: Will man eine verlässliche Vorhersage treffen, müssen diese Parameter mit besonderer Genauigkeit bestimmt werden.
- Insgesamt kann man damit auch Fehlerabschätzungen realisieren, d.h. wie stark sich *Unbekanntheiten* bei den Parametern auswirken. Bis hin zur Unbrauchbarkeit der Vorhersage.
- Eine weitere Erkenntnis bei dem Populationsmodell scheint zu sein, dass sich trotz vieler Variationen der Parameter final immer irgendein Gleichgewicht einzustellen scheint. Solche Systeme kann man bzgl. der Vorhersage als *stabil* bezeichnen.

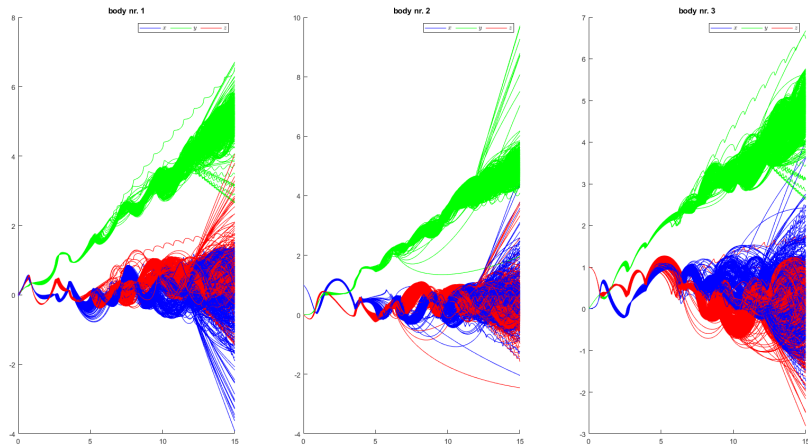
Stochastische Modelle mit Monte–Carlo–Simulationen: Sensitivitätsanalyse an zufälligen Dgln (RDE)

Als zweites Beispiel, welches qualitativ anders reagiert, wollen wir hier noch einmal das Dreikörper–Problem aus dem ODE–Abschnitt betrachten. Für dieses Modell werden ausschließlich die Anfangswerte auf einer sehr kleinen Skala variiert.

Analog zur vorherigen Simulationen wollen wir statistische Auswertungen für die Lösung dieses ODE–Systems durchführen. Es gibt dabei 9 errechnete Funktionen, die x –, y – und z –Werte von jedem der 3 Körper, welche sich durch Gravitation beeinflussen. Es gibt insgesamt 18 Startwerte $\vec{y}_0$, nämlich die Anfangsorte und –Geschwindigkeiten der 3 Körper.

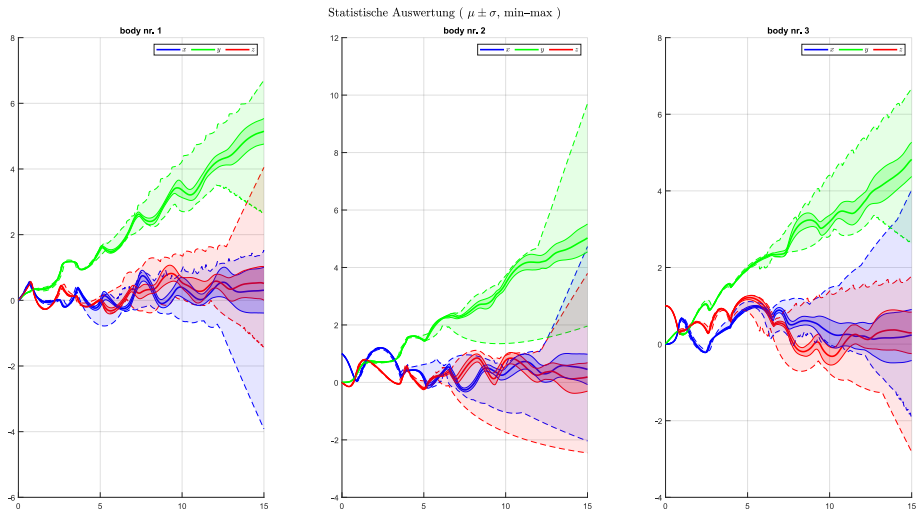
Stochastische Modelle mit Monte-Carlo-Simulationen: Sensitivitätsanalyse an zufälligen Dgln (RDE)

⇒ Plot aller Lösungen wenn nur der x -Startwert des 1. Körpers mit $\sigma = 0.01$ variiert wird.



Stochastische Modelle mit Monte-Carlo-Simulationen: Sensitivitätsanalyse an zufälligen Dgln (RDE)

⇒ Darstellung der statistischen Auswertung zu diesem Szenario



Es drängen sich folgende Erkenntnisse auf:

- Das System ist in höchstem Maße instabil bzgl. Vorhersagen, da trotz minimaler Änderungen die Vielfalt der Lösungen drastisch steigt.
- Praktisch bedeutet dies, dass jede noch so kleine Unkenntnis zu Beginn der Berechnungen irgendwann immer zu einem komplett anderem Verlauf führen können.
- Noch praktischer: Da wir immer nur endliche Genauigkeiten beim Betrachten der Natur haben, können wir den Lauf unseres Sonnensystems nicht beliebig lange in die Zukunft vorhersagen (viele Körper + Asteroiden + ..). Diese Simulation hilft uns jedoch zu verstehen, wie schnell unsere Vorhersagen bedeutungslos werden.
- Eine weitere Erkenntnis ergibt sich noch einmal beim Betrachten der direkt geplotteten Lösungen: Es scheint bei den y -Koordinaten des 2. Körpers im Bereich $t = 7$ Entscheidungspfade zu geben, an denen sich das System drastisch anders verhält. Solche spontanen Entscheidungspfade können typisch sein für hochgradig nichtlineare Systeme.

Aufgabe: Betrachten wir eine Anwendung mit elektrischen Schaltungen. In einem Lastenheft werden in einer Anwendung konkrete Spezifikationen bzgl. eines Schwingungs– bzw. Resonanzverhaltens gefordert. Sie haben mehrere Güteklassen für die Bauteile (Widerstand, Spule, Kondensator). Natürlich gilt: Je höhere Qualität die Güteklasse hat, desto teurer das Bauteil. Wie bestimmen Sie, mit Hilfe von Monte–Carlo–Simulationen welche Bauteilgüte Sie für das wirtschaftlich attraktivste System benötigen?

Bemerkung 3.24 (Parameterverteilungen in Differentialgleichungen)

Das Besondere an dieser **Modellierung** ist bspw., dass

- es leicht ist ein gegebenes durchaus kompliziertes Modell mit Hilfe von Monte–Carlo–Simulationen zu variieren.
- Die Komplexität der Modelle steigt dadurch nicht an, sondern es wird das gleiche Modell einfach nur wiederholt aufgerufen.

Bzgl. der **Simulation** kann man feststellen, dass

- es neue Qualitäten von System–Eigenschaften gibt: Wie bspw. *Stabilität* bzw. *Instabilität* bei Vorhersagen mit Modellen. Diese Instabilitäten können auch *spontan zu neuen Entscheidungspfaden* führen.
- Umgekehrt sind damit auch Sensitivitäten im Rahmen einer *Sensitivitätsanalyse* bzw. *Spezifikationen* für gewisse Parameter in einem Auslegungsprozess ableitbar.
- Die Darstellung der statistischen Auswertungen sind hier von besonderer Bedeutung und bedürfen besonderer Sorgfalt, damit die Ergebnisse gut verständlich bleiben.

Stochastische Modelle mit Monte–Carlo–Simulationen: Brown'sche Bewegung

Wie schon im vorhergehenden Abschnitt erwähnt, kann man nicht nur die Auswirkung der *Unbekanntheit von Parametern* simulieren, sondern auch Vorgänge, die tatsächlich *in sich Störungen* enthalten, sog. *stochastische Prozesse*.

Eine der bekanntesten stochastischen Prozesse ist die *Brown'sche Bewegung* B_t , welche die zufällige Bewegung von Partikeln beschreiben kann.

Wir definieren für den 1D–Fall:

- Der Startwert ist $B_0 = 0$.
- Zwischen zwei Zeitpunkten $s < t$ gilt, dass der Zuwachs *stationär* ist, d.h. von Zeitpunkt s zu Zeitpunkt t mit $B_t - B_s \sim \mathcal{N}(0, t - s)$ normalverteilt ist ($\mu = 0, \sigma = \sqrt{t - s}$) (sog. „weißes Rauschen“), und
- die Zuwächse *endlich vieler, nicht überlappender* Zeitintervalle unabhängig voneinander sind.

Stochastische Modelle mit Monte–Carlo–Simulationen: Brown'sche Bewegung

Man kann diese *Brown'sche Bewegung* mit Hilfe eines *Random Walks* simulieren, welcher wie folgt funktioniert:

- Wähle den zu simulierenden Zeitraum $[0, T]$.
- Wähle die Anzahl der Zeitpunkte N die man simulieren möchte.
- Erzeuge N unabhängige Ziehungen x_i von $\mathcal{N}(0, T/N)$.
- $b_k = \sum_{i=1}^k x_i$ ($k = 1, \dots, N$) ist dann eine diskrete Annäherung an eine Realisierung des kontinuierlichen Prozesses B_t .

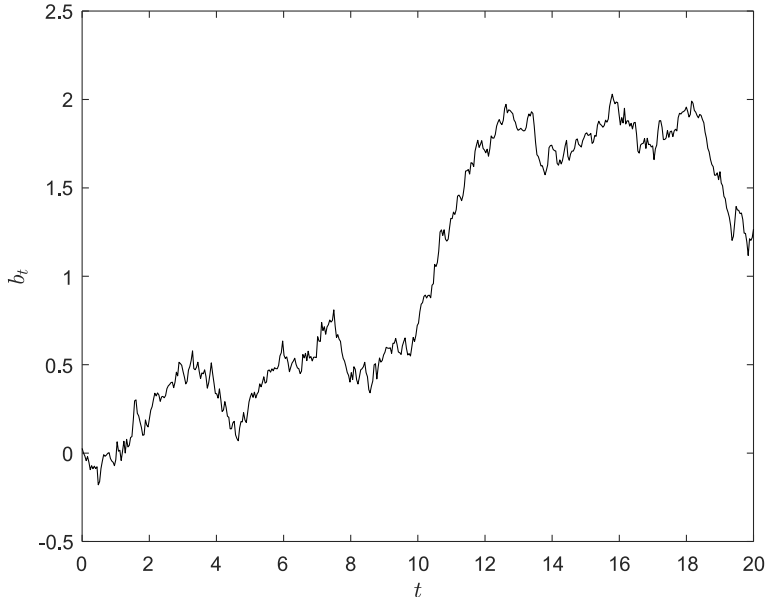
Beispielsweise kann man das in einem Code wie folgt ausführen:

Matlab/Octave

```
T = 20; N = 500;  
t = linspace(0,T,N);  
x = sqrt(T/N)*randn(size(t));  
b = cumsum(x)
```

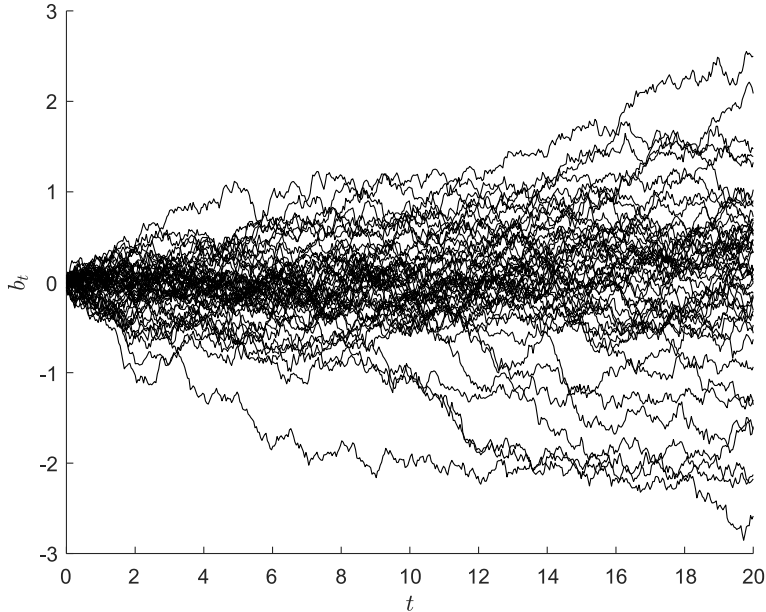
Stochastische Modelle mit Monte-Carlo-Simulationen: Brown'sche Bewegung

⇒ Darstellung eines einzelnen Realisierungspaths erinnert an einen *Aktienkurs*:



Stochastische Modelle mit Monte-Carlo-Simulationen: Brown'sche Bewegung

⇒ Darstellung von 50 Realisierungspfaden lässt die statistische *Einhülle* erahnen:



Stochastische Modelle mit Monte–Carlo–Simulationen: Brown'sche Bewegung

Eine Erweiterung ist die zufällige Bewegung von Partikeln in einem Raum. Hier wird einfach für jede Raumkoordinate eines Partikels eine unabhängige Brown'sche Bewegung angenommen. Beispielsweise für die x -Koordinate $B_{x,t}$ und für die y -Koordinate $B_{y,t}$.

Dies kann beispielsweise wie folgt simuliert werden:

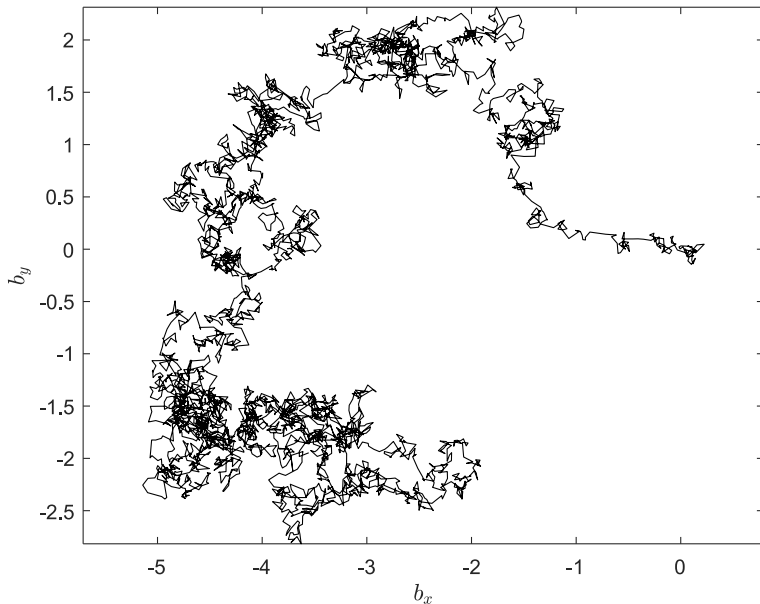
Matlab/Octave

```
T = 200;  
N = 4000;  
t = linspace(0,T,N);  
x = sqrt(T/N)*randn(size(t));  
bx = cumsum(x);  
y = sqrt(T/N)*randn(size(t));  
by = cumsum(y);
```

Die Beschreibung von Staub– bzw. Pollen–Bewegungen war auch die ursprüngliche *Motivation* für die mathematische Beschreibung der Brown'schen Bewegung.

Stochastische Modelle mit Monte-Carlo-Simulationen: Brown'sche Bewegung

⇒ Betrachten wir einen einzelnen Realisierungspfad für (b_x, b_y) :



Stochastische Modelle mit Monte–Carlo–Simulationen: Simulation stochastischer Dgln (SDE)

Neben den deterministischen *gewöhnlichen* (ODEs) und *partiellen* (PDEs) Differentialgleichungen und deren Sensitivitätsanalysen (als RDEs) gibt es auch noch eine Erweiterung zu *stochastischen* gewöhnlichen (SDEs) oder partiellen (SPDEs) Differentialgleichungen.

Die Idee ist wie folgt: Wir kombinieren die Dynamik von Dgln mit sich zeitlich änderndem zufälligem Rauschen.

Dies ist etwas anderes als die Sensitivitätsanalyse der Parameter mit RDEs inkl. statistischer Auswertung, da bei der Sensitivitätsanalyse jeder einzelne Simulationslauf der *deterministischen* Dgln folgt. Hier geht es jetzt jedoch um eine stochastische Störung **während der Entwicklung der Dynamik** in der Differentialgleichung.

Dies ist mathematisch / konzeptionell jedoch sehr anspruchsvoll, da ein „Rauschen“ typischerweise nicht differenzierbar im gewöhnlichen Sinn der Analysis ist. D.h. diese beiden Konzepte scheinen sich auszuschließen.

Stochastische Modelle mit Monte–Carlo–Simulationen: Simulation stochastischer Dgln (SDE)

Eine scheinbare Auflösung dieses Problems erreicht eine Umformulierung bspw. einer expliziten ODE:

$$z'(t) = \frac{dz(t)}{dt} = g(t, z(t))$$

wird symbolisch umgeformt zu

$$dz(t) = g(t, z(t)) dt ,$$

d.h. die differentielle Änderung des Werte $z(t)$ hängt von dem Vorfaktor $g(t, z(t))$ und der differentiellen Zeitänderung dt ab.

Hier ist nun die Idee von SDEs, dass man die differentielle Wertänderung nun mit stochastischen Störungen belegt, bspw.

$$dz(t) = g(t, z(t)) dt + h(t, z(t)) dB_t ,$$

mit der differentiellen Änderung der *Brown'schen Bewegung* dB_t . D.h. die momentane Wertänderung hängt nun zusätzlich konkret von dem Vorfaktor $h(t, z(t))$ und dem momentanen dB_t ab. Wir sagen, die SDE beschreibt einen *dynamischen stochastischen Prozess*.

Stochastische Modelle mit Monte–Carlo–Simulationen: Simulation stochastischer Dgln (SDE)

Die mathematische Theorie für Lösungen solcher SDEs ist sehr anspruchsvoll. Der Vorteil ist hier wieder einmal, dass die Modellierung und Simulation solcher SDEs jedoch verhältnismäßig einfache Simulationsverfahren ermöglicht.

Wir betrachten nun das Näherungsverfahren nach *Euler–Maruyama* für konkrete Simulationen. Dabei diskretisieren wir die Zeitschritte $t_i = t_0 + i \cdot \Delta t$ und erhalten damit auch einen diskreten Wertzuwachs von t_i nach t_{i+1} :

$$\begin{aligned} z(t_{i+1}) - z(t_i) &= g(t_i, z(t_i)) \Delta t + h(t_i, z(t_i)) (B_{t_{i+1}} - B_{t_i}) \\ \Leftrightarrow \quad z(t_{i+1}) &= z(t_i) + g(t_i, z(t_i)) \Delta t + h(t_i, z(t_i)) (B_{t_{i+1}} - B_{t_i}) . \end{aligned}$$

Hier zeigt sich nun die Einfachheit der *Brown'schen Bewegung*, da ja unabhängig für jeden Zeitfortschritt gilt

$$B_{t_{i+1}} - B_{t_i} \sim \mathcal{N}(0, \Delta t) ,$$

welche wir wieder durch einfache Pseudo-Zufallszahlen simulieren können.

Stochastische Modelle mit Monte-Carlo-Simulationen: Simulation stochastischer Dgln (SDE)

Dies kann beispielsweise wie folgt für die (Riccati'sche) SDE

$$dz(t) = (z^2(t) - t) dt + dB_t ,$$

simuliert werden:

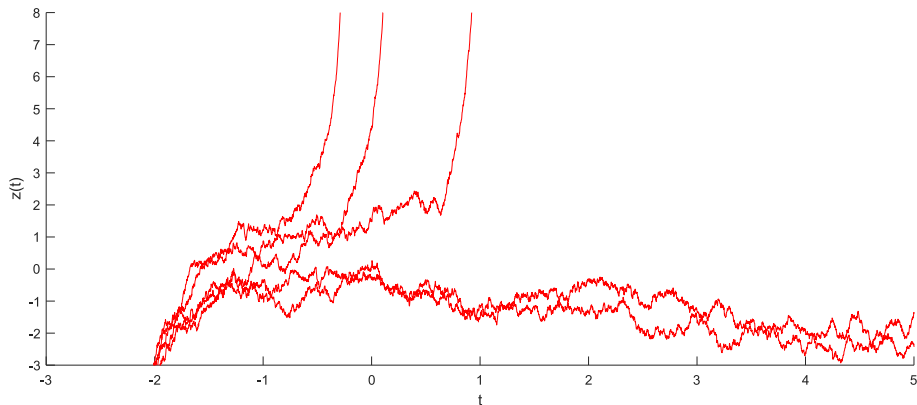
```
Matlab/Octave
start_val = [-2.1,-5];
N = 2^12; t = linspace(start_val(1),5,N);
Xem = zeros(1,N);

T = t(end)-t(1); dt = T/N;
dW = sqrt(dt)*randn(1,N); % brownian noise

% Euler-Maruyama-Method
Xtemp = start_val(2);
for j = 1:N
    Xtemp = Xtemp + dt*(Xtemp^2-t(j)) + dW; % Riccati'sche SDE
    Xem(j) = Xtemp;
end
```

Stochastische Modelle mit Monte-Carlo-Simulationen: Simulation stochastischer Dgln (SDE)

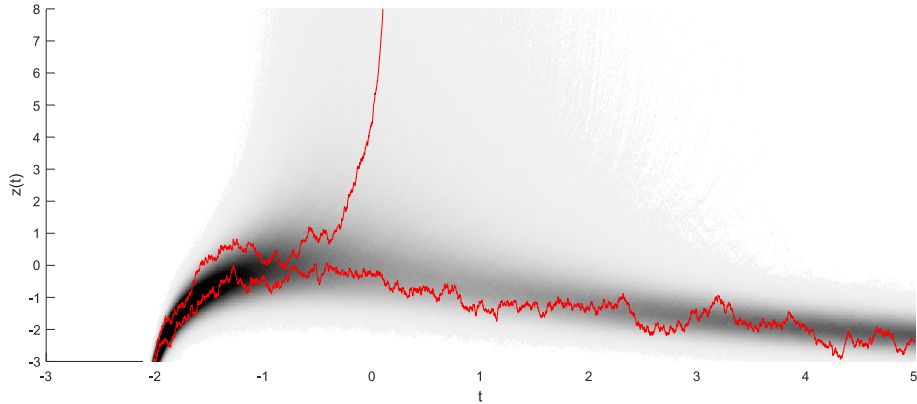
⇒ Führt man das bspw. 5 mal aus, ergeben sich aus dem Zusammenspiel von Dynamik und Rauschen folgende Realisierungspfade:



Bei genauer Betrachtung gibt es scheinbar zwei Möglichkeiten: Entweder die Werte steigen stark (Richtung unendlich) an oder sie laufen konstant nach rechts in einen *Attraktor*. Jeder Pfad ist jedoch höchst individuell und erlaubt keine allgemeinen Aussagen.

Stochastische Modelle mit Monte-Carlo-Simulationen: Simulation stochastischer Dgln (SDE)

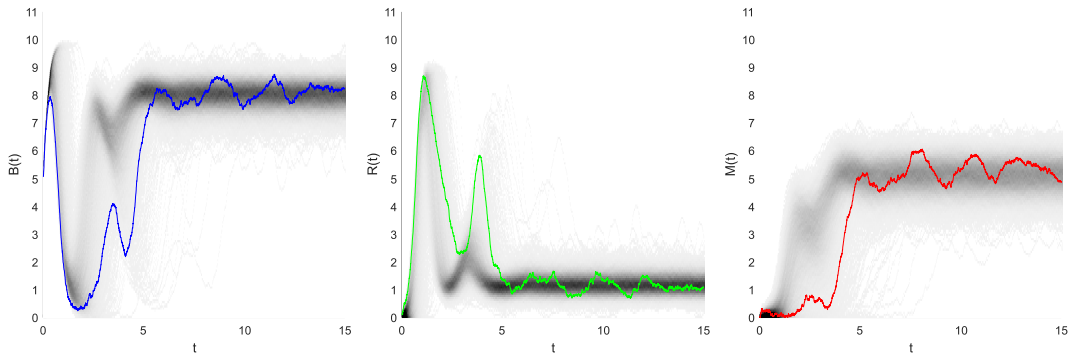
⇒ Führt man das nun 50000 mal durch und wertet die Ergebnisse mit einem 2D-Histogramm aus, so ergibt sich ein tieferer Blick in die Eigenschaften der SDE:



Wir sehen nun, wie alle Pfade beim Startwert beginnen (schwarz) und dann statistisch in einem breiten Korridor entweder stark (gegen unendlich) steigen oder nach rechts zu einem Attraktor konvergieren.

Stochastische Modelle mit Monte-Carlo-Simulationen: Simulation stochastischer Dgln (SDE)

⇒ Eine analoge statistische Untersuchung für das Räuber-Beute-Modell mit hinzugefügtem weißen Rauschen pro Population (und sonst gleiche Parameter wie aus der Sensitivitätsanalyse zuvor) mit 5000 Läufen, ergibt für die drei Populationen $B(t)$, $R(t)$ und $M(t)$:



Wir sehen wieder, wie alle Pfade beim Startwert beginnen und nach einer anfangs starken Dynamik auch wieder statistisch in die bekannten Gleichgewichtsbereiche konvergieren. Dies drückt in einer alternativen Betrachtungsweise die Stabilität dieser Populationsgleichgewichte aus.

Bemerkung 3.25 (Brown'sche Bewegung und SDEs)

Das Besondere an dieser **Modellierung** ist bspw., dass

- man echt zufällige Vorgänge wie die Bewegung von Partikeln durch ihre Temperatur mit Hilfe von stochastischen Prozessen beschreiben kann,
- welche mathematisch grundsätzlich nicht schwer zu formulieren sind. Die tatsächliche stochastische Untersuchung dieser Prozesse kann jedoch aufwändig sein.
- Mit den *stochastischen Differentialgleichungen* (SDEs) gibt es eine Erweiterung, welche die Dynamik eines Systems mit zufälligen Störungen des Systems zu jedem Zeitpunkt verbindet und zu einer Vielzahl von realen Beobachtungen passt.

Bzgl. der **Simulation** kann man feststellen, dass

- sie mit Hilfe von bspw. eines *Random Walks* sehr leicht umzusetzen ist.
- Durch eine Vielzahl solcher einzelner Realisierungspfade kann man wieder statistische Auswertungen durchführen, und damit auch für zufällige, dynamische Prozesse Kennzahlen (wie bspw. mit welchen Wahrscheinlichkeiten landet man wo in dem Prozess) generieren.

3. Modelle und ihre Simulation

Diskrete Modelle mit Dynamik

Diskrete Modelle mit Dynamik: Zelluläre Automaten

Nicht immer ist es möglich vollständige Gleichungen für interessante Modelle aufzustellen. Dies ist insbesondere der Fall, wenn einzelne lokale Einheiten des Systems im Laufe einer Simulation an die momentane Situation angepasste Entscheidungen treffen.

Solche Systeme beschreiben typischerweise eine *mikroskopische* Sichtweise auf eine reale Situation. Betrachten wir ein uns schon bekanntes Beispiel:

- Bei den Populationsmodellen hatten wir *makroskopische* ODE-Gleichungen betrachtet, die die Population als eine beliebig zerteilbare Größe betrachtet hat. Hätten wir jedoch nur bspw. 10 Individuen in einer Population, dann wäre es eher ungünstig diese mit dem ODE-System zu beschreiben, da bspw. das Aussterben der Population automatisch bei Populationsgröße < 1 geschehen wäre – d.h. die Diskretheit dominiert in diesem Fall. Es wäre daher eine *mikroskopische* Betrachtung mit den Individuen im Fokus zu bevorzugen.
- Dass die *mikroskopische* und *makroskopische* Sichtweise bei mittleren Größen ineinander übergehen ist meist eine sehr gute Überprüfung der Modelle.

Diskrete Modelle mit Dynamik: Zelluläre Automaten

Ein nützliche Gruppierung von diskreten Simulationen werden unter dem Begriff **zelluläre Automaten** zusammengefasst. Diese charakterisiert insbesondere:

1. Ein diskreter Zellraum. Dabei sind alle Zellen identisch und geometrisch gleich aufgebaut. Diese Zellen entsprechen dabei meist einer *Diskretisierung des Raums*. Je nach Anwendung können sie daher ein- oder mehrdimensional sein.
2. Jede Zelle kann nur eine diskrete Anzahl von Zuständen annehmen, d.h. bspw. *binär* („ist belegt bzw. nicht belegt“, oder „ist an bzw. aus“) oder *diskret mehrwertig*.
3. Die geometrische Anordnung der Zellen führt zu einer direkten Nachbarschaftsumgebung jeder Zelle. Die Nachbarschaft hängt von der geometrischen Form der Zellen ab und wird für alle Zellen gleich definiert.
4. Die Zeit wird ebenso gleichförmig diskretisiert. D.h. die Entscheidung über Zustandsveränderungen in allen Zellen findet gleichzeitig statt.
5. Die Zustände einer Zelle verändern sich pro Zeitschritt durch vorgegebene Regeln aufgrund des aktuellen Zustands der Zelle und nur seiner direkten Nachbarn.

Diskrete Modelle mit Dynamik: Zelluläre Automaten

Aufgabe: Welchen Gewinn bzw. welche Einschränkungen bzgl. der Anwendung von *zellulären Automaten* sehen Sie in der Simulation von realen Problemstellungen?

Diskrete Modelle mit Dynamik: Verkehrssimulationen: Das Nagel–Schreckenberg–Modell

Eine in der Vergangenheit wichtige Frage in der Verkehrssimulation war: Wie kommt es zu Staus? Ein wichtiges Phänomen ist dabei der „Stau aus dem Nichts“.

Wie dieser entstehen kann wurde mit dem eindimensionalen *Nagel–Schreckenberg–Modell* eindrucksvoll nachgewiesen. Das Modell ist ein zellulärer Automat und beruht auf folgenden Vorgaben:

- Eine einspurige Straße wird simuliert. Hierzu wird die Straße in N gleich großen Zellen unterteilt. Die Idee ist, dass in einer Zelle genau eine Verkehrsteilnehmer:in (bspw. ein Auto) Platz findet.
- Alle M Verkehrsteilnehmer:innen haben eine Position zu Beginn der Simulation und eine momentane Fahrgeschwindigkeit v_m ($m = 1, \dots, M$).
- Wir wählen die Zeitschritte bzw. möglichen Geschwindigkeiten so, dass eine Teilnehmer:in pro Zeitschritt nur ganzzahlig entweder $1, 2, \dots, v_{\max}$ Zellen vorrücken kann. Diese Bewegung wird jedoch dadurch beschränkt, dass eine andere Teilnehmer:in vorne weg fährt und damit die max. Geschwindigkeit beschränkt (Bremsen durch Kollisionsfreiheit).
- Es werden K solcher Zeitschritte simuliert.

Diskrete Modelle mit Dynamik: Verkehrssimulationen: Das Nagel–Schreckenberg–Modell

Betrachten wir einen Ausschnitt des Codes für das Setzen der Anfangsbedingungen für Positionen P und Geschwindigkeiten V pro Teilnehmer:in:

```
% Parameter
M = 180; % Anzahl Teilnehmer
v_max = 5*ones(1,M); % Maximalgeschwindigkeit pro Teilnehmer:in

N = 500; % Anzahl Zellen
K = 100; % Anzahl Zeitschritte

% Anfangsverteilung Positionen P und Geschwindigkeiten V
P = unique(ceil(rand(1,M)*N));
while length(P) ~= M
    P = [P,ceil(rand(1)*N)];
    P = unique(P);
end
P = sort(P);
V = zeros(1,M);
for i=1:M
    V(i) = ceil(rand(1)*(v_max(i)+1))-1;
end
```

Der ganze Ablauf lässt sich damit wie folgt zusammenfassen:

Algorithmus 3.5 (Regeln des vereinfachten NaSch–Modells, aus Bungartz et al. (2013))

In jedem Zeitschritt $k = 1, \dots, K$ gibt es ein Update für Fahrzeug $m \in \{1, \dots, M\}$, welches folgende Regeln durchläuft:

1. **Beschleunigen:**

$$v_m = \min\{v_m + 1, v_{\max}\}$$

2. **Bremsen/Kollisionsfreiheit:**

$$v_m = d(m, m+1), \text{ falls } v_m > d(m, m+1)$$

mit $d(m, m+1)$ der Abstand zwischen Fahrzeug m und $m+1$.

3. **Bewegung:**

Fahrzeug m bewegt sich v_m Zellen vorwärts.

Zur Vereinfachung formulieren wir folgende einfache *periodische Randbedingung*: Falls eine Verkehrsteilnehmer:in rechts aus dem Bild rausfährt, fährt er bzw. sie wieder links rein. Dies bedeutet, wir simulieren eine *Ringstraße*.

Ein zugehöriger Code für diese Schritte könnte bspw. wie folgt aussehen:

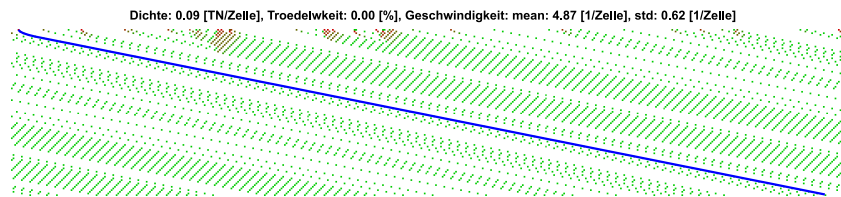
```
                                Matlab/Octave
% Zeitfortlauf
P_container = zeros(K,M); % Alle Positionen aller M Teilnehmer:innen
P_container(1,:) = P;
V_container = zeros(K,M); % Alle Geschwindigkeiten aller M Teilnehmer:innen
V_container(1,:) = V;
for k = 2:K
    % Beschleunigung
    V_new = V+1; V_new( V_new > v_max ) = v_max( V_new > v_max );
    % Periodische Fortsetzung + Kollisionsfreiheit (Bremsen)
    P_period = [P,P(1)+N]; P_dist = (P_period(2:end)-P_period(1:end-1))-1;
    V_new( V_new > P_dist ) = P_dist( V_new > P_dist );
    % Bewegung
    P = P+V_new;
    P(P>N) = P(P>N) - N;

    [P,idx] = sort(P); % Neusortierung
    V = V_new(idx);

    P_container(k,:) = P; % Ablegen aller aktueller Positionen
    V_container(k,:) = V; % Ablegen aller aktueller Geschwindigkeiten
end
```

Diskrete Modelle mit Dynamik: Verkehrssimulationen: Das Nagel–Schreckenberg–Modell

Betrachten wir das Simulationsergebnis mit $M = 45$ Verkehrsteilnehmer:innen:

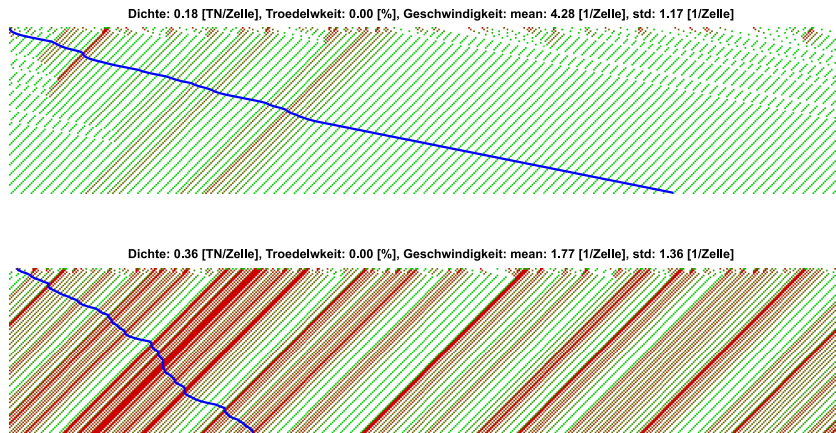


In dieser Darstellung sehen wir:

- Jede Verkehrsteilnehmer:in m zu einem Zeitpunkt k als farbiges Kästchen auf der Zeit–Orts–Tabelle mit Zeitpunkt $k = 1$ in der ersten Zeile und den M Verkehrsteilnehmer:innen in dieser Zeile auf N Zellen aufgeteilt.
 - Die Färbung der Kästchen gibt an, wie die momentane Geschwindigkeit des Fahrzeugs ist (rot für stehen, grün für fahren).
 - Eine Beispieltrajektorie eines Verkehrsteilnehmers ist durch die blaue Linie dargestellt.
- ⇒ Ganz konkret erkennt man damit, dass trotz anfänglicher Beschleunigungen der ersten Verkehrsteilnehmer:innen (welche nicht mit $v_{\max}$ fahren) sich für alle ein gleichmäßiges Tempo einstellt.

Diskrete Modelle mit Dynamik: Verkehrssimulationen: Das Nagel–Schreckenberg–Modell

Für höhere Verkehrsdichten $M = \{90, 180\}$ sehen wir folgende Darstellungen:



Dies bedeutet, qualitativ erkennt man, dass durch das höhere Verkehrsaufkommen tatsächlich die durchschnittliche Geschwindigkeit sich reduziert und es für individuelle Teilnehmer:innen durchaus zu Stauphasen kommt.

Diese Staus sind im Wesentlichen durch die Anfangsverteilung vorgegeben. Man erkennt dennoch schon, dass die Position des Staus wandert.

Das Besondere am Nagel–Schreckenberg–Modell ist jedoch, dass es auch „Staus aus dem Nichts“ erklären konnte. Hierzu musste man einen sog. *Trödelfaktor* einbauen:

Algorithmus 3.6 (Regeln des NaSch–Modells, aus Bungartz et al. (2013))

In jedem Zeitschritt $k = 1, \dots, K$ gibt es ein Update für Fahrzeug $m \in \{1, \dots, M\}$, welches folgende Regeln durchläuft:

1. **Beschleunigen:**

$$v_m = \min\{v_m + 1, v_{\max}\}$$

2. **Bremsen/Kollisionsfreiheit:**

$$v_m = d(m, m+1), \text{ falls } v_m > d(m, m+1)$$

mit $d(m, m+1)$ der Abstand zwischen Fahrzeug m und $m+1$.

3. **Trödeln:**

$$v_m = \max\{v_m - 1, 0\} \text{ mit Wahrscheinlichkeit } p < 1.$$

4. **Bewegung:**

Fahrzeug m bewegt sich v_m Zellen vorwärts.

Ein zugehöriger Code für diese Schritte könnte bspw. wie folgt aussehen:

```
Matlab/Octave
...
for k = 2:K
    % Beschleunigung
    V_new = V+1; V_new( V_new > v_max ) = v_max( V_new > v_max );
    % Periodische Fortsetzung + Kollisionsfreiheit (Bremsen)
    P_period = [P,P(1)+N]; P_dist = (P_period(2:end)-P_period(1:end-1))-1;
    V_new( V_new > P_dist ) = P_dist( V_new > P_dist );
    % Trödelei mit W'keit "troedel_wkeit"
    troedel = find(rand(1,M) < troedel_wkeit);
    idx_troedel_positive = (V_new(troedel) > 0);
    V_new(troedel(idx_troedel_positive)) = V_new(troedel(idx_troedel_positive))-1;
    % Bewegung
    P = P+V_new;
    P(P>N) = P(P>N) - N;

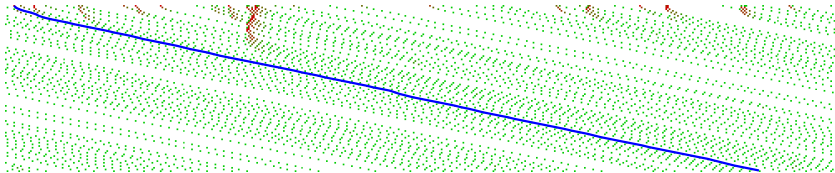
    [P,idx] = sort(P); % Neusortierung
    V = V_new(idx);

    P_container(k,:) = P; % Ablegen aller aktueller Positionen
    V_container(k,:) = V; % Ablegen aller aktueller Geschwindigkeiten
end
```

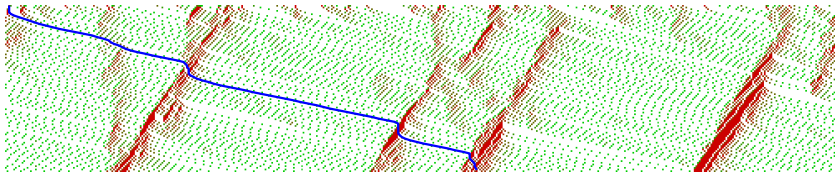
Die Ergebnisse hierzu sehen wie folgt aus:

Diskrete Modelle mit Dynamik: Verkehrssimulationen: Das Nagel-Schreckenberg-Modell

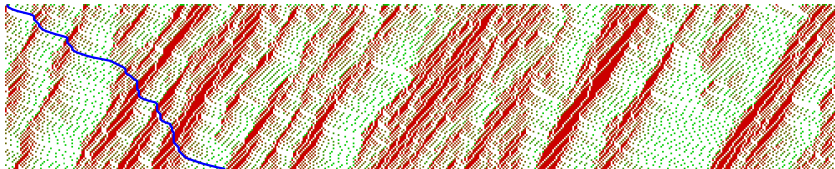
Dichte: 0.09 [TN/Zelle], Troedelwkeit: 20.00 [%], Geschwindigkeit: mean: 4.59 [1/Zelle], std: 0.83 [1/Zelle]



Dichte: 0.18 [TN/Zelle], Troedelwkeit: 20.00 [%], Geschwindigkeit: mean: 3.01 [1/Zelle], std: 1.79 [1/Zelle]

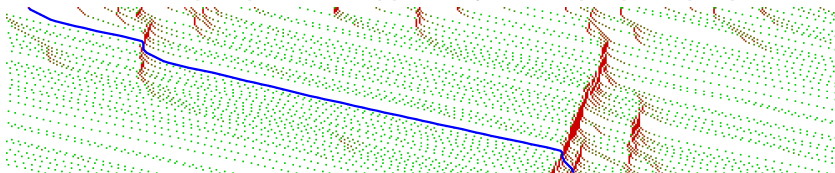


Dichte: 0.36 [TN/Zelle], Troedelwkeit: 20.00 [%], Geschwindigkeit: mean: 1.23 [1/Zelle], std: 1.31 [1/Zelle]

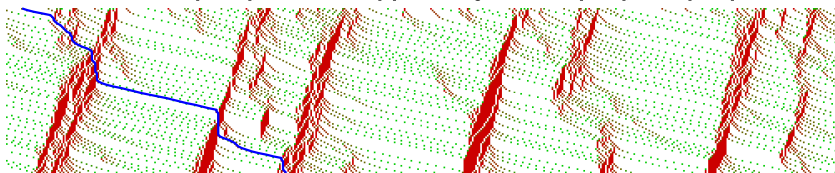


Diskrete Modelle mit Dynamik: Verkehrssimulationen: Das Nagel-Schreckenberg-Modell

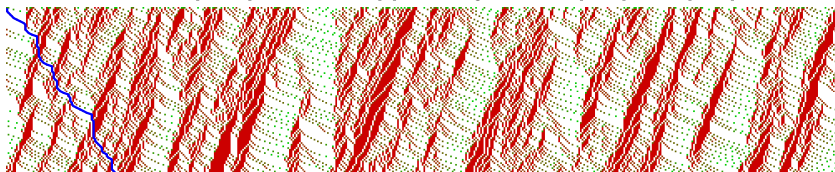
Dichte: 0.09 [TN/Zelle], Troedelwkeit: 50.00 [%], Geschwindigkeit: mean: 3.45 [1/Zelle], std: 1.61 [1/Zelle]



Dichte: 0.18 [TN/Zelle], Troedelwkeit: 50.00 [%], Geschwindigkeit: mean: 1.61 [1/Zelle], std: 1.71 [1/Zelle]



Dichte: 0.36 [TN/Zelle], Troedelwkeit: 50.00 [%], Geschwindigkeit: mean: 0.71 [1/Zelle], std: 0.97 [1/Zelle]



Diskrete Modelle mit Dynamik: Verkehrssimulationen: Das Nagel–Schreckenberg–Modell

In diesen Ergebnissen erkennt man trotz der vereinfachenden Modellannahmen:

- Allein das (leichte) Trödeln führt dazu, dass Staus aus dem Nichts entstehen können, selbst bei sehr geringem Verkehrsaufkommen.
- Und die Durchschnittsgeschwindigkeit bei geringer Dichte verringert sich von nahezu $v_{\max}$ um ca. 30 Prozent bei einer Trödelwahrscheinlichkeit von 50%.
- Daraus könnte man schließen: Würden alle Verkehrsteilnehmer:innen immer so schnell wie nur gerade möglich fahren, gäbe es weniger Staus. Da eine solche koordinierte Leistung bei so vielen individuellen Verkehrsteilnehmer:innen jedoch nicht möglich ist, muss man mit diesen Staus leben.
- Eine technische Abhilfe hierzu ist ein Tempomat mit Abstandssensor.

Diskrete Modelle mit Dynamik: Verkehrssimulationen: Das Nagel–Schreckenberg–Modell

Dies ist natürlich nur ein vereinfachtes Modell für eine Ringstraße. Sehr leicht kann man sich hierfür Erweiterungen vorstellen.

Aufgabe: Welche wesentlichen Komponenten fehlen dann im Modell noch, damit man bspw. eine realistische Simulation in einer Stadt simulieren kann?

Diskrete Modelle mit Dynamik: Beispiele für Zelluläre Automaten in 2D

Zelluläre Automaten in 2D sind interessant, da es

- eine deutlich reichhaltigere Nachbarschaftsbeziehung gibt, und
- man sich die Ergebnisse als Bilder sehr schön veranschaulichen kann.

Im Folgenden wollen wir die mögliche Vielfalt der Anwendung an drei Beispielen demonstrieren.

Diskrete Modelle mit Dynamik: Beispiele für Zelluläre Automaten in 2D

In der 1. Anwendung zeigen wir, wie *Gebäudeschutz* realisiert werden kann. Hierzu stellen wir uns folgendes Szenario vor:

In einem Gebäude tritt an einer Stelle eine Flüssigkeit oder ein Gas aus. Das Gas oder die Flüssigkeit kann sich nicht durch die Wände, jedoch durch die nicht dichten Türen verbreiten. Deshalb gibt es eine individuell an dem Grundriss orientierte Ausbreitung.

Praktische Fragen aus dem Gebäudeschutz könnten dabei sein:

- In Abhängigkeit vom Austrittsort X, wie lange benötigt bspw. ein Gas sich bis in den Raum Y auszubreiten?
- Praktisch: Wie lange hat man Zeit für Fluchtmöglichkeiten? Was sind sinnvolle Fluchtwege?
- Wo bringt man am effizientesten Warnmelder an (d.h. so wenig wie möglich, jedoch an den wichtigsten Stellen)?
- Wo muss man Notausgänge oder Lüftungen planen?

Die Regeln des zellulären Automaten sind dabei einfach:

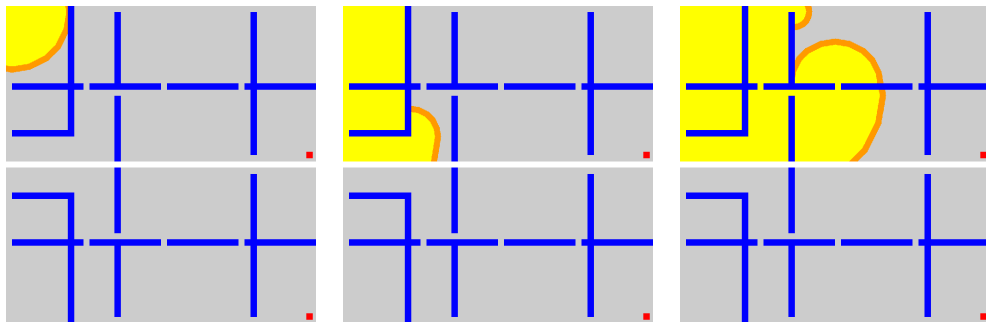
Algorithmus 3.7 (Zellulärer Automat für den Gebäudeschutz)

1. Ausgehend von dem realen Grundriss, diskretisiere jedes Geschoß mit seinem Grundriss als Pixel-Bild. Hierbei ist der Grundriss als eine Barriere-Matrix (bspw. barrier) festzulegen.
2. Definiere den Startort des Austritts und setze dort die Werte der Ergebnismatrix zum Zeitpunkt $k = 1$ (bspw. $\text{area}\{1\}$) auf 1 – sonst überall 0. Definiere einen Verbindungsort zwischen den Geschoßen.
3. Folge dann pro diskretem Zeitschritt folgenden Regeln:
 - Überall, wo $\text{area}\{k-1\}$ zum Zeitpunkt $k-1$ eine 1 liegt, wird mit vorgegebenen Ausbreitungsradius pro Zeitschritt zum Zeitpunkt k ein ausgefüllter Kreis um diese Pixel in $\text{area}\{k\}$ auf 1 gesetzt.
 - Ist eine Barriere (bspw. Wand) an den Pixeln vorhanden, die gerade auf 1 gesetzt wurden, dann setze diese wieder auf 0.
 - Wiederhole die Schritte für jeden Zeitpunkt k und überprüfe den Geschoßübergang.

Diskrete Modelle mit Dynamik: Beispiele für Zelluläre Automaten in 2D

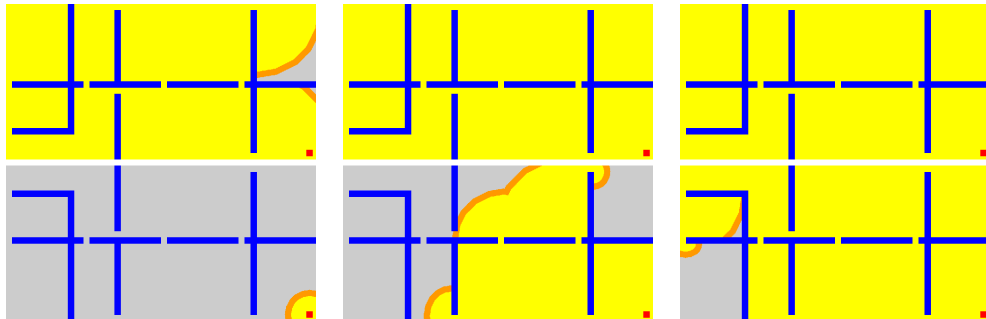
In der folgenden Simulation werden zwei Geschosse dargestellt (1. Zeile, 2. Zeile) zu jeweils drei Zeitpunkten pro Bild (1.–3. Spalte). Die Farbkodierung lautet:

- grau: Zellen ohne Gas/Flüssigkeit ($\text{area}\{k\}==0$).
- blau: der Gebäudegrundriss (barrier).
- gelb: Zellen mit Gas/Flüssigkeit ($\text{area}\{k\}==1$).
- orange: Zellen mit Gas/Flüssigkeit die im aktuellen Zeitschritt neu hinzukamen.
- rot: Der Geschoßübergang.



Diskrete Modelle mit Dynamik: Beispiele für Zelluläre Automaten in 2D

Es zeigt sich, wie ganz konkret sich das Fluid vollkommen über die zwei Geschoße ausbreitet.



Offensichtlich hilft diese Simulation wichtige Schlüsse für den Gebäudeschutz zu ziehen.

Ein Code könnte bspw. wie folgt aussehen:

```
Matlab/Octave
% .. "barrier"-Matrizen mit Grundriss schon definiert
area{1}{1} = zeros(M,N); % Startwerte im Geschoss 1
area{2}{1} = zeros(M,N); % Startwerte im Geschoss 2

idx = ( (X-20).^2 + (Y-20).^2 <= 15.^2 ); % Setze Austrittsort im Geschoss 1
area{1}{1}(idx) = 1;

[Mx,My] = meshgrid([-20:20],[-20:20]); % Aus Recheneffizienz: Definiere Nachbarschaft
idx = ( (Mx).^2 + (My).^2 <= 20.^2 );
Mx_rel = Mx(idx); My_rel = My(idx);

for k=2:K % Schleife über alle Zeitschritte
    for e = 1:E % Starte neuen Zeitschritt mit Ergebnis des vorherigen
        area{e}{k} = area{e}{k-1} ;
    end

    for e = 1:E % Schleife über alle Geschosse
        if k < 3 % nur auf neu hinzugekommenen Bereichen rechnen (Effizienz)
            new_area = area{e}{k-1};
        else
            new_area = area{e}{k-1};
            new_area( logical(area{e}{k-2}) ) = 0;
        end
    end
end

...
```

Diskrete Modelle mit Dynamik: Beispiele für Zelluläre Automaten in 2D

Matlab/Octave

```
...
    for i=1:M % Schleife über alle Pixel
        for j=1:N
            if new_area(i,j) == 1 % Rechnung nur, falls Pixel zuletzt hinzugekommen
                Mx_rel_cur = Mx_rel+j;
                My_rel_cur = My_rel+i;
                idx = (Mx_rel_cur < 1) | (Mx_rel_cur > N) | (My_rel_cur < 1) | My_rel_cur > M;
                Mx_rel_cur( idx ) = [];
                My_rel_cur( idx ) = [];
                ind = sub2ind([M,N],My_rel_cur,Mx_rel_cur);
                area{e}{k}(ind) = 1; % Setze alle Werte in dieser Nachbarschaft auf 1
            end
            if e==1 % Geschossuebergang definieren
                if area{e}{k}(480,980) == 1
                    area{2}{k}(480,980) = 1;
                end
            end
        end
    end
    area{e}{k}(barrier{e}) = 0; % Setze alle Werte in der Barriere auf 0
end
```

Diskrete Modelle mit Dynamik: Beispiele für Zelluläre Automaten in 2D

In der 2. Anwendung zeigen wir, wie sich der zelluläre Automat besonders effizient eignet, *Ausbreitungsverhalten von Infektionskrankheiten* (oder alternativ Gerüchten) in einem Netzwerk von Personen zu beurteilen.

Wichtige Fragen, die dabei interessant sind:

- Wie verhalten sich Infektionen unterschiedlicher R -Werte (oder Gerüchte unterschiedlicher Brisanz) in einer Nachbarschaft? (R -Wert: wie viele Personen steckt eine infizierte Person im Mittel an?) Bei wie vielen möglichen Übertragungskontakten breitet sich die Krankheit wie schnell aus?
- Oder umgekehrt: Wie viele Personen müssen immun gegenüber der Krankheit (geimpft oder genesen, etc.) sein, damit sich die Infektionsdynamik statistisch betrachtet zuverlässig stets selbst zum Erliegen kommt?
- Wie schnell/stark breitet sich die Infektion (oder das Gerücht) konkret aus?

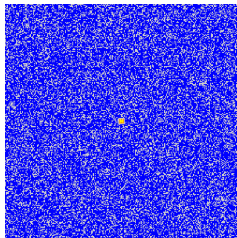
Die Regeln des zellulären Automaten sind dabei etwas aufwändiger:

Algorithmus 3.8 (Zellulärer Automat für den Infektionsschutz)

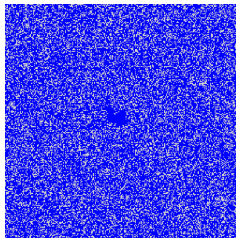
1. Es wird eine gewisse Zahl an zufällig verteilten immunen Personen gewürfelt und in (bspw. barrier) abgelegt.
2. Definiere den Startort des Austritts und setze dort die Werte der Ergebnismatrix zum Zeitpunkt $k = 1$ (bspw. $\text{area}\{1\}$) auf 1 – sonst überall 0.
3. Folge dann pro diskretem Zeitschritt folgenden Regeln:
 - Überall, wo in $\text{area}\{k-1\}$ zum Zeitpunkt $k - 1$ eine 1 liegt, wird (mit R -Wert und stat. Verteilung) die Anzahl der nächsten potenziell infizierten Nachbarn festgelegt (zwischen 1 und 8). Diese Nachbarn werden dann per Zufall bestimmt und die zugehörigen Werte in $\text{area}\{k\}$ auf 1 gesetzt.
 - Ist eine Barriere (bspw. immune Person) an den Pixeln vorhanden, die gerade auf 1 gesetzt wurden, dann setze diese wieder auf 0.
 - Wiederhole die Schritte für jeden Zeitpunkt k und setze Infizierte nach 3 Zeitschritten als Barriere („immun durch genesen“).

Diskrete Modelle mit Dynamik: Beispiele für Zelluläre Automaten in 2D

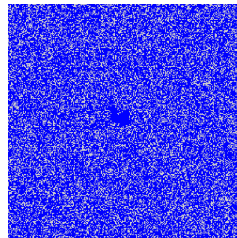
time step 3, impfdurchschnitt 70.0%, R-Wert 5.0



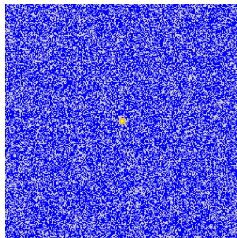
time step 50, impfdurchschnitt 70.0%, R-Wert 5.0



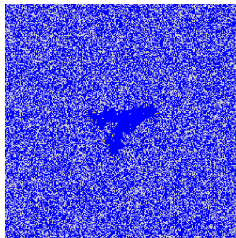
time step 100, impfdurchschnitt 70.0%, R-Wert 5.0



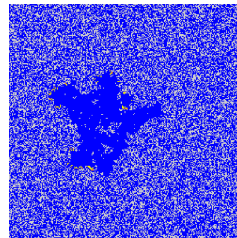
time step 3, impfdurchschnitt 60.0%, R-Wert 5.0



time step 50, impfdurchschnitt 60.0%, R-Wert 5.0

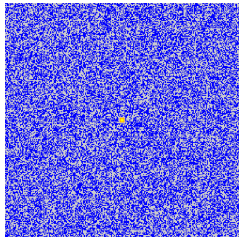


time step 100, impfdurchschnitt 60.0%, R-Wert 5.0

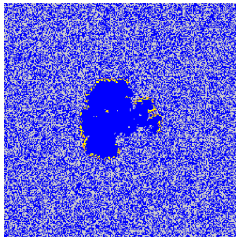


Diskrete Modelle mit Dynamik: Beispiele für Zelluläre Automaten in 2D

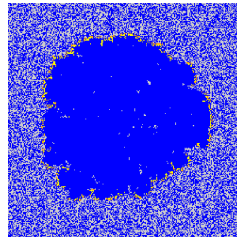
time step 3, impfdurchschnitt 50.0%, R-Wert 5.0



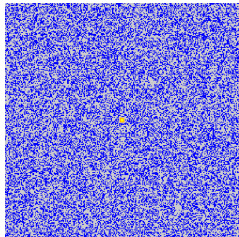
time step 50, impfdurchschnitt 50.0%, R-Wert 5.0



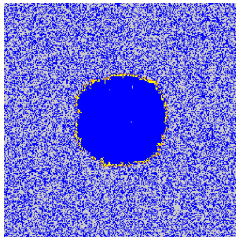
time step 100, impfdurchschnitt 50.0%, R-Wert 5.0



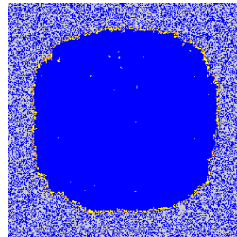
time step 3, impfdurchschnitt 40.0%, R-Wert 5.0



time step 50, impfdurchschnitt 40.0%, R-Wert 5.0



time step 100, impfdurchschnitt 40.0%, R-Wert 5.0



Diskrete Modelle mit Dynamik: Beispiele für Zelluläre Automaten in 2D

Ein Code könnte bspw. wie folgt aussehen:

```
Matlab/Octave
barrier_wkeit = 0.7;
barrier{1} = rand(M,N)<barrier_wkeit; % Setze zufällig verteilte Immunität

area{1} = zeros(M,N); % Setze Startwert der Infektionskette
area{1}(M/2,N/2) = 1;
barrier{1}(M/2+[-3:3],N/2+[-3:3]) = false; % Ermögliche initale kleine Ausbreitung

R_value = 5; % Setze R-Parameter der Infektion (bspw. covid delta variant)
std_R_value = 1; % statistische Verteilung der Infektionsausbreitung (std)

for k=2:K % Schleife über alle Zeitschritte
    area{k} = area{k-1};
    barrier{k} = barrier{k-1};
    for i=1:M % Schleife über alle Personen
        for j=1:N
            if area{k-1}(i,j) == 1 % Aus Recheneffizienz: Rechne nur dort wo Infektion
                min_i = max([1,i-1]); % Definiere Nachbarschaft
                min_j = max([1,j-1]);
                max_i = min([M,i+1]);
                max_j = min([N,j+1]);

                idx = [ [min_i,i,max_i]', [min_j,min_j,min_j]'; ...
                        [min_i,i,max_i]', [max_j,max_j,max_j]'; min_i,j; max_i,j ];

                ...
            end
        end
    end
end
```

Diskrete Modelle mit Dynamik: Beispiele für Zelluläre Automaten in 2D

```
...                               Matlab/Octave
R_value_cur = round(R_value + std_R_value*randn(1)); % individuelle Ansteckungsrate
if R_value_cur > 8 % max. 8 Nachbarn, mind. 1 Ansteckung
    R_value_cur = 8;
elseif R_value_cur < 1
    R_value_cur = 1;
end

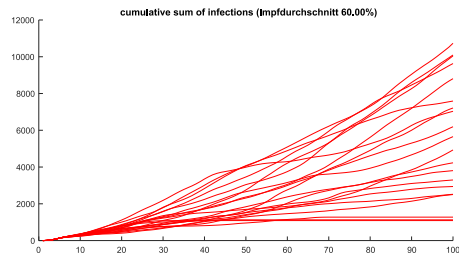
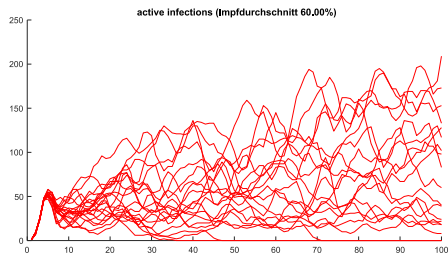
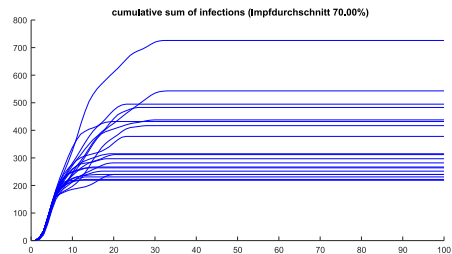
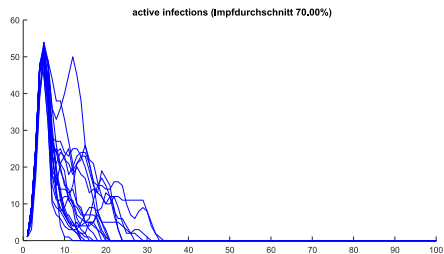
idx_sub = ceil(8*rand(R_value_cur,1)); % Wähle zufällige Nachbarn aus
idx_sub = unique(idx_sub);
while length(idx_sub) ~= R_value_cur
    idx_sub = [idx_sub; ceil(8*rand(R_value_cur-length(idx_sub),1))];
    idx_sub = unique(idx_sub);
end
for n = idx_sub % Setze die ausgewählten Nachbarn auf infiziert
    area{k}(idx(n,1),idx(n,2)) = 1;
end
end
if k > 3 % Führe nach 3 Zeitschritten alle Infizierten als Genesene der Barriere zu
    if area{k-3}(i,j) == 1
        barrier{k}(i,j) = true;
    end
end
end
end
area{k}(barrier{k}) = 0; % Setze alle Immunen auf nicht infiziert
end
```

Diskrete Modelle mit Dynamik: Beispiele für Zelluläre Automaten in 2D

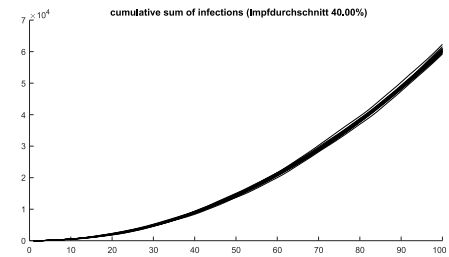
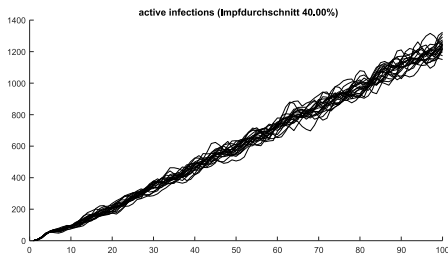
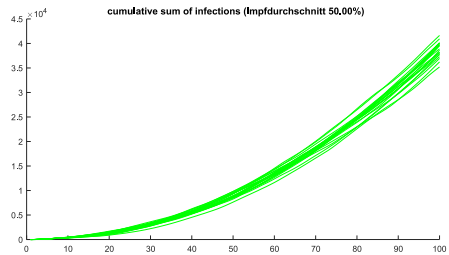
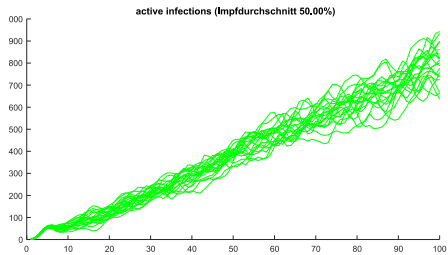
Man erkennt, in diesen 4 Szenarien, wie sich die Krankheit bei unterschiedlichen Impfdurchschnitten (zufällige Verteilung von Immunen als Teil der „Barriere“ in blau) ausbreitet (Infizierte in Gelb/Orange). Ab unter 50% Impfdurchschnitt scheint ein zuverlässiges, kaum gehindertes Wachstum möglich.

Das muss man statistisch jedoch noch etwas untermauern. Hierzu kann man diese Simulationen einfach pro Impfdurchschnitt mehrfach wiederholen und sich die Anzahl der Infizierten pro Zeitschritt abspeichern. Über bspw. 20 Simulationen pro Impfdurchschnitt ergeben sich folgende Darstellungen über die Zeit (links: Anzahl aktuell Infizierte, rechts: Gesamtzahl Infizierte).

Diskrete Modelle mit Dynamik: Beispiele für Zelluläre Automaten in 2D



Diskrete Modelle mit Dynamik: Beispiele für Zelluläre Automaten in 2D



Diskrete Modelle mit Dynamik: Beispiele für Zelluläre Automaten in 2D

Dies ist also eine Monte–Carlo–Simulation kombiniert mit zellulären Automaten.

Daraus ergibt sich, dass eine zuverlässige Eindämmung („Herdenimmunität“) ab ca. 70% Immunität in der Bevölkerung möglich ist bei $R = 5$. Bei 60% ist man in einer *transitiven* Situation (manchmal Ausbreitung, manchmal Erliegen des Infektionsgeschehens). Ab 50% und weniger herrscht exponentielles Wachstum unterschiedlicher Grade.

Diskrete Modelle mit Dynamik: Beispiele für Zelluläre Automaten in 2D

In der 3. Anwendung von zellulären Automaten betrachten wir das berühmte *Conway's Game of Life*: ein stark vereinfachtes biologisches Modell. Die Bedeutung von *Conway's Game of Life* ist vielfältig und zeigt u.a. wie aus Einfachheit Komplexität entstehen kann – eine Erkenntnis, die bspw. für die Evolutionstheorie von besonderer Bedeutung ist.

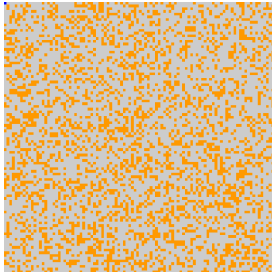
Das Spiel ist sehr einfach:

Algorithmus 3.9 (Zellulärer Automat: Conway's Game of Life)

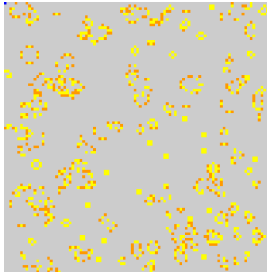
1. Bestimme zu Beginn eine Menge von lebenden Zellen, der Rest ist tot.
2. Folge dann pro diskretem Zeitschritt folgenden Regeln:
 - Jede Zelle die genau 3 lebende Nachbarn hat wird auch zum Leben erweckt („Geburt“).
 - Jede Zelle, welche keinen oder max. 1 lebenden Nachbarn hat stirbt („Tod durch Einsamkeit“).
 - Jede Zelle, welche mehr als 3 lebende Nachbarn hat stirbt („Tod durch Überbevölkerung“).
 - Alle anderen Zellen bleiben im vorherigen Zustand („stabile Lebensbedingung“).

Diskrete Modelle mit Dynamik: Beispiele für Zelluläre Automaten in 2D

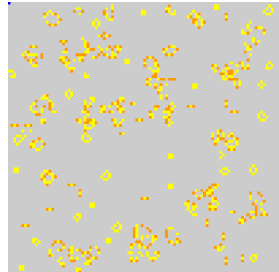
time step 1



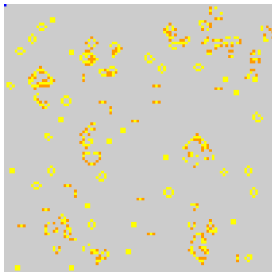
time step 50



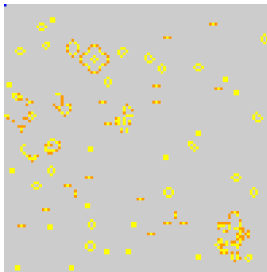
time step 100



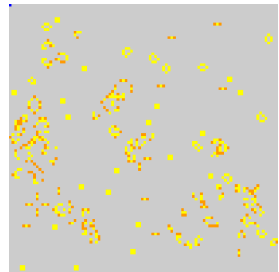
time step 200



time step 300



time step 500



Diskrete Modelle mit Dynamik: Beispiele für Zelluläre Automaten in 2D

```
Matlab/Octave
for k=2:K % Schleife über alle Zeitschritte
    area{k} = area{k-1};
    for i=1:M % Schleife über alle Zellen
        for j=1:N
            min_i = max([1,i-1]); % definiere Nachbarschaft
            min_j = max([1,j-1]); max_i = min([M,i+1]); max_j = min([N,j+1]);
            idx = [ [min_i,i,max_i]', [min_j,min_j,min_j]'; ...
                [min_i,i,max_i]', [max_j,max_j,max_j]'; min_i,j; max_i,j ];
            idx = unique(idx,'rows'); % vermeide Doppelzählung am Rand

            livingneighbor_count = 0; % Zähle lebende Nachbarn
            for n = 1:size(idx,1)
                if area{k-1}(idx(n,1),idx(n,2)) == 1
                    livingneighbor_count = livingneighbor_count+1;
                end
            end

            if livingneighbor_count == 3 % Geburt
                area{k}(i,j) = 1;
            elseif livingneighbor_count < 2 % Tod durch Einsamkeit
                area{k}(i,j) = 0;
            elseif livingneighbor_count > 3 % Tod durch Überbevölkerung
                area{k}(i,j) = 0;
            end
        end
    end
end
end
```

Diskrete Modelle mit Dynamik: Beispiele für Zelluläre Automaten in 2D

Es zeigt sich, dass nach einer großen Anzahl von Zeitschritten bei einer dichten Population eine Ausdifferenzierung stattfindet. Die konkreten Veränderungen (orange für im Zeitschritt neu hinzugekommene lebende Zellen und gelb sonstige lebende Zellen) hören nicht auf und stoßen sich stets gegenseitig an.

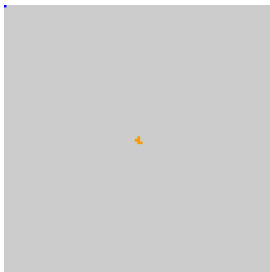
Offensichtlich ist hier eine große Vielfalt an *lebenden* Strukturen dauerhaft möglich. Gleichzeitig gibt es evolutionäres Entstehen von (unter diesen Regeln) stabilen Zellsystemen.

Im *Game of Life* entstehen vielfältige Strukturen, deren Komplexität nicht anhand der einfachen Regeln erahnbar sind und somit überraschen (siehe bspw. „Life in Life“).

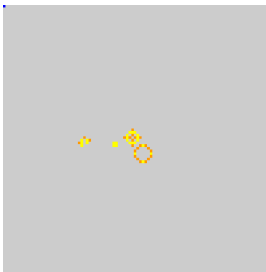
Ein Beispiel hierfür ist das sog. „r-Pentomino“. Aus diesem kleinen Zellhaufen entsteht unter diesen Regeln unerwartete komplexe Strukturen.

Diskrete Modelle mit Dynamik: Beispiele für Zelluläre Automaten in 2D

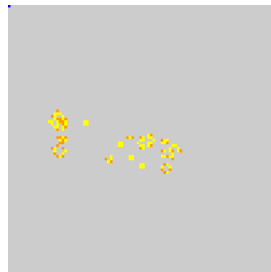
time step 1



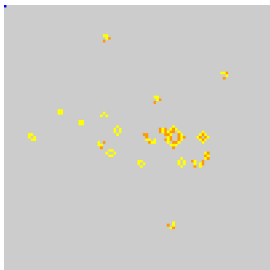
time step 50



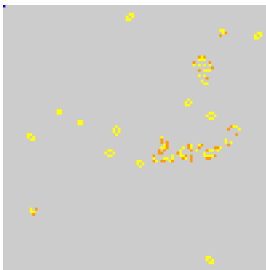
time step 100



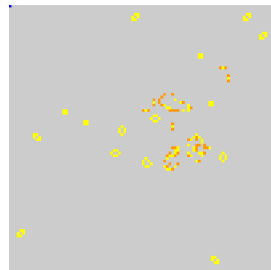
time step 200



time step 300



time step 500



Bemerkung 3.26 (Zelluläre Automaten)

Das Besondere an dieser **Modellierung** ist bspw., dass

- es vielfältige Anwendung dieser sehr einfachen Spiele gibt, die durchaus tiefgreifende oder nichttriviale Bedeutungen haben.
- Die Regeln sind in all diesen Spielen sehr einfach anzuwenden auf die direkte Nachbarschaft.

Bzgl. der **Simulation** kann man feststellen, dass

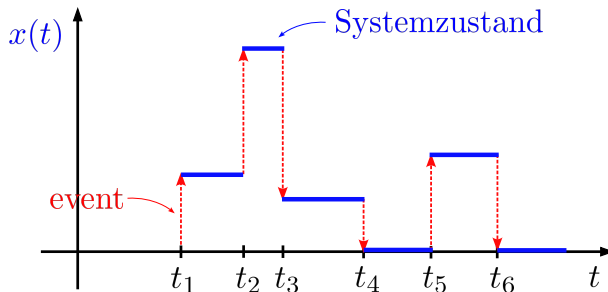
- man in diesen Simulationen effiziente Datenstrukturen benötigt, damit die Rechenzeit nicht zu groß wird.
- Man kann dabei jedoch auch unintuitives, überraschendes globales Verhalten entdecken, welches wir nicht anhand der Regeln erraten und uns mehr über die grundlegende Natur gewisser Modellierungen liefert („Komplexität aus Einfachheit“).

Diskrete Modelle mit Dynamik: Discrete Event Simulation (DES)

Eine weitere, recht große Klasse an Simulationsmethoden sind diskrete eventbasierte Simulationen (*Discrete Event Simulation, DES*).

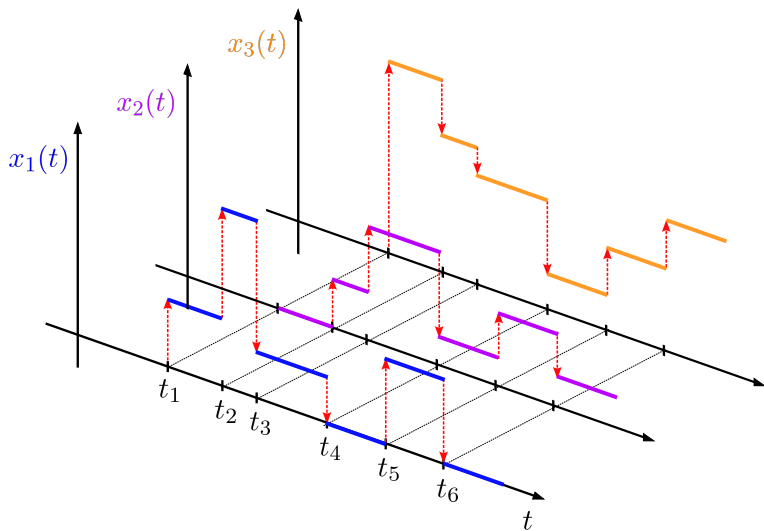
In diesen Simulationen modelliert man typischerweise reale Systeme nach, welche

- eine kontinuierliche Zeit t haben, jedoch
- in dieser Zeit nur zu diskreten Zeitpunkten $t_1, t_2, ..$ einzelne Ereignisse (*events*) stattfinden.
- Ein Ereignis verändert dabei Variablen des Systems $x(t)$, die sog. Einheiten des Systems. Zwischen Ereignissen bleibt der Systemzustand konstant.



Diskrete Modelle mit Dynamik: Discrete Event Simulation (DES)

Im Normalfall gibt es nicht nur eine Systemvariable, sondern einen Vektor $\vec{x}(t) \in \mathbb{R}^n$, welcher den Systemzustand in seinen Komponenten (*=Einheiten*) erfasst.



Beispiel 3.19 (DES)

- *Fahrsstuhl-Simulation*: $x_1(t)$ ist die Zielposition des Fahrschuhls, $x_2(t)$ Gewicht im Fahrsstuhl
- *Warteschlangen-Simulation*: $x_1(t)$ Anzahl Personen in der Warteschlange, $x_2(t)$ Kasse belegt oder nicht
- *Bedarfsampel-Simulation*: $x_1(t)$ Zustand Fußgängerampel, $x_2(t)$ Zustand Fahrzeugampel, $x_3(t)$ Anzahl Fahrzeuge an der Ampel

Bemerkung 3.27 (Simulation von DES)

Eine wichtige Erkenntnis von DES: Da zwischen Ereignissen der Systemzustand im Rahmen der Modellbildung konstant bleibt, kann man sich beim *Ablauf der Simulation* von Ereignis zu Ereignis hangeln. Dies ermöglicht eine effiziente Implementierung dieser Simulation.

Die wesentliche Vereinfachungen dieser Modellierung: Der Systemzustand bleibt ohne Ereignis konstant und Ereignisse werden sofort ausgeführt.

Diskrete Modelle mit Dynamik: Discrete Event Simulation (DES)

Die wesentliche Idee diese Simulation durchzuführen ist die zukünftige Ereignisliste (*Future Event List, FEL*). Die FEL beinhaltet alle Informationen über die Ereignisse, insbesondere: *Wann* es stattfindet und *Wie* sich der Systemzustand ändert.

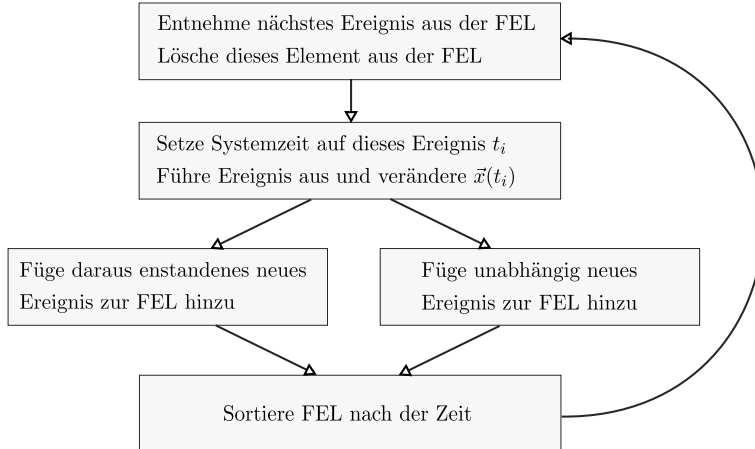
Zeitpunkt	Zustandsveränderung
t_0	$\vec{x} = \vec{0}$
t_1	$x_1 \rightarrow x_1 + 1, x_3 \rightarrow x_3 + 5$
t_2	$x_1 \rightarrow x_1 + 1.5, x_2 \rightarrow x_2 + 0.5, x_3 \rightarrow x_3 - 1$
t_3	$x_1 \rightarrow x_1 - 2, x_2 \rightarrow x_2 + 0.5, x_3 \rightarrow x_3 - 0.5$
t_4	$x_1 \rightarrow x_1 - 0.5, x_2 \rightarrow x_2 - 0.7, x_3 \rightarrow x_3 - 2$
t_5	$x_1 \rightarrow x_1 + 1.5, x_2 \rightarrow x_2 + 0., x_3 \rightarrow x_3 + 1$
t_6	$x_1 \rightarrow x_1 - 1.5, x_2 \rightarrow x_2 - 0.5, x_3 \rightarrow x_3 + 1$
$\vdots$	$\vdots$

Diskrete Modelle mit Dynamik: Discrete Event Simulation (DES)

Bei gegebener unveränderlicher FEL wäre diese Form der Simulation jedoch recht langweilig, da ja alles determiniert wäre.

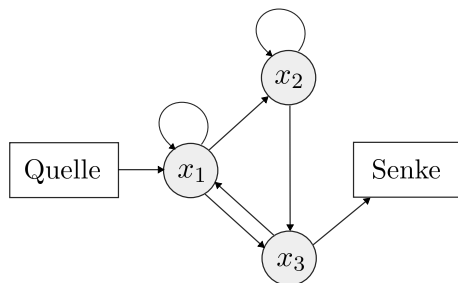
Eine wesentliche Komponente der *Dynamik*: Ein Ereignis kann auch andere zukünftige Ereignisse auslösen, welche einfach an die FEL angehängt werden.

Der Ablauf der Simulation ist dabei beispielsweise wie folgt:



Diskrete Modelle mit Dynamik: Discrete Event Simulation (DES)

Eine Möglichkeit die Abhängigkeiten in der Struktur der Dynamik bei einer endlichen Zahl von Systemvariablen $x_1, x_2, \dots$ darzustellen sind *Graphen*, wie Sie sie evtl. schon bei Markow-Ketten in der Anwendung der Linearen Algebra kennen gelernt haben. Die Knoten sind dabei die (sich ändernden) Systemvariablen und die Pfeile zeigen alle möglichen Ereignisse, die zu einer Veränderung der Systemvariablen führen.



Diese Darstellungen sind oft hilfreich für das Verständnis *was alles passieren kann*, jedoch werden dabei die zeitlichen Komponenten, d.h. die reale Dynamik, vernachlässigt und man weiß nicht *wie und wann sich die Systemvariablen ändern*. Es stellt damit eine *Topologie der Ereignisabhängigkeiten der Systemvariablen* dar. Oft dienen diese Graphen als *Startpunkt* der Modellierung einer DES.

Diskrete Modelle mit Dynamik: Discrete Event Simulation (DES)

Ein weiterer wesentlicher Bestandteil von DES sind *zufällige Komponenten*, wie bspw. *zufälliges Ziehen*

- neuer Zeitpunkte t_i für die FEL
- der Größe der Veränderung der Einheiten $x_j(t_i)$
- von Entscheidungspfaden bei Aufzweigungen welche x_j Änderungen erfahren

D.h. damit wird daraus auch eine Monte-Carlo-Simulation.

Bemerkung 3.28

Das Zusammenspiel von

- *Dynamik* (Ereignisse erzeugen neue Ereignisse, oder es gibt unabhängige Ereignisgeneratoren)
- *Systemabhängigkeiten* (Topologie)
- *Zufallsgrößen* (Inter-Arrival-Times, zufällige Zustandsänderungen)

führt zu unübersichtlichen Situationen im zeitlichen Gesamtverlauf des Systems, welche wir jedoch simulativ mit der FEL elegant simulieren können.

Beispiel 3.20 (Komplexe DES)

- *Logistik in Produktions-Netzwerken*: Ereignisse sind die Verschickungen von Arbeitspaketen, in einem Produktionsnetzwerk. Systemvariablen sind bspw. Lagerauslastung, Belegung von Verarbeitungseinheiten, Warteschlangen, etc.
- *Computer / Drucker Netzwerke*: Ereignisse sind Verschickung und Verarbeitung von Datenpaketen, Datenpaket wird als fehlerhaft erkannt. Systemvariablen sind bspw. Speicherauslastung, Druckerbelegung, etc.

Fallen Ihnen weitere Beispiele für komplexe DES-Anwendungen ein?

Bemerkung 3.29 (Auswertung von DES)

- Statistische Auswertung von Systemvariablen, bspw. Auslegung von Kapazitäten / Lagern / Speichern / Transferkanäle oder durchschnittliche Wartezeiten / Systemdurchlaufzeiten.
- Feststellen von Engpässen (=Bottle-Necks) in Netzwerken, und daraus Modellierung von Topologie-Anpassungen.

Beispiel 3.21 (Monte-Carlo-Simulation zur Auslastung auf Graphen)

Im Folgenden werden wir ein recht generisches Beispiel zur Simulation der Auslastung in Graphen betrachten. Zuerst definieren wir uns einen Beispiel-Graphen anhand der *Adjazenz-Matrix* eines *gewichteten, gerichteten Graphen*:

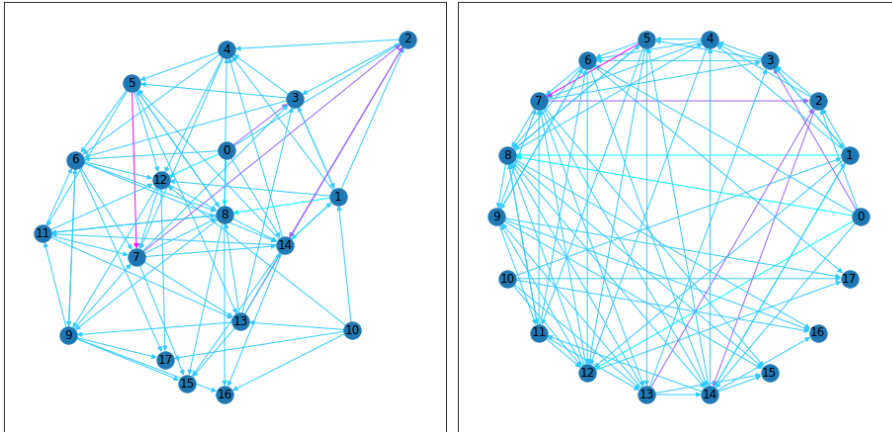
```

Python
A = np.array([[0., 0., 1., 2., 0., 0., 1., 0., 0.5, 0., 0., 0., 0.5, 0., 1. , 0., 0., 0.],
              [0., 0., 1., 1., 1., 0., 0., 0., 0.5, 0., 0., 0., 1. , 0., 0.5, 0., 0., 0.],
              [0., 0., 0., 1., 1., 0., 0., 0., 0. , 0., 0., 0., 0. , 0., 2. , 0., 0., 0.],
              [0., 1., 0., 0., 1., 1., 1., 0., 0. , 0., 0., 0., 0. , 0., 0. , 0., 0., 0.],
              [0., 0., 0., 0., 0., 1., 1., 1., 0. , 0., 0., 0., 1. , 0., 0. , 0., 0., 0.],
              [0., 0., 0., 0., 0., 0., 1., 3., 1. , 0., 0., 1., 1. , 1., 0. , 0., 0., 0.],
              [0., 0., 0., 0., 0., 0., 0., 1., 1. , 1., 0., 0., 1. , 0., 1. , 0., 0., 1.],
              [0., 0., 2., 1., 0., 0., 0., 0., 1. , 0., 0., 0., 0. , 1., 1. , 1., 0., 0.],
              [0., 0., 0., 0., 1., 1., 0., 0., 0. , 1., 0., 1., 0. , 0., 1. , 1., 1., 0.],
              [0., 0., 0., 0., 0., 0., 1., 1., 0. , 0., 0., 1., 0. , 0., 0. , 1., 1., 1.],
              [0., 1., 0., 0., 0., 0., 0., 0., 0. , 0., 0., 0., 1. , 1., 0. , 0., 1., 1.],
              [0., 0., 0., 0., 0., 0., 1., 1., 1. , 0., 0., 1., 1. , 1., 0. , 0., 0., 0.],
              [0., 0., 0., 0., 0., 0., 0., 1., 1. , 1., 0., 0., 1. , 0., 0. , 0., 0., 1.],
              [0., 0., 2., 1., 0., 0., 0., 0., 1. , 1., 0., 0., 0. , 1., 1. , 1., 0., 0.],
              [0., 1., 0., 0., 1., 1., 0., 0., 0. , 0., 0., 1., 0. , 0., 1. , 1., 1., 0.],
              [0., 0., 0., 0., 0., 0., 0., 0., 0. , 0., 0., 0., 0. , 0., 0. , 0., 0., 0.],
              [0., 0., 0., 0., 0., 0., 0., 0., 0. , 0., 0., 0., 0. , 0., 0. , 0., 0., 0.],
              [0., 0., 0., 0., 0., 0., 0., 0., 0. , 0., 0., 0., 0. , 0., 0. , 0., 0., 0.]])

```

Diskrete Modelle mit Dynamik: Discrete Event Simulation (DES)

Welchen wir in zwei Varianten darstellen:



Die Färbung der Kanten gibt das Gewicht (=Durchflusskapazität) an. Es gibt Knoten in denen Pakete nur einfließen (0 und 10) oder nur abfließen (15,16 und 17).

Zwischen allen anderen Knoten verteilt sich der Fluss der Pakete zufällig anhand der Durchflusskapazität der Kanten.

Diskrete Modelle mit Dynamik: Discrete Event Simulation (DES)

In dieser Simulation werden mehrere zufällige Größen nach Monte-Carlo erzeugt:

- Die Zeitabstände neuer Pakete in den Zuflussknoten sind *exponentialverteilt* gewürfelt.
- Die Bearbeitungszeit jedes Knotens (= Aufenthaltsdauer eines Pakets in einem Knoten) wird für jedes Paket ebenso *exponentialverteilt* gewürfelt.
- Die Paketgröße (=Skalierungsfaktor für die Aufenthaltsdauer eines Pakets in einem Knoten) wird *gleichverteilt* gewürfelt.
- Die Entscheidung an welchen Knoten das Paket danach weitergeht wird anhand der Verhältnisse der Durchflusskapazität abfließender Kanten gewürfelt.

Durch

- die unübersichtliche *Topologie* des gewichteten, gerichteten Graphen, und
- die dominanten *zufälligen Monte-Carlo* Anteile,

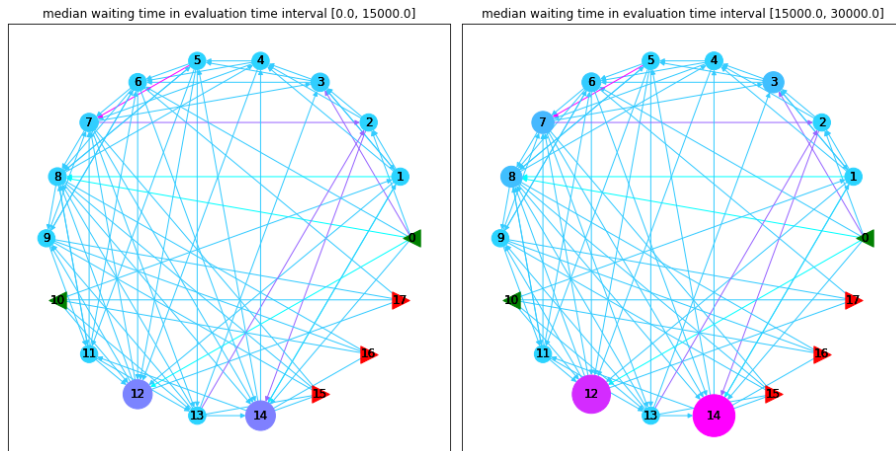
ist es unklar, wie sich die Flussdynamik der Pakete verhält.

Eine typischerweise wichtige Frage hierzu lautet: *Stauen sich die Pakete an gewissen Knoten?*

Eine Antwort hierzu liefert beispielsweise die *durchschnittliche Wartezeit* pro Paket an jedem Knoten im Fortlauf der Simulation.

Diskrete Modelle mit Dynamik: Discrete Event Simulation (DES)

Wir stellen die durchschnittliche Wartezeit in der ersten Hälfte der Simulation (links) und in der zweiten Hälfte der Simulation (rechts) dar:

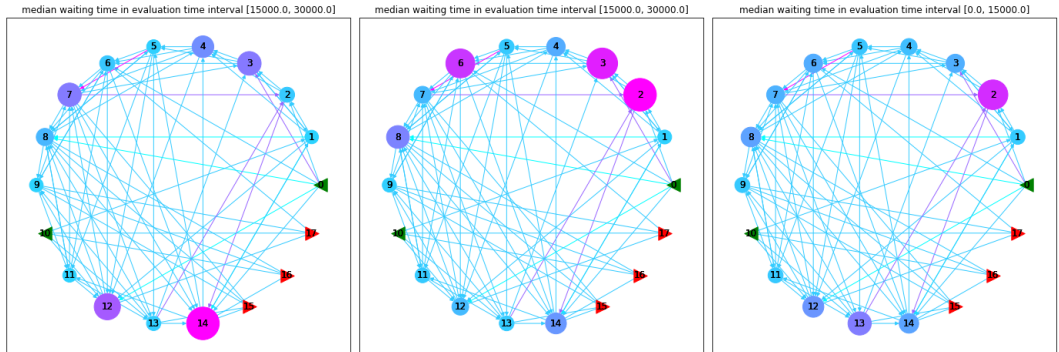


Die Färbung und Größe der Knoten deutet die durchschnittliche Wartezeit an.

Diskrete Modelle mit Dynamik: Discrete Event Simulation (DES)

Wir erkennen: 1. An Knoten 12 und 14 erhöht sich die Wartezeit dramatisch im Verhältnis zu den anderen (= „*Bottle-Necks*“). 2. An den restlichen Knoten ist die Bilanz zwischen eingehenden und ausgehenden Paketen ausgeglichener.

Passt man nun bspw. die Gewichte im Graphen nur leicht an, können sich schnell andere *Bottle-Necks* ergeben:



Diskrete Modelle mit Dynamik: Discrete Event Simulation (DES)

Aufgabe: Was sind die Stärken und Schwächen dieses (so vorgestellten) Modells: DES auf Graphen?

Bemerkung 3.30 (Diskrete Event-Simulation, DES)

Das Besondere an dieser **Modellierung** ist bspw., dass

- man nur mehr in Events zu gewissen (kontinuierlichen) Zeitpunkten denkt.
- Grundsätzlich sind die Regeln zur Event-Erstellung dabei meist durch lokal recht einfach definierte Folge-Events und eine *Topologie* (zumindest einfach beschreibbar) definiert.
- Grundlage der Denkweise ist eine zentrale *Future Event List* (FEL).

Bzgl. der **Simulation** kann man feststellen, dass

- dies nur mehr mit ausgeklügelten Softwarepaketen (bspw. *simpy* in Python) möglich ist, welche sich um die Logistik und das kontinuierliche Anpassen der FEL kümmert.
- In Situationen komplizierter Topologien und starker zufälliger Abhängigkeiten (bspw. DES auf großen Graphen), wird die graphische Aufarbeitung der statistischen Auswertung besonders wichtig, um bspw. *Bottle-Necks* schnell zu erkennen.

3. Modelle und ihre Simulation

Ein Warnschuss: Komplexität und Chaos im Einfachsten

Ein Warnschuss: Komplexität und Chaos im Einfachsten

Diese Vorlesung soll Ihnen ein „Warnschuss“ sein, damit Sie erkennen, wie schwer es ist ein gutes Modell aufzustellen, und dass es auch erfahrenen Modellierer:innen immer wieder schwer fällt alle Konsequenzen eines Modells zu kennen. Insbesondere:

- Es gibt viele Beispiele von einstürzenden Brücken und Gebäuden, welche durch entweder fehlerhafte mathematische Modellierung oder durch unbeabsichtigtes Weglassen von Einflussfaktoren der Realität auf ein Modell entstanden sind. Hierzu zählen bspw. unerwartete Resonanzeffekte in Bauteilen – dies führt häufig dazu, dass große Maschinen mittlerweile routinemäßig einer Reihe von mechanischen Stabilitätssimulationen unterzogen werden.
- In vielen Anwendungsbeispielen herrscht eine „Modellverliebtheit“, d.h. man findet gewisse mathematische Modelle spannend und lernt sehr viel über diese und zieht dann *ohne adäquaten Validierungsschritt* sofort Schlüsse für die Realität. Dies ist ein stark handwerklicher Fehler.
- Dem letzten wichtigen Hinweis wollen wir in dieser Vorlesung nachgehen: „Der Illusion der Kontrolle“. Nur weil Sie ein Modell aufgestellt haben, heißt es nicht, dass Sie es „beherrschen“. Ganz im Gegenteil, überraschen sehr häufig die Modelle die Modellierer:in während ihrer Simulation. Hier ist eine gewisse Offenheit zum Fehler und ein empirischer Zugang hilfreich.

Ein Warnschuss: Komplexität und Chaos im Einfachsten

Im ersten Teil möchte ich einen Erkenntnis aus den vorhergehenden Vorlesungen noch etwas gründlicher nachgehen: „Komplexität aus der Einfachheit“.

Stellen wir uns einmal das einfachste deterministische Modell vor: Beginnen wir mit den sehr einfachen *zellulären Automaten in 1D*. Betrachten wir hierzu folgende Rahmenbedingungen:

1	0	0	1	0	1	0	0	1	0	1	1
---	---	---	---	---	---	---	---	---	---	---	---

1. Jeder Zustand der 1D-Zelle (array) kann nur den Wert 0 oder 1 annehmen.
2. Wir betrachten immer 3er-Nachbarn einer Generation m und berechnen daraus den mittleren Wert der nächsten Generation $m + 1$.
3. Es gibt insgesamt $2^3 = 8$ Konstellationen, in denen 3er-Nachbarn vorliegen können $(0, 0, 0), \dots, (1, 1, 1)$. Eine Regel sagt nun, wie der mittlere Wert der nächsten Generation aussehen muss für alle 8 möglichen Konstellationen.
4. Da jeder dieser 8 Werte 0 oder 1 sein kann, gibt es nach Definition genau $2^8 = 256$ verschiedene Regeln. Diese nennen wir (nach dem Begründer dieser Untersuchungen) *Wolfram's Rules*.

Ein Warnschuss: Komplexität und Chaos im Einfachsten:

Eine eher „unelegante“, aber einfach zu lesende Version für die Anwendung der Regel ist in folgendem Code gegeben.

Matlab/Octave

```
function val = ApplyRule(p,rule)
```

```
if      all(p==[1,1,1])
    val = rule(1);
elseif all(p==[1,1,0])
    val = rule(2);
elseif all(p==[1,0,1])
    val = rule(3);
elseif all(p==[1,0,0])
    val = rule(4);
elseif all(p==[0,1,1])
    val = rule(5);
elseif all(p==[0,1,0])
    val = rule(6);
elseif all(p==[0,0,1])
    val = rule(7);
elseif all(p==[0,0,0])
    val = rule(8);
end
```

```
end
```

Ein Warnschuss: Komplexität und Chaos im Einfachsten:

Insgesamt ist der Code angewendet auf ein Startpixel in der Mitte wie folgt gegeben:

Matlab/Octave

```
N = 1001; M = 400; % Dimensions
```

```
v = zeros(M,N); % empty bitmap
```

```
% rule = [0,0,0,1,1,1,1,0]; % 30
```

```
% rule = [0,1,0,1,1,0,1,0]; % 90
```

```
rule = [1,0,0,1,0,1,1,0]; % 150
```

```
v(1,floor(N/2)) = 1; % starting point at center
```

```
for m = 2:M
```

```
    for n = 2:N-1
```

```
        v(m,n) = ApplyRule(v(m-1,n-1:n+1),rule);
```

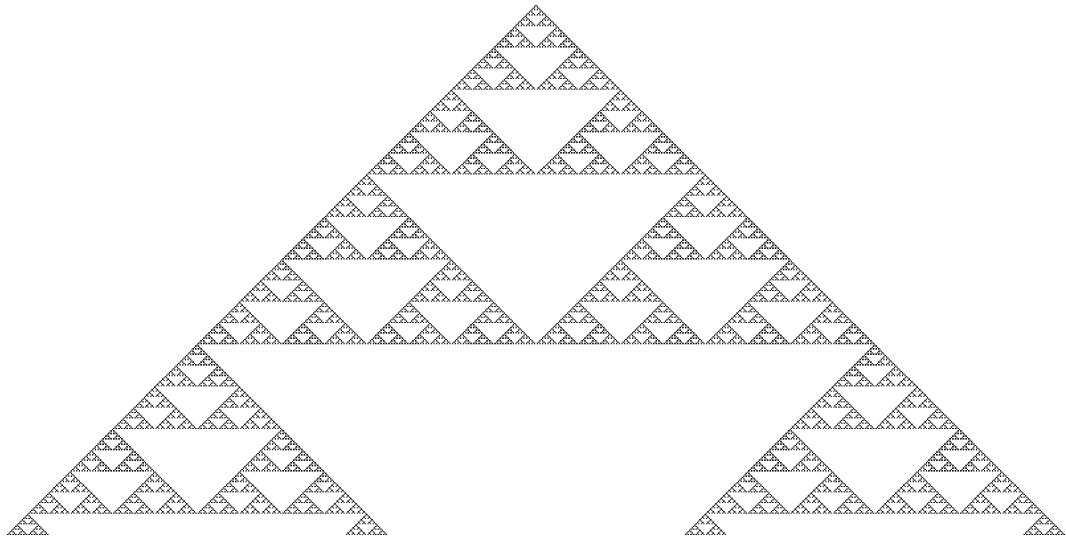
```
    end
```

```
end
```

Final stellen wir die Schwarz–Weiß–Bitmap v in folgenden Bildern dar. Die meisten Regeln sind dabei trivial und führen zu einfachen periodischen Mustern, oder zur Einfarbigkeit. Es ergibt sich jedoch auch bspw. für die folgenden Regeln:

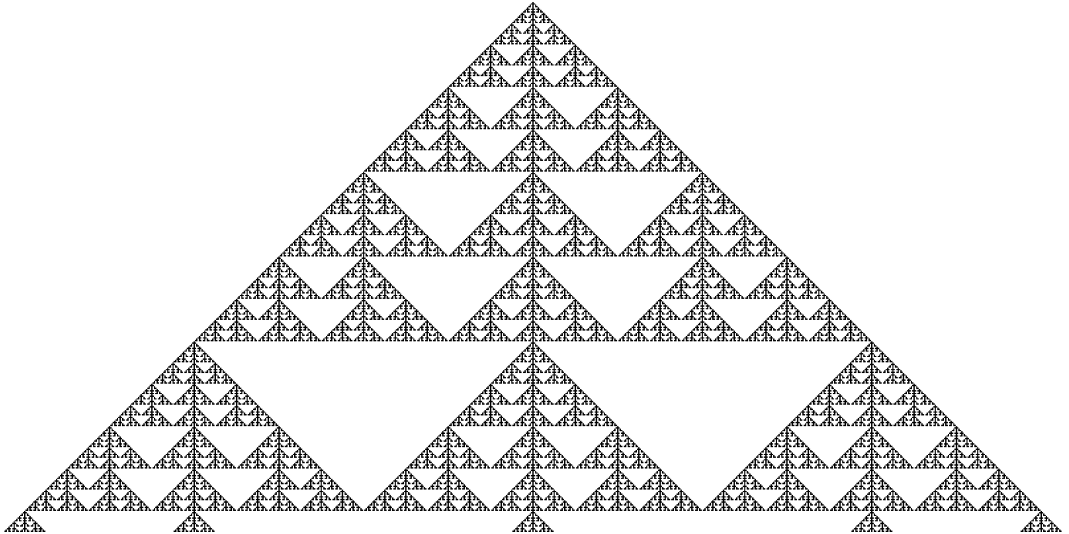
Ein Warnschuss: Komplexität und Chaos im Einfachsten

Wolfram's Rule 90:



Ein Warnschuss: Komplexität und Chaos im Einfachsten

Wolfram's Rule 150:



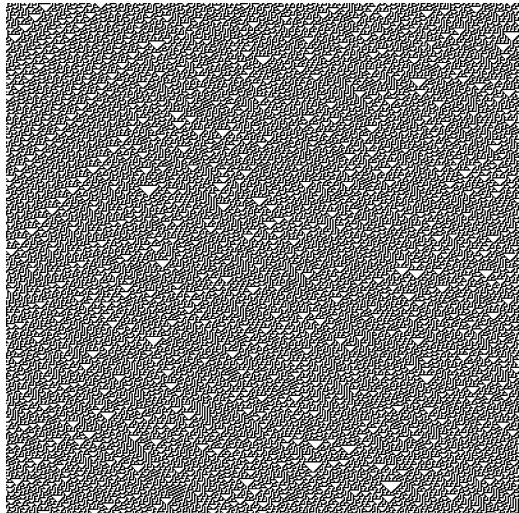
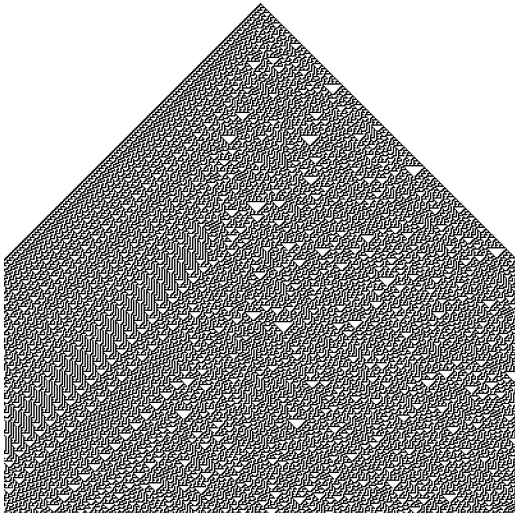
Ein Warnschuss: Komplexität und Chaos im Einfachsten

Dies sind in ihrer Komplexität sehr unerwartete Strukturen (wir müssen dabei bedenken, dass wir hier mit sehr einfachen Regeln arbeiten).

Deutlich verstörender ist folgende Regel, welche sich ebenso aus dieser einfachen Modellierung ergibt.

Ein Warnschuss: Komplexität und Chaos im Einfachsten

Wolfram's Rule 30:



Ein Warnschuss: Komplexität und Chaos im Einfachsten

Diese Regel ist ein sehr eindrückliches Beispiel. An ihr kann man am deutlichsten erkennen, wie aus *einfachsten Modellierungen sehr komplexe Strukturen* entstehen können. Eine der verblüffenden Eigenschaften ist dabei, dass sich dieses Muster scheinbar nicht wiederholt.

Ein Warnschuss: Komplexität und Chaos im Einfachsten

An dieser Stelle wollen wir uns noch einmal etwas allgemeiner über den Begriff „Chaos“ unterhalten.

- Dieses tritt insbesondere bei *deterministischen nichtlinearen dynamischen Systemen* auf. Bspw. wie bei den 1D zellulären Automaten von *Wolfram*.
- Der Begriff „Chaos“ ist nicht einheitlich definiert, jedoch wird er fälschlicherweise oft mit *Zufall* in Verbindung gebracht, was hier grundsätzlich falsch ist. Es geht um deterministische Systeme, welche – grob gesagt – ein Verhalten aufzeigen, dass keiner einfach erkennbaren Vorhersage mehr folgt – obwohl sie deterministisch sind!

Ein Warnschuss: Komplexität und Chaos im Einfachsten

Wie erkennt man nun ob *chaotische Elemente* bei einem System vorliegen?

- Das System reagiert *sehr empfindlich auf leichte Änderungen* der Anfangsbedingungen (hohe Sensitivität). Nach einigem Fortlauf des Systems kann man nicht mehr erkennen, ob zwei Simulationsläufe nur leichte oder größere unterschiedliche Anfangsbedingungen hatten, da die errechneten Zustände komplett anders aussehen.
- Häufig treten zusätzlich *seltsame Attraktoren* auf, d.h. das System scheint sich „global“ betrachtet in seiner Verhaltensweise abschätzen zu lassen. Bspw. die Zustände des dynamischen Systems verhalten sich nach einer Einpendelzeit immer ähnlich (näheren sich sozusagen einem *Attraktor* an), jedoch gilt weiterhin der obige Punkt: leichte Änderungen der Anfangsbedingungen führen zu beliebig unterschiedlichen Zuständen nach endlicher Zeit (d.h. sie sind „lokal“ bzw. „konkret“ nicht vorhersagbar).
- Zusätzlich tritt auch häufig eine sog. *Selbstähnlichkeit* auf, d.h. diese Systeme scheinen sich in unterschiedlichen Bereichen qualitativ immer wieder ähnlich zu verhalten, aber diese Ähnlichkeit ist nicht vorhersagbar und offensichtlich. Diese Eigenschaft wird graphisch am eindrucklichsten deutlich.

Ein Warnschuss: Komplexität und Chaos im Einfachsten

Im zweiten Teil betrachten wir erneut so ein deterministisches Modell, nämlich das logistische Wachstum, welches in der Simulation zum sog. *Feigenbaum*–Diagramm führt (ein Standard–Einführungsbeispiel in die *Chaostheorie*).

Wir betrachten eine Iterationsfunktion

$$x_{n+1} = \Phi(x_n) ,$$

bei der man beginnend von einem Startwert x_0 aus einem Zustand x_n den neuen Zustand x_{n+1} berechnet. Dieser ist natürlich wieder der Input für die Berechnung von x_{n+2} und so fort.

Konkret betrachten wir die Iterationsfunktion des *logistischen Wachstums* mit freiem Parameter r :

$$\Phi(x) = r \cdot x (1 - x) ,$$

mit $r \in \mathbb{R}$. $\Phi(x)$ ist offensichtlich eine nach unten geöffnete Parabel mit Werten $\Phi(0) = 0$ und $\Phi(1) = 0$. Das Maximum liegt bei $\Phi(1/2) = r/4$.

Ein Warnschuss: Komplexität und Chaos im Einfachsten

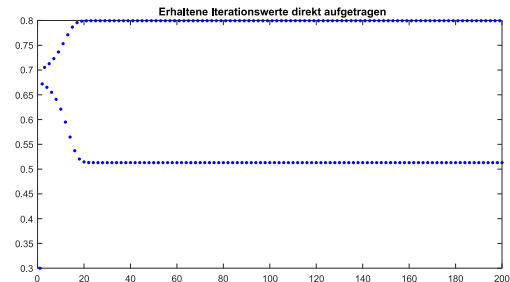
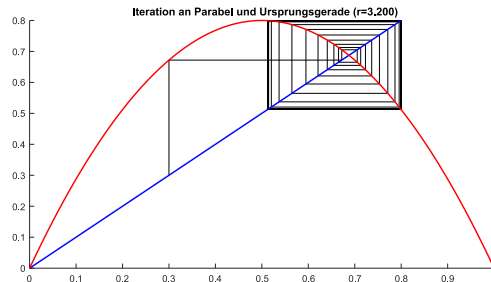
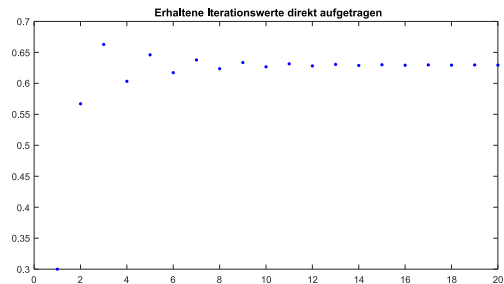
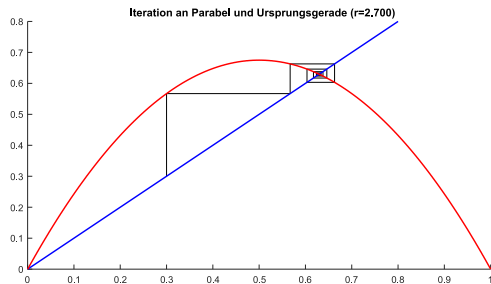
Daraus ergibt sich schon einmal folgende wichtige Erkenntnis: Solange $r \in [0, 4]$, werden Werte des Intervalls $[0, 1]$ wieder auf Werte des Intervalls $[0, 1]$ abgebildet (Maximum ≤ 1).

Dies hilft uns natürlich enorm die Iteration besser zu verstehen: Gilt $r \in [0, 4]$, werden alle Werte der Iteration in $[0, 1]$ liegen.

Kurz: Das globale Verhalten scheint schon einmal sehr einfach abschätzbar.

Betrachten wir folgende Simulationen mit Startwert $x_0 = 0.3$ und unterschiedlichen Werten für $r \in [0, 4]$. Links ist das Fortschreiten der Iteration graphisch dargestellt und rechts sieht man der Reihe nach x_n für $n = 0, 1, 2, \dots$ aufgetragen.

Ein Warnschuss: Komplexität und Chaos im Einfachsten

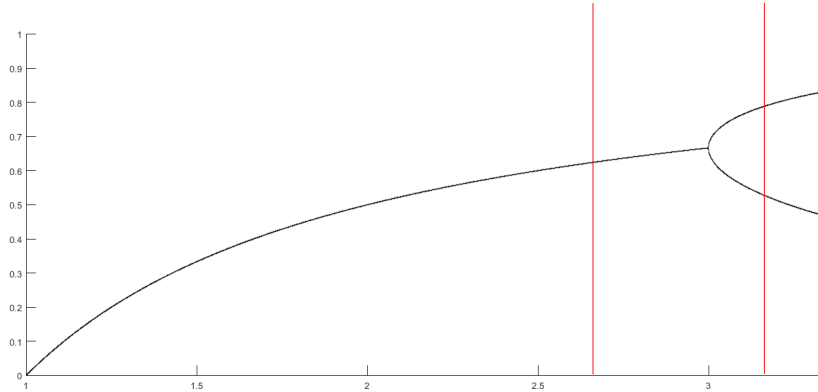


Ein Warnschuss: Komplexität und Chaos im Einfachsten

Man erkennt: Für manche r gibt es einen einzigen Attraktor, bei anderem r zwei Attraktoren, zwischen denen das System in seinen Iterationen pendelt.

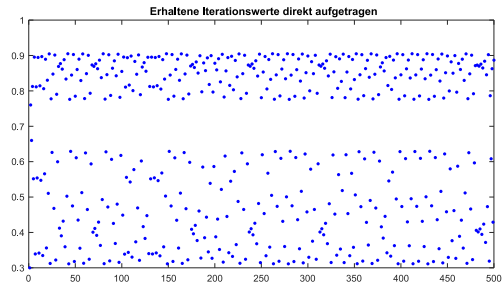
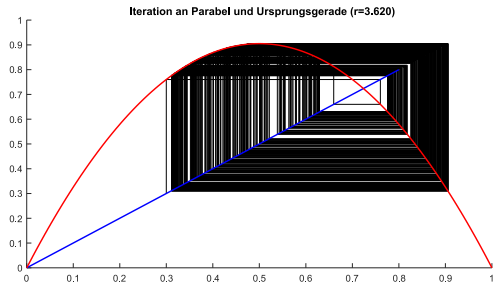
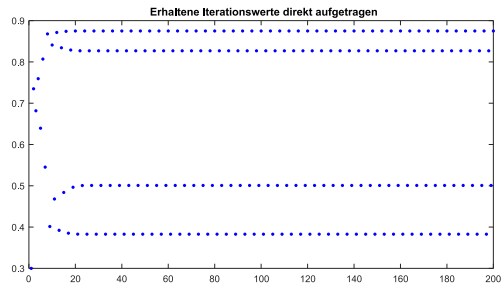
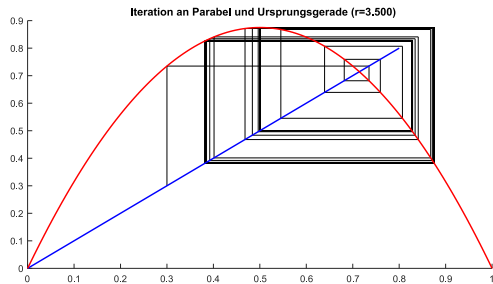
Stellt man nun die Iterationswerte nach dem anfänglichen Einschwingen für beliebige $r \in [1, 3.4]$ (Rechtswertachse) dar, so erhält man das sog. *Feigenbaum*–Diagramm. Darin erkennt man bspw. die beiden gerade gezeigten Fälle an den beiden Stellen mit der roten Markierung. Interessant ist ein Punkt in der Nähe der 3, bei dem plötzlich aus einem Attraktor zwei Attraktoren werden. Hier verändert sich unerwartet das Ergebnis *qualitativ*. Man nennt dies eine *Bifurkation* – dies ist jedoch per se noch nicht „chaotisch“.

Ein Warningschuss: Komplexität und Chaos im Einfachsten



Spielt man dieses System weiter für andere r erhält man folgende Beobachtungen:

Ein Warningschuss: Komplexität und Chaos im Einfachsten



Ein Warnschuss: Komplexität und Chaos im Einfachsten:

Insgesamt ist ein Code zur Berechnung des *Feigenbaum*-Diagramms über r :

```
Matlab/Octave
r_vals = linspace(3.6325,3.6328,5000); % Auswertebereich für r
y_lim = [0.8274,0.8278]; % y-Fenster im Plot
val_length = 4000; % Anzahl der Ergebnispunkte nach transienter Phase
transient = 4000; % Anzahl Punkte bis transiente Phase abgeklungen

N = val_length+transient; % Gesamtzahl Iterationen

vals = zeros(1,N);
f = @(x,r) r*x*(1-x); % Logistisches Wachstum, allgemein

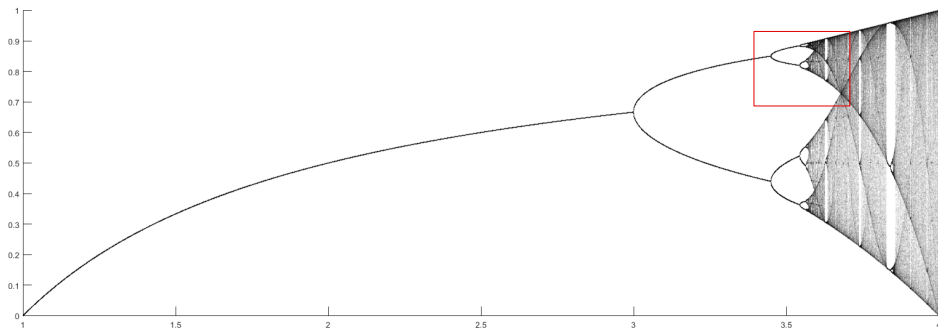
for j = 1:length(r_vals)
    vals_final{j} = [];

    vals(1) = 0.3;
    for i = 2:N % Iteration
        vals(i) = f(vals(i-1),r_vals(j));
    end

    vals_final{j} = vals(transient+1:transient+val_length); % Werte nach transienter Phase
end
```

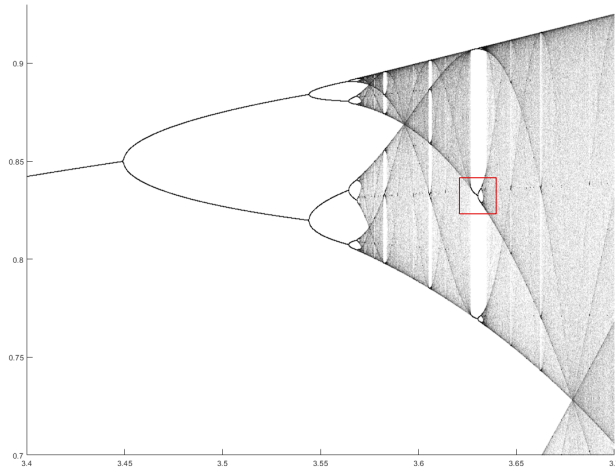
Ein Warnschuss: Komplexität und Chaos im Einfachsten

Feigenbaum $r \in [1, 4]$:



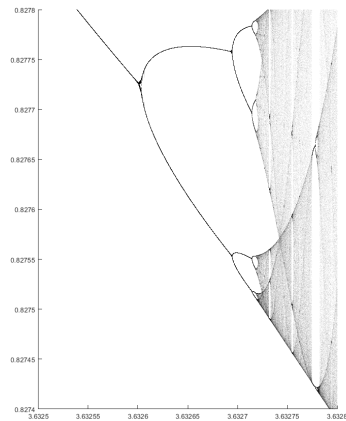
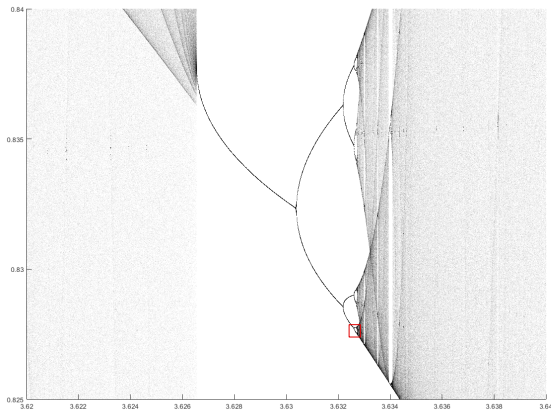
Ein Warnschuss: Komplexität und Chaos im Einfachsten

Feigenbaum $r \in [3.4, 3.7]$:



Ein Warningschuss: Komplexität und Chaos im Einfachsten

Feigenbaum $r \in [3.62, 3.64]$ (links) bzw. $r \in [3.6325, 3.6328]$ (rechts):



Ein Warnschuss: Komplexität und Chaos im Einfachsten

Hier erkennt man die typischen Eigenschaften des Chaos:

- Für kleinste Änderungen von r geht man von klaren, endlichen vielen Attraktoren über in scheinbar unendlich viele Zustände ohne konkreter Regel.
- Dennoch werden selbst in diesen Bereichen gewisse Zustände x_n systematisch nicht angenommen, wohingegen andere x_n durchaus vielfältig angenommen werden können. D.h. Die globale Erkenntnis „alle $x_n \in [0, 1]$ “ lässt sich durchaus noch deutlich genauer sagen: Man kann gewisse Intervalle für x_n angeben. Innerhalb dieser Intervalle gibt es jedoch keine einfach vorhersagbare Regel mehr – ein sog. *seltsamer Attraktor*.
- Durch den starken Zoom mit den roten Kästchen wird eine *Selbstähnlichkeit* des Systems deutlich. Es scheint sich immer wieder ähnlich zu verhalten, ohne dass man das vorhersagen könnte. Beeindruckend ist dabei, wie für kleine Änderungen von r aus einem seltsamen Attraktor wieder ein klassischer Attraktor und dieser dann wieder in einen seltsamen Attraktor übergeht – es wirkt wie „Inseln der Regelmäßigkeit inmitten des Chaos“.

Ein Warnschuss: Komplexität und Chaos im Einfachsten

Das logistische Wachstum ist jedoch alles andere als ein „kompliziertes“ System – es ist sehr einfach. Trotzdem stoßen wir schon bei diesem System auf das „Chaos“. Was sagt uns das über anwendungsnähere komplizierte nichtlineare Systeme aus?

Mehr noch: Was sagt uns das über die Anwendung an sich aus? Was erfahren wir damit über die Natur bzw. unsere „Welt“ in der ständig Dinge dynamisch und nichtlinear auseinander hervorgehen?

Bemerkung 3.31 (Komplexität und Chaos im Einfachsten)

Es ergeben sich folgende Erkenntnisse:

- Mit Sicherheit können wir sehr schwierige Aufgaben durch Modellierung und Simulation heute mit Hilfe von Computern lösen, welche auf den ersten Blick sehr kompliziert oder praktisch unlösbar erscheinen.
- Manchmal muss man jedoch auch mit Demut erkennen: Schon in einfachsten Modellen gibt es vollkommen unbeherrschbare, überraschende Situationen.
- Zwischen diesen beiden Polen liegt die notwendige Erfahrung und Fehlerkultur guter Modellierer:innen und Simulierer:innen.
- Daraus folgt: Eine gute Modellierer:in ist stets suchend und offen für die Veränderung seiner/ihrer Modelle. Er/Sie unterschätzt die Komplexität selbst in einfachen Modellen nicht und wagt dennoch auch die schwierigen Probleme der Anwendung sinnvoll anzunähern.
- Auf diesem Weg unterstützen die Modellierer:in *Verifikation*, *Validierung* und die Diskussion mit dem/der Aufgabensteller:in (auch bezüglich der verfügbaren Ressourcen).